PRODUCT DOCUMENTATION

HORIZON GRID

Every Signal. One Operational Picture.

Version: 0.2.5

Windows Installer: HORIZON-GRID-Setup-0.2.5.exe

Installer SHA-256: 98dee90cfb16a9c9b2bd56611ba024225fa0261c20386765171fe79e01d198c2

Documentation prepared: 2026-08-23

PREPARED FOR EVALUATION

Table of Contents

PART I — PRODUCT GUIDE

PART II — TECHNICAL APPENDIX

Executive Overview

HORIZON GRID is a self-hosted application that takes a single indicator of compromise -- an **IOC** (Indicator of Compromise), meaning a piece of evidence such as an IP address, domain name, URL, file hash, or CVE ID (a publicly catalogued software vulnerability, e.g. "CVE-2021-44228") -- and looks it up simultaneously against a wide range of third-party threat-intelligence sources. Rather than opening a dozen browser tabs and pasting the same indicator into a dozen different tools, an analyst pastes it once into a single search box. The platform queries every relevant provider in parallel, waits for their answers, automatically looks for relationships between what those providers reported, and then asks an AI model to summarize the whole picture in plain language.

This tool is built for security analysts doing IOC triage: the day-to-day work of taking an indicator that showed up in an alert, a phishing report, an incident, or a threat feed, and quickly forming a judgment about whether it is dangerous, and why. It is equally usable by a solo analyst working a single alert and by a small security team that wants a shared, self-hosted place to run lookups, keep notes, and track cases -- because the platform runs entirely on infrastructure the organization controls, nothing about the indicator or the investigation needs to leave that environment except the outbound calls to the third-party providers the analyst has chosen to configure.

The value proposition in one sentence: **one search, many providers, correlated, AI-summarized, and evidence-backed** -- an analyst gets in seconds what used to take many separate tools and a lot of manual cross-referencing to piece together, along with a way to check, item by item, exactly where every claim in the summary came from.

The Problem

Investigating a single indicator properly means checking it against several independent sources, because no one source sees everything. A file hash might be flagged by one antivirus engine and ignored by another. An IP address might be a known Tor exit node according to one list and a clean, unremarkable address according to another. A domain might have no reputation history anywhere but still be tied to a live vulnerability. Getting a trustworthy picture means checking multiple sources and weighing what they collectively say -- not any single answer in isolation.

In practice, that means an analyst opens a reputation lookup site, then a sandbox or malware database, then a certificate-transparency search, then a vulnerability database, then a passive-DNS tool, then a DNS blocklist -- copying and pasting the same IOC into each one, reading each result in its own format, and holding all of it in their head (or in a scratch document) well enough to notice when two sources disagree or, more importantly, when two sources are quietly describing the *same underlying thing* from different angles. That second part is the real cost: correlations -- "this IP resolves to infrastructure tied to this malware family, which is associated with this ATT&CK technique, which matches this hash" -- are exactly the kind of connection that's easy to spot when everything is laid out side by side, and easy to miss when it is spread across six browser tabs opened at six different times.

Multiply this by every indicator that needs triage in a day, and the real cost isn't just the minutes spent per lookup -- it's the tabs, the context-switching, the copy-paste errors, and the correlations that quietly get missed because no human is going to manually cross-reference a dozen sources for every single IOC that crosses their desk.

The Solution

HORIZON GRID collapses that whole workflow into a single search box. An analyst submits one IOC, and the platform:

- **Queries every applicable provider in parallel** -- rather than one at a time, so results arrive within seconds of submission instead of after a long serial wait, with each provider's individual verdict shown side by side as it comes in.
- **Correlates the results automatically** -- extracting relationships such as shared IPs, related hashes, malware families, MITRE ATT&CK techniques, and CVEs across the providers that reported them, and surfacing where multiple independent sources corroborate the same fact.
- **Summarizes the findings with AI** -- both a short summary of each provider's own result, and one consolidated final assessment that reasons over all of the provider summaries together with the correlation output, written in plain language instead of a wall of raw JSON.

That last point comes with an important caveat, and the platform is built around it rather than around hiding it: **AI output is analytical assistance, not unquestionable truth.** The AI model can misread evidence, and providers themselves can disagree -- a genuine, documented example is the IOC `8.8.8.8` (Google's public DNS resolver), where one provider's DNS blocklist flags it as malicious due to a query-permission error while two other providers report it as clean, and the AI's own executive summary has to explicitly reconcile that disagreement rather than paper over it. Because of cases exactly like this, every AI-generated claim on an investigation page is traceable back to the underlying evidence it was built from, through the **Evidence Ledger** -- a deterministic, non-AI-generated list of the concrete facts (provider results, correlation edges, and summaries) that any AI explanation is required to cite. An analyst never has to take the AI's word for it; they can always check the receipts.

Key Capabilities

- **Multi-provider IOC lookup** -- submit one indicator (IP, domain, URL, file hash, or CVE) and query every configured, applicable threat-intelligence provider at once, with results streaming in live as each provider responds.
- **AI-assisted analysis grounded in evidence** -- a plain-language summary per provider plus one consolidated final assessment, with dedicated views for "Why this verdict?", score explanations, provider disagreements, false-positive checks, and a "Challenge This Verdict" option -- all designed to be checked against real evidence, not taken on faith.
- **Correlation and relationship graph** -- automatic detection of relationships between providers' findings (shared infrastructure, related hashes, malware families, ATT&CK techniques, CVEs), visualized as a relationship graph the analyst can explore.
- **Case management** -- create a case, attach IOCs to it as the investigation grows, and add freeform analyst notes to build a record of the work.
- **IOC Basket** -- a saved, running list of IOCs an analyst wants to keep an eye on or revisit, with the ability to investigate or compare selected items together.
- **JSON and Markdown export** -- download an investigation's results in either format for use outside the platform.
- **Windows installer with a guided setup wizard** -- a standard installer followed by a step-by-step wizard that creates the administrator account, configures the AI backend and threat-intelligence providers (with live "Test" buttons for each), reviews network ports, and installs the platform end to end.

The User Journey

An orientation map, not a manual: what a session with HORIZON GRID looks like end to end, from one-time setup through a routine day of investigating indicators.

Setup, Once

An admin installs the platform via the Windows installer, then runs a setup wizard collecting an administrator account, an AI backend choice, and API keys for whichever threat-intelligence providers are available. After that, day-to-day use is just opening a browser tab.

Signed in, an analyst sees the home page: a single search box, ready for the first IOC (Indicator of Compromise — an IP, domain, URL, file hash, or CVE ID — a Common Vulnerabilities and Exposures identifier for a known software vulnerability — worth investigating).

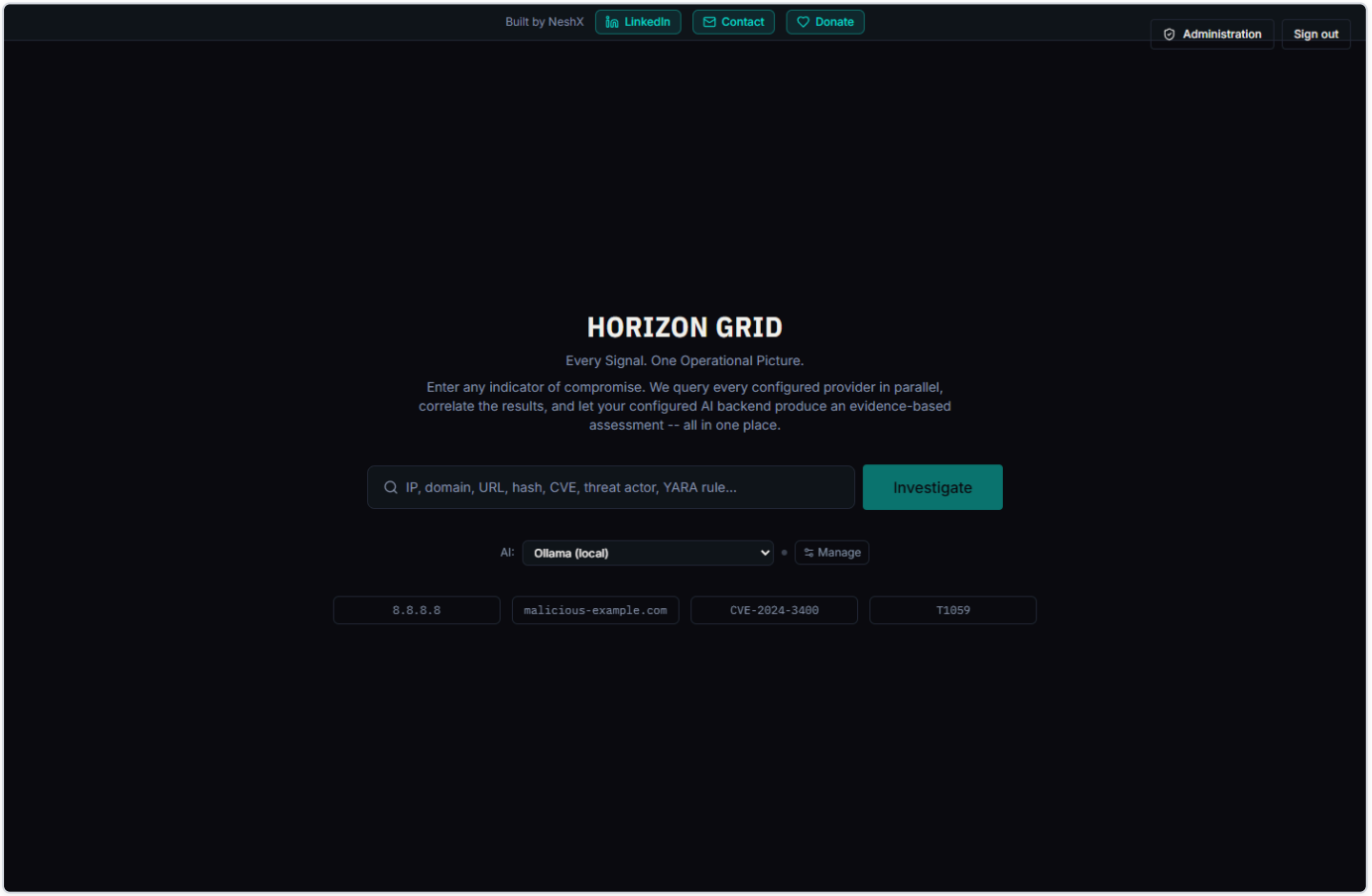

Figure 1 — The platform's home page after signing in — a single search box ready to accept an IOC.

From here, the new **Dashboard** entry in the global navigation is a natural first stop too, before diving into a specific IOC — an at-a-glance operational view of active investigations, case load, provider health, and AI reliability across the whole team.

The Daily Loop: Submit an Indicator, Watch Evidence Arrive

This is the core workflow, repeated many times a day. The analyst types or pastes an IOC and submits it. The platform classifies the indicator type and queries every configured provider that supports it in parallel, not one after another — results stream in as each finishes. As each result arrives, the AI briefly summarizes that provider's own findings; once all providers report in, one consolidated AI call produces a Final Assessment: a verdict, a risk score, and a plain-language summary weighing everything together.

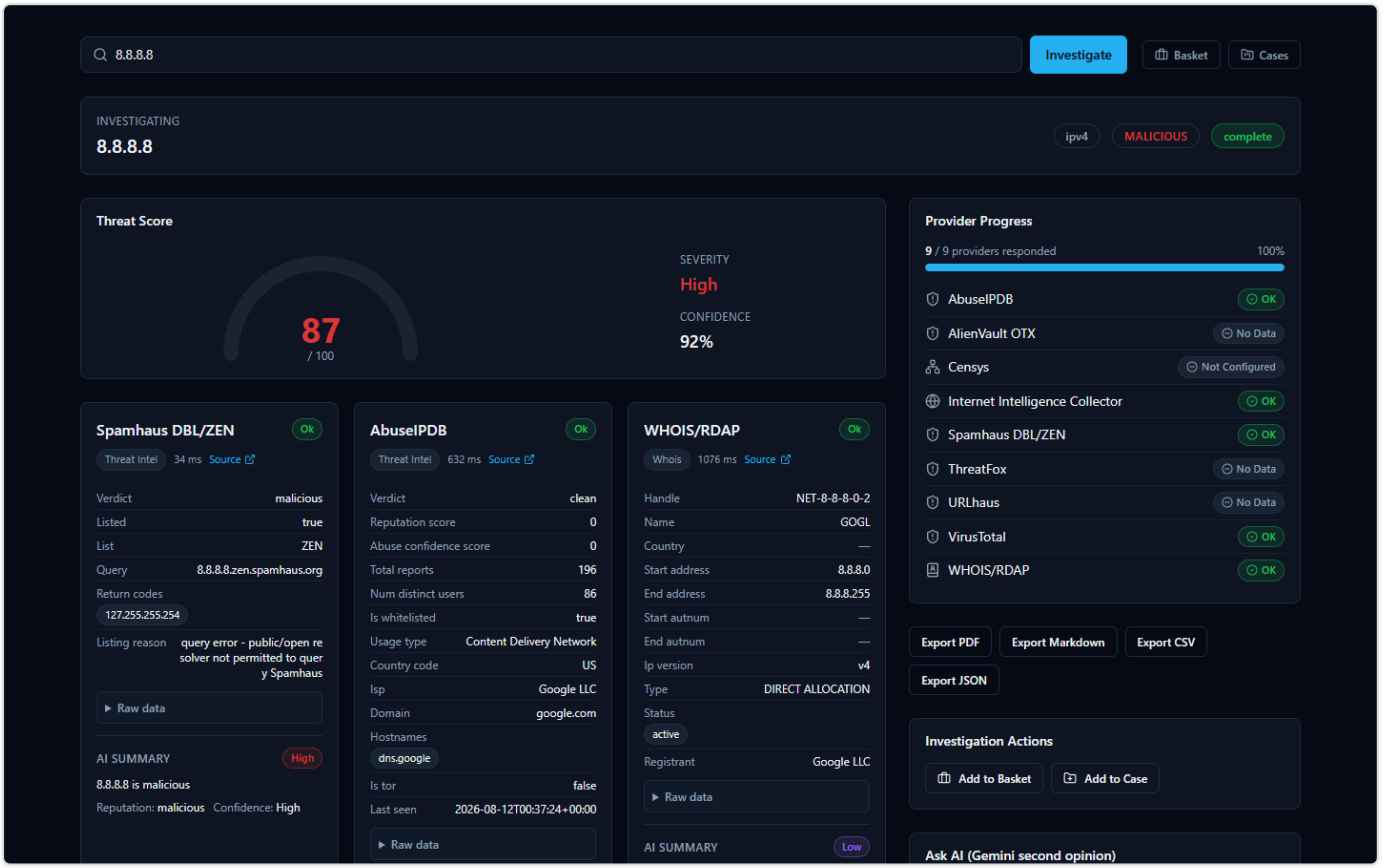

Figure 2 — A lookup of 8.8.8.8 (Google's public DNS resolver) mid-flight, with provider verdict cards arriving in parallel — Spamhaus flags it malicious over an unrelated DNS-policy reason, while AbuseIPDB shows it clean with zero abuse reports.

Reading the AI's Summary — and Verifying It

The AI summary is a fast way to orient, not a verdict to accept blindly — providers genuinely disagree, and the AI's job is to weigh that disagreement, not hide it. The 8.8.8.8 case above is a real example: Spamhaus calls the address malicious (it rejects lookups from public resolvers, unrelated to actual malice), while AbuseIPDB and VirusTotal both call it clean, and the Final Assessment says so explicitly. This is why every investigation includes an Evidence Ledger — non-AI-generated facts with confidence scores. When a claim looks surprising, the analyst opens the ledger and checks it against the real provider data before trusting it.

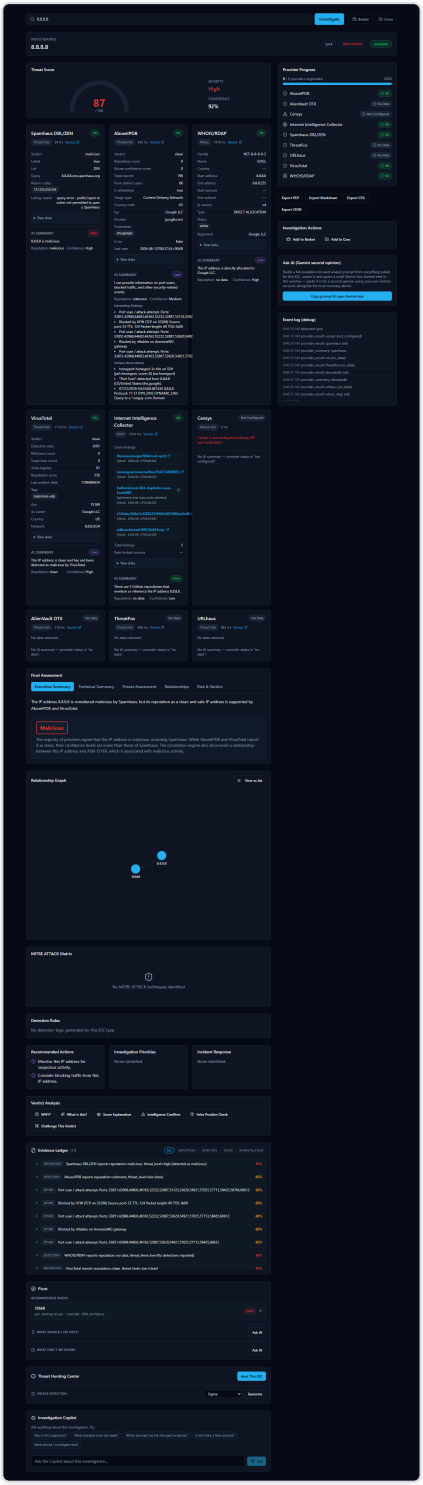

Figure 3 — The full 8.8.8.8 investigation page, with the Final Assessment's summary alongside the Evidence Ledger and Relationship Graph used to check any claim against the underlying evidence.

Following the Trail, Then Keeping Track

The Relationship Graph shows indicators connected to the one just searched — for 8.8.8.8, its parent ASN (Autonomous System Number) — letting the analyst pivot onward without a fresh search. Interesting IOCs go into the IOC Basket, a running list serving the

purpose of a personal watchlist, or get attached to a Case with analyst notes when tied to a specific incident.

Taking Findings Elsewhere

Investigations export cleanly as JSON or Markdown today. PDF and CSV buttons exist but return an honest "Export format not yet available" message — a known gap, not a hidden failure.

Installation Guide

What the Installer Actually Does

HORIZON GRID is a **self-hosted** tool: instead of sending your indicators to someone else's cloud service, you run the entire product on a Windows machine you control. "IOC" stands for **Indicator of Compromise** — a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID (a public vulnerability identifier) that a security analyst wants to investigate.

When you run the installer, you are not just copying a single program onto your machine. You are setting up a small, self-contained application stack — the platform's web interface, its own database, and several supporting background services — all bundled together and running only on this one Windows computer. The internal databases behind the platform are locked to this computer only and are never reachable from anywhere else on your network. The platform's own web interface, however, is genuinely usable from other devices on the same local network, not only from this computer — the installer sets this up automatically (a Windows Firewall rule scoped to your Private network, plus automatic detection of this machine's network address), and the app itself figures out the right address to talk to no matter which device opened it. See "Accessing From Another Computer" below for how to actually do this; if you'd rather keep the platform usable only from this machine, restrict access at your Windows firewall.

You do not need to understand how those internal pieces fit together to install and use the product. This guide only covers what you, the administrator, will see and click. A full technical breakdown of the underlying architecture is provided separately for readers who want it.

Before You Begin: Prerequisites

Two things matter before you start the installer:

- **Administrator privileges.** You must run the installer as a Windows administrator. The installer checks for this and will not proceed without it.
- **Docker Desktop, installed and running.** This is the important one to understand: the platform's real, working services — its database, its background job runners, its web server — do not run as ordinary Windows programs. They run inside **Docker containers**, which is a standardized way of packaging and running software in isolated, self-contained units. Docker Desktop is the engine that makes those containers run on Windows. If Docker Desktop is not installed and running before you start, the installer's prerequisite check will flag it and prompt you to fix it before continuing.

Think of the installer as putting the platform's files in place and Docker Desktop as what actually breathes life into them afterward. Both are required.

Step-by-Step: Running the Installer

The installer is a standard Windows setup wizard (built with Inno Setup, a common Windows installer framework). It walks you through a short series of screens before copying any files.

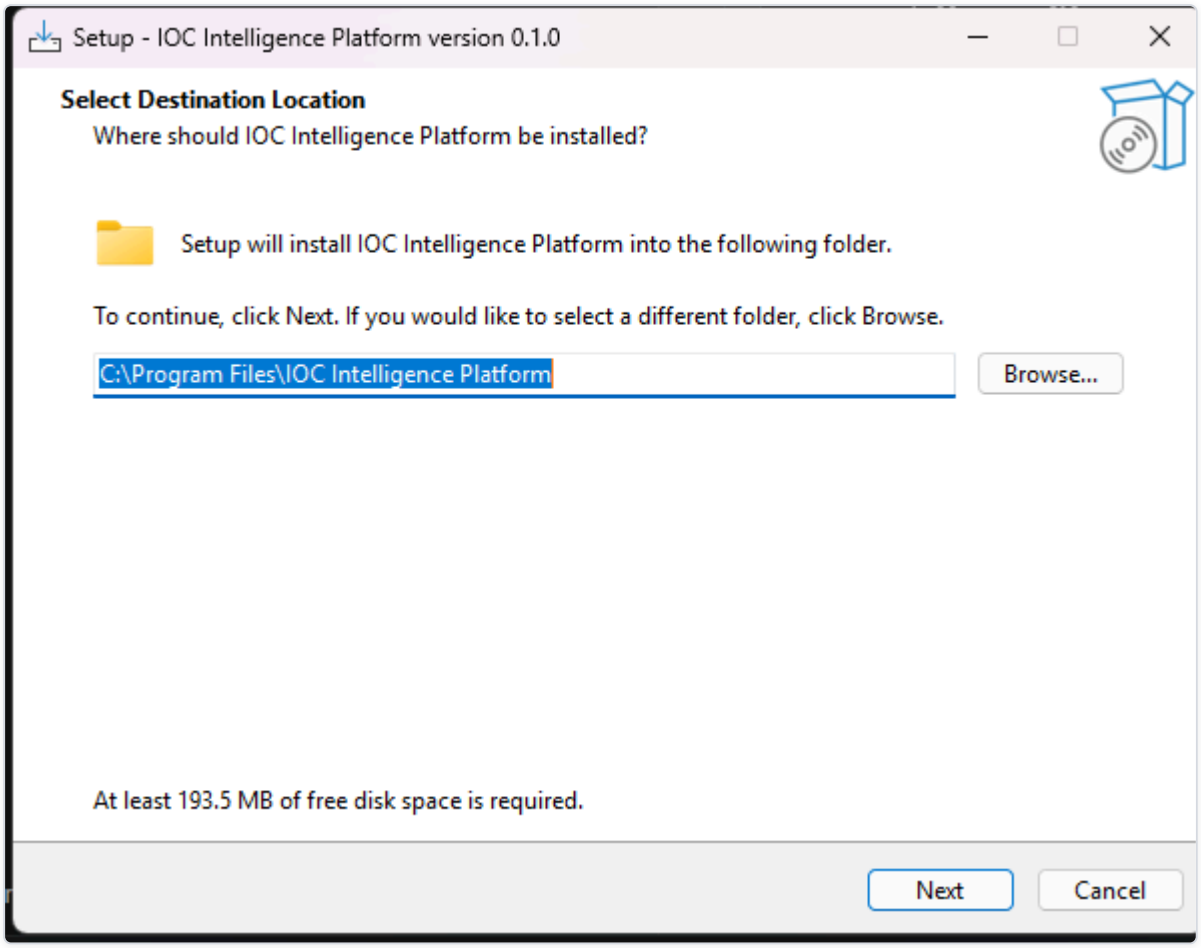

Figure 4 — The installer's "Select Destination Location" page, where the administrator confirms or changes the folder on this machine where the platform's program files will be installed.

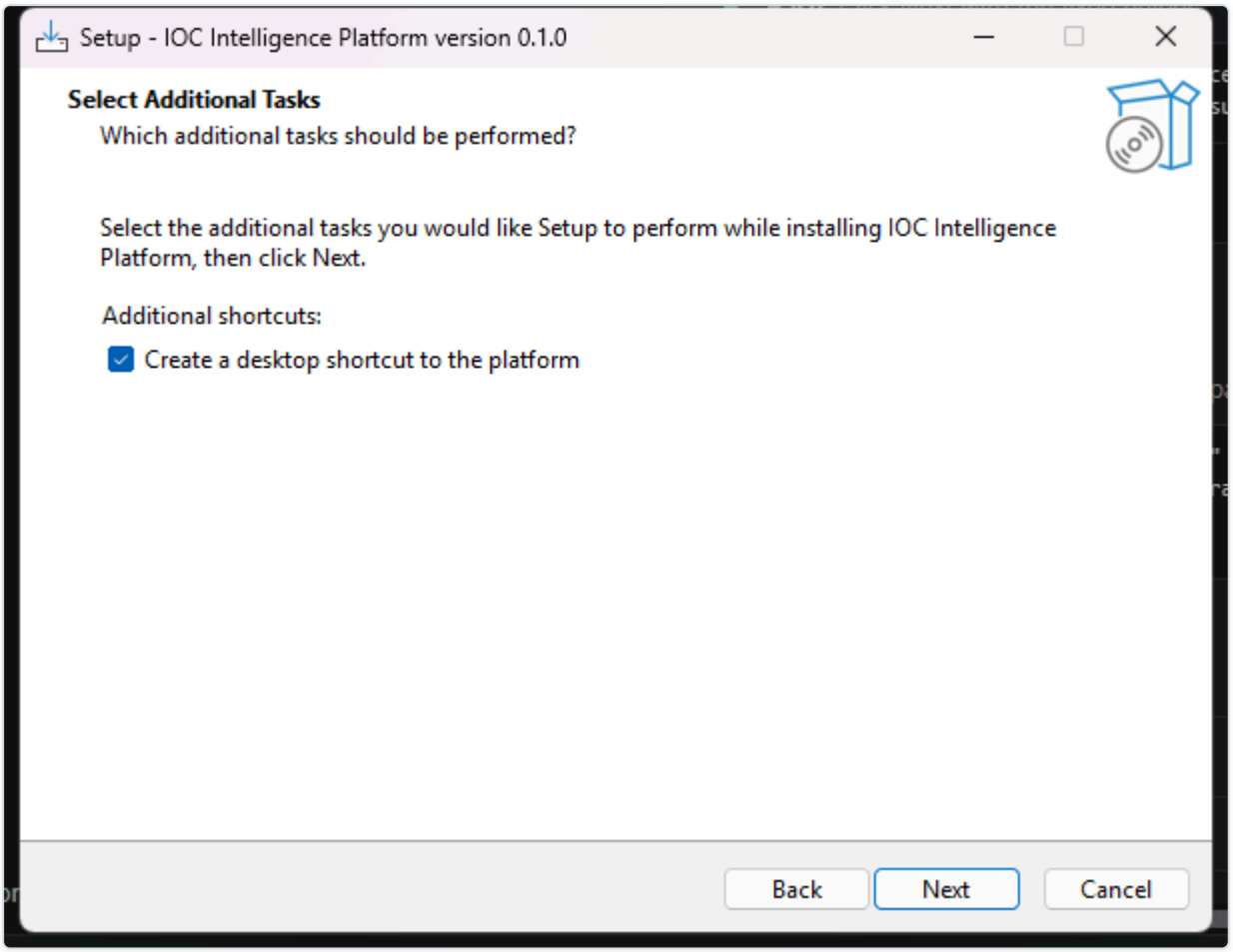

Figure 5 — The installer's "Select Additional Tasks" page, where the administrator chooses optional extras — such as creating a desktop icon — before installation begins.

On the next screen, the installer summarizes everything it is about to do before it touches your system.

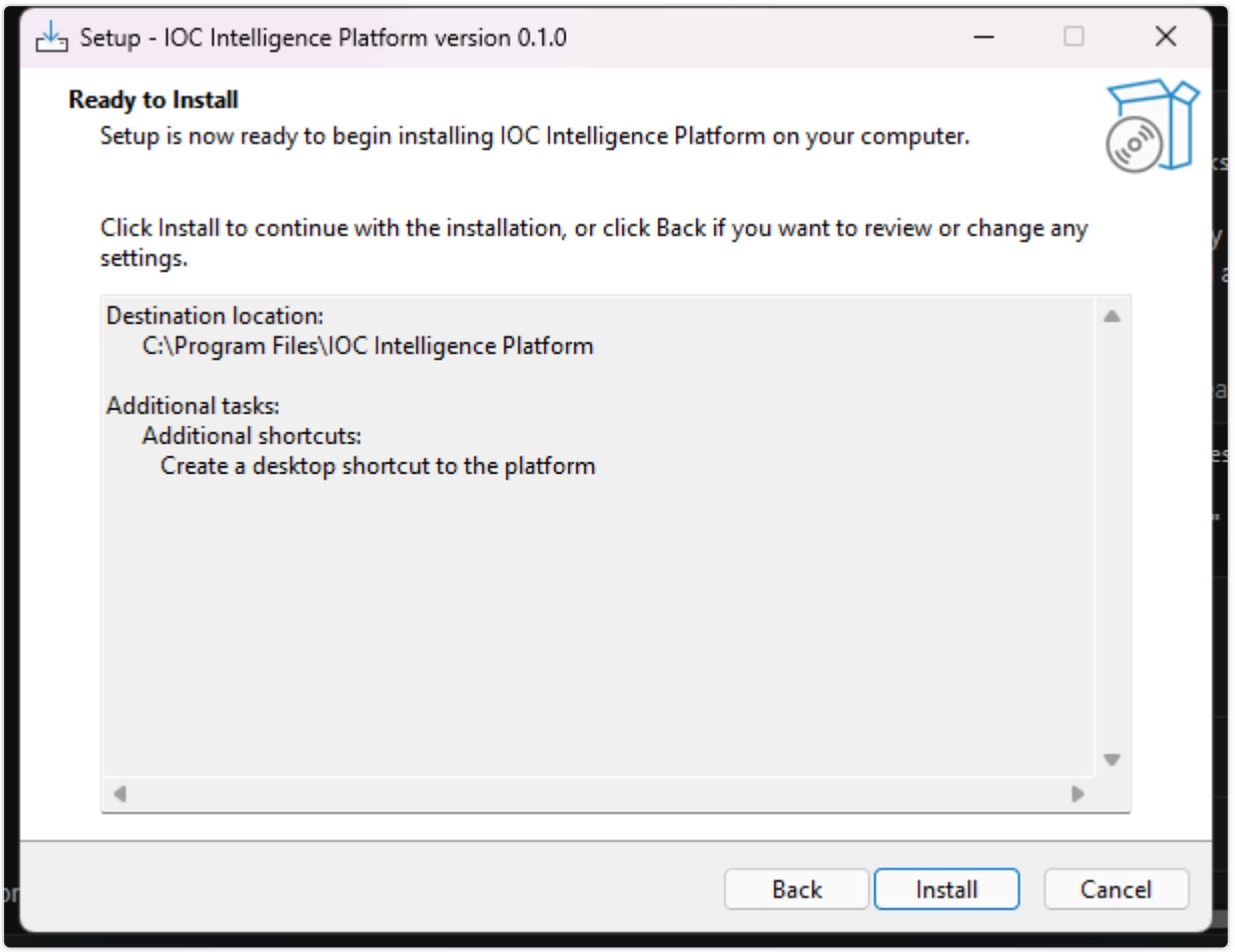

Figure 6 — The installer's "Ready to Install" summary page, giving the administrator a final review of the chosen destination and selected tasks before clicking Install.

After you click through, the installer copies the platform's program files to your chosen location. This step only places files on disk — it does not yet start any services or ask you for any configuration. When it's done, you'll see the final screen:

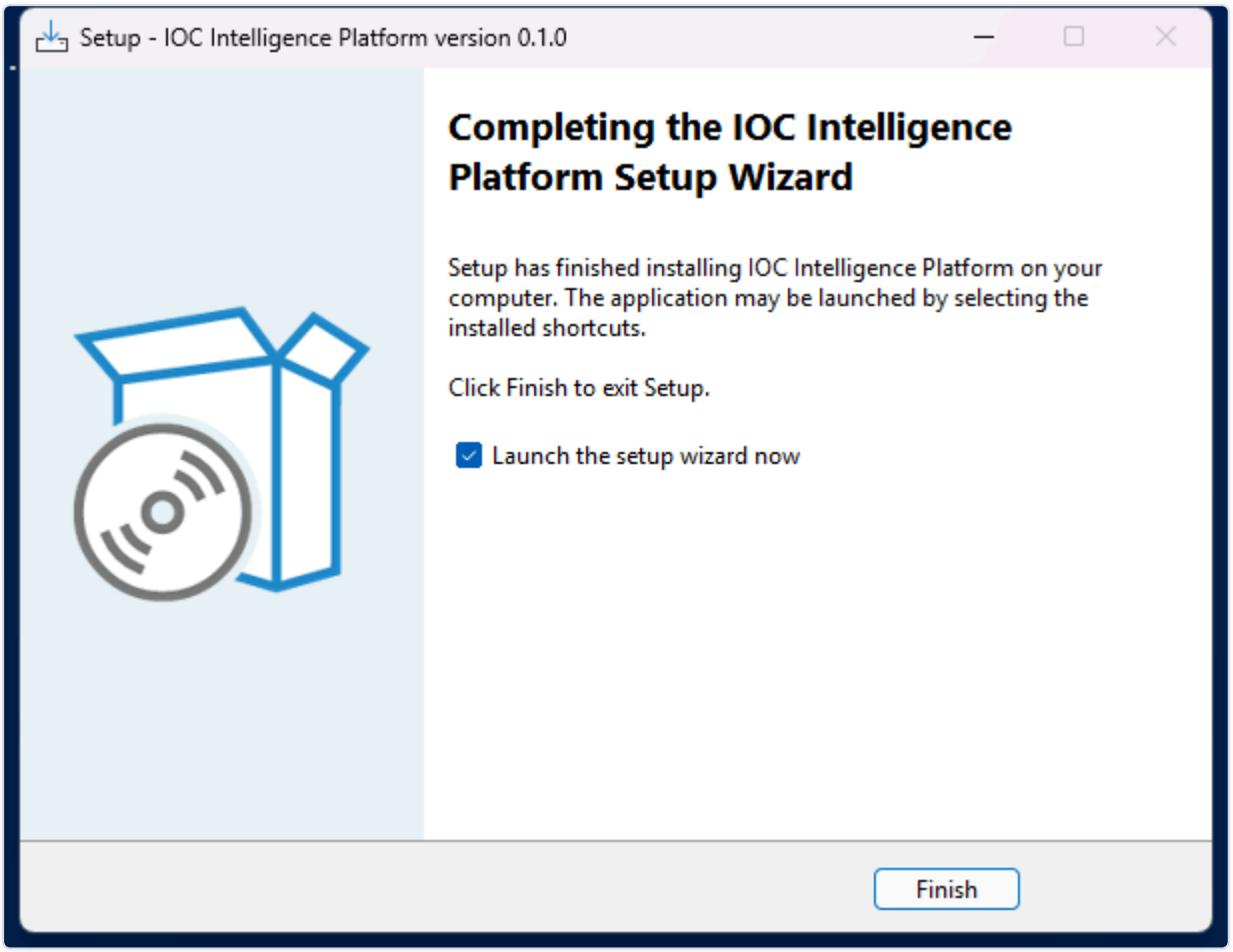

Figure 7 — The installer's "Completing Setup" page, confirming the file copy has finished successfully.

What Happens Next

Once you close the installer, the **Setup Wizard launches automatically**. This is a separate, second stage — a dedicated desktop application (not a web page) that collects your administrator account details, your AI configuration, and your threat-intelligence provider API keys, and then does the actual work of building and starting the Docker containers described above. That process is covered in the next section, "Setup Wizard."

Accessing From Another Computer

Once setup finishes, the platform is usable both from this computer and from any other device on the same local network — a second laptop, a colleague's desktop, even a phone or tablet's browser, as long as it's connected to the same Wi-Fi or Ethernet network. Nothing needs to be edited or reconfigured to make this work; it's set up automatically during installation.

How to connect from another device. The Setup Wizard's final page shows two links: one for using the platform on this computer, and a second one labeled for other devices on the network. Open that second link's address (it looks like `http://` followed by a set of numbers and a port, for example `192.168.1.125:3000`) in a browser on the other device. You'll land on the same sign-in page described in the next chapter, and everything from there on works identically to using the platform locally.

If you don't have that page open anymore, the same address is always available afterward from inside the app itself: sign in, open **Providers** in the top navigation, and select the **Network Access** tab. It shows the address for this computer and the address for other

devices side by side, with a button to copy the second one so you can send it to whoever needs it.

What makes this work. The installer creates a Windows Firewall rule that allows the platform's two ports through, but only on networks Windows classifies as **Private** (the setting you're asked about the first time you connect to a new Wi-Fi network or Ethernet connection) — never on Public networks, and never open to the wider internet. This is deliberate: the feature is meant for a trusted home or office network, the same network your other devices are already on, not for remote access from elsewhere.

If a second device can't connect:

- Make sure both devices are actually on the same network, and that the network is set to **Private** in Windows (check under Settings → Network & Internet on the computer running the platform) — the firewall rule intentionally will not open the ports on a network marked Public.
- If the address shown doesn't work, your router may have assigned this computer a new network address since setup last ran (this can happen after a restart). Re-run **Configuration** from the Start Menu (Start Menu → IOC Intelligence Platform → Configuration) to re-detect it — no need to reinstall.
- The Network Access tab will say "Not detected" if the wizard couldn't determine a network address automatically (uncommon, but possible on unusual network setups). The platform still works fine from this computer either way; re-running Configuration is worth trying to pick it up.

First-Run Configuration

Before you ever open a browser and type in an IOC (Indicator of Compromise -- a piece of evidence such as an IP address, domain, or file hash that a security analyst wants to investigate), you'll meet HORIZON GRID's Setup Wizard. This is a native Windows desktop application (built the traditional WinForms way, not a web page you'd load in a browser) that runs once, right after the installer finishes, and walks you through every decision the platform needs before it can start working: who the administrator is, which AI model will do the analysis, which third-party threat-intelligence sources you want to plug in, and which network ports it should use on this machine.

None of these choices are permanent traps -- everything set here can be revisited later by re-running the same wizard from the "Configuration" shortcut the installer creates. But for a first-time administrator, this is where the platform's whole configuration model gets introduced, so it's worth walking through page by page.

Welcome

The wizard opens with a simple Welcome page that confirms you're about to configure HORIZON GRID and sets expectations for what's coming next: an administrator account, an AI model choice, provider setup, a network port review, and finally installation itself.

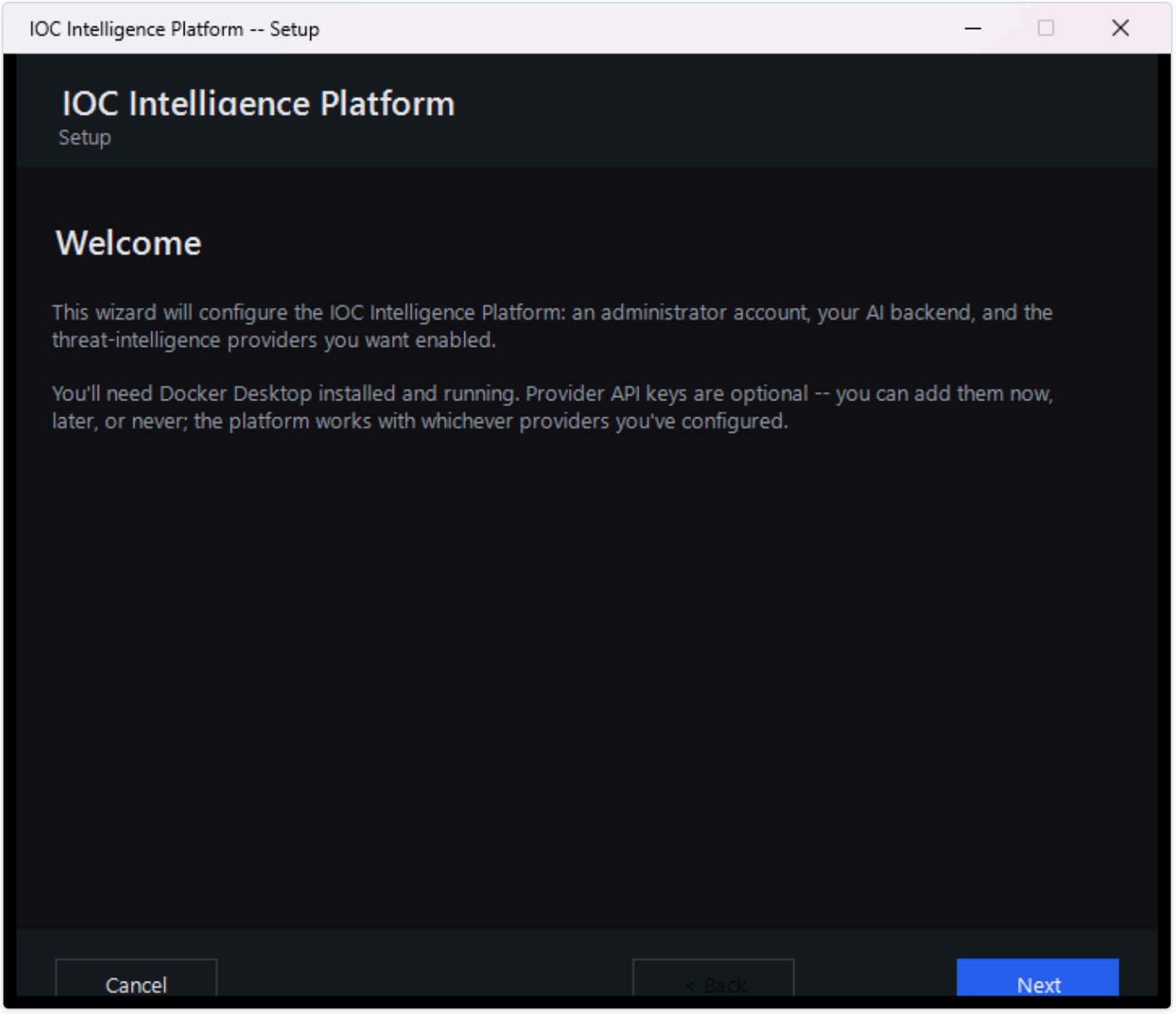

Figure 8 — The Setup Wizard's Welcome page, the first screen a new administrator sees after the installer finishes.

Setting Up the Administrator Account

The next page asks for the details of the very first user account: an email address, an optional full name, and a password (typed twice, to confirm it).

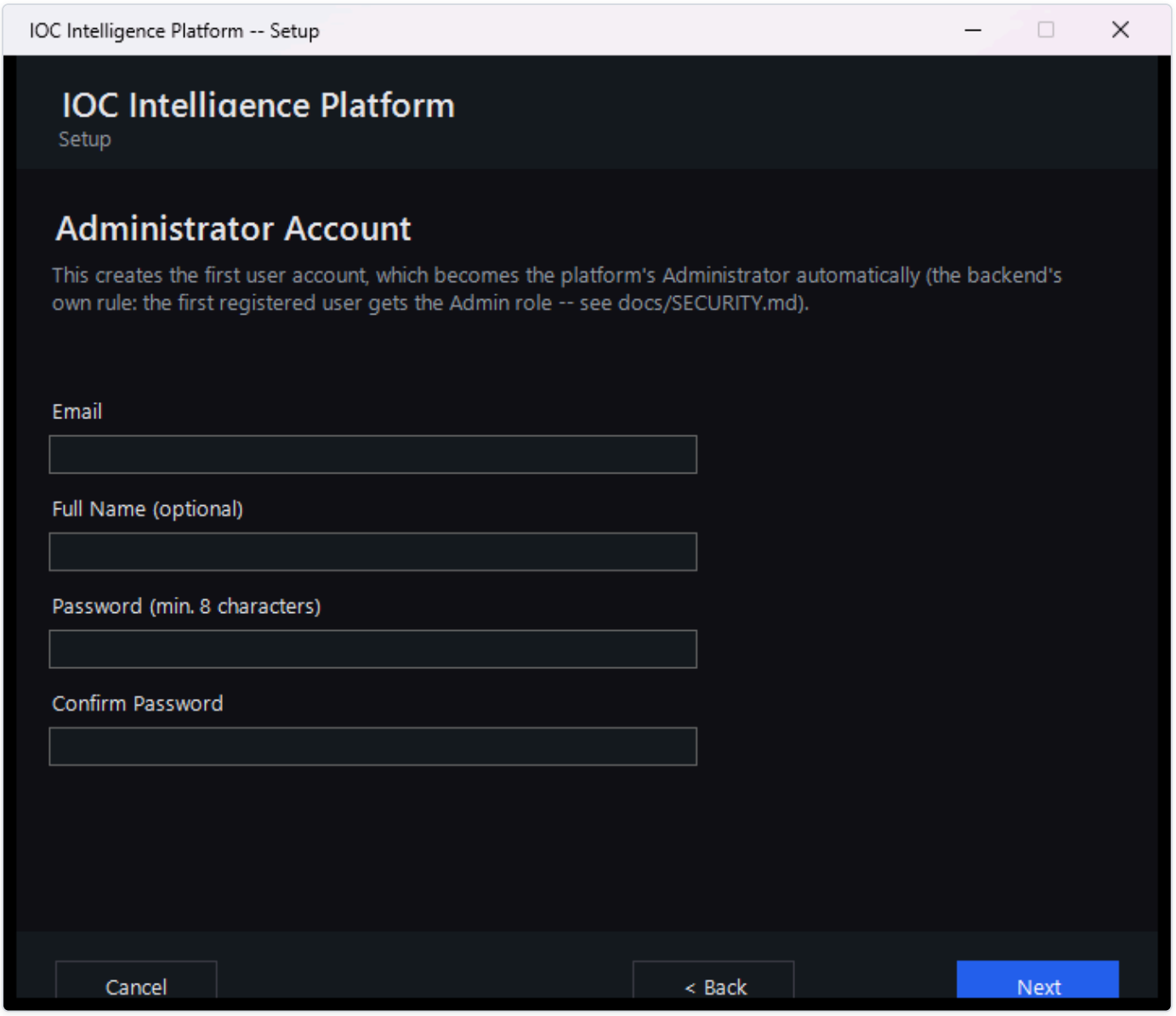

Figure 9 — The Administrator Account page with its fields still blank, showing exactly what information the wizard needs to create the first user.

This page's own note that the first account "gets the Admin role -- see docs/SECURITY.md" is pointing at a file inside the product's source code, for developers who have it checked out -- not something you need to go find. Everything that file says about roles is covered in plain language in this manual's Security and Data Handling section, so nothing is missing if you don't have code access.

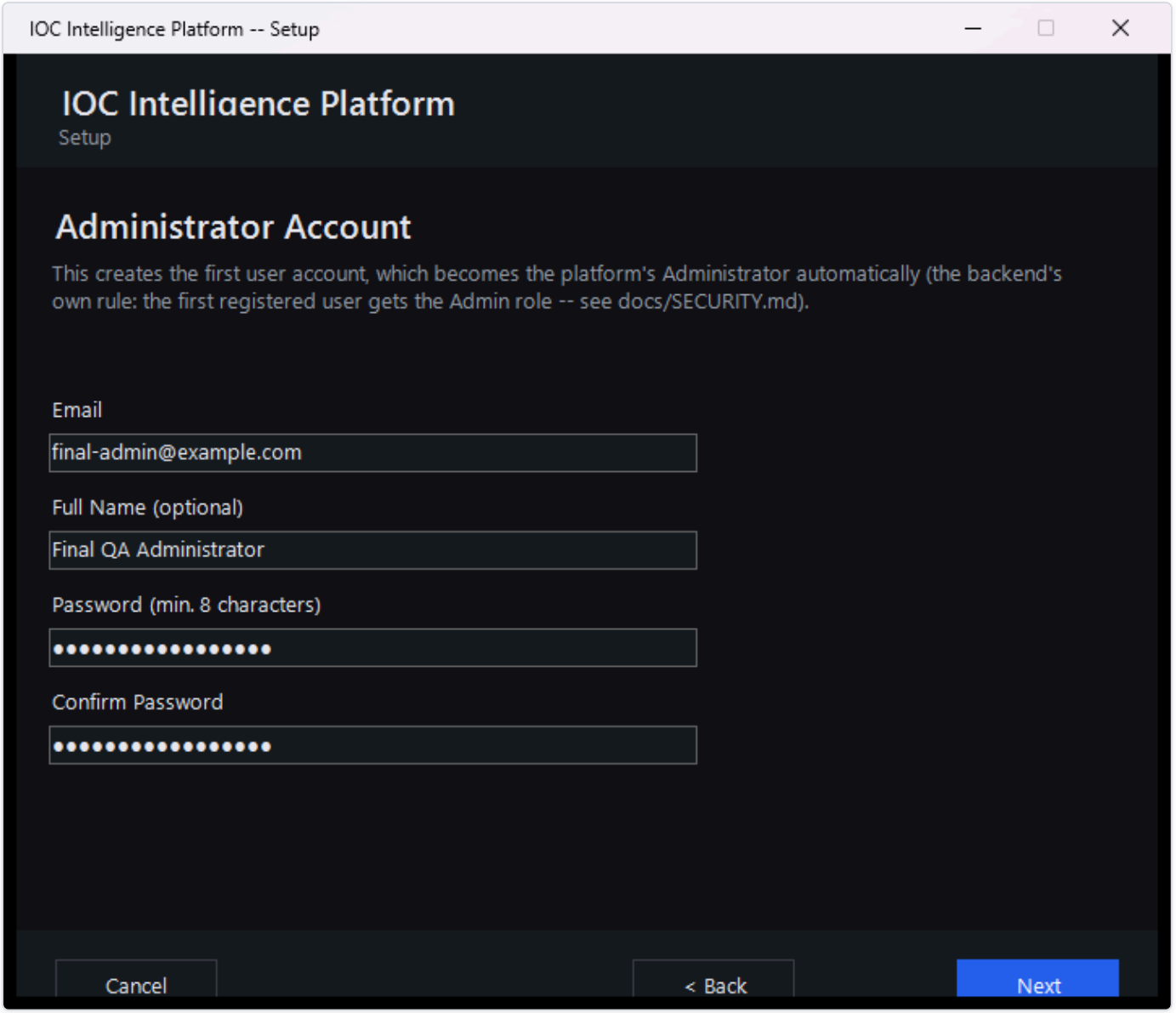

Figure 10 — The same Administrator Account page after an email address, name, and password have been entered.

This page looks like an ordinary "create your account" form, but it quietly does something more important: **whoever registers here becomes the platform's administrator, automatically, with no extra step.** The platform has a simple, fixed rule for this -- the very first account ever registered on a fresh installation is automatically granted administrator rights. The moment that happens, the public sign-up page closes itself: anyone who tries to register through the web app afterward (for example, a colleague who wants an account of their own) is turned away and told to ask an administrator instead, rather than quietly being handed a lesser account. There's no separate "make me an admin" checkbox or database edit required; it's purely a matter of being first. Because the Setup Wizard registers your account before anyone else can reach the platform, filling in this page is what makes you the administrator -- and every colleague's account after that has to be created for them from the Administration page, where you pick their role up front.

If you run the wizard again later to *reconfigure* an installation that's already up and running, this page behaves a little differently, since an administrator account already exists. You can simply leave the email and password fields blank to keep that account exactly as it is. Or, if you'd like to sign in for this session -- which is what enables the live Test Connection buttons on the pages that follow -- re-enter your existing email and password here; the wizard will log you in without creating a new account or changing the existing one.

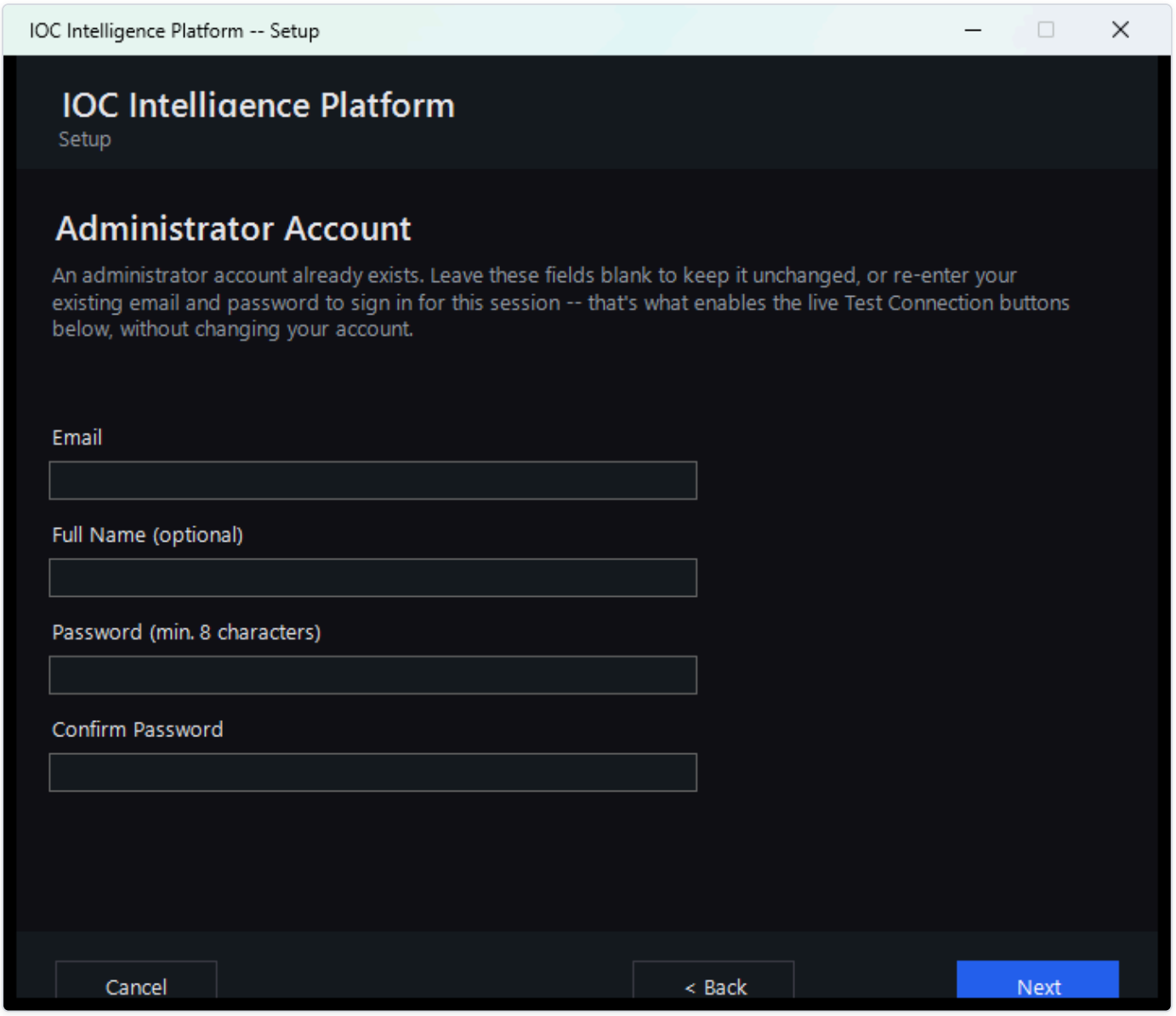

Figure 11 — The Administrator Account page on a reconfigure run, with guidance explaining that leaving the fields blank keeps the existing account unchanged, while re-entering the existing email and password signs in for the session and enables the Test Connection buttons below.

Choosing an AI Model

Every investigation the platform runs ends with an AI-generated summary that pulls together what all the different intelligence sources found. This page is where you decide which AI does that work.

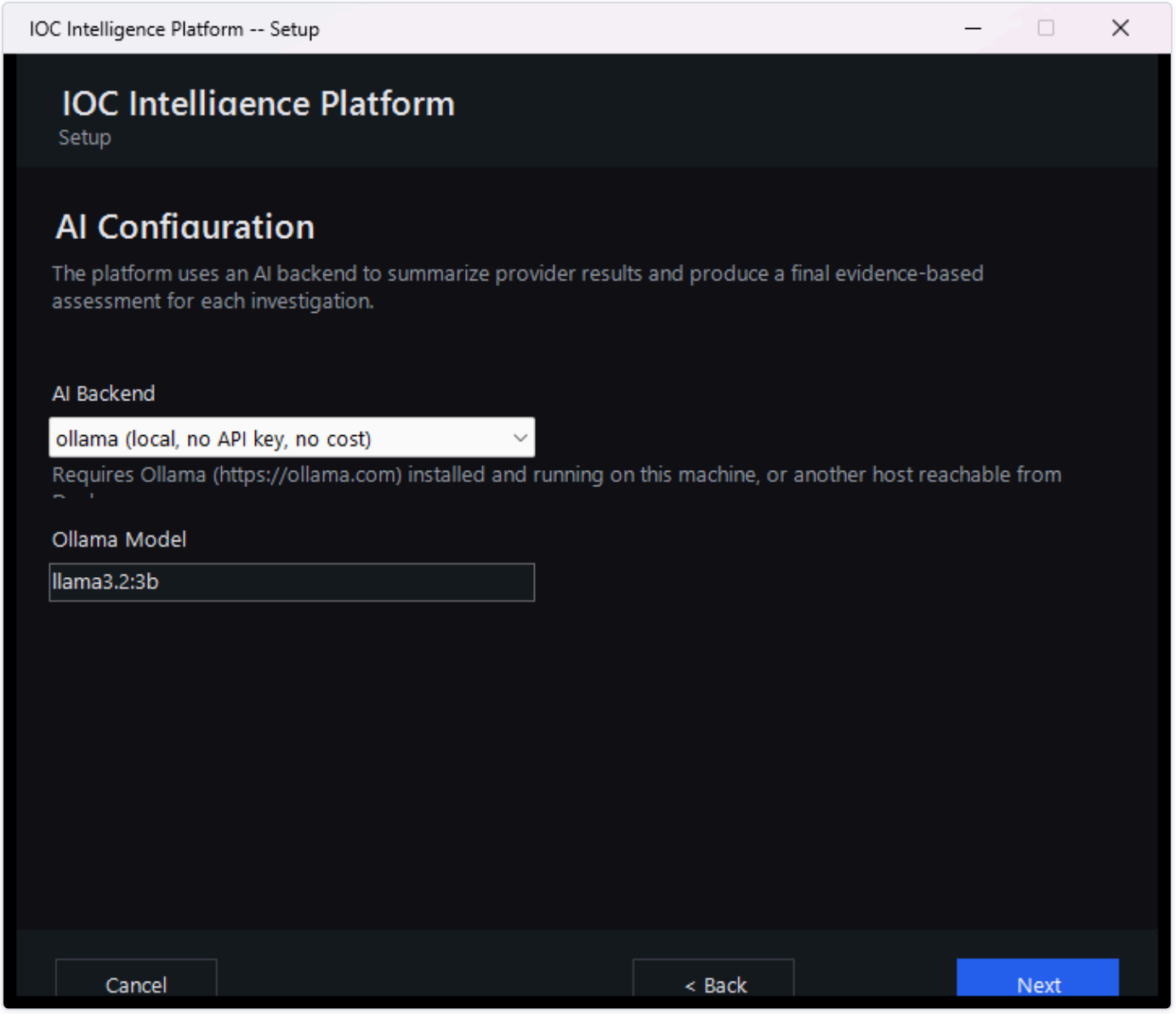

Figure 12 — The AI Configuration page, where the administrator chooses which AI model will generate the AI-written summaries and final assessment for every investigation. (This screenshot predates the Groq backend described below; the page now offers a fifth choice alongside Ollama, Anthropic, Bedrock, and Gemini.)

You have two broad options here:

- **A free local AI model**, which runs entirely on your own machine via Ollama. It costs nothing, requires no account or API key, and keeps everything on-premises -- a good default if you just want to get started.
- **A cloud AI provider** instead, if you'd rather use a more capable hosted AI model from a third-party service. The wizard currently supports four such providers: Anthropic, Amazon Bedrock, Google Gemini, and Groq (a fast-inference API service). Each requires an account and an API key with that provider.

Whichever backend you choose, this page now has a live **Test Connection** button that sends a minimal, real request to that provider and reports a genuine result -- not a placeholder or an automatic green checkmark. A working setup reports something like "Connected. Model replied: 'pong' (model: llama-3.3-70b-versatile, 245 ms)", including the real model that answered and the actual round-trip latency. A broken one reports a specific, real reason instead of a vague failure -- for example, an invalid key, an unrecognized model name, or a provider rate limit.

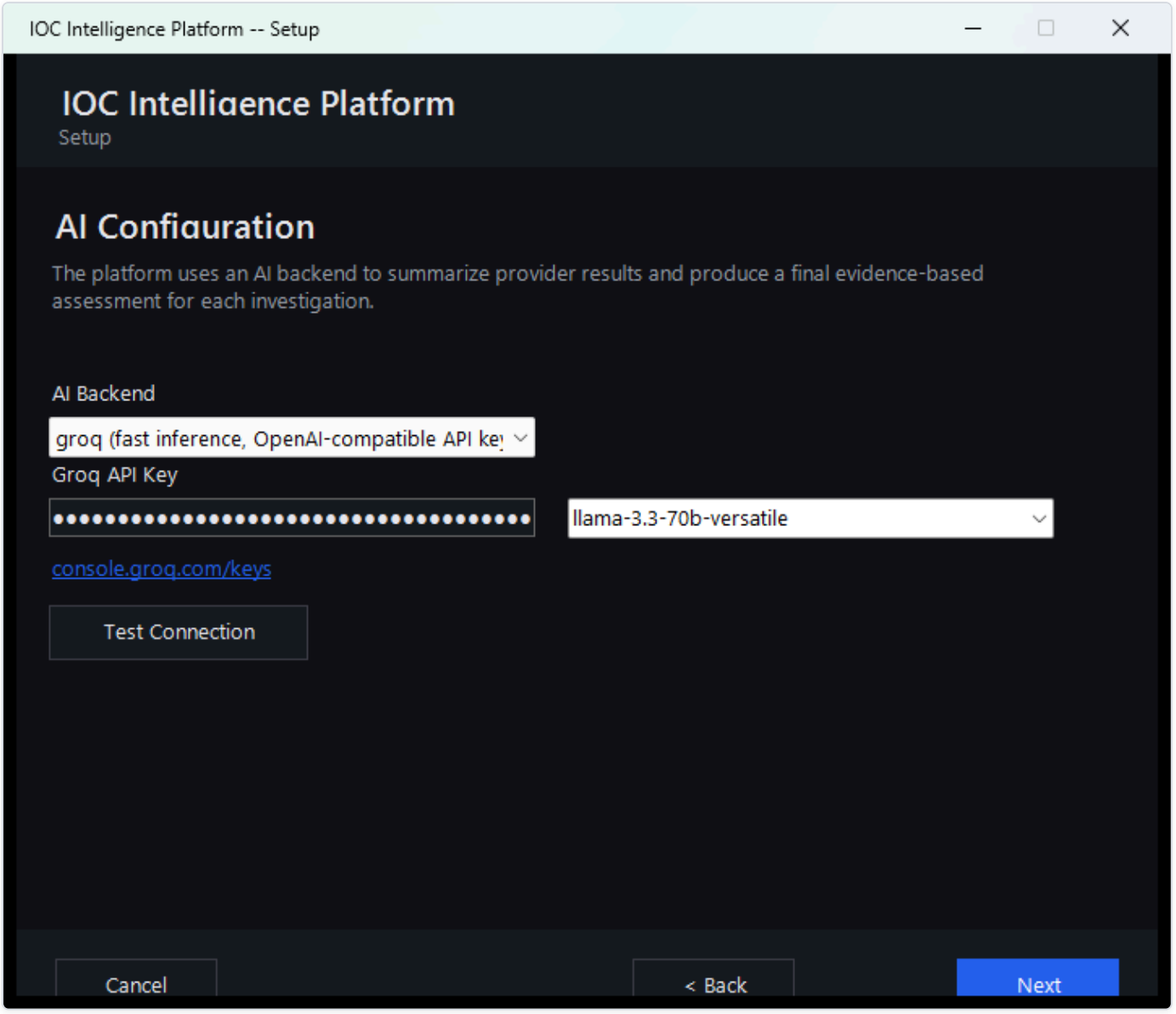

Figure 13 — The AI Configuration page with the Groq backend selected, showing the masked API key field and the Model dropdown set to llama-3.3-70b-versatile.

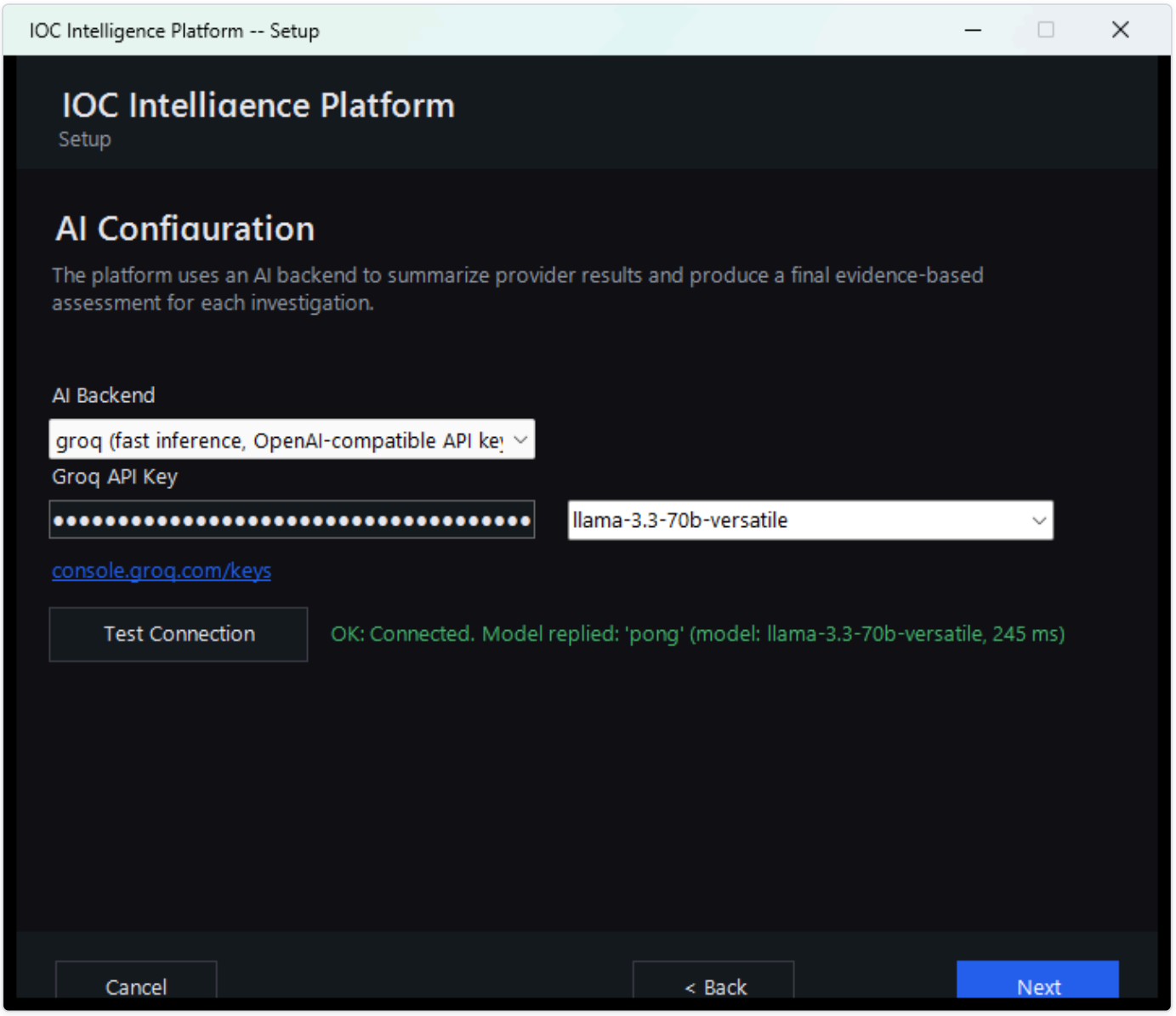

Figure 14 — The AI Configuration page after clicking Test Connection for the Groq backend, showing the real success result: "OK: Connected. Model replied: 'pong' (model: llama-3.3-70b-versatile, 245 ms)".

Either way, it's worth setting expectations early: the AI's job throughout this platform is to help you read and correlate a lot of raw intelligence data faster -- it is analytical assistance, not an unquestionable verdict. Later in the platform, every AI-written summary and verdict comes paired with an Evidence Ledger and "show your reasoning"-style verification tools that let you check exactly which underlying data the AI actually drew on, rather than just taking its word for it. You'll see this play out directly later in this guide, including a real case where two intelligence sources disagreed about the same IP address and the AI's own verdict was worth double-checking against the evidence.

Threat Intelligence Providers

This page is where you connect the platform to outside sources of threat intelligence -- third-party services and databases (like VirusTotal or AbuseIPDB) that have already collected information about known-malicious IPs, domains, files, and more. The wizard lets you configure up to eight such providers here.

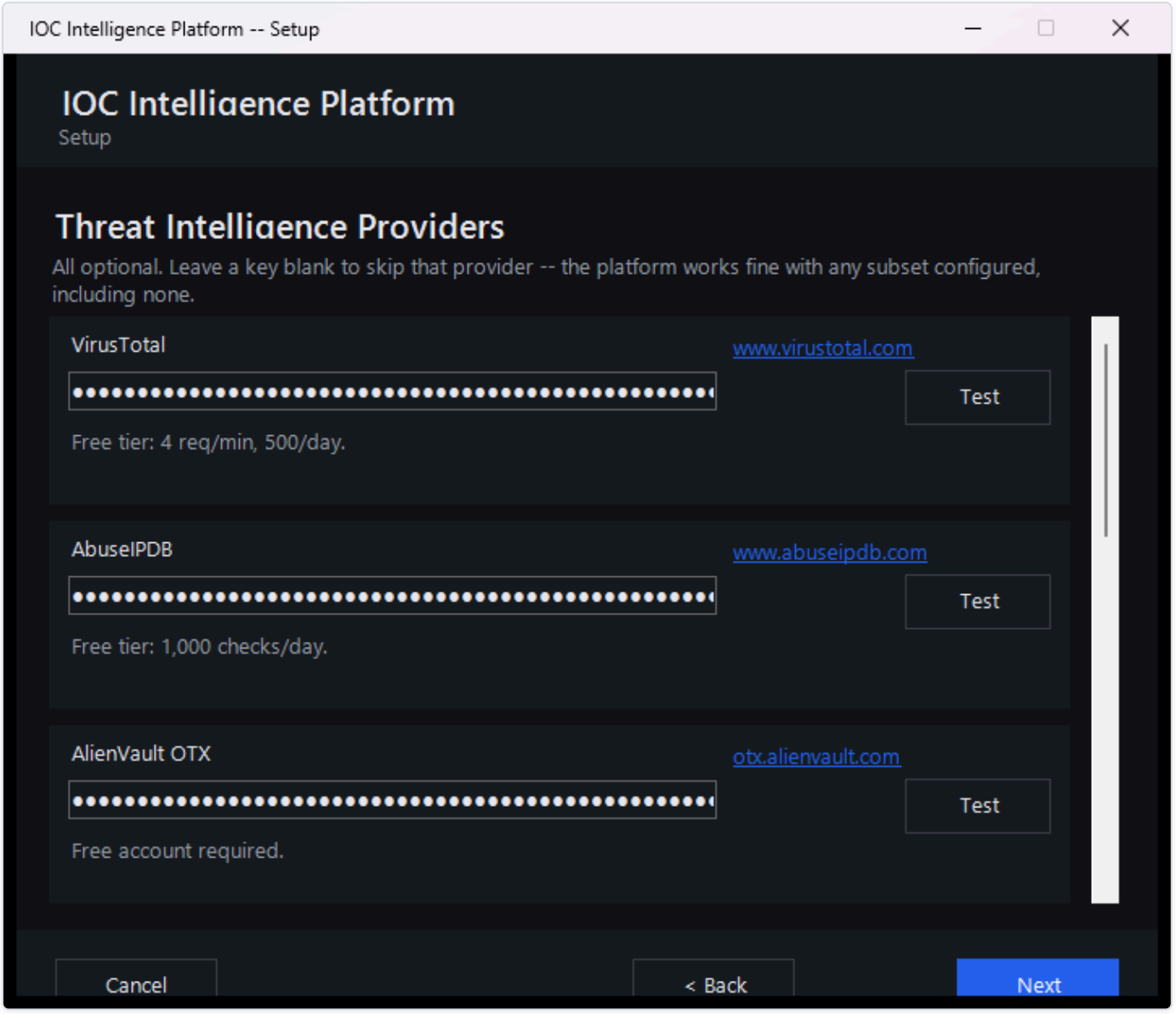

Figure 15 — The Threat Intelligence Providers page, with API keys entered for VirusTotal, AbuseIPDB, and AlienVault OTX -- shown here as masked dots rather than the real key text.

A few things worth knowing about this page:

- **Every provider is optional.** You don't need to fill in all eight, or even any of them, to finish setup. A couple of the providers listed here (for example, the U.S. government's NVD vulnerability database) work out of the box with no key at all -- an API key for those just raises how many requests per minute you're allowed, it doesn't switch the provider on or off.
- **Each key is entered once and masked.** Once typed, a key shows only as dots on screen, the same way a password field would, so it isn't left visible.
- **Each provider has a live "Test" button.** Clicking it immediately checks whether the key you just entered actually works against that provider's real service, so you find out about a typo or an expired key during setup rather than during your first real investigation.

In the interest of full transparency: the screenshot above is from this very evaluation of the platform, and the keys entered for VirusTotal, AbuseIPDB, and AlienVault OTX are real, working keys used to test the product -- not placeholder or example values.

Network Ports

The next page lets you review which network ports on this Windows machine the platform will use once it's installed and running.

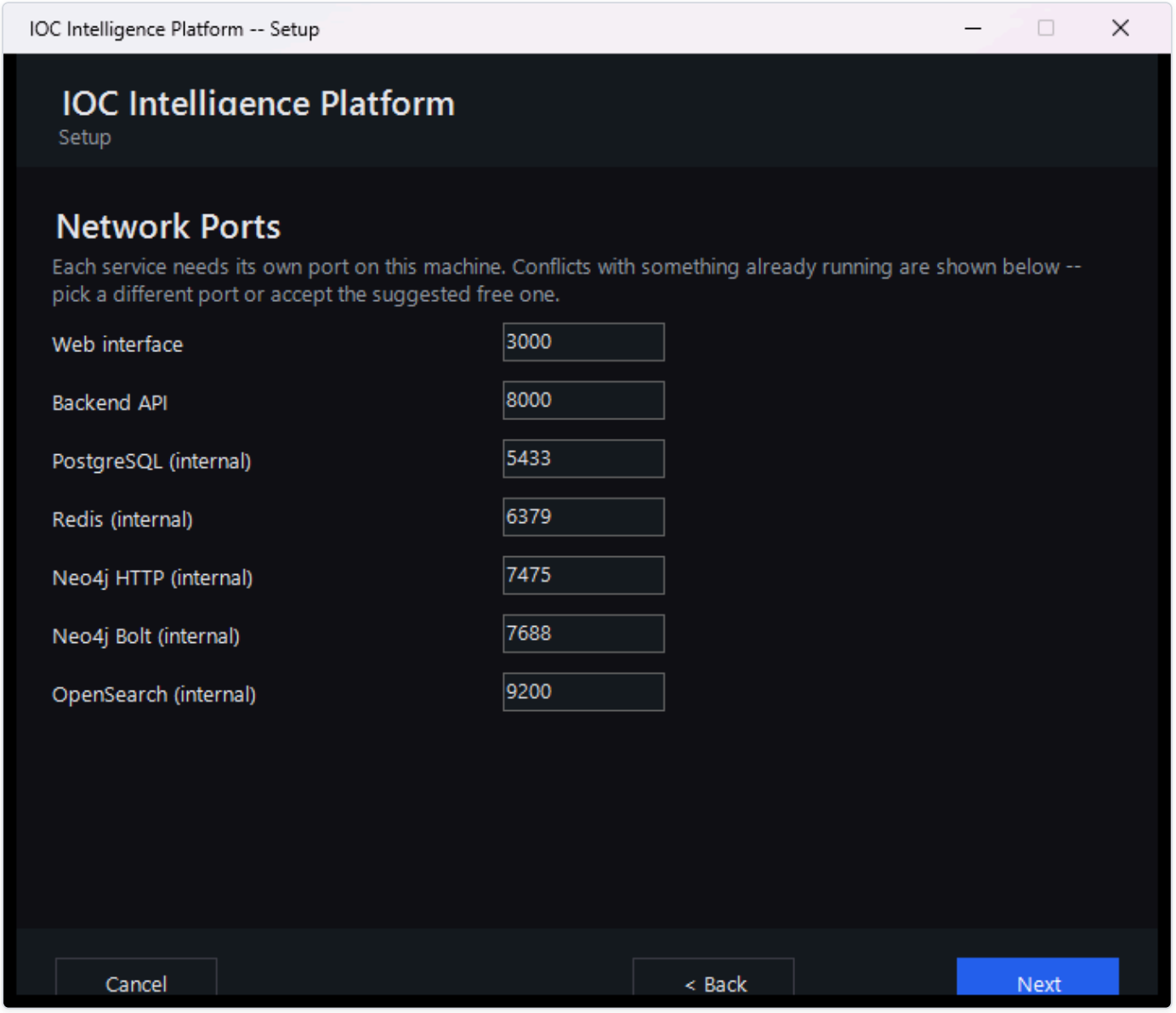

Figure 16 — The Network Ports page, where the administrator can review (and change, if needed) which ports the platform will use on this machine.

Most administrators can simply accept the defaults shown here and move on. This page exists mainly for the case where something else already running on the same machine is using one of the same ports -- in which case you can change the platform's port here before installation begins, rather than running into a conflict afterward.

Reviewing the Summary

Before anything is actually installed, the wizard shows a summary of every choice made so far -- the administrator account, the AI model selection, the ports, and how many threat-intelligence providers were configured.

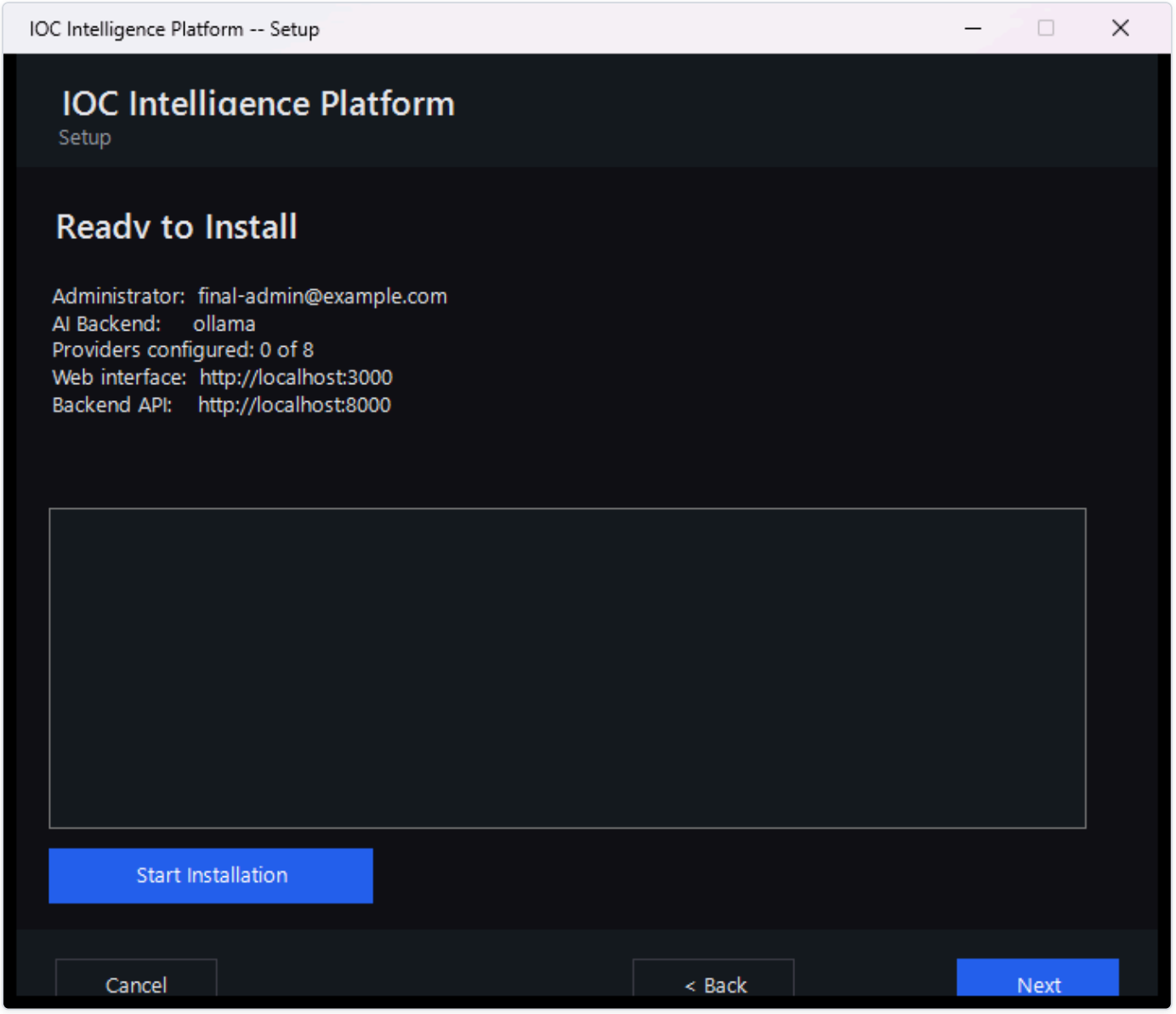

Figure 17 — The Ready to Install summary page, giving the administrator one last look at every setting before installation begins.

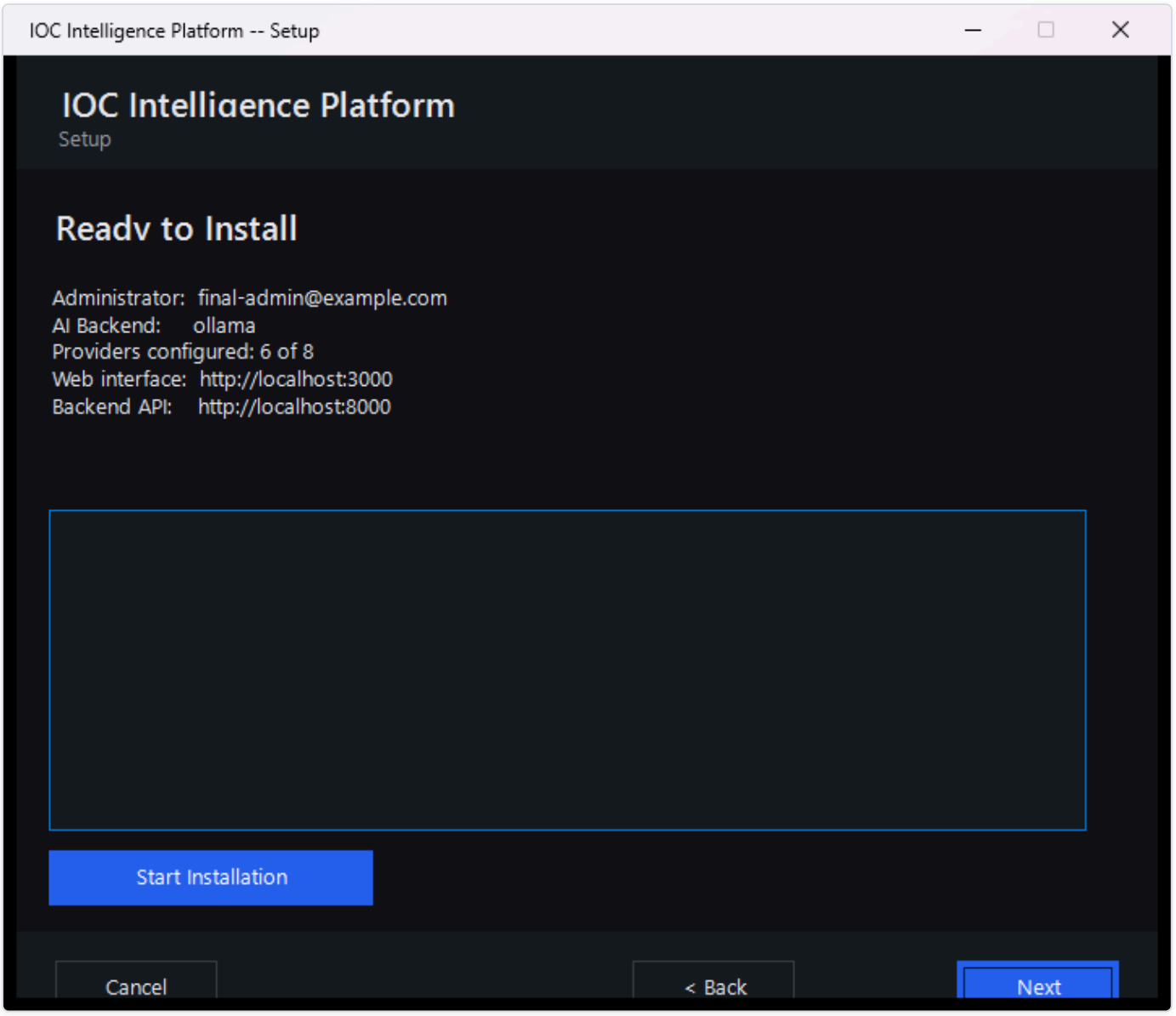

Figure 18 — The same summary page reporting "Providers configured: 6 of 8" for this evaluation.

That "6 of 8" line is a good illustration of how flexible provider setup really is: it counts both the providers where a real key was entered and the providers that work with no key required at all, added together. Your own installation might show a different number -- three, six, or zero -- and the platform will install and run just as well either way. Nothing on this page is a point of no return; if something looks wrong, you can still go back and change it before clicking through to install.

Installation Complete

Once you confirm the summary, the wizard takes over and does the heavy lifting on its own: it pulls down and starts every service the platform needs, waits until the platform reports itself healthy, and then registers your administrator account using the exact details you typed in earlier -- so by the time this page appears, your account already exists and works.

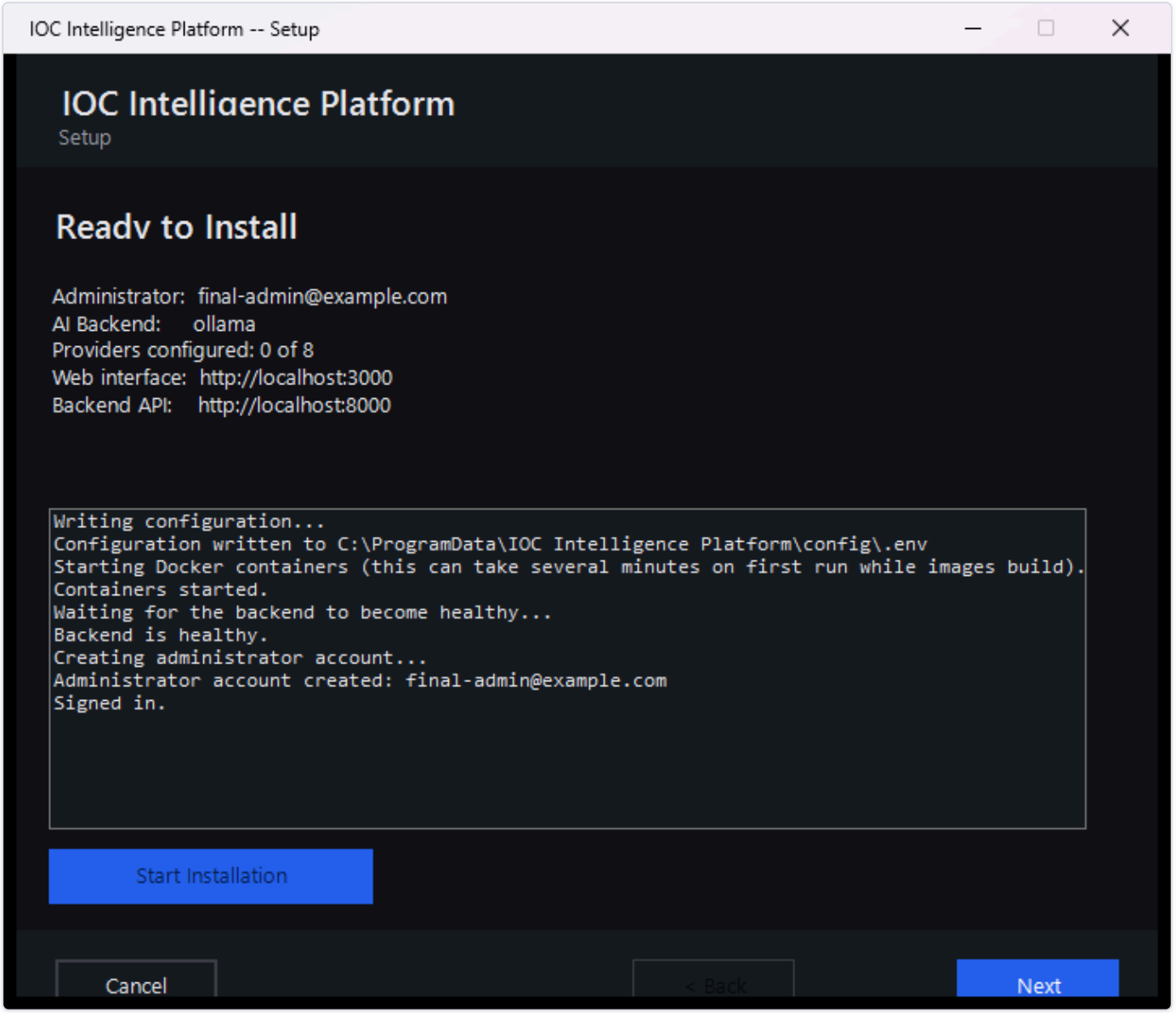

Figure 19 — The Setup Wizard's final page, confirming that installation finished successfully and the platform is ready to use.

This step can take a few minutes the first time, since it's downloading and starting up everything the platform depends on. Once it's done, you're ready to open the platform in a browser and sign in with the administrator account you just created.

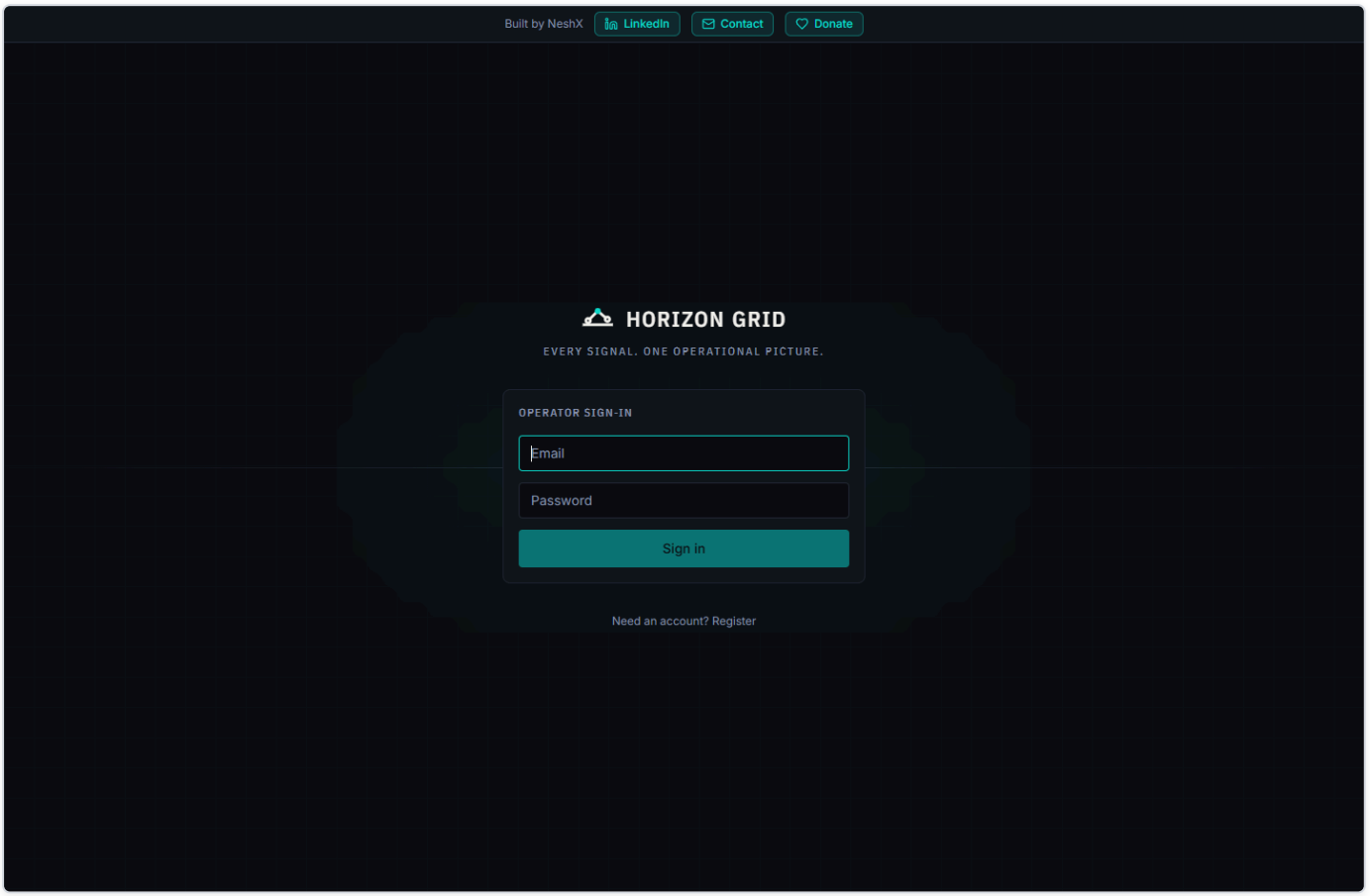

Figure 20 — The web app's Sign in page. Enter the administrator email and password you set on the wizard's Admin Account page and click Sign in -- this is the screen you'll land on every time you open the platform in a browser from now on.

IOC Investigation

This is the core of the product: you give it one IOC, and it tells you — with receipts — what dozens of threat-intelligence sources and an AI model think about it.

What Is an IOC?

IOC (Indicator of Compromise) — a piece of evidence such as an IP address, a domain name, a URL, a file hash, or a **CVE** (Common Vulnerabilities and Exposures — a public catalog ID for a known software vulnerability, e.g. `CVE-2021-44228`) that a security analyst wants to investigate. An IOC by itself doesn't prove anything is wrong; it's a lead. Maybe it showed up in a firewall log, an email attachment, or a colleague's incident notes. The question an analyst always has to answer is: *is this thing actually dangerous, and what do I know about it?*

HORIZON GRID exists to answer that question fast, by querying many independent, real third-party sources at once, correlating what they say, and having an AI model summarize the result — while still giving you everything you need to check the AI's work yourself.

Starting an Investigation

There are two ways to kick off a lookup:

- **Type the IOC into the search box** on the platform's home page and submit it. You can paste in an IP address, a domain, a URL, a file hash, a CVE ID, or any of the other supported types (see the list at the end of this section) — the platform automatically detects what kind of IOC you gave it before it decides which providers to query.
- **Click one of the suggested example IOCs** offered on the same page. This drops that value straight into the search box for you — submit it the same way you would your own IOC. It's the fastest way to see the platform in action before you commit a real indicator from your own environment, and once submitted it runs an actual, live lookup, not a canned demo.

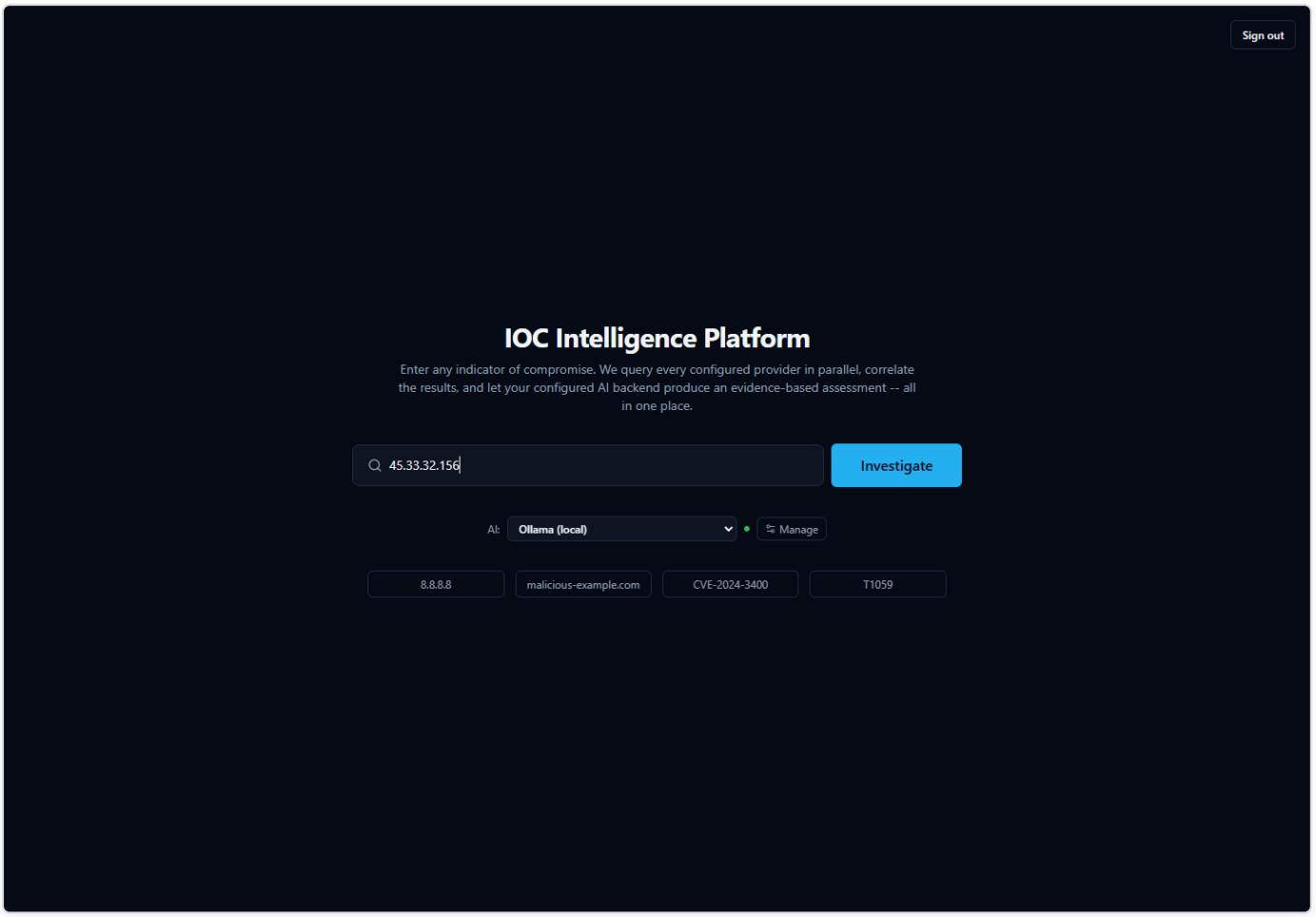

Figure 21 — An IP address typed into the home page search box, ready to submit — the "AI:" selector alongside it shows which backend will analyze this investigation once submitted.

Once you submit an IOC, the platform fans out to every configured provider that supports that IOC type at the same time (not one after another), so results stream back into the page within seconds rather than making you wait for the slowest source.

Anatomy of a Real Investigation: 8.8.8.8

To show what an investigation actually looks like, walk through a real one: the IOC `8.8.8.8` — the IPv4 address of Google's Public DNS resolver, a well-known, legitimate internet service used by billions of devices. It's a genuinely interesting example, because the sources don't fully agree with each other — and that disagreement is exactly the kind of thing an analyst needs to know how to read.

The First Few Seconds

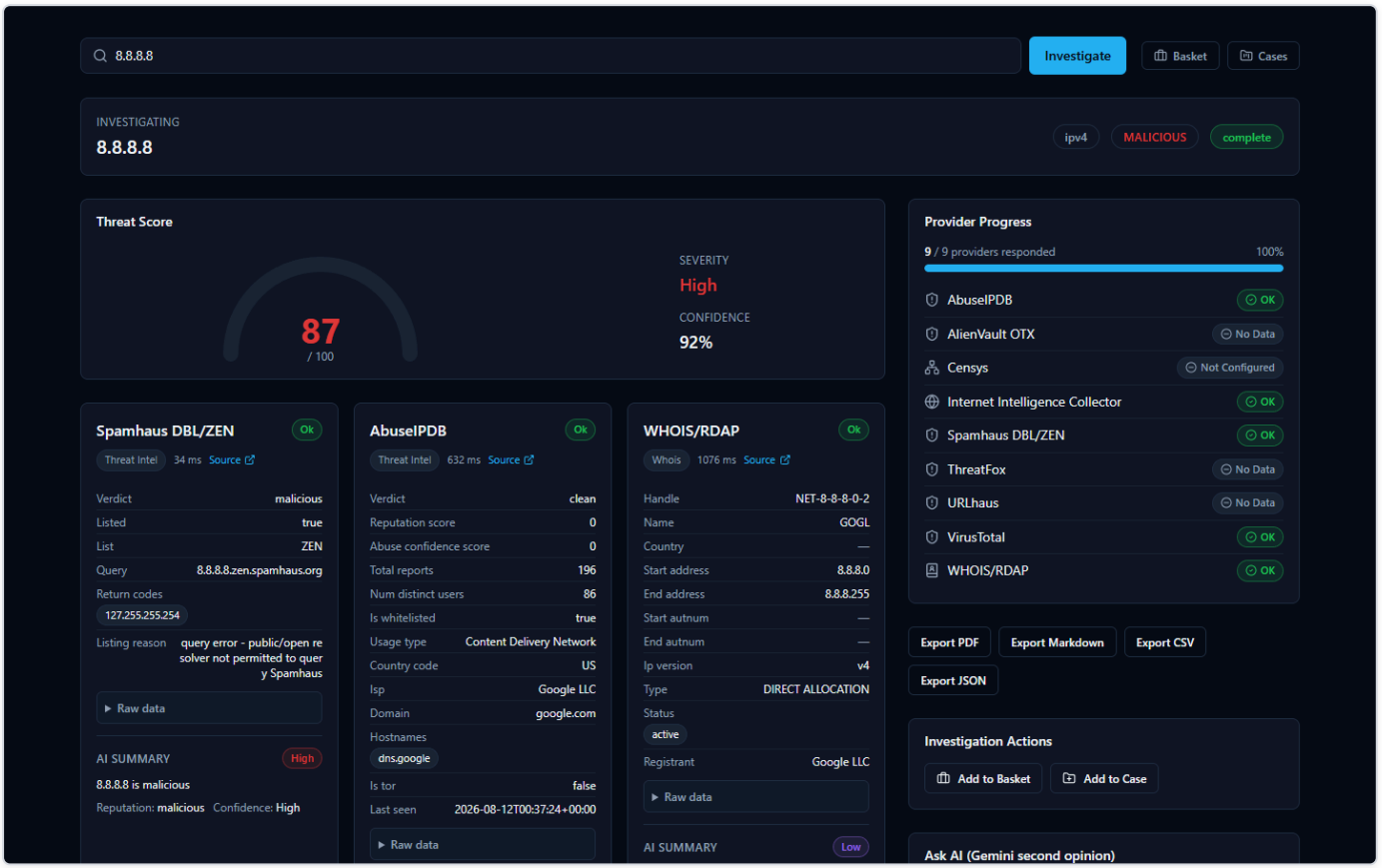

Figure 22 — Seconds after submitting 8.8.8.8, the investigation page fills in live: a Threat Score gauge, a Provider Progress list, and individual result cards from Spamhaus, AbuseIPDB, and WHOIS/RDAP.

This compact view appears almost immediately and updates as more providers respond:

- **Threat Score gauge** — a single 0–100 number with a plain-language severity label. In this investigation it reads **87/100, "High."** This number is the platform's aggregated read on the IOC once results start coming in; it is a starting signal, not a final judgment — hold that thought, because this exact example is about to show why.
- **Provider Progress list** — a running checklist of every provider the platform queried for this IOC type, ticking off as each one finishes. Because providers are queried concurrently, you see them complete in whatever order they actually respond, not a fixed sequence.
- **Per-provider result cards** — one card per source, each showing that provider's own verdict and supporting detail:
  - **Spamhaus DBL/ZEN** shows verdict **malicious**, with a reason string attached: "*query error – public/open resolver not permitted to query Spamhaus.*"
  - **AbuseIPDB** shows verdict **clean**, with **0 abuse reports** on file.
  - **WHOIS/RDAP** shows registration/network ownership data for the address (who the IP block is allocated to).

Already, two cards disagree with each other. Keep reading — the full page explains why, and it isn't a bug.

The Full Picture

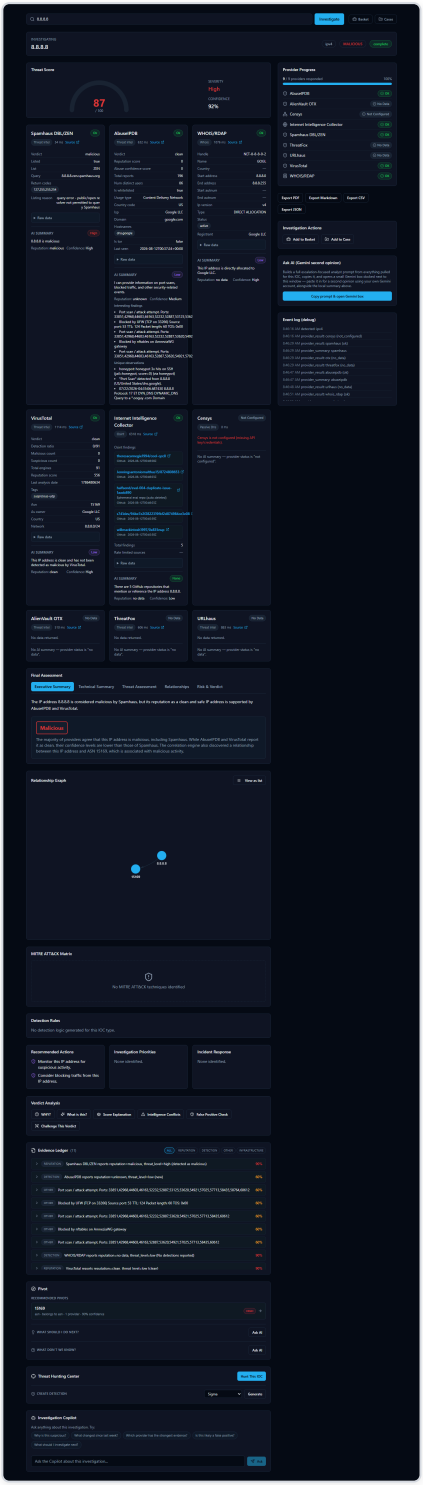

Figure 23 — The complete 8.8.8.8 investigation page: additional provider cards, Final Assessment tabs, a two-node Relationship Graph, MITRE ATT&CK Matrix, Detection Rules, Recommended Actions, Verdict Analysis tools, an 11-item Evidence Ledger, Pivot, Threat Hunting Center, and an Investigation Copilot chat box.

Scroll down (or wait for later providers to finish) and the rest of the investigation appears:

- **More provider cards**: **VirusTotal** reports **clean**; the **Internet Intelligence Collector** (the platform's own OSINT — open-source intelligence, meaning publicly available, non-classified sources — crawler, pulling from sources like GitHub, Reddit, RSS feeds, and

paste sites) reports its findings; **Censys** shows **"Not Configured"** — meaning that particular provider needs API credentials entered before it will run at all, not that it looked and found nothing.

- **Final Assessment tabs** — the AI's consolidated read on the whole investigation, written only after every provider has reported back. For 8.8.8.8, the **Executive Summary** tab states, in effect, that the IP "is considered malicious by Spamhaus, but its reputation as a clean and safe IP address is supported by AbuseIPDB and VirusTotal." The AI is not hiding the disagreement — it's telling you about it directly.
- **Relationship Graph** — a visual map of what this IOC connects to. Here it's simple: two nodes, `8.8.8.8` and its owning network, **ASN 15169** (Google's autonomous system number). On IOCs with more history, this graph can grow to show shared infrastructure, related files, or campaigns.
- **MITRE ATT&CK Matrix** — MITRE ATT&CK is an industry-standard catalog of known attacker tactics and techniques. This tab highlights any techniques the investigation's evidence actually supports. For 8.8.8.8 it correctly reads **"No MITRE ATT&CK techniques identified"** — there's nothing here to map to attacker behavior.
- **Detection Rules** — a tab that can generate ready-to-use detection logic (for a SIEM — Security Information and Event Management system, the kind of tool a SOC uses to watch for threats — or similar tool) when the evidence supports it. For this IOC it reads **"No detection logic generated for this IOC type"** — again, appropriately, since there's no malicious behavior to write a detection rule against.
- **Recommended Actions** — AI-suggested next steps for the analyst, grounded in what was actually found.
- **Verdict Analysis tools** — this is the toolbox that exists specifically so you don't have to just trust the Threat Score number. Each tab lets you interrogate the AI's own conclusion:
  - **Why?** — asks the AI to justify its verdict, citing the actual evidence behind it.
  - **What's this?** — a plain-language explanation of what the IOC is.
  - **Score Explanation** — breaks down how the numeric score was arrived at.
  - **Intelligence Conflicts** — specifically surfaces where providers disagree with each other (exactly the Spamhaus-vs-AbuseIPDB/VirusTotal situation in this example).
  - **False Positive Check** — asks the AI to assess how likely it is that the flagged verdict is wrong.
  - **Challenge This Verdict** — a deliberate red-team tool: it asks the AI to argue *against* its own conclusion, rather than defend it.
- **Evidence Ledger** — for this investigation, **11 items**, each with its own confidence percentage. This list is built directly and deterministically from the real provider data and correlation results — it is explicitly not itself written by the AI. That's the point: it exists so every claim the AI makes elsewhere on the page can be checked against a real, traceable record rather than taken on faith.
- **Pivot** — lets you jump from this IOC to a related one (for example, the ASN) to start a fresh investigation from something this one turned up.
- **Threat Hunting Center** — generates hunting queries or leads an analyst could run in their own environment, built only from indicators actually present in this investigation's evidence.
- **Investigation Copilot** — a chat box for asking follow-up questions about this specific investigation. Its answers are grounded in this lookup's own evidence rather than general knowledge, and unsupported citations get filtered out before you see them.

Why You Must Read the Evidence, Not Just the Score

This 8.8.8.8 example is worth sitting with, because it's a genuine, non-staged illustration of the platform's most important lesson: **a single verdict number is a summary, not a substitute for reading the evidence.**

Here's what actually happened. Spamhaus operates a DNSBL (a DNS-based block list) — a lookup service you query to ask "is this IP known to be bad?" That service refuses to answer queries that come from public, open DNS resolvers, as an anti-abuse measure protecting its own infrastructure. 8.8.8.8 *is* exactly that kind of public resolver — it's Google's own open DNS service. So when the platform queried Spamhaus about it, Spamhaus didn't report finding malicious activity at all; it rejected the query itself, with the reason

"query error – public/open resolver not permitted to query Spamhaus." The platform recorded that rejection as a **malicious** verdict card, because that's the status Spamhaus returned — but the underlying reason has nothing to do with 8.8.8.8 doing anything harmful.

Meanwhile, AbuseIPDB checked its abuse-report database and found zero reports. VirusTotal checked its own reputation data and came back clean. Two independent, purpose-built sources say this address is clean; one source's result is actually a permission error dressed up as a verdict.

Notice, too, that the Threat Score gauge on the first screen still read **87/100, "High"** — pulled up by that one Spamhaus flag — even though the fuller picture tells a different story. That's not a flaw the product is hiding: it's precisely why the Executive Summary text spells out the disagreement in plain words rather than quietly averaging it away, and why tools like **Intelligence Conflicts** and **Challenge This Verdict** exist on the same page. The AI's output here is analytical assistance — a fast, well-organized starting point — not an unquestionable ruling. An analyst who stopped at the gauge would walk away thinking Google's DNS resolver is a threat. An analyst who reads the provider cards, the Executive Summary, and the Evidence Ledger sees the real picture in under a minute.

Supported IOC Types

The platform automatically detects what kind of IOC you've submitted and routes it only to the providers that support that type. Types with dedicated provider coverage include:

- **IP addresses** (IPv4 and IPv6)
- **Domains** and **hostnames**
- **URLs**
- **File hashes** (MD5, SHA-1, SHA-256, SHA-512)
- **CVE identifiers** (e.g. `CVE-2021-44228`)
- **MITRE ATT&CK techniques**
- **TLS/SSL certificates**
- **Autonomous System Numbers (ASN)**
- **Malware family names**
- **Threat actor names**
- **Campaign names**
- **File names**

This is not an exhaustive list of every type the platform can classify internally, but it covers the types you'll actually get provider results back for.

Security Assessment: Active Checks Against the Target

Every provider covered so far is **passive** — it asks a third party what they already know about your indicator. Once an investigation completes for an IP, domain, hostname, or URL, a **Security Assessment** panel appears further down the page offering a genuinely different kind of check: sending real traffic to the target itself. Four checks are available — a Nmap port/service scan, a DNS record lookup, a TLS certificate inspection, and an HTTP security-header check — each described in plain language when you select it.

This never happens automatically. Unlike every passive provider, a security assessment is something you must deliberately start, and the platform requires two explicit confirmations before it will run anything:

1. **Retype the exact target** in the confirmation box — this is a safeguard against accidentally scanning the wrong thing.
2. **Check the authorization box**, confirming you're actually allowed to run active checks against this target. Only scan systems you own or have explicit permission to test.

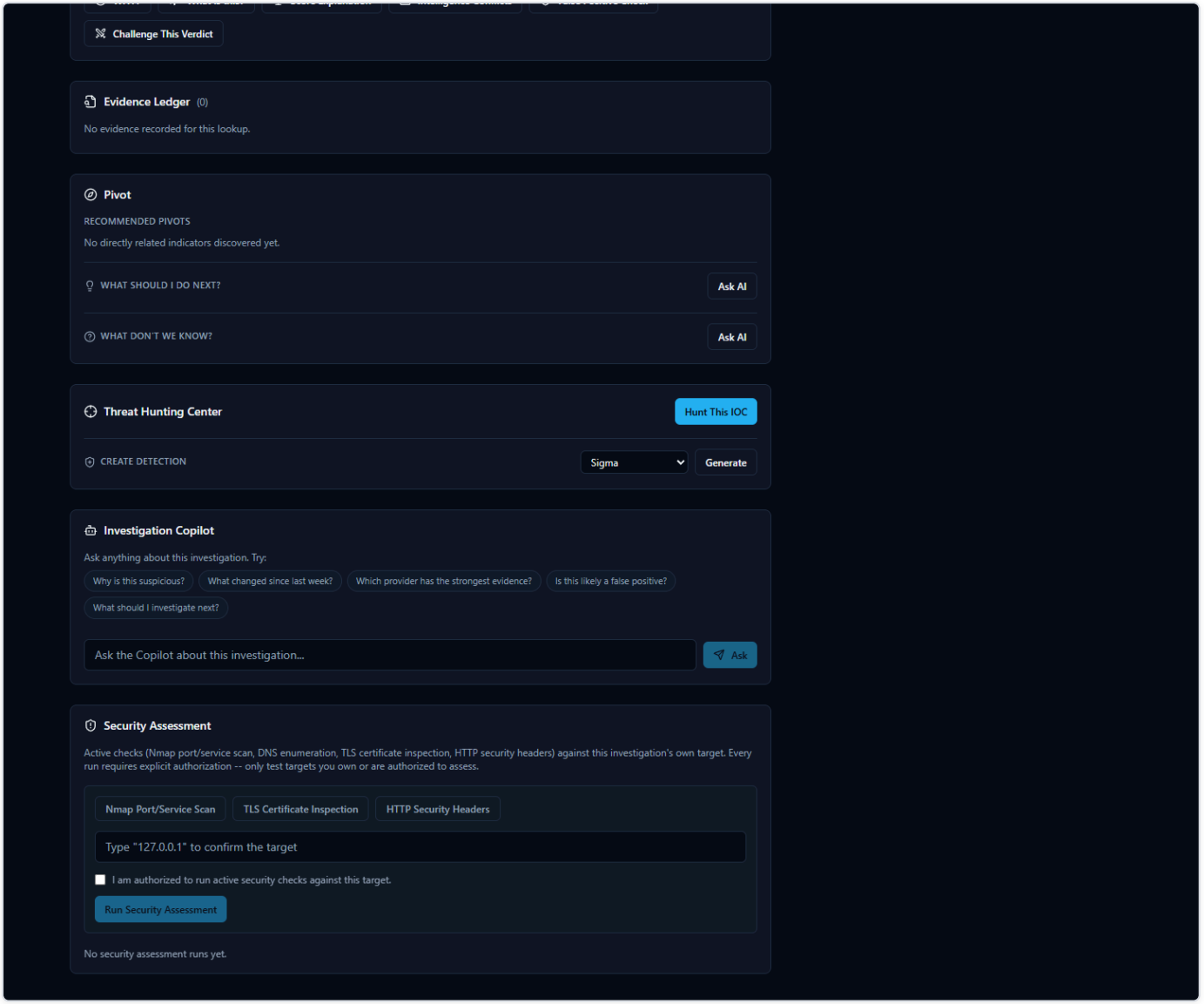

Figure 24 — The Security Assessment panel: tool selection, the target-confirmation box, and the authorization checkbox.

Once you start a run, results appear in a findings table with a severity badge on each row (from informational up to critical) — severities are assigned by fixed, documented rules based on what was actually found (e.g. an expired certificate, or an open port running software with a known vulnerability), never guessed. Click any finding to see exactly what was observed.

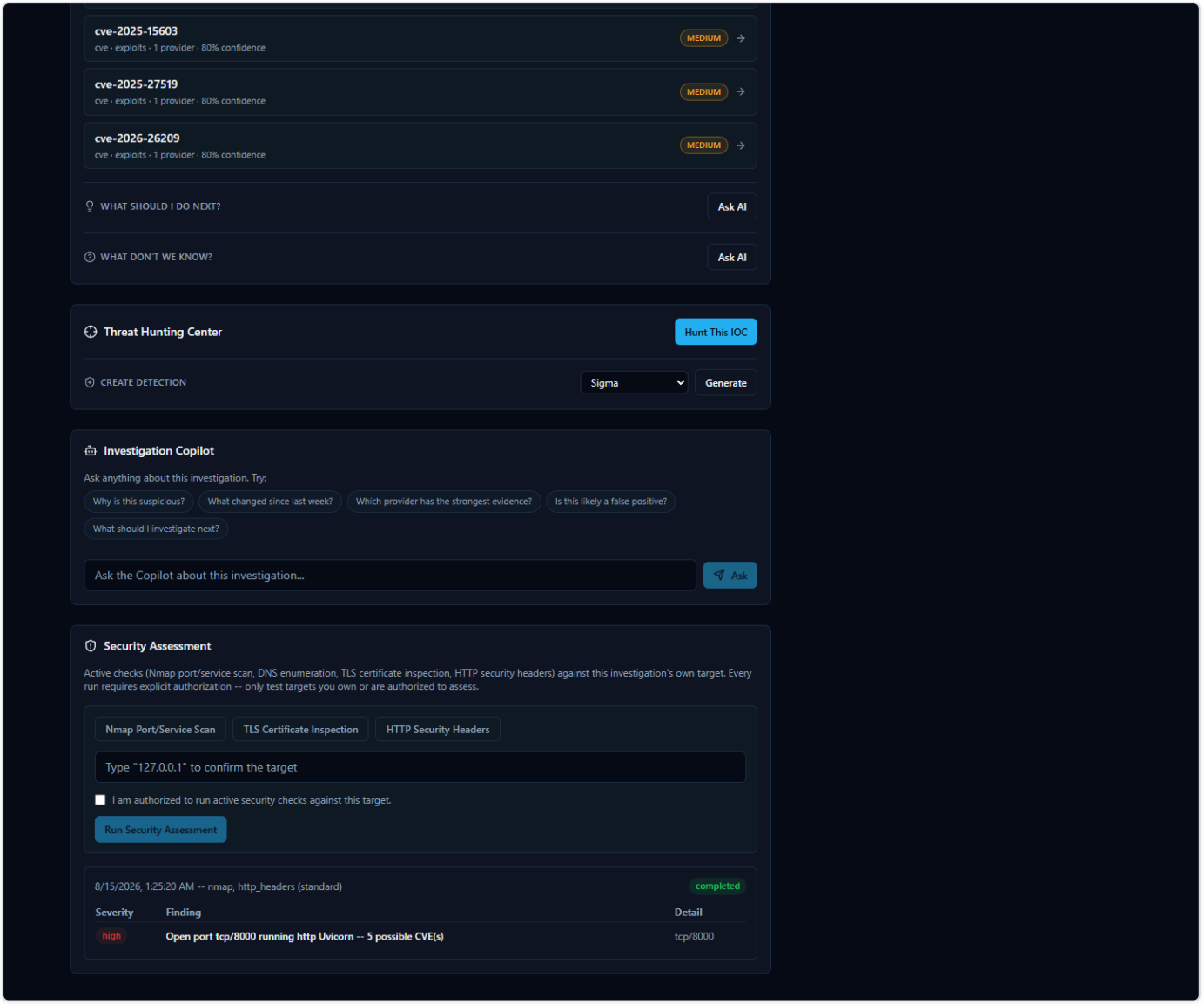

Figure 25 — A completed run's findings table, including a real port-scan result with a matched CVE.

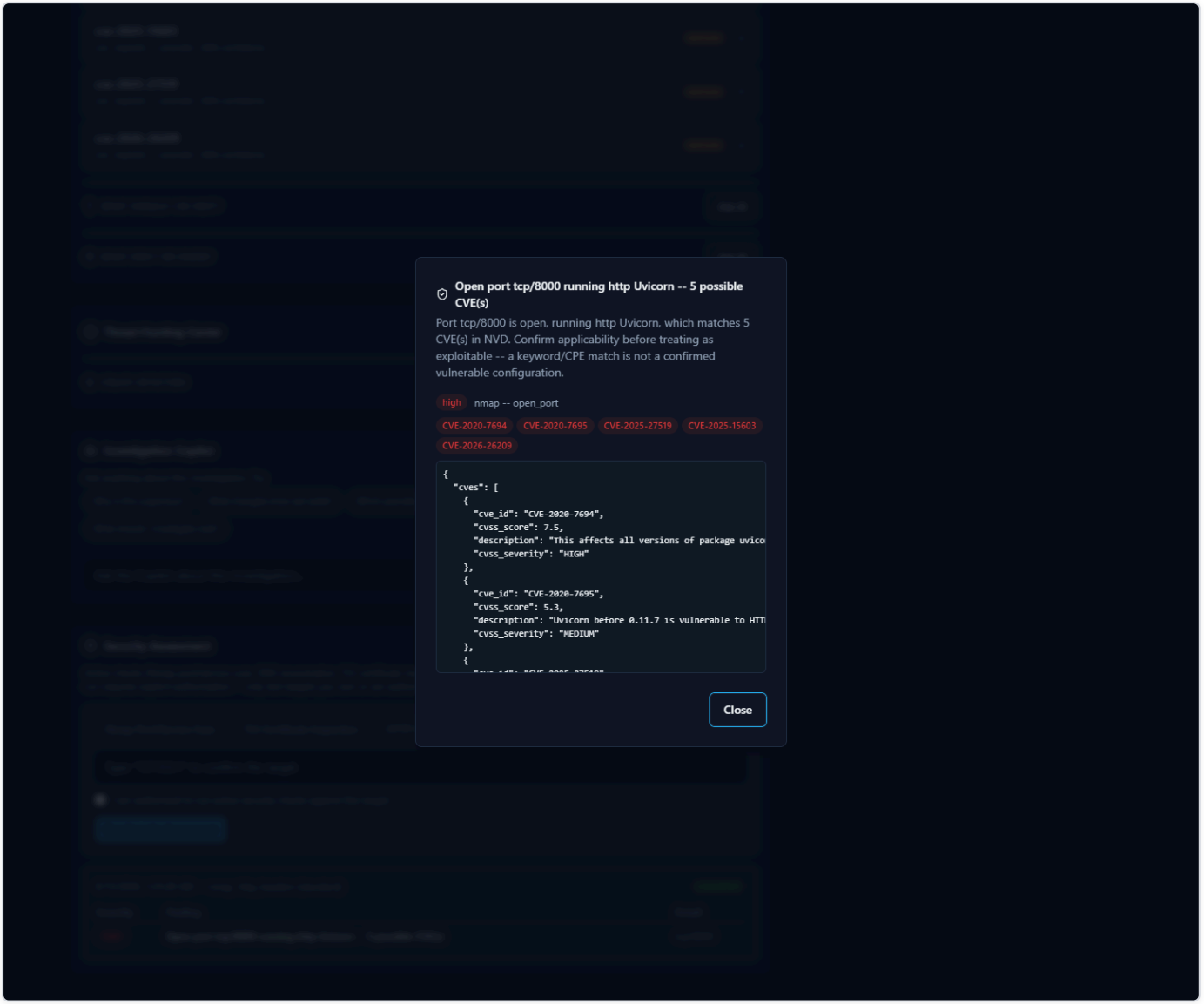

Figure 26 — Clicking a finding opens its full detail: the matched CVEs and the raw evidence the severity was based on.

A completed run's findings feed back into the same investigation — the AI-generated assessment, risk score, and verdict at the top of the page are refreshed to account for what the security assessment found, exactly as if a new provider had reported in.

What a port scan actually does

The Nmap port/service scan is the only one of the four checks that has more than one option — a **profile** you pick before running it:

- **Quick scan** — the 100 most common ports, no service/version detection. Fastest, lowest footprint.
- **Standard scan** (the default) — the 1,000 most common ports, *with* service/version detection (identifying what software is actually answering on an open port, e.g. "Uvicorn" or "nginx"). Slower than Quick, but far more informative — a service/version match is what lets the platform cross-reference known CVEs against it.
- **Web service scan** — only the common web ports (80, 443, 8080, 8443), with service/version detection. The fastest way to check specifically web-facing exposure.

A port only ever shows up in the findings table if it's genuinely **open**. Closed and filtered ports are not reported as findings at all — they're the expected, uninteresting majority of any scan, not evidence of anything. **If a scan completes and the findings table is empty, that means every port it checked came back closed or filtered — a real, successful result, not a failure.** A failed scan looks different: the run's status badge itself shows "failed" (in red) with an explanatory error message underneath, not an empty table with no explanation.

Scan statuses

A run moves through a small set of statuses, shown as a colored badge:

Status	Meaning
Pending	Accepted, waiting to start. Usually only visible for a moment.
Running	Actively in progress. The page checks for updates automatically every 2 seconds — no need to refresh.
Completed	Finished normally. Check the findings table (which may legitimately be empty — see above).
Failed	Something went wrong before a result could be produced (e.g. the scanner isn't installed on this deployment, or the target couldn't be reached). The specific reason is shown under the badge.
Cancelled	You (or another analyst — findings and runs are visible to your whole team, not private to whoever started them) stopped the scan before it finished.

Cancelling a scan

A **Cancel Scan** button appears next to any run that's still Pending or Running. Clicking it stops the scan immediately — including the real underlying scan process, not just the page's display of it — and the run's status changes to Cancelled. This is useful if you started the wrong profile by mistake, or a scan against a large/slow target is taking longer than you want to wait. A cancelled scan can simply be started again with different settings; nothing about the investigation itself is affected.

If a tool shows as "unavailable"

Each tool button in the panel is disabled with an "(unavailable)" label if that specific check isn't currently usable on this deployment — hovering over it explains why. For the Nmap port scanner specifically, this means the `nmap` program isn't present on the machine actually running the backend. On the standard Windows and Linux installations of this platform, `nmap` is installed automatically as part of the backend container and should never need any manual setup; seeing "unavailable" on a standard install is itself worth reporting, not something to work around.

Intelligence Providers

What Is a "Provider," Exactly?

Every time you look up an IOC (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID that a security analyst wants to investigate) in HORIZON GRID, the tool doesn't ask one source what it thinks. It asks many.

Each of those outside sources — a malware database, a government vulnerability catalog, a domain registration lookup, a live web crawler — is called a **provider**. A provider is simply a specialized service, database, or feed that knows something specific about the internet: who owns this IP address, has this file hash been seen in malware before, is this CVE (Common Vulnerabilities and Exposures — a standardized ID number for a publicly known security flaw, e.g. "CVE-2021-44228") actively being exploited right now, and so on.

The platform has **18 providers built in**. When you submit an IOC, it queries every provider that's relevant to that IOC's type — for example, a file hash won't be sent to a domain-registration lookup, but it will be sent to a malware-sample database — and streams each provider's answer back to you as it arrives, before combining everything into one assessment.

This section walks through what each of the 18 real providers is, in plain language: what it is, what kind of intelligence it supplies, why a SOC (Security Operations Center) analyst would care, and roughly what comes back.

Malware, IP, and URL Reputation Providers

These are the "has anyone seen this before, and was it bad?" sources — the closest thing to a criminal record check for an IP, domain, URL, or file.

- **VirusTotal** — A well-known, multi-engine file/URL scanning service: when you submit a hash, domain, IP, or URL, VirusTotal reports back what dozens of antivirus engines and URL scanners think of it. For a SOC analyst, this is one of the fastest ways to get a broad consensus opinion. What it returns: a detection ratio (e.g. how many engines out of how many flagged it as malicious), plus a verdict.
- **AbuseIPDB** — A community-driven database where network administrators and analysts report IP addresses that have attacked or abused them (brute-force logins, spam, port scanning, etc.). It only covers IP addresses. This is useful because it reflects real-world abuse reports rather than automated scanning alone. What it returns: an abuse-report count and a verdict (e.g. "clean, 0 reports").
- **AlienVault OTX (Open Threat Exchange)** — A large, community-contributed threat-intelligence exchange covering IPs, domains, hostnames, URLs, and file hashes. Analysts and vendors publish "pulses" describing malware campaigns, and OTX links IOCs to them. This is valuable for context: OTX can name the actual malware family or threat actor associated with an indicator, not just say "bad." What it returns: a verdict plus, when available, malware family names and threat-actor attributions.
- **URLhaus** — An abuse.ch project that tracks URLs actively distributing malware. It checks URLs, domains, and IPs. This is useful for catching live malware-hosting infrastructure specifically, rather than general reputation. What it returns: whether the URL/domain/IP appears in the malware-distribution database.
- **ThreatFox** — Another abuse.ch project, a shared IOC (Indicator of Compromise) database contributed to by the security community, covering IPs, domains, URLs, and MD5/SHA256 hashes. It's a good complement to URLhaus because it's broader than just distribution URLs. What it returns: a verdict, and often the associated malware family.
- **MalwareBazaar** — The third abuse.ch project, a repository of actual malware sample hashes (MD5, SHA1, SHA256, SHA512). If a file hash you're investigating shows up here, someone has already submitted that exact malicious file to the community. What it returns: a verdict and malware family/tag information for known samples.

- **Hybrid Analysis** — An online malware sandbox: files are actually executed in an isolated environment and their behavior is recorded. In this platform, it's used to check SHA256 hashes only (the connector's own documentation notes that hash-search-by-MD5/SHA1 and URL search are no longer supported by the underlying service). This is useful because it can reflect actual observed behavior, not just static signatures. What it returns: a verdict and summary from a prior sandbox run of that exact file, if one exists.
- **Spamhaus** — A long-established DNSBL (DNS-based blocklist) — a reputation list checked via a plain DNS query rather than a web API — covering IPs and domains, widely used across the email and network security industry to flag spam sources and malicious infrastructure. It's a fast, no-configuration check. What it returns: a verdict (listed / not listed), and sometimes a specific reason string from the lookup itself.
- **PhishTank** — A community-reported database of phishing URLs. Since phishing pages are often short-lived, having a dedicated, frequently updated phishing list matters. What it returns: whether the URL has been reported and confirmed as phishing.
- **urlscan.io** — Submits a URL or domain for a real, live sandbox scan rather than checking it against a pre-built database, then polls for the result (capped at 60 seconds). This is useful when you want to see what a page actually does right now, not just whether it was flagged before. It checks URLs and domains only. What it returns: a scan verdict once the sandbox run completes; a scan that isn't ready by the timeout is reported honestly as timed out, never as clean.
- **Google Safe Browsing** — Google's own URL/domain reputation check (the same list of malware and phishing threat types your web browser itself would normally warn you about), covering malware, social engineering, unwanted software, and potentially harmful applications. It checks URLs and domains only. What it returns: a verdict based on whether Google's own threat lists have a match — and, by design, absolutely nothing else (a failed check, a network error, or an unexpected response is always reported as unknown/error, never mistaken for "safe," specifically so a broken or misconfigured key can never look like a clean scan).

Vulnerability Intelligence Providers

These answer a different question: not "is this indicator bad," but "how serious is this known software vulnerability, and is it actually being used in the wild?"

- **NIST NVD (National Vulnerability Database)** — The U.S. government's official, authoritative catalog of publicly disclosed software vulnerabilities. Given a CVE ID, it returns the full vulnerability description and its CVSS (Common Vulnerability Scoring System) severity score. This is often an analyst's starting point for understanding what a vulnerability actually is and how bad it could theoretically be. What it returns: description, CVSS score/severity (e.g. 10.0, CRITICAL), and related metadata.
- **CISA KEV (Known Exploited Vulnerabilities catalog)** — Also run by a U.S. government agency (CISA), but answering a much sharper question: is this vulnerability *actually* being exploited by attackers right now, not just theoretically dangerous? A CVE appearing here means it's confirmed to be actively used in real attacks — which is exactly the kind of finding that should jump a vulnerability to the top of a patching queue. What it returns: a verdict, plus vendor/product details and CISA's recommended remediation.

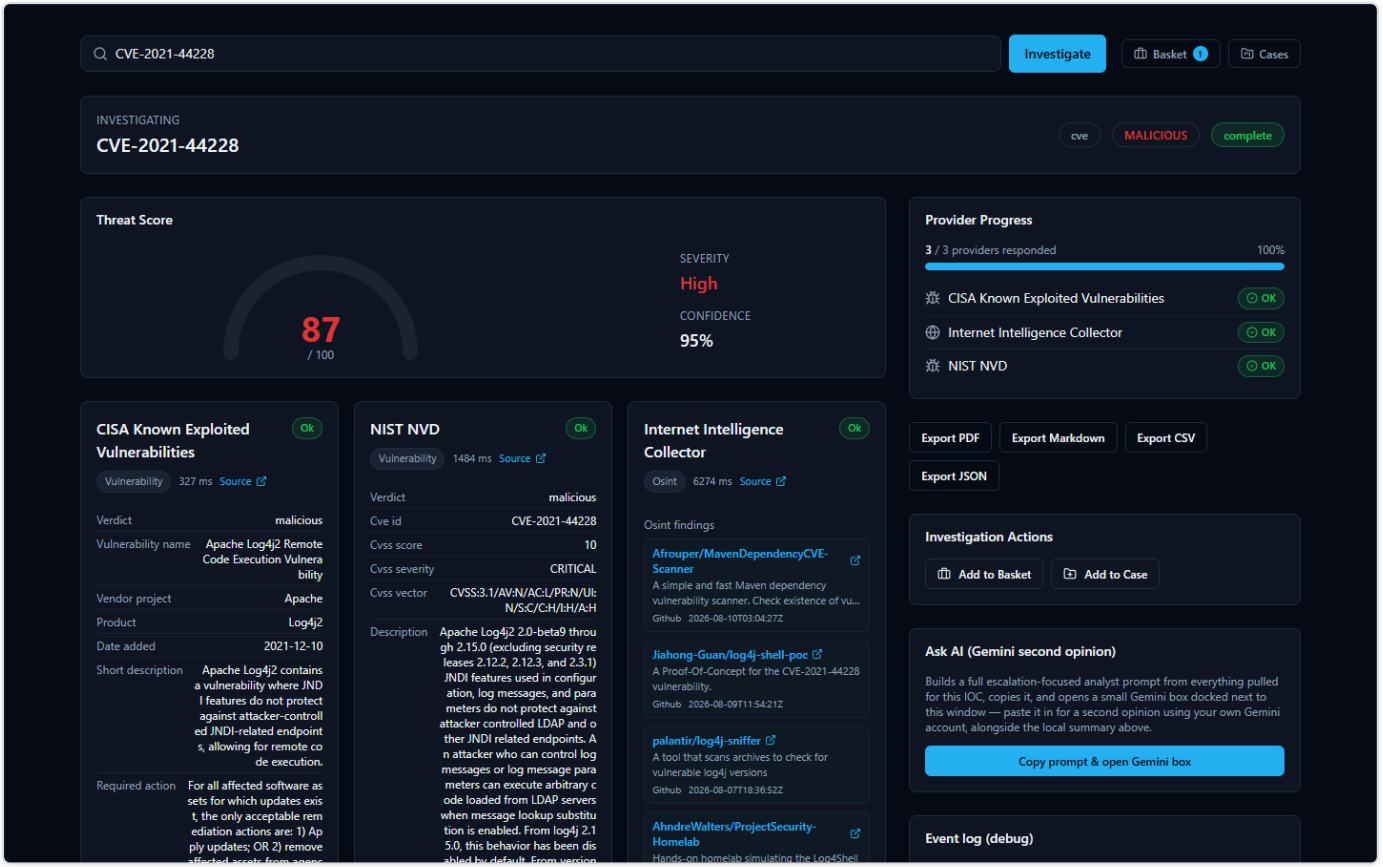

Figure 27 — Investigating CVE-2021-44228 (Log4Shell) returns cards from CISA Known Exploited Vulnerabilities, NIST NVD, and the Internet Intelligence Collector side by side — CISA KEV confirms real-world exploitation, NVD supplies the CVSS 10.0 "CRITICAL" score and full description, and the Internet Intelligence Collector adds live-fetched OSINT findings, all without any provider requiring a credential.

Live Web / OSINT Intelligence

- **Internet Intelligence Collector** — This is the platform's own OSINT (Open-Source Intelligence — information gathered from publicly available sources rather than a paid feed) crawler. It actively searches public sources like GitHub, Reddit, RSS feeds, and paste sites for recent chatter about a domain, IP, malware family, threat actor, campaign, CVE, or filename. Unlike the other providers, which query a fixed, pre-built database, this one goes out and fetches live results at the moment you run the investigation. For an analyst, this is useful precisely because it can surface very recent, informal discussion — like a proof-of-concept exploit script just published on GitHub — that a slower-moving commercial feed hasn't caught up to yet. What it returns: a list of real, live-fetched findings (e.g. matching GitHub repositories) relevant to the IOC.

Attack Technique Reference

- **MITRE ATT&CK** — MITRE ATT&CK is the industry-standard framework that catalogs known attacker tactics and techniques (for example, "T1055 — Process Injection"). This provider looks up a MITRE technique ID directly against the official ATT&CK knowledge base. It's mainly used behind the scenes to enrich technique references the platform's correlation engine has already identified, giving analysts the real, official name and description rather than a bare ID.

Domain, IP, and Certificate Registration Providers

These answer "who owns this, and what else is tied to it?" — infrastructure-and-ownership questions rather than reputation questions.

- **WHOIS/RDAP** — WHOIS and RDAP (Registration Data Access Protocol, the modern successor to WHOIS) are the standard ways to look up who registered a domain, or who an IP address / ASN (Autonomous System Number — an ID assigned to a network operator) block belongs to. This is often the very first thing an analyst checks: is this a brand-new domain registered yesterday, or a major cloud provider's IP range? What it returns: registrar/owner details, registration dates, and organizational information.
- **crt.sh (Certificate Transparency)** — Every publicly trusted TLS certificate (the certificate that secures a website's HTTPS connection) gets logged in public, tamper-evident "Certificate Transparency" logs. crt.sh searches those logs by domain. This is a well-known technique for discovering other subdomains or infrastructure tied to the same certificate issuer or domain — useful for mapping out an attacker's broader infrastructure. What it returns: a list of matching certificates and the domains/subdomains they cover.
- **Censys** — An internet-wide scanning service that catalogs what's actually running on IP addresses across the internet (open ports, services, certificates) — sometimes described as passive DNS / internet asset intelligence. This is useful for understanding what an IP is actually hosting, beyond just a reputation score. Censys is the one provider in this platform that needs **two** separate credentials to work — a Personal Access Token *and* an Organization ID — both are required, and having only one leaves it not configured.

All 18 Providers at a Glance

Provider	What It Checks	Key Required?
VirusTotal	IPs, domains, URLs, file hashes — multi-engine scan verdicts	Yes
AbuseIPDB	IPs — community abuse reports	Yes
AlienVault OTX	IPs, domains, hostnames, URLs, hashes — community threat exchange	Yes
URLhaus	URLs, domains, IPs — malware-distribution tracking	Yes (shared abuse.ch key)
ThreatFox	IPs, domains, URLs, MD5/SHA256 hashes — shared IOC database	Yes (shared abuse.ch key)
MalwareBazaar	File hashes (MD5/SHA1/SHA256/SHA512) — known malware samples	Yes (shared abuse.ch key)
crt.sh	Domains, TLS certificates — Certificate Transparency logs	No
NIST NVD	CVE IDs — vulnerability description & CVSS score	No (optional key raises rate limit)
CISA KEV	CVE IDs — confirmed active exploitation	No
MITRE ATT&CK	MITRE technique IDs — official attack-technique reference	No
WHOIS/RDAP	Domains, IPs, ASNs — registration/ownership records	No
Hybrid Analysis	SHA256 file hashes — malware sandbox behavior	Yes
Spamhaus	IPs, domains — DNS-based blocklist	No
PhishTank	URLs — community-reported phishing	No (optional key raises rate limit)
Censys	IPs — internet-wide host/certificate scan data	Yes (Personal Access Token **and** Organization ID, both required)
Internet Intelligence Collector	Domains, IPs, malware families, threat actors, campaigns, CVEs, filenames — live OSINT crawl	No
urlscan.io	URLs, domains — live sandbox scan	Yes (no wizard entry — see below)
Google Safe Browsing	URLs, domains — Google's malware/phishing threat lists	Yes (no wizard entry — see below)

Why the Setup Wizard Only Shows 8 Providers, Not 18

If you've been through the installation wizard, you may have noticed its provider page only lists 8 entries to fill in — not 18. This is a deliberate, verified design detail, not a missing feature.

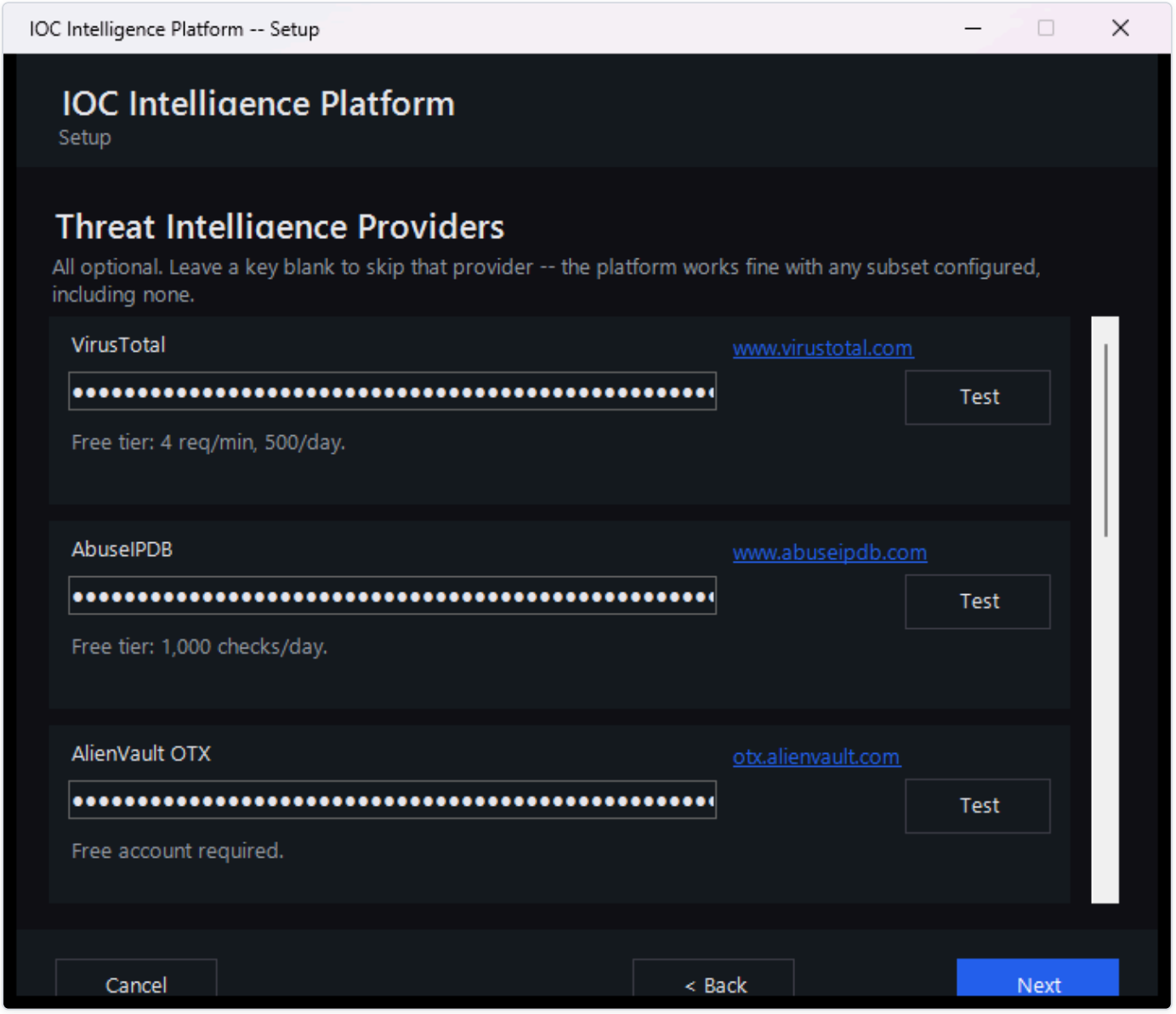

Figure 28 — The Setup Wizard's Threat Intelligence Providers page, where an administrator enters credentials (shown as masked dots, never real key text) for providers such as VirusTotal, AbuseIPDB, and AlienVault OTX.

Here's the honest breakdown of why the numbers work out that way:

- **6 of the 18 providers need no credential at all and have no entry on the wizard's page**: crt.sh, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, and the Internet Intelligence Collector. Since there's nothing to type in, there's simply nothing for a setup screen to ask for — these providers are already active the moment the platform starts, with no setup step required. (NIST NVD and PhishTank also work with no credential, but the wizard still gives each of them an optional key field purely to raise their rate limit — so they're counted in the wizard's 8 entries below, not in this list of 6.)

- The remaining **10 providers with an actual credential requirement are covered by only 8 wizard entries**, because three of them — URLhaus, ThreatFox, and MalwareBazaar — all share a single abuse.ch "Auth-Key." Entering that one key on the wizard's single abuse.ch field activates all three providers at once.
- **2 providers need a credential but have no wizard entry at all: urlscan.io and Google Safe Browsing.** Both are configured exclusively after install, from the app's own Providers page (sign in → Providers) — the same runtime, database-backed, no-restart configuration described below. This isn't a Windows-versus-Linux gap; both installers' wizards omit these two identically, and both say so explicitly in their own setup-time messaging.

So the math is: 8 wizard entries → 10 providers with a wizard entry (7 individual + 3 sharing one abuse.ch key) + 6 always-on providers with nothing to configure + 2 providers that need a credential but are set up after install instead of during the wizard = **18 providers total**, all working once configured. Nothing is hidden or broken — the shorter wizard list is simply an accurate reflection of which providers actually have a setting on that specific page.

Managing Providers From the App (No Restart)

The Setup Wizard's provider page is a **first-run bootstrap**, not the only place these settings live. Once the platform is up and running, every credential, model choice, and enabled/disabled state — for both IOC providers and AI backends — can be viewed and changed from inside the app itself, at any time, with no restart, no re-installation, and no editing of any configuration file by hand. This is the same underlying configuration the wizard wrote on first install; the wizard just gets you to a working starting point, and this page is where you go afterward whenever something needs to change.

You'll find it via the **Providers** link in the workspace navigation, which opens a dedicated page organized into four tabs.

AI Providers. This tab lists the platform's AI backends and their current status — for example, showing which ones are "Not configured," which are "Configured," and which one is currently "Active."

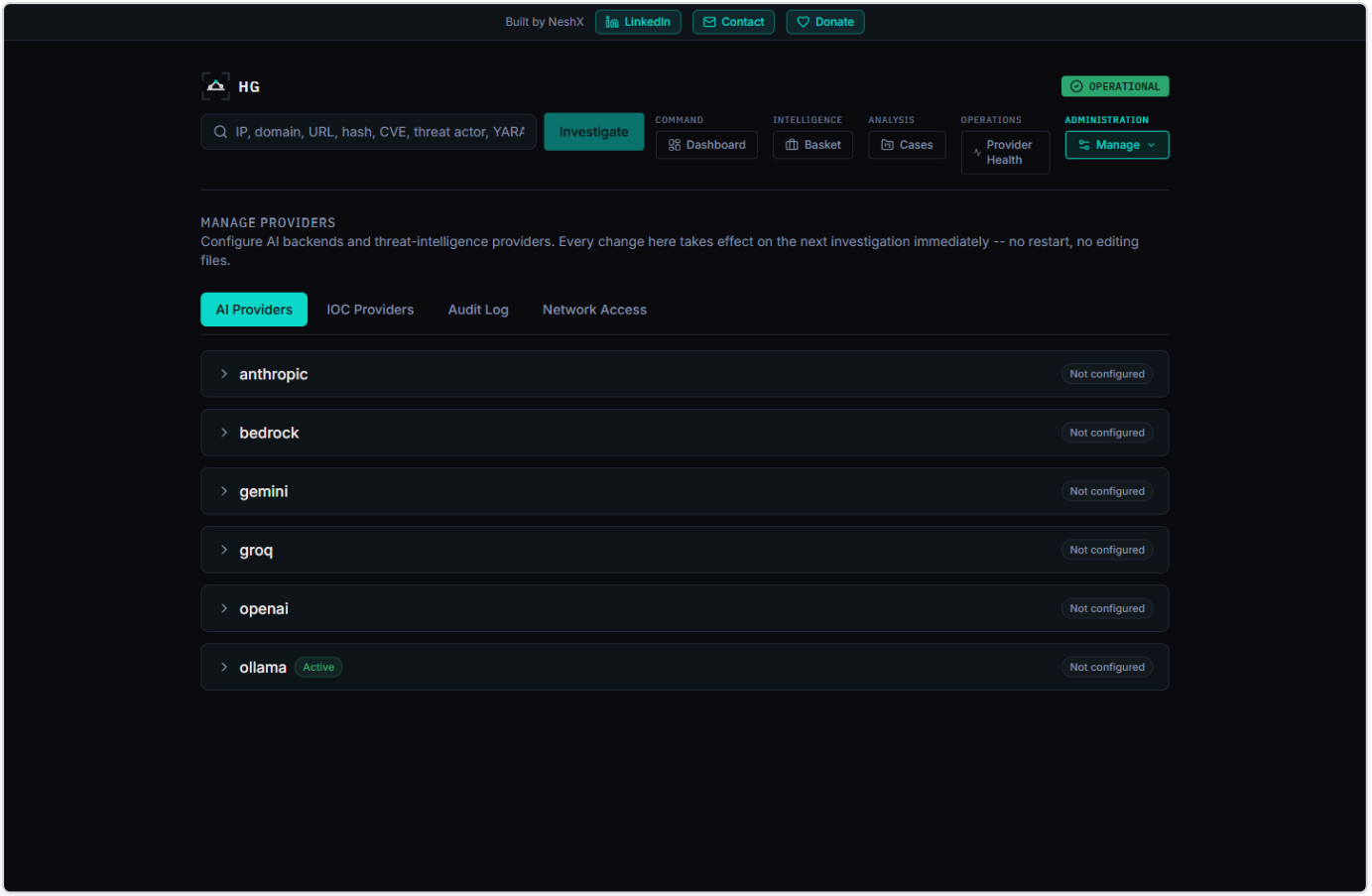

Figure 29 — The /providers page, AI Providers tab, listing each AI backend with its live configured/active status.

Expanding a backend's row lets you enter or update its credentials and model. An already-saved credential is never shown in plain text -- it displays as a row of masked dots followed by the last few characters, the same partial-reveal convention used by most login and payment forms, so you can recognize which key is saved without the full value ever being visible on screen or recoverable from it. From that same expanded row you can click **Test Connection** to confirm the credentials actually work before relying on them, and **Set Active** to make that backend the one the platform uses for new investigations — immediately, without restarting anything.

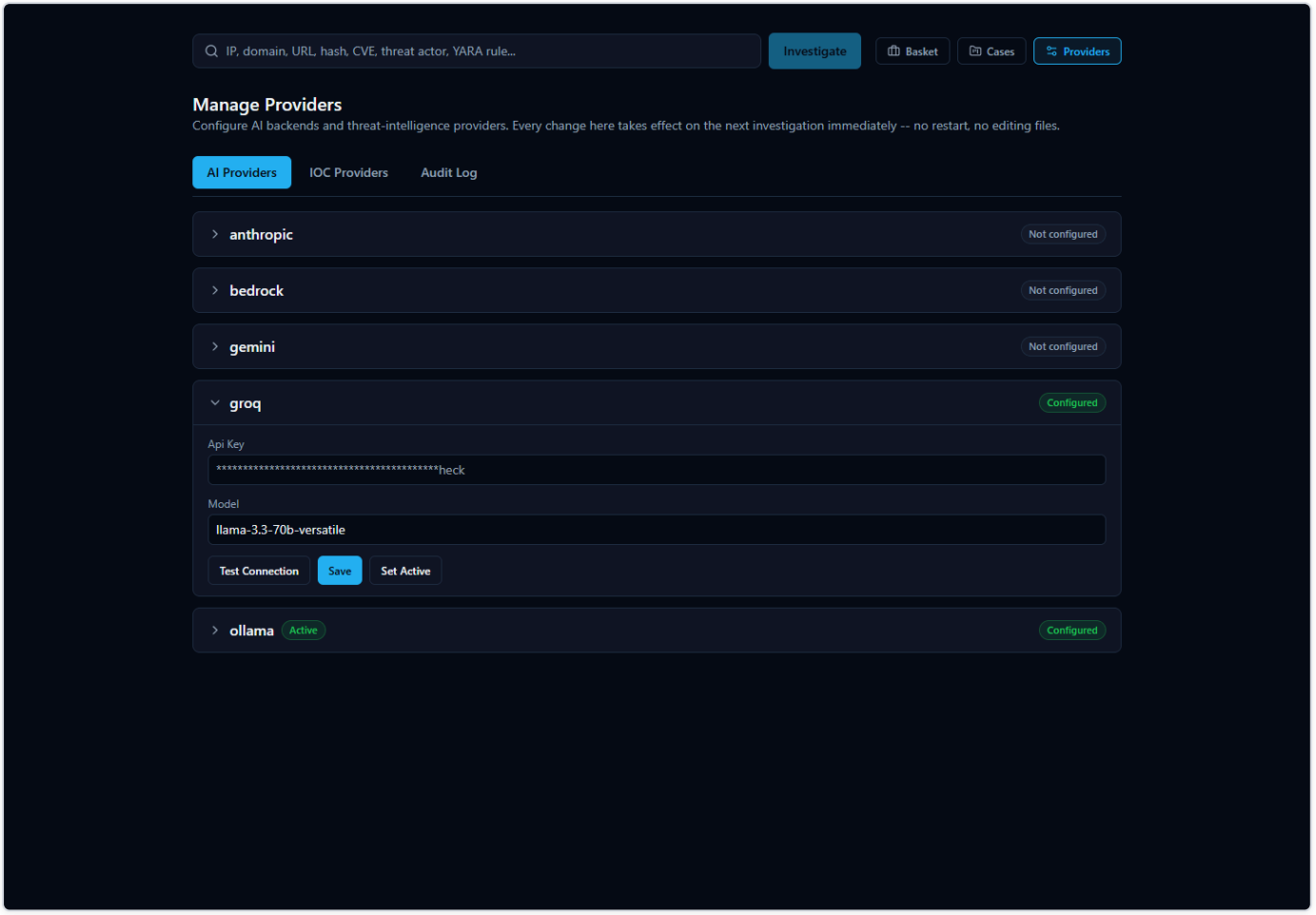

Figure 30 — An AI backend's row expanded for editing, showing the masked API key field, the model field, and the Test Connection / Save / Set Active controls.

IOC Providers. This tab covers all 16 built-in providers described earlier in this section — each one configurable, testable, and individually enabled or disabled, right from this screen. Because each provider's row only asks for the exact fields that provider actually needs (a single credential for most, the shared abuse.ch key for URLhaus/ThreatFox/MalwareBazaar, or the Personal Access Token *and* Organization ID pair for Censys), there's no guesswork about what to fill in. Providers that need no credential at all still appear here with an enable/disable toggle, since "on or off" remains a setting worth controlling even when there's nothing to type in.

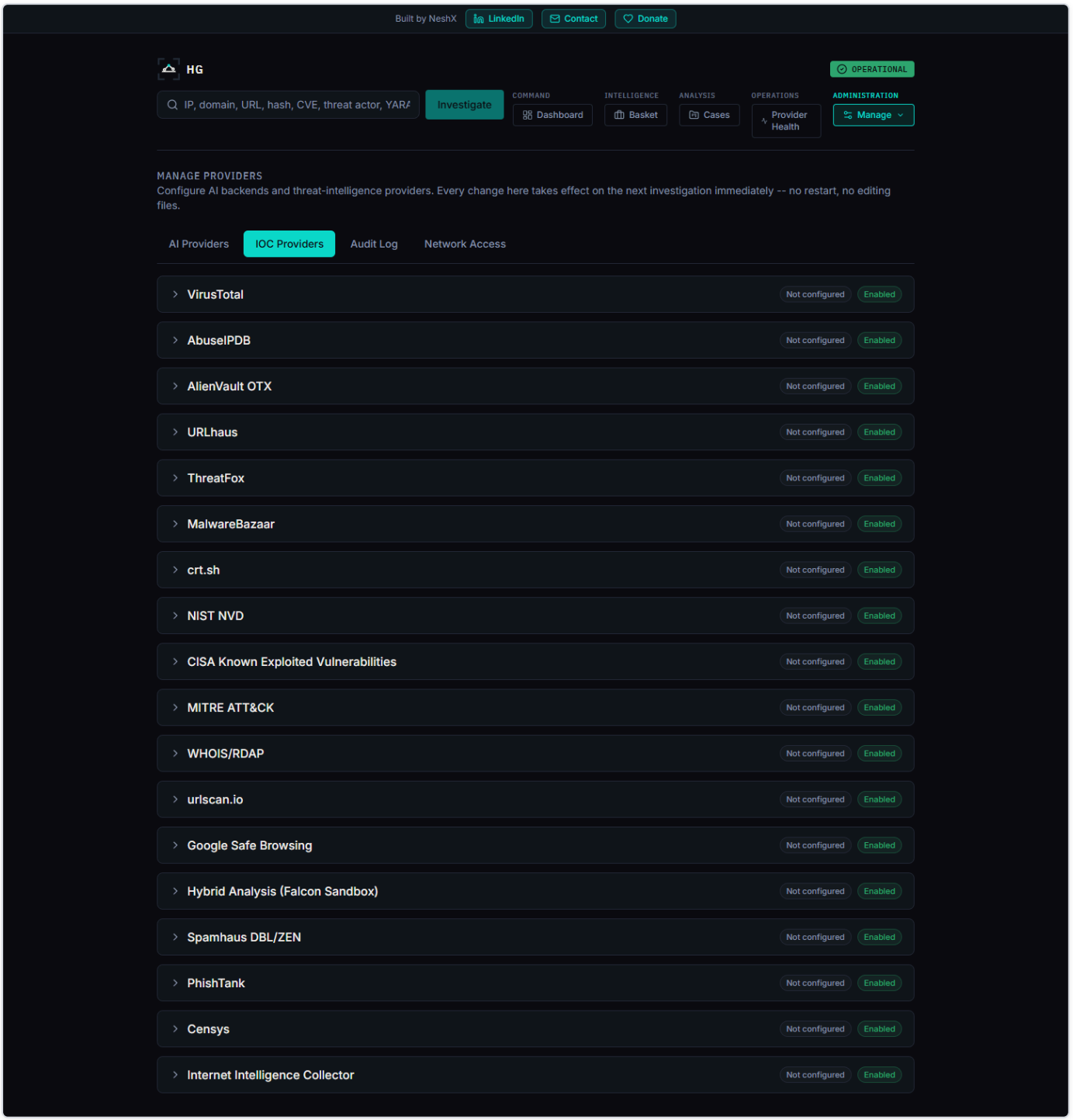

Figure 31 — The IOC Providers tab, listing all 18 registry providers with their real configured/enabled status.

Expanding an IOC provider's row works the same way as an AI backend's: a masked credential field for exactly the input that provider needs, plus Test Connection, Save, and Enable/Disable controls. The screenshot below shows AbuseIPDB expanded with a key typed in, ready to test or save — the same flow used whether you're setting up a provider for the first time or changing an existing key.

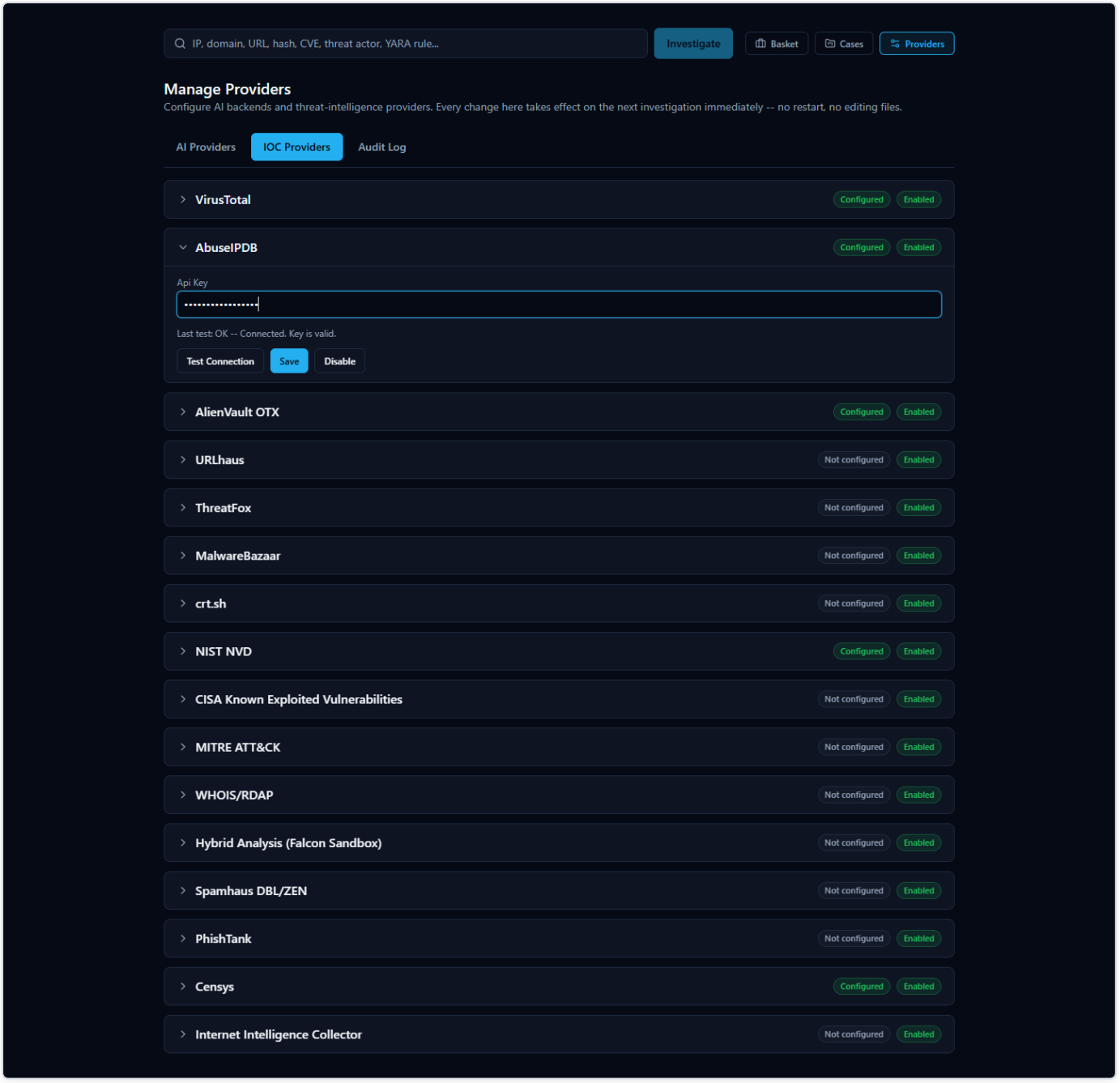

Figure 32 — An IOC provider's row expanded for editing (AbuseIPDB), with the masked API key field and Test Connection / Save / Disable controls.

Test Connection always makes a real request to the provider rather than just checking that a field is non-empty, so an incorrect or revoked key is caught immediately with a specific reason, not a vague failure. The screenshot below shows a genuine failed test — a real "Authentication failed" response from the provider, not a placeholder message:

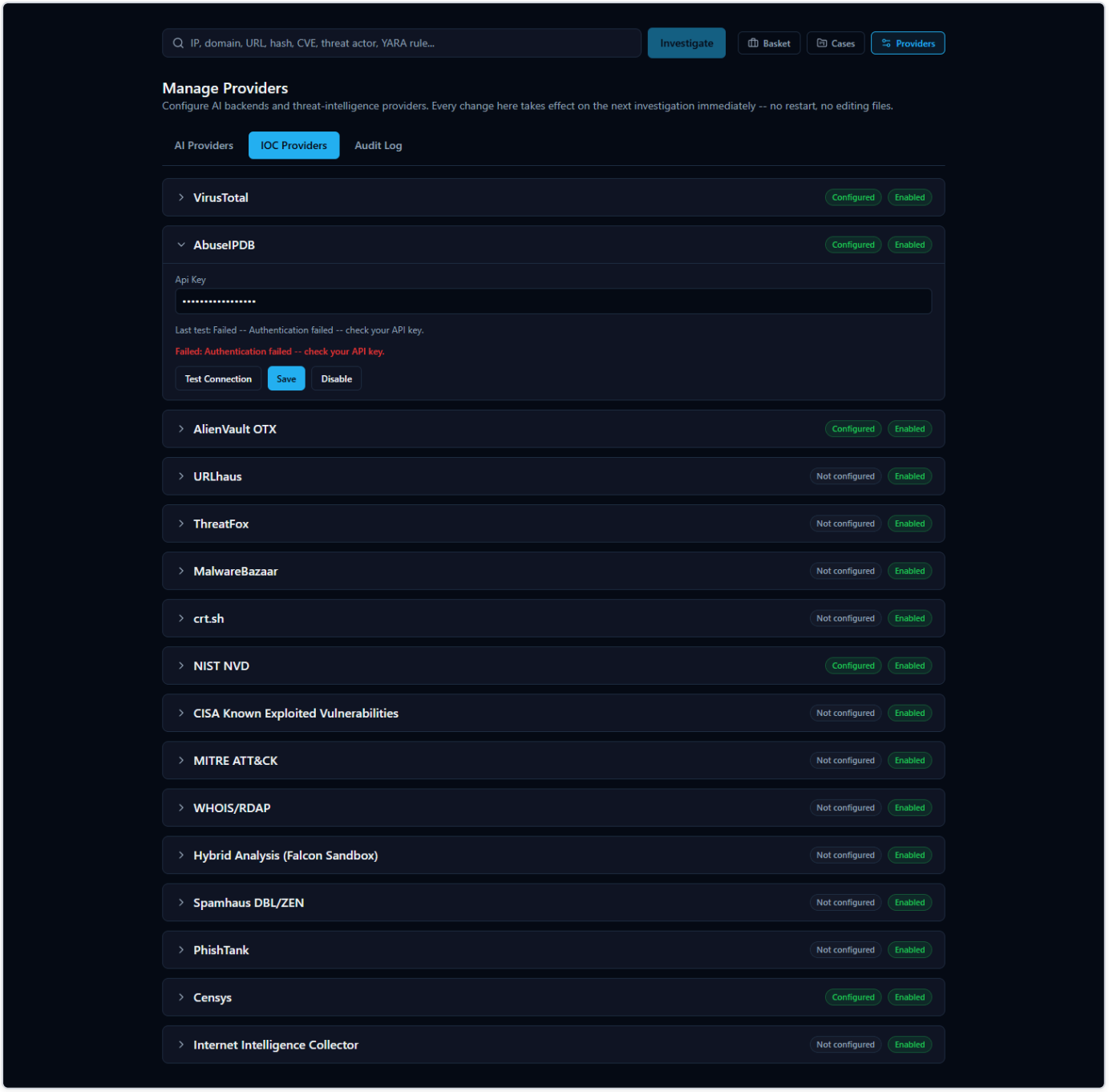

Figure 33 — A real failed Test Connection result: "Failed: Authentication failed -- check your API key," shown in place, with the provider's exact response reflected back rather than a generic error.

Audit Log. Every configuration change made through this page — configuring a provider, enabling or disabling one, switching the active AI backend, running a connection test — is recorded here in plain, human-readable language: who made the change and what it was. Consistent with how the platform treats credentials everywhere else, the audit log **never records the actual credential value** — only that a credential was configured or updated, never what it is.

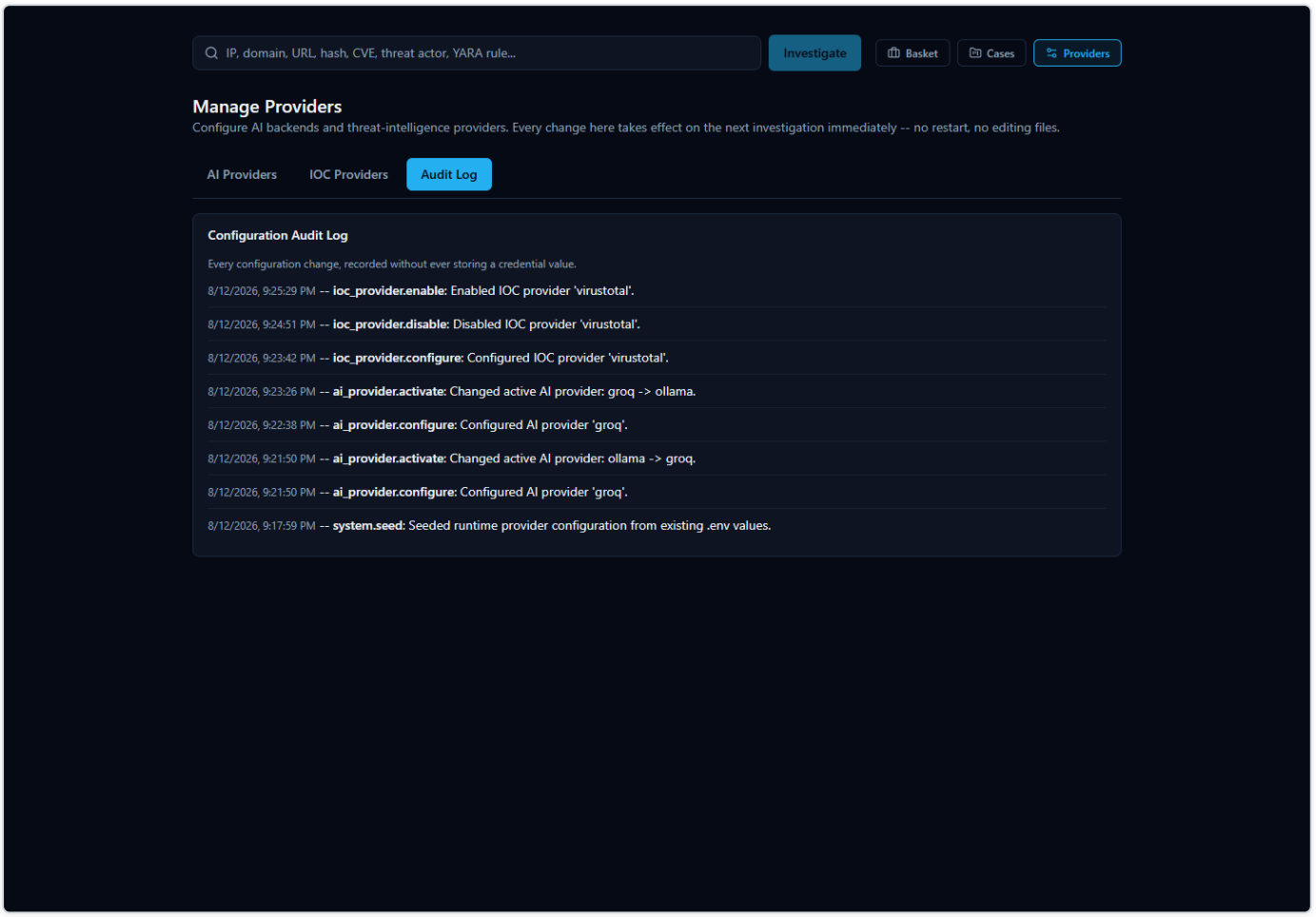

Figure 34 — The Audit Log tab, showing recorded configuration actions with human-readable detail text and no credential values.

Network Access. This tab shows the address other devices on your local network can use to reach the platform (detected automatically during setup), the ports in use, and whether the local Windows Firewall rule needed for LAN access is in place — see "Accessing From Another Computer" for the full walkthrough.

A Note on Provider Disagreement

Because the platform asks many independent providers at once, they won't always agree — and that's normal, useful behavior, not a malfunction. Reputation feeds, blocklists, and community-reported databases each have their own methodology, so it's entirely possible for one provider to flag an indicator as malicious while another calls the exact same indicator clean.

When that happens, the platform's AI-generated summary tries to explain the disagreement in plain language rather than silently picking a side. That AI output should be treated as **analytical assistance, not an unquestionable verdict** — it's a starting point for your own judgment, not a replacement for it. Every investigation includes an Evidence Ledger (and a "Show receipts"-style breakdown of how the score and verdict were reached) so you can trace any AI claim back to the specific, real provider data behind it, rather than just taking the summary's word for it.

AI-Assisted Analysis

When you look up an **IOC** (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID that a security analyst wants to investigate) in HORIZON GRID, the platform doesn't just show you a pile of raw results from a dozen different sources and leave you to make sense of them yourself. It also uses an AI model to read that data and write a plain-language summary of what it means.

This section explains, honestly, what that AI layer actually does, what it deliberately does **not** do, and — most importantly — how you as an analyst can check any AI-written claim against the real evidence behind it rather than simply trusting it.

What the AI Actually Does

The platform's AI analysis happens in two distinct passes over the same investigation:

1. **A per-provider summary.** After each individual data source (VirusTotal, AbuseIPDB, Spamhaus, WHOIS, and so on) returns its result, the AI writes a short summary of *that one provider's findings* — nothing else. It is only shown that provider's own data when writing this summary, so a per-provider summary can never blend in information from a different source by mistake.
2. **One consolidated final assessment.** Once every provider has finished responding, the AI makes a second, separate call — this time it is given all of the per-provider summaries together, plus a correlation step that has already mechanically compared the providers against each other (looking for things they agree on, disagree on, or both independently point to, such as a shared malware family or IP relationship). From that combined picture, the AI writes one overall assessment: a verdict, a risk score, and prose explaining how the providers relate to and support (or contradict) one another.

Both passes run on whichever AI backend the platform has been configured to use. The platform supports eleven interchangeable backends: Ollama (a locally-hosted model with no external API calls), Anthropic, Amazon Bedrock, Google Gemini, Groq (a fast-inference provider with an OpenAI-compatible API, defaulting to the `llama-3.3-70b-versatile` model), OpenAI, Kimi (Moonshot AI), DeepSeek, xAI (the company behind Grok — a different product from Groq above, despite the similar name), Mistral AI, and OpenRouter (a meta-router that gives access to hundreds of underlying models from many different companies through one API). Whichever backend is selected on the AI Configuration page of the setup wizard is the one actually used for both the per-provider summaries and the final assessment; the platform never silently mixes or falls back to a different provider mid-investigation. See `standalone-ai-guide.md` for how each backend is actually configured, credentialed, tested, and switched.

Both of these appear on the investigation page, and they answer different questions. A per-provider AI summary answers "what did this one source say?" The final assessment answers "now that I've seen everything, what's the overall picture?" You'll see one summary box per provider card, and then a single Final Assessment section (with its own Executive Summary) further down the page that speaks for the investigation as a whole.

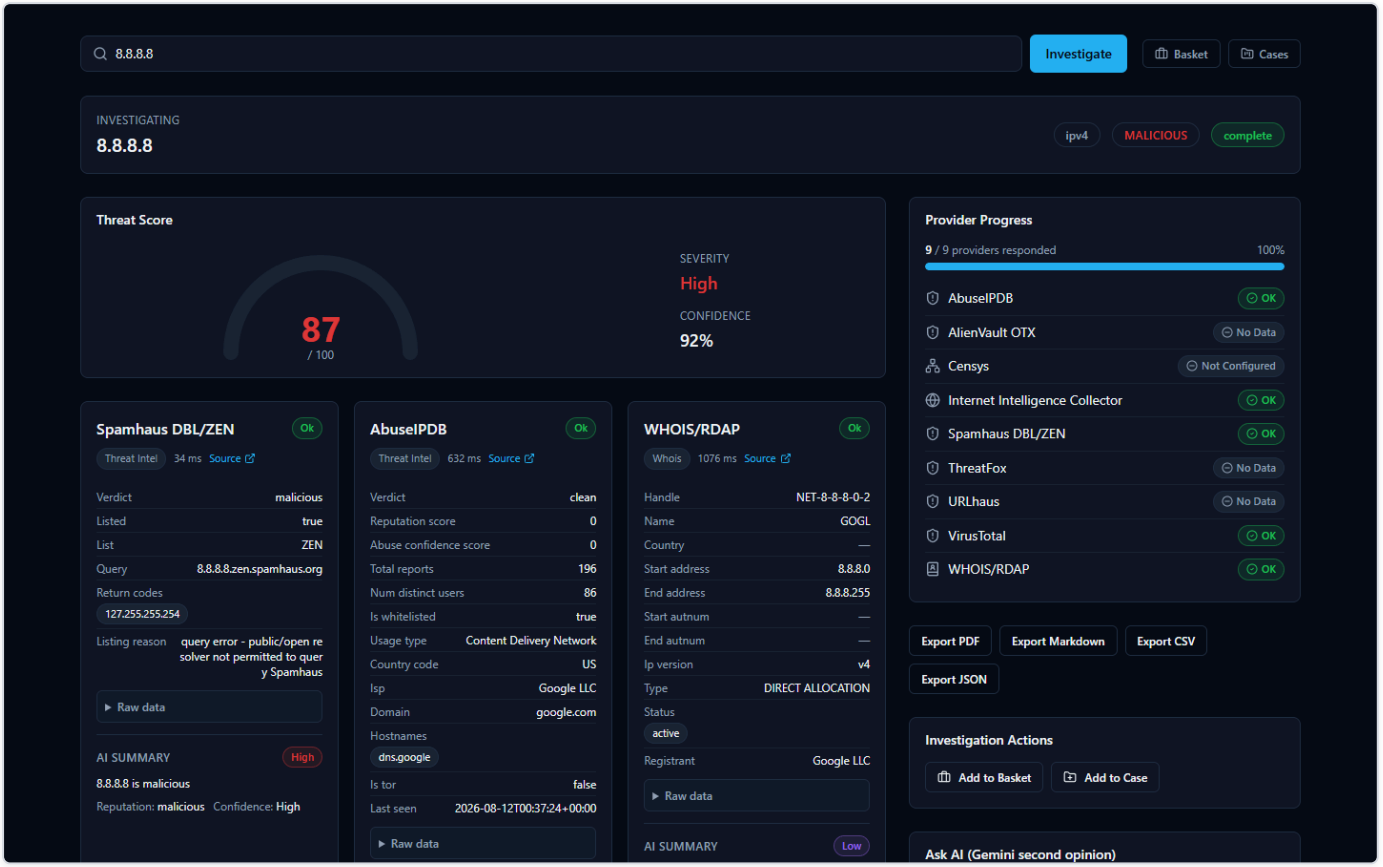

Figure 35 — An investigation of the IOC 8.8.8.8 (Google Public DNS) shown moments after submission, with a Threat Score gauge and individual provider result cards (Spamhaus, AbuseIPDB, WHOIS/RDAP) displayed side by side as each provider finishes responding.

What the AI Does NOT Do

It's just as important to understand the boundaries of this feature as it is to understand what it produces:

- **The AI does not scan any files itself.** It has no antivirus engine, no sandbox, and no ability to independently examine a file, URL, or IP. Every fact it summarizes came from one of the third-party providers the platform queried — VirusTotal, AbuseIPDB, Spamhaus, NIST NVD, and so on.
- **The AI has no threat feed or knowledge base of its own that it consults for a verdict.** It doesn't maintain a private list of known-bad indicators. Its job is strictly to read and reason over the results the providers already returned for *this specific lookup* — never to supply new "knowledge" of its own about whether something is malicious.
- **The AI is explicitly prevented from inventing a verdict when there is no real evidence.** This is a hard, code-level safeguard, not just a polite instruction to the model, and it exists precisely because the alternative was tested and failed. During validation, the platform's real EICAR test file was looked up by its actual MD5 hash (EICAR is a standard, harmless, industry-wide test file used to safely trigger antivirus and threat-intelligence detections without touching real malware). In that specific test, none of the configured providers were actually turned on for that lookup, so there was zero real evidence of any kind — no provider data, no correlation between sources. Despite that, the AI model still produced a verdict of "highly malicious," a fabricated 92% probability of maliciousness, and invented prose claiming an "association with ransomware and trojans" — none of it grounded in anything that had actually been returned. This was caught, and the platform now includes a deterministic check that runs *before* the AI is ever asked to weigh in: if there are no per-provider findings and no correlated relationships between providers, the platform skips the AI call entirely and returns a fixed, honest result — a verdict of "unknown," a message explaining that no provider returned

usable data for the indicator, and the risk fields left at zero — rather than letting a model guess from its own general training data. In other words, when there's nothing to go on, the platform says so, instead of making something up.

How the AI Handles Disagreement Between Providers

Threat intelligence sources don't always agree, and the platform doesn't hide that when it happens — it says so directly. A good real example is the IOC `8.8.8.8` (Google's public DNS resolver): Spamhaus flagged it as malicious, but the actual reason given was a query-permission error ("public/open resolver not permitted to query Spamhaus"), not a genuine detection — while AbuseIPDB reported zero abuse reports and VirusTotal returned a clean result. The AI's Executive Summary for that investigation says so plainly: the IP "is considered malicious by Spamhaus, but its reputation as a clean and safe IP address is supported by AbuseIPDB and VirusTotal."

That's not a bug in the product — it's the AI doing exactly what it's supposed to do: surfacing a genuine conflict between sources instead of quietly picking one side, so the analyst can see the disagreement and judge it for themselves (in this case, a known, benign, heavily-used public DNS server being caught by a resolver-permission rule rather than an actual threat detection).

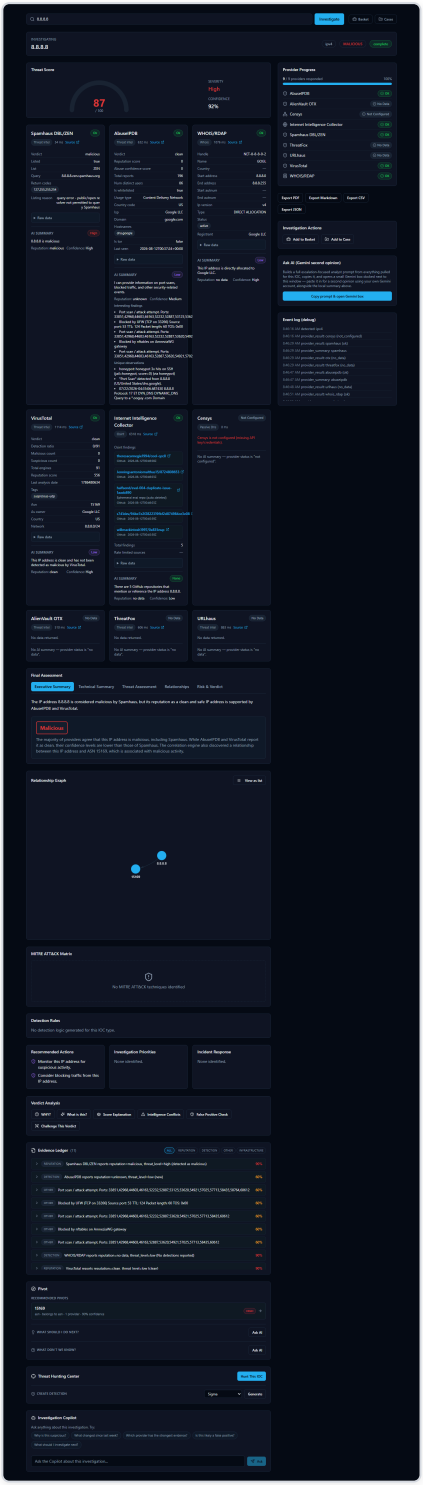

Figure 36 — The same 8.8.8.8 investigation, scrolled further, showing the Final Assessment's Executive Summary describing the disagreement between Spamhaus (malicious) and AbuseIPDB/VirusTotal (clean), alongside the Evidence Ledger, Verdict Analysis tabs, and Relationship Graph.

Verifying an AI Claim Instead of Trusting It Blindly

Every investigation gives you concrete ways to check an AI-written statement against the real underlying evidence, rather than asking you to just take its word for it:

- **The Evidence Ledger.** Every investigation includes a running list of individual, numbered evidence items — each one tied to a real source and carrying its own confidence percentage. This is the platform's version of "show receipts": any specific claim the AI makes should be traceable back to one or more of these concrete, checkable entries rather than floating free as an unsupported assertion.
- **Verdict Analysis tools.** Alongside the Final Assessment, the investigation page offers a set of tabs specifically for interrogating the verdict rather than passively reading it, including **"Why?"** (asks the AI to justify the verdict by pointing at specific evidence), **"Challenge This Verdict"** (deliberately pushes back on the AI's own conclusion to see if it holds up), and **"False Positive Check"** (asks the AI to reason about whether this could be a benign result mislabeled as a threat). These exist so that an analyst's natural next question — "why should I believe this?" — has a direct, built-in answer instead of requiring you to take the summary on faith.
- **The AI provenance badge.** Every AI-generated Final Assessment carries a small badge naming exactly which AI backend and model produced it — for example, `groq / llama-3.3-70b-versatile` or `ollama / llama3.2:3b`. This matters because, as noted above, the platform never silently switches or falls back between backends mid-investigation, so the badge is a dependable record rather than a guess: an analyst can always see precisely which AI generated a given conclusion, and can factor that into how much weight to give it (a larger hosted model and a small local model won't necessarily reason about the same evidence with the same thoroughness).

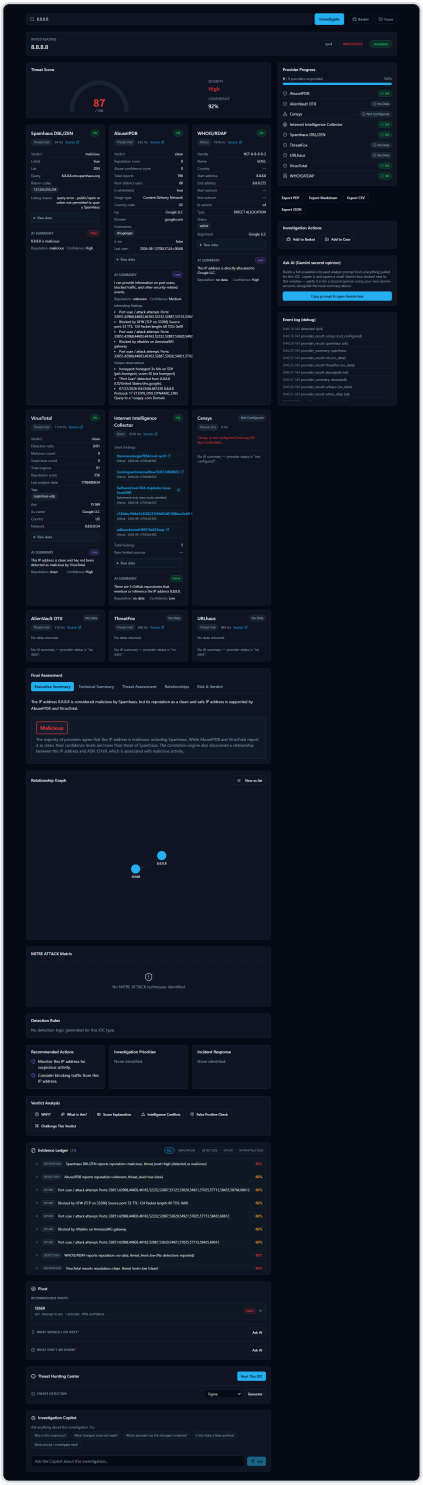

Figure 37 — The Evidence Ledger and Verdict Analysis tabs (Why?, What's this?, Score Explanation, Intelligence Conflicts, False Positive Check, Challenge This Verdict) on the 8.8.8.8 investigation, giving an analyst tools to check the AI's verdict against the underlying evidence.

Switching AI Backends and Getting a Second Opinion

Because the platform supports several interchangeable AI backends, changing which one analyzes your work is a day-to-day action, not an administrator task. A compact **"AI: [dropdown]"** control sits right next to the search bar on the home page, showing whichever

backend is currently selected along with a colored dot — green if that backend is actually configured with working credentials, gray if it isn't — and a **Manage** link through to the full provider configuration page. Picking a different backend from the dropdown changes which AI analyzes your next investigation immediately: no restart, no editing configuration files, no detour through a separate settings page.

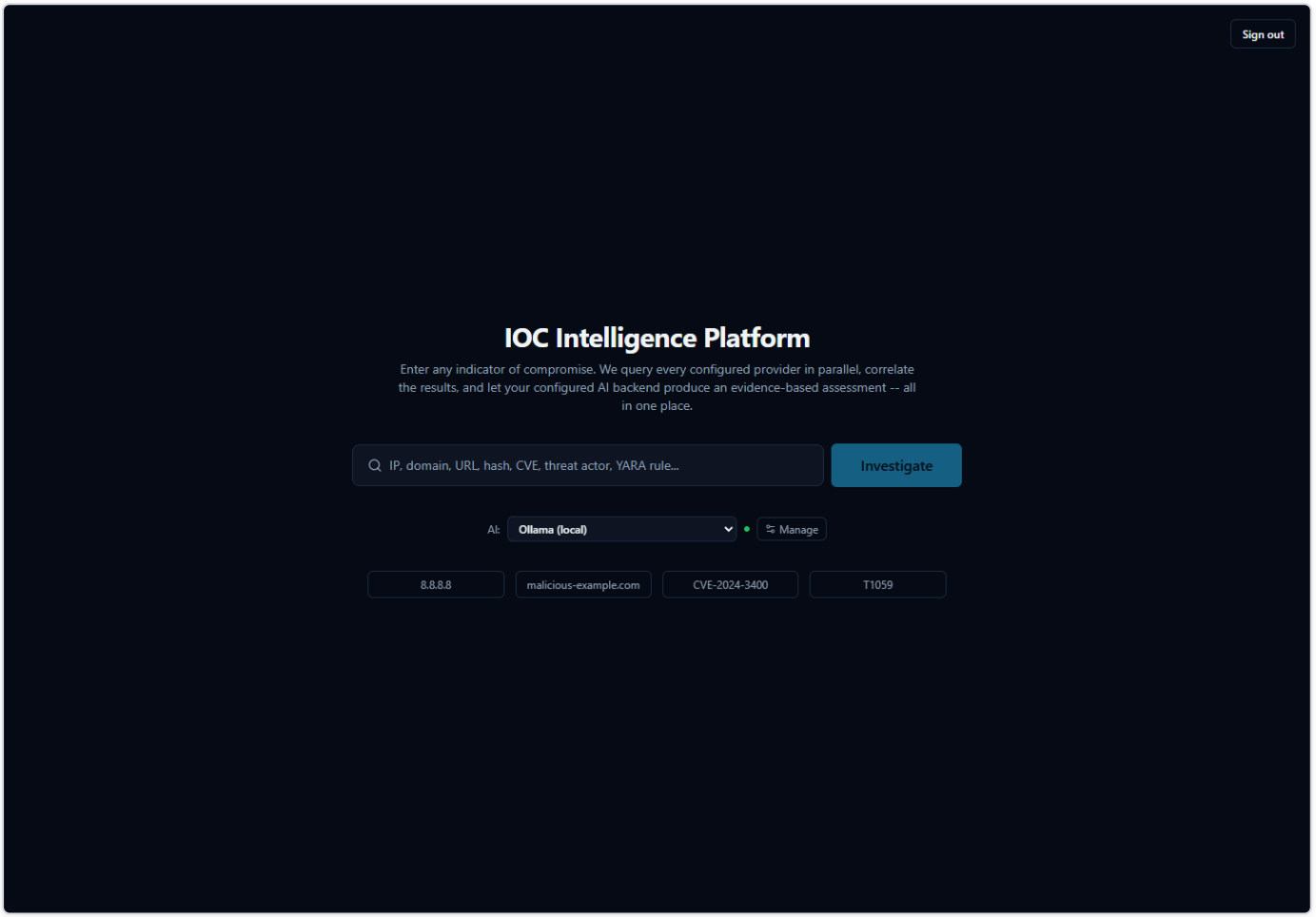

Figure 38 — The home page showing the compact "AI:" dropdown (currently "Ollama (local)") and "Manage" control next to the search bar, used to switch which AI backend analyzes the next investigation without leaving the page.

After picking a different entry from the dropdown, the control immediately reflects the new selection and its real configured status — here, switched to Groq:

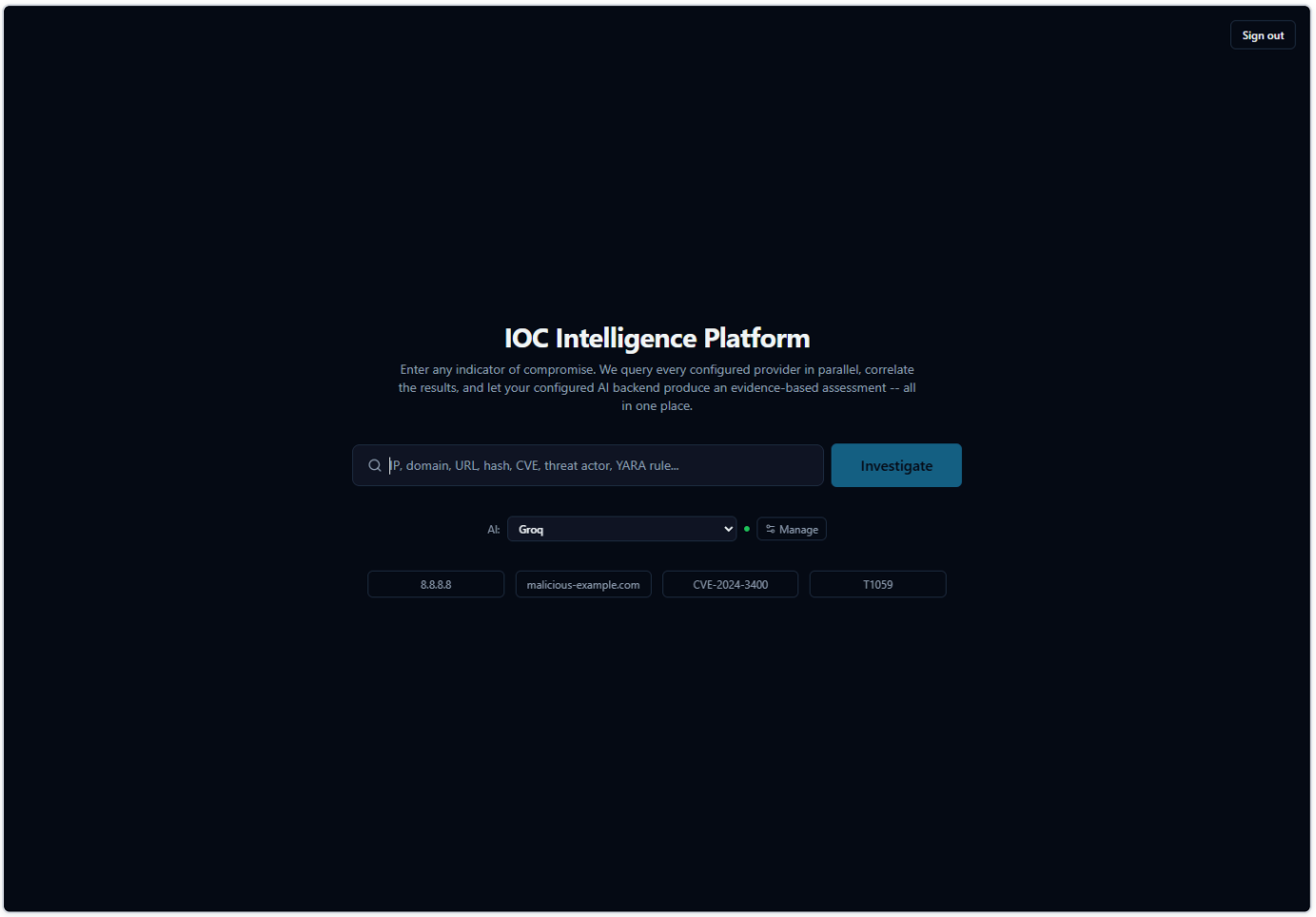

Figure 39 — The same "AI:" control after switching to Groq — the dropdown and its green configured-status dot update immediately, and this backend is what the next investigation will use.

This connects directly to the point above about not trusting an AI claim blindly: if you want to check whether a verdict reflects the evidence itself rather than one particular model's read of it, you can ask a second AI to look at the same evidence and see whether it agrees. On a completed investigation's page, the **AI Comparison** panel lets you pick a different backend and click **"Analyze with [backend]"**; the platform re-runs only the final-assessment step against the evidence already collected — it does not re-query any provider — and appends that backend's independent result below the original, with each result clearly labeled by exactly which AI and model produced it. Neither result is presented as more correct than the other; the value is in having a second opinion to weigh against the first, which is especially useful for sanity-checking a verdict, or for spotting cases where two backends actually disagree.

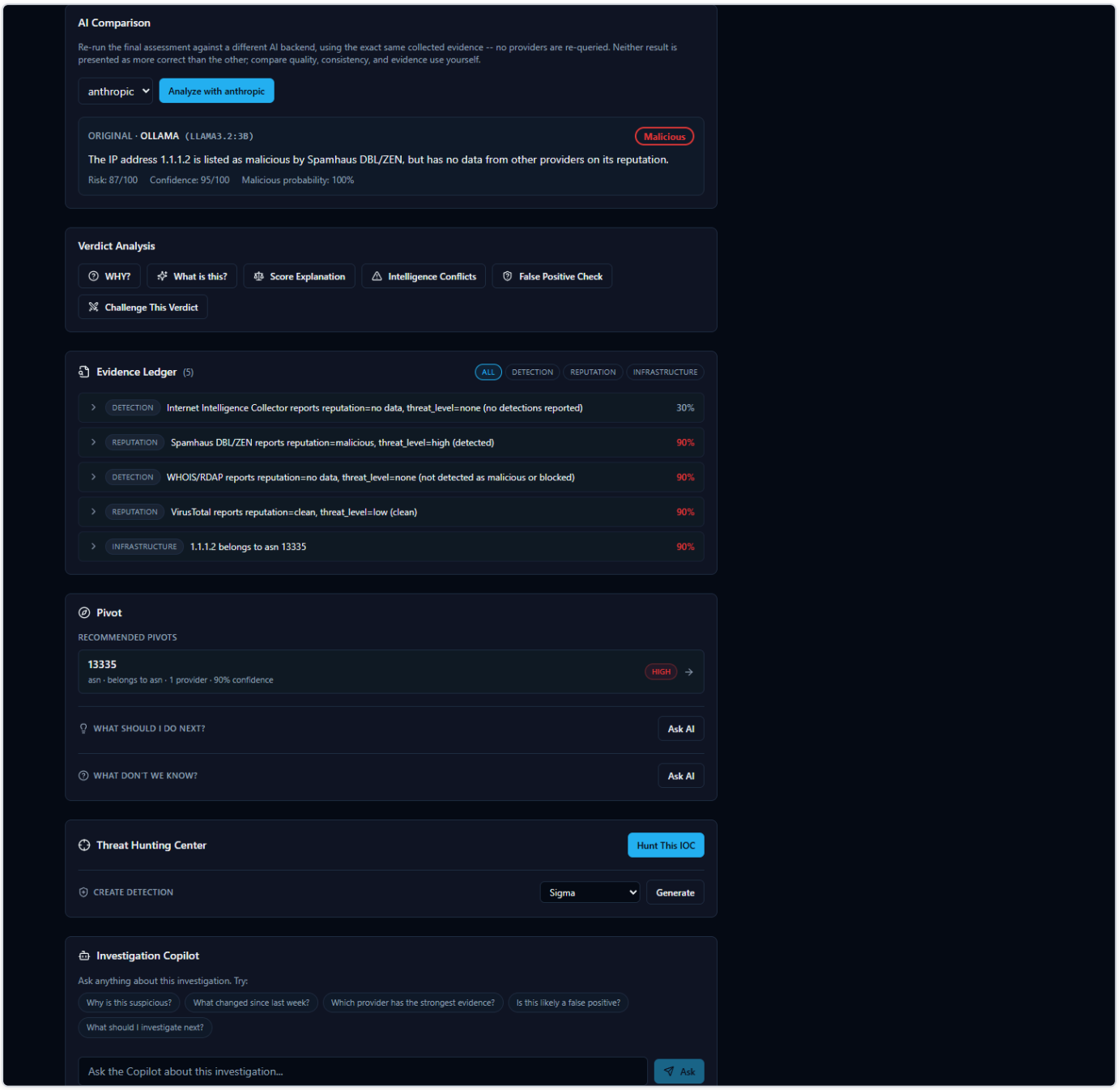

Figure 40 — A completed investigation's AI Comparison panel, showing the original Ollama-generated assessment (verdict, risk score, executive summary) with a ready "Analyze with anthropic" control to append an independent second opinion from a different backend.

Ask AI (Gemini Second Opinion) -- a separate, manual escalation path

Alongside AI Comparison, every investigation page (both while it is still running and afterward) shows a smaller **"Ask AI (Gemini second opinion)"** panel in the sidebar. This is a distinct feature, not another view of AI Comparison, and works differently: clicking **"Copy prompt & open Gemini box"** builds a single, detailed analyst-escalation prompt from everything pulled for this IOC so far (provider results, per-provider summaries, and correlation), copies that prompt to your clipboard, and opens a small popup window pointed at `gemini.google.com`. You then paste the prompt into that popup yourself and read Gemini's answer there, signed in with your own personal Google account.

This matters for two reasons an analyst should know before using it. First, it is entirely manual and entirely outside the platform: the platform never sends your IOC data to Gemini itself, has no API credential or integration with Gemini for this feature, and never sees or stores whatever Gemini replies with in the popup -- unlike AI Comparison's result, a Gemini second opinion obtained this way is not saved to the investigation record. Second, because the copied prompt includes the IOC value and every provider result pulled so far, treat it the same way you would treat pasting investigation data into any other outside AI chat tool -- follow your organization's policy on what may leave the platform this way before using it on sensitive investigations.

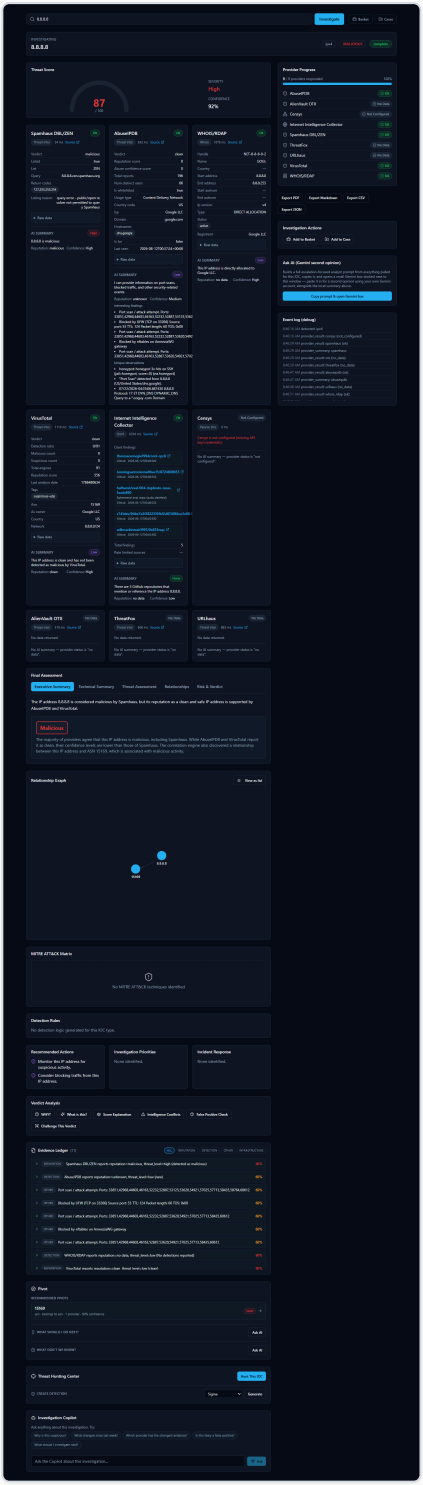

Figure 41 — The full investigation page for 8.8.8.8, with the "Ask AI (Gemini second opinion)" panel and its "Copy prompt & open Gemini box" button visible in the top-right sidebar, below Export and Investigation Actions.

AI analysis in this product is intended as analytical assistance, not unquestionable truth, and every AI-generated claim is designed to be traceable back to real evidence an analyst can independently check.

Evidence and Receipts

Every AI claim comes with a receipt

When the platform finishes investigating an IOC (Indicator of Compromise -- a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID that a security analyst wants to investigate), it doesn't just hand you a paragraph of AI-written analysis and expect you to take it on faith. Underneath every AI explanation sits an **Evidence Ledger**: a list of individual, checkable evidence records, each one built directly from what a specific provider (one of the many third-party threat-intelligence services the platform queries, such as VirusTotal or AbuseIPDB) actually returned, or from a connection the platform's correlation step found between two providers' results.

The Evidence Ledger itself is not written by the AI. It is assembled deterministically from the real provider data and the real correlation output, precisely so that when the AI *does* write a summary, a verdict, or an answer to a question, every meaningful claim in that text can point back to one of these real records instead of being a bare assertion. Each item carries its own source, its own confidence score, and the underlying data it came from -- nothing in the ledger is invented after the fact to make an AI claim look more supported than it is.

On top of the ledger sit several different views an analyst can open on the same evidence -- for example, "Why?", "What's this?", "Score Explanation," "Intelligence Conflicts," "False Positive Check," and "Challenge This Verdict." These are different lenses on the same underlying evidence, not separate, independent opinions. Where an AI explanation cites specific evidence for a specific sentence, "Show receipts" takes you straight from that sentence to the exact evidence record(s) it's based on, instead of leaving you to wonder where a number or a claim came from.

Why this matters: never take the AI's word for it

This design exists for one reason: an analyst should never have to simply trust an AI's conclusion. The AI is analytical assistance -- it reads, summarizes, and correlates a lot of provider data quickly -- but the underlying evidence, not the AI's prose, is the actual source of truth. If a claim in a summary can't be traced to a real evidence record, that's a signal to question it, not accept it.

A genuine example from this platform makes the point well. Looking up `8.8.8.8` (Google's Public DNS server, a benign, well-known service) produces a Threat Score of 87 out of 100, rated "High" -- because the Spamhaus provider marked it malicious. But the actual reason string behind that verdict, visible in the evidence itself, is "query error -- public/open resolver not permitted to query Spamhaus" -- a policy rejection from Spamhaus's own lookup service, not a detection of malicious behavior. Meanwhile AbuseIPDB reports zero abuse reports and VirusTotal calls the address clean. The platform's own AI-written Executive Summary is honest about this: it describes the IP as "considered malicious by Spamhaus, but its reputation as a clean and safe IP address is supported by AbuseIPDB and VirusTotal" -- disagreement and all.

This isn't a bug to hide -- it's exactly the kind of case the Evidence Ledger exists for. An analyst who opens the underlying evidence record can read the literal reason for the Spamhaus verdict, recognize it as a lookup-policy quirk rather than a real threat finding, and decide for themselves whether the high score is warranted. That is the whole point: the platform surfaces disagreement and lets you check it, rather than smoothing it over into a single unquestionable number.

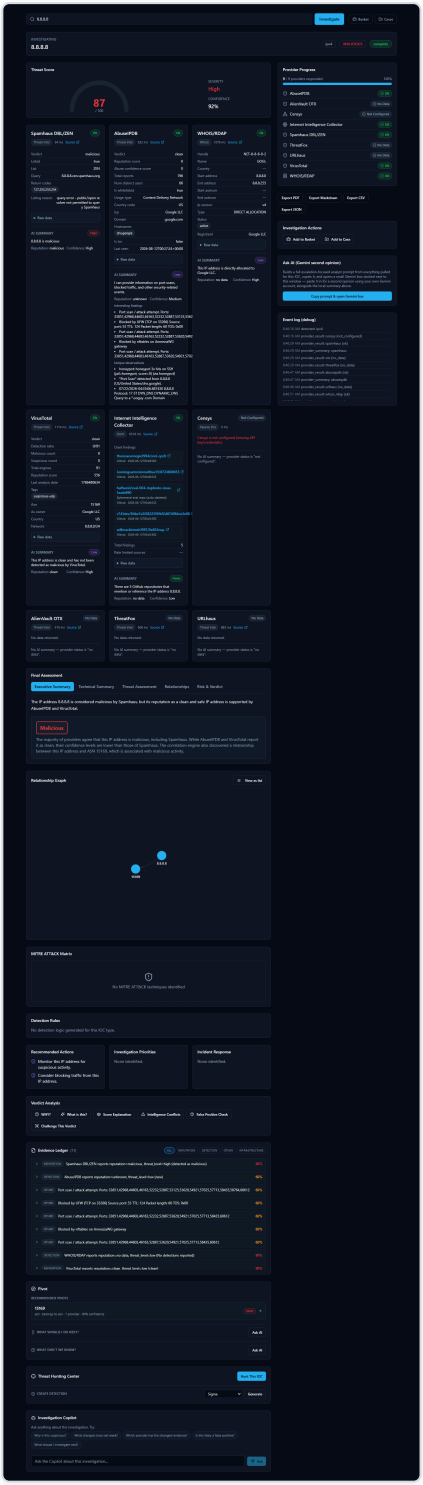

Figure 42 — For the IOC 8.8.8.8 (Google's public DNS resolver), the Evidence Ledger lists individual, checkable evidence records with confidence scores, alongside the Verdict Analysis tabs (including "Why?" and "Challenge This Verdict") an analyst can use to trace exactly which evidence backs the Executive Summary's statement that Spamhaus flagged the IP while AbuseIPDB and VirusTotal call it clean.

Correlation and the Relationship Graph

What "correlation" means here

Every provider the platform queries only knows about its own narrow slice of the picture. AbuseIPDB knows abuse reports. WHOIS/RDAP knows registration and network-ownership details. VirusTotal knows antivirus-engine detections. On their own, these are just separate lists of facts from separate sources.

Correlation is the step where the platform automatically looks across everything the providers returned and notices when two separate findings are actually pointing at the same related fact. A common example: an IP address and the ASN (Autonomous System Number -- an identifier for the network block or organization that owns a range of IP addresses) it belongs to. If one provider reports the IP address and another (or the same provider's WHOIS/RDAP data) reports which network owns it, the platform draws that as a connection rather than leaving you to notice the overlap yourself while reading two separate provider write-ups.

This happens automatically after every provider has finished responding, before the platform writes its final consolidated assessment -- so the AI's overall summary is correlation-aware, not just a stitched-together list of per-provider opinions. When more than one provider corroborates the same underlying fact, the platform's confidence in that connection increases accordingly, since agreement across independent sources is more meaningful than a single provider's claim alone.

The Relationship Graph: seeing connections instead of reading walls of text

The **Relationship Graph** is where these correlated connections become visible. Instead of an analyst reading through several separate provider cards and mentally cross-referencing which facts relate to which, the graph draws each fact as a node and each discovered connection as a line between nodes -- so relationships that would otherwise be buried across multiple paragraphs of provider text are visible at a glance.

In the simplest case, this might be no more than two connected nodes -- for example, an IP address and the single ASN it belongs to. In a richer investigation involving more providers and more shared facts -- such as a vulnerability with related findings pulled together from several sources -- the same graph grows to reflect however many real, corroborated connections the correlation step actually found. In every case, the graph is a visualization of the same underlying evidence discussed above, not a separate or less-verified layer -- it shows relationships the platform can already point to specific evidence for.

Figure 43 — For the Log4Shell vulnerability (CVE-2021-44228), the investigation view's Relationship Graph visualizes the correlated connections the platform found around the CVE, letting an analyst see how findings relate to one another at a glance instead of reading each provider's write-up separately to piece the picture together.

Case Management

Why group IOCs together at all?

A single **IOC** (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID that a security analyst wants to investigate) rarely tells the whole story on its own. A real incident is usually made up of several related pieces of evidence: the CVE (Common Vulnerabilities and Exposures identifier — a public catalog number for a known software vulnerability) behind an exploit attempt, the malicious IP address that tried to use it, the file hash of a payload that was dropped, and the analyst's own observations along the way.

If a SOC (Security Operations Center) analyst investigates each of those as a separate, disconnected search, several problems show up quickly:

- **Context gets lost.** A week later, nobody remembers that the "8.8.8.8 lookup" and the "CVE-2021-44228 lookup" and the "note about the firewall log" were all part of the same incident.
- **Handoffs are painful.** A second analyst picking up the investigation has no single place to see everything that's already been found — they have to be told, or re-discover it themselves.
- **Nothing to report from.** When it's time to write up what happened, the evidence is scattered across a browser history of individual lookups instead of living in one record.

The platform's **Case** feature solves this by giving an analyst one container — a "case" — that IOCs and notes can all be attached to as an investigation grows. Instead of "three unrelated searches I happened to run this week," it becomes "one case, with three pieces of evidence attached to it."

Walking through a real case

The screenshot below shows a case immediately after it was created:

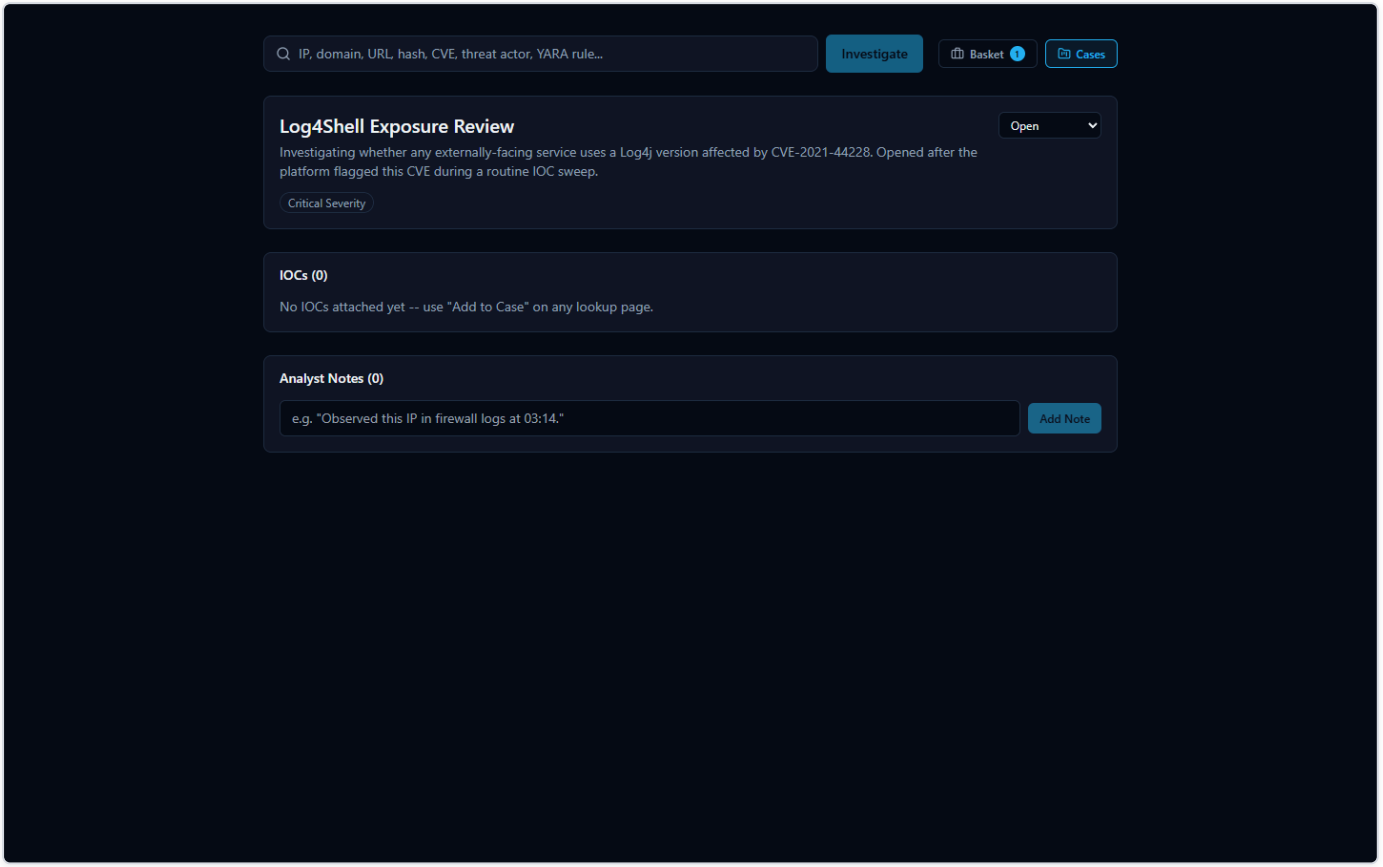

Figure 44 — A newly created case titled "Log4Shell Exposure Review," set to Critical severity and Open status, with a written description — an empty container ready to have IOCs and notes attached to it.

At this point the case is just a title, a severity level (here, Critical), an Open status, and a description of what the analyst is tracking — no evidence has been linked to it yet.

From here, an analyst builds the case up over the course of the investigation:

- **Attaching an IOC:** every investigation page in the platform (the page that shows a lookup's results) has an **"Add to Case"** button. When an analyst is looking at the results for an IOC — for example, the CVE-2021-44228 (Log4Shell) investigation — clicking "Add to Case" links that IOC directly to the case, so it no longer has to be tracked as a separate, disconnected search.
- **Adding a note:** the case also supports freeform analyst notes — short written observations that don't belong inside an automated provider result, such as "confirmed this CVE affects our externally-facing logging service; escalating to patch team."

The second screenshot shows the same case after both of those actions have happened:

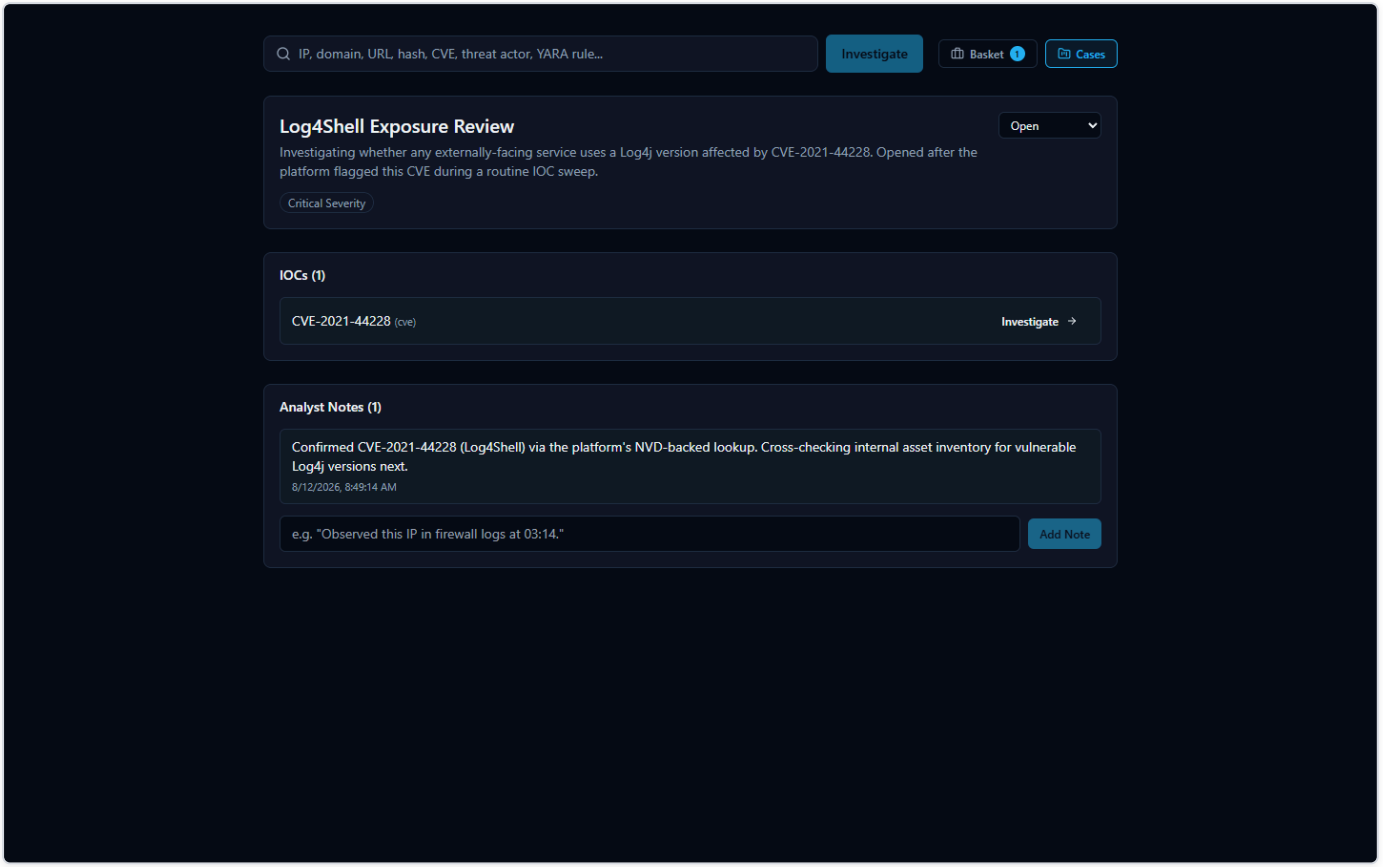

Figure 45 — The same "Log4Shell Exposure Review" case after the CVE-2021-44228 IOC was attached via the investigation page's "Add to Case" button and an analyst note was added — the case now shows 1 attached IOC and 1 note.

The case now shows one attached IOC (CVE-2021-44228) and one analyst note. As an investigation continues, more IOCs and more notes can be attached the same way, so that everything relevant to "Log4Shell Exposure Review" stays in one place rather than spread across separate, easily-forgotten lookups.

IOC Basket

What it is (and what it isn't called)

While working an investigation, an analyst often wants to keep track of a handful of IOCs they're personally paying attention to — not necessarily a formal case yet, just a running list of "things I'm keeping an eye on." The platform's real name for this feature is the **IOC Basket** (or just "Basket"). It is a personal scratch-list: an analyst adds IOCs to it while they investigate, and the Basket holds onto them for later.

It's worth being precise about naming here: this feature is **not** called a "Watchlist" anywhere in the product. "Basket" is its actual name. It serves a purpose an analyst might informally think of in watchlist terms — a personal running list of IOCs to come back to — but the platform's own term for it is Basket, and that's the name used throughout its interface.

Using the Basket

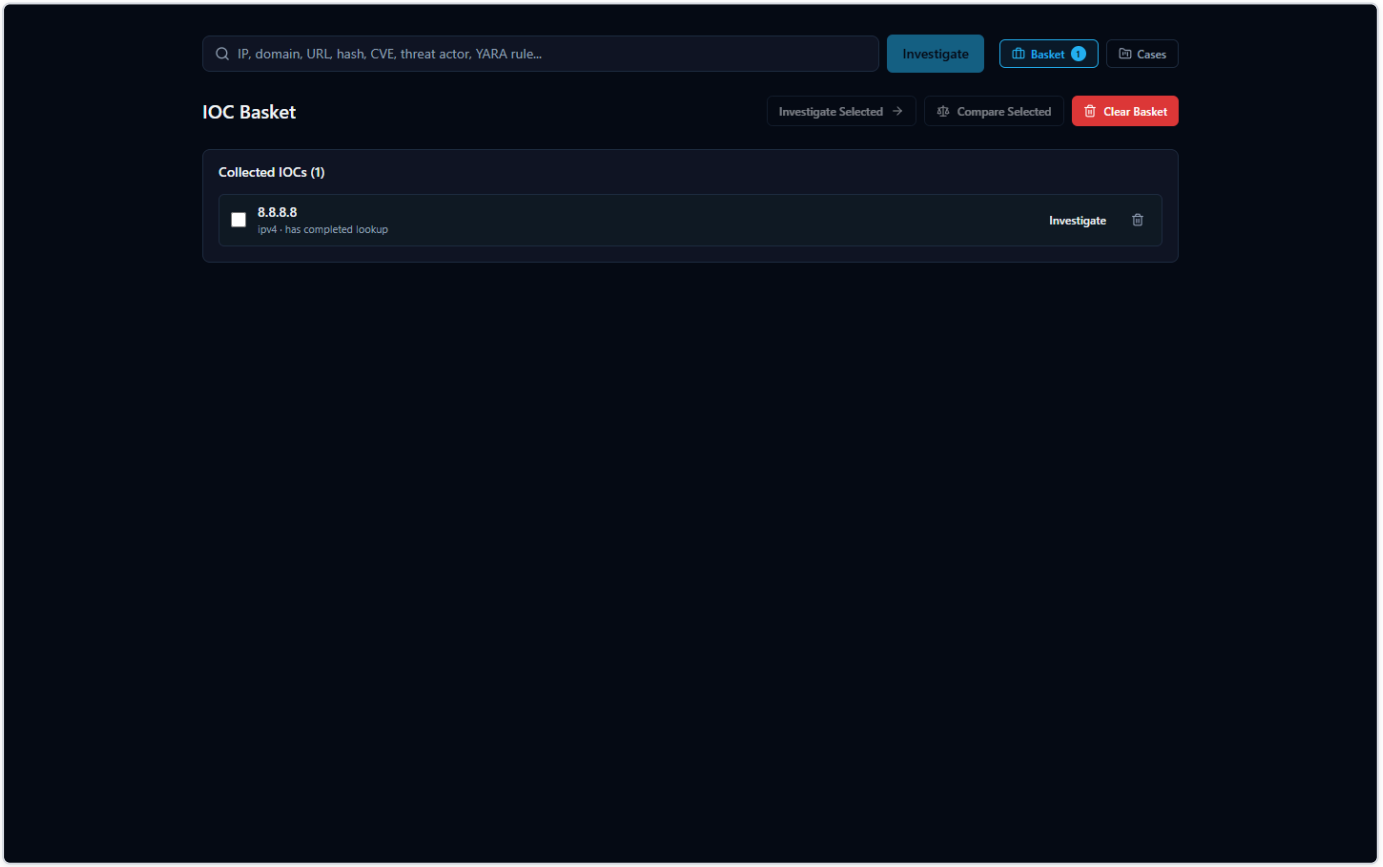

Figure 46 — The IOC Basket page, showing one collected IOC (8.8.8.8) with a status tag indicating its lookup has already completed, alongside "Investigate Selected," "Compare Selected," and "Clear Basket" buttons.

In the screenshot above, the Basket holds one collected IOC, 8.8.8.8, tagged as having already completed a lookup — so the analyst can see at a glance which basket entries already have results waiting versus which still need to be investigated.

The Basket page offers a small set of actions for working with the IOCs an analyst has collected:

- **Investigate Selected** — run (or re-run) a lookup on the IOCs currently selected in the Basket, rather than having to re-enter each one manually on the search page.
- **Compare Selected** — look at more than one saved IOC side by side, useful when an analyst wants to check whether two or more indicators they've been tracking relate to each other or share characteristics.
- **Clear Basket** — remove everything from the personal scratch-list once it's no longer needed.

Because the Basket is a personal list rather than a shared case record, it's best suited for the everyday "let me keep track of these while I work" habit — when something in the Basket turns out to matter for a real incident, it can then be formally attached to a Case (see the Case Management section above) so the finding isn't lost once the Basket is cleared.

Exporting Findings

Every investigation the platform runs produces a lot of material worth keeping: a verdict, a risk score, per-provider evidence, correlation results, and an AI-written summary. Once you've reviewed an investigation, you'll usually want to get some or all of that out of the browser and into wherever the rest of your work lives — a ticket, an incident report, a chat message to a teammate, or another tool entirely.

The export controls for this live in the sidebar of every investigation page. Opening them shows four buttons: **Export JSON**, **Export Markdown**, **Export PDF**, and **Export CSV**.

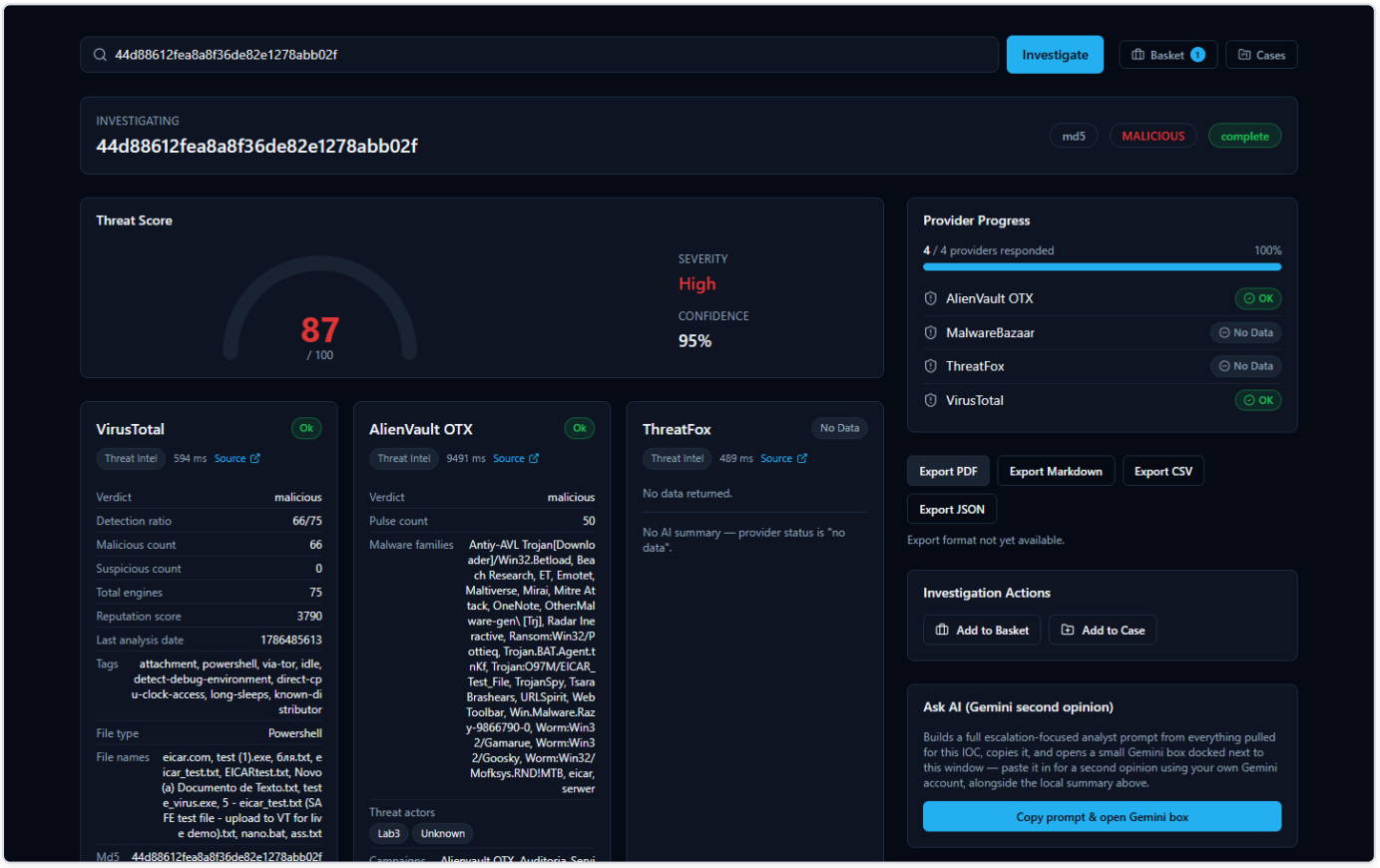

Figure 47 — The export sidebar on an investigation page, showing all four export buttons and the real in-product message displayed when an unavailable format is clicked.

Two of these — JSON and Markdown — work today and produce a real file. Two of them — PDF and CSV — are visible in the interface but not yet built. This section covers both honestly, because that's exactly what you'll see if you click each one.

What works today: Export JSON and Export Markdown

Clicking **Export JSON** or **Export Markdown** downloads a real file to your browser's Downloads folder, built from the same investigation you're looking at on screen. Both formats carry the same underlying content, just structured for different purposes:

- **Export JSON** is the machine-readable version — the same assessment fields shown on screen, structured for a script, a SIEM (Security Information and Event Management system — the log-aggregation tool many SOCs use as their central alerting hub) ingestion pipeline, or another tool that expects structured data rather than prose.
- **Export Markdown** is the human-readable version — the same content laid out as readable text, suitable for pasting straight into a ticket, a wiki page, or an email.

Either export includes:

- The **verdict** and **risk score** the platform settled on for the IOC (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID that an analyst wants to investigate).
- The **executive and technical summary** — a plain-language explanation of what the IOC is and why it received the verdict it did, alongside the more detailed technical reasoning.
- A **supporting evidence** list — specific findings the AI pulled out of the per-provider results and correlation data to back up its verdict. This is a curated, AI-written excerpt list, not the full structured Evidence Ledger; for the complete per-provider findings with confidence scores on each item, use the Evidence Ledger inside the app itself (covered elsewhere in this document) — it isn't part of either export.
- Any **MITRE ATT&CK mappings** — techniques the evidence supports, when applicable.
- The **recommended actions** — practical next steps suggested for an analyst handling this IOC.
- Any **detection rules** the platform generated for that IOC, when the IOC type and available evidence supported generating one.

In other words, whichever format you pick, you get the verdict, risk score, and the AI's full written reasoning in one file — enough to hand this investigation off to a colleague, attach it to a ticket, or drop it into another tool without re-typing anything. If your use case specifically needs the full per-item Evidence Ledger with confidence scores rather than the AI's supporting-evidence excerpts, view that in the app itself — it isn't part of either export.

What's not available yet: Export PDF and Export CSV

Export PDF and **Export CSV** appear as buttons in the same sidebar, but neither is built yet. Clicking either one shows the real message the product displays: **"Export format not yet available."** No file downloads, and nothing crashes — the platform tells you plainly that this format isn't ready rather than pretending it worked or failing silently.

This is a known, documented scope boundary, not a defect: the two working formats (JSON and Markdown) already cover both the machine-readable and human-readable cases most analysts need, and PDF/CSV support is simply not built out yet. If your workflow depends specifically on a PDF for a report template or a CSV for spreadsheet import, use Export Markdown or Export JSON as the source material for now and convert it with whatever tool your workflow already uses.

A note on the AI-written portions of an export

Nearly everything in either export — the executive summary, technical summary, supporting-evidence excerpts, MITRE ATT&CK mappings, recommended actions, and any detection rules — comes from a single consolidated AI call, not hand-typed by a human analyst, and it is a separate thing from the platform's own non-AI-generated **Evidence Ledger**. Like any AI-generated analysis, an export is a starting point for your judgment, not a substitute for it. The platform's own Evidence Ledger and verdict-explanation tools (covered elsewhere in this document) are how you check an AI-written claim against the real, underlying evidence before you rely on it or pass it along in an exported report. Providers genuinely disagree on real IOCs — the 8.8.8.8 (Google Public DNS) investigation shown elsewhere in this document is a good real example, where one provider flagged it malicious and two others called it clean — and the platform's summaries are written to reflect that disagreement rather than paper over it, but it's still worth reading the underlying evidence yourself before treating an exported verdict as final.

A Real-World SOC Workflow

Every feature described so far is easier to trust once you see it used the way a SOC (Security Operations Center) analyst actually works: not by opening one tool at a time, but by following a single lead wherever it goes. The walkthrough below is not a hypothetical — it is the exact sequence an analyst followed in the platform, start to finish, while triaging a routine batch of indicators.

The lead: a CVE turns up in a routine sweep

It starts the way most real investigations do — unglamorously. The analyst is working through a list of IOCs (Indicators of Compromise: IP addresses, domains, hashes, and CVE IDs pulled from recent alerts and vendor advisories) that need to be checked against threat intelligence before the shift ends. Most will turn out to be noise. One entry in the list is `CVE-2021-44228` — a CVE (Common Vulnerabilities and Exposures) identifier, the standard way the industry names a specific, publicly tracked software vulnerability. The analyst doesn't recognize the number offhand, so they submit it to the platform the same way they would any other IOC.

The results come back aggregated from multiple authoritative sources in one view. This is the payoff of an IOC lookup platform in the first place: instead of opening a CISA advisory in one tab, a NIST database in another, and a search engine in a third, the analyst gets all of it side by side, in the time it takes the providers to respond.

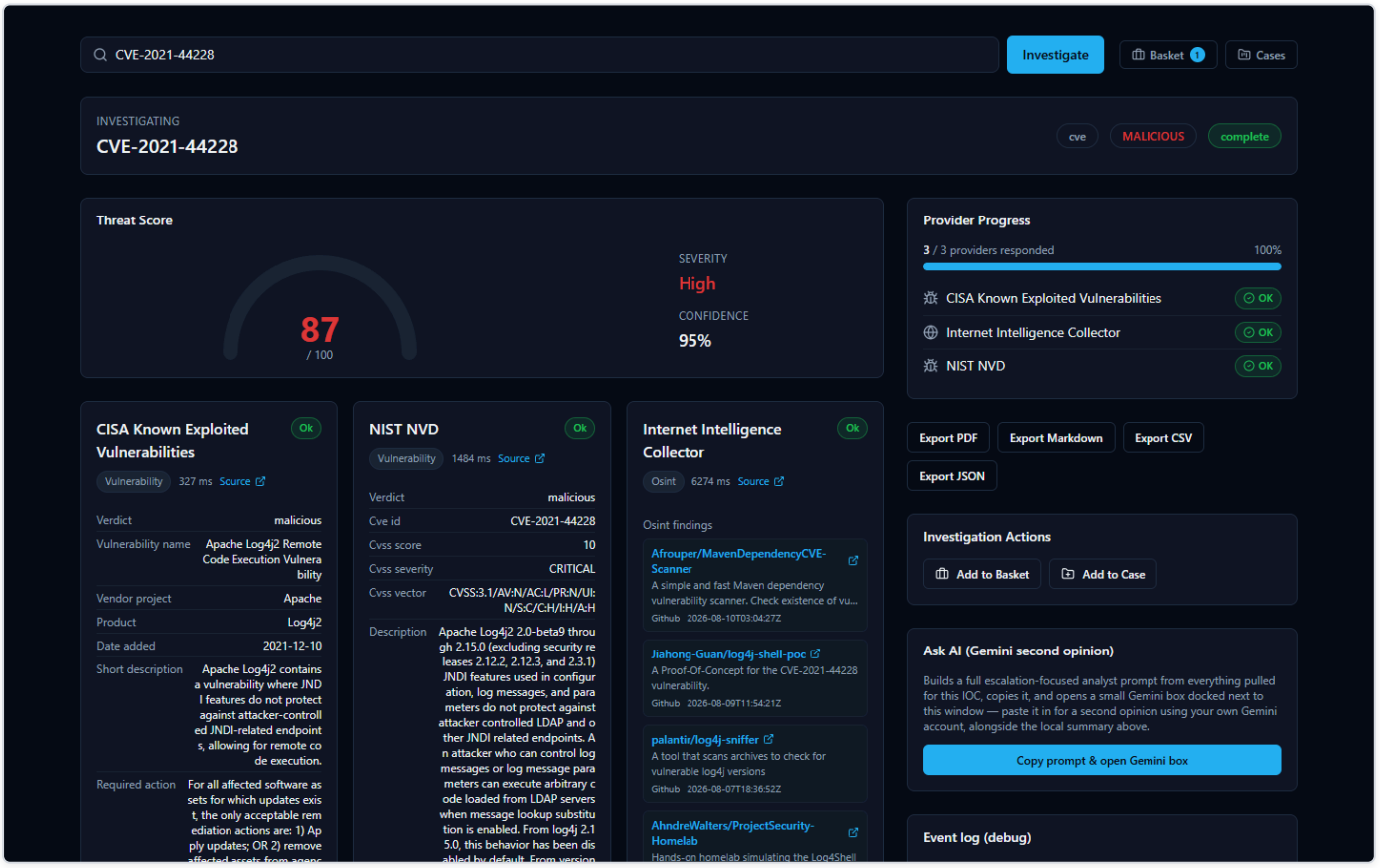

Figure 48 — Looking up CVE-2021-44228 returns a CISA Known Exploited Vulnerabilities card confirming active exploitation with real CVSS, vendor, and product data; a NIST NVD card showing a CVSS score of 10 and CRITICAL severity with the full vulnerability description; and an Internet Intelligence Collector card of live, freshly-fetched OSINT (Open Source Intelligence) findings from public GitHub repositories discussing this exact CVE.

The CISA (Cybersecurity and Infrastructure Security Agency) Known Exploited Vulnerabilities card is the first thing that raises the stakes: CISA doesn't list a CVE unless it has confirmed real-world exploitation, and here the verdict is unambiguous — malicious, with the vendor, affected product, and remediation guidance pulled directly from that catalog. The NIST NVD (National Vulnerability Database) card independently corroborates the severity: a CVSS (Common Vulnerability Scoring System) score of 10 out of 10, rated CRITICAL, with the full official vulnerability description. Underneath both, the Internet Intelligence Collector — the platform's own OSINT crawler — has pulled in live findings from GitHub repositories actively discussing this CVE, showing the analyst that it isn't just historically significant, it's still a live topic. The analyst recognizes the number now: this is Log4Shell, the Log4j remote-code-execution vulnerability. Two independent, authoritative government-maintained sources agreeing on maximum severity is exactly the kind of signal that turns a routine sweep item into a priority.

Scrolling further confirms there's substance behind the verdict, not just a score:

Figure 49 — The full Log4Shell investigation page adds a Final Assessment, a Relationship Graph, a MITRE ATT&CK Matrix, an Evidence Ledger, and a generated Detection Rule titled "Direct Class Unloader EJP Proxy Vulnerability" that gives the analyst a concrete starting draft to bring to a detection engineering team.

The Detection Rules panel is a concrete, useful artifact: a real generated rule that gives the analyst a working draft to review and adapt rather than having to write one from scratch. As with any AI-assisted output in the platform, this rule and the Final Assessment above it are analytical assistance, not a verdict to take on faith — every claim on this page can be traced back to a real provider record or correlation edge in the Evidence Ledger below, so the analyst (or a teammate reviewing the case later) can check exactly where a

specific statement came from rather than just trusting the summary text. It's a head start for the detection engineering team, not a substitute for their review before it ships to production.

Escalating: opening a case

A CVSS 10, confirmed-exploited, still-actively-discussed vulnerability isn't a one-off lookup to close and forget — it needs to be tracked, assigned a severity, and worked as its own investigation. The analyst opens the platform's Cases feature and creates a new case.

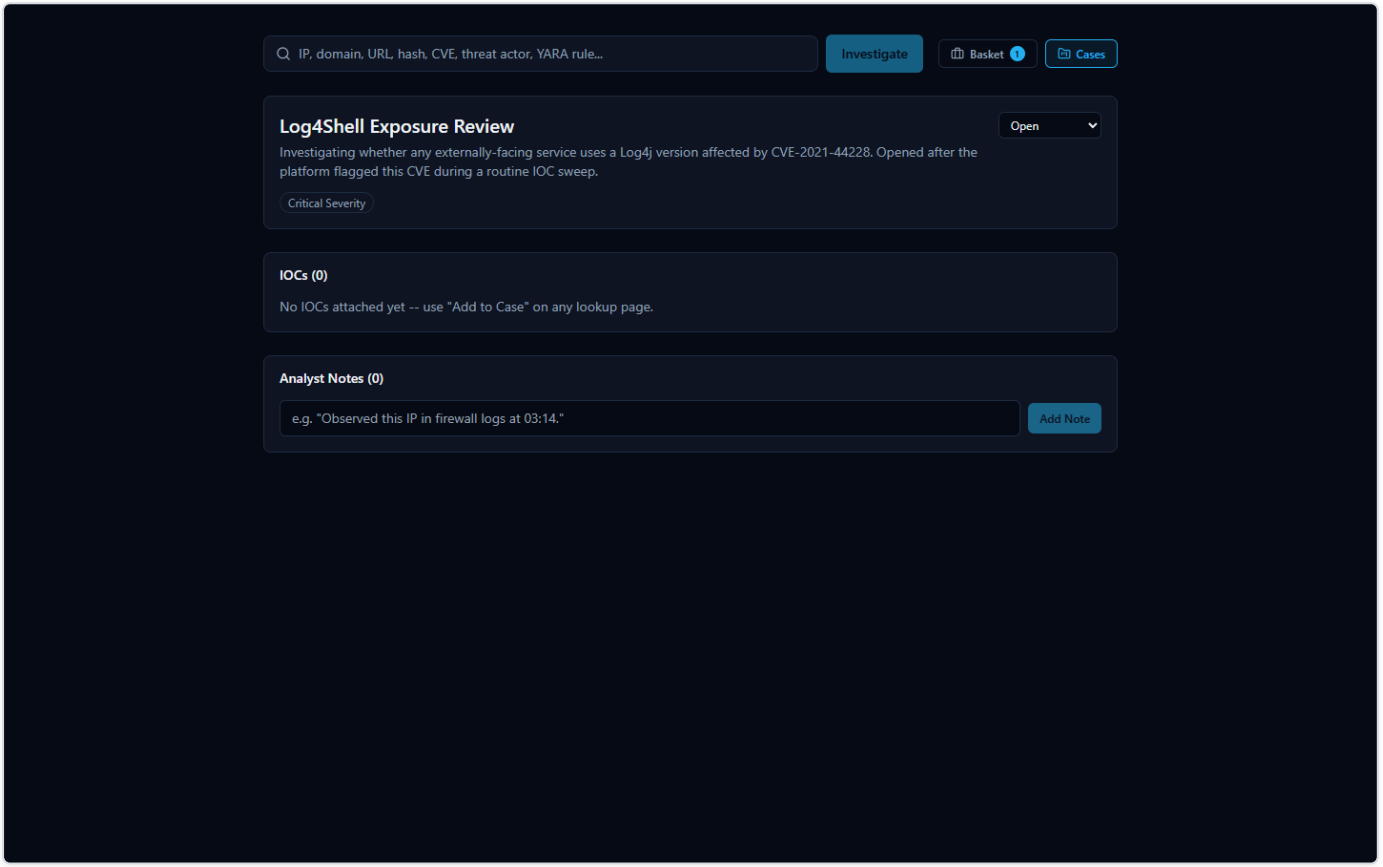

Figure 50 — A new case titled "Log4Shell Exposure Review" is created with Critical severity and Open status, giving the investigation a home separate from the one-off IOC lookup that started it.

Naming the case, setting its severity to Critical, and leaving it Open turns a single search result into a tracked piece of work — something that can be picked up again tomorrow, handed to another analyst, or referenced from a ticketing system, instead of living only in the analyst's memory or a screenshot.

Attaching evidence and recording the next step

From the CVE-2021-44228 investigation page, the analyst attaches the CVE directly to the new case using the "Add to Case" button, then records what they're doing about it as an analyst note before moving on.

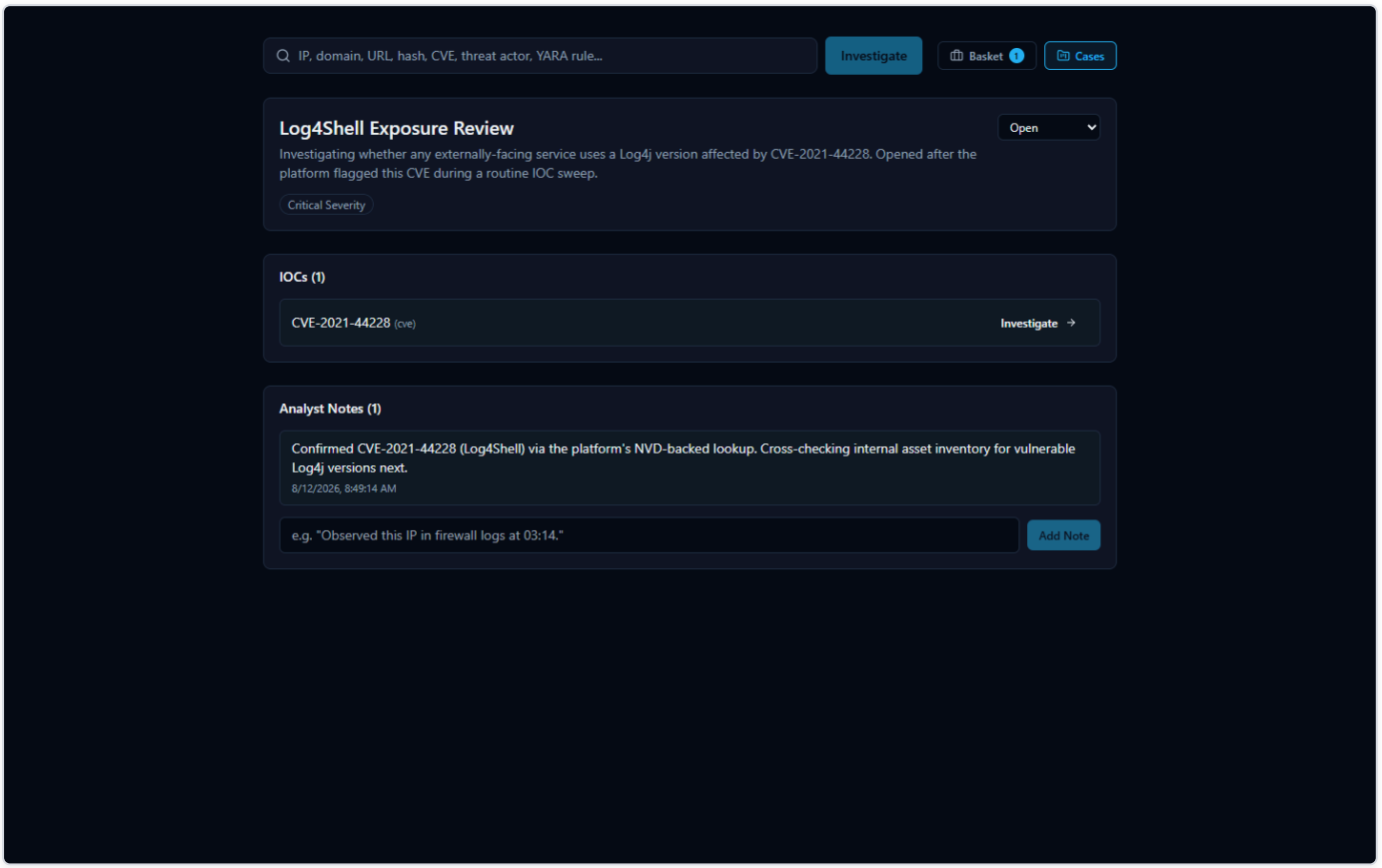

Figure 51 — The "Log4Shell Exposure Review" case now shows one attached IOC, CVE-2021-44228, and one analyst note documenting the next investigative step.

The note reads, in full: "*Confirmed CVE-2021-44228 (Log4Shell) via the platform's NVD-backed lookup. Cross-checking internal asset inventory for vulnerable Log4j versions next.*" This is a small thing, but it is the difference between institutional knowledge and a Slack message that scrolls away: the case now carries a permanent record of what was confirmed, from where, and what happens next — reviewable by a teammate, a manager, or the analyst's own future self without anyone having to reconstruct it from memory.

Handing it off

The investigation still needs to leave the platform and land in wherever the team tracks incident tickets. The analyst exports the findings as Markdown, which attaches cleanly to a ticket or incident-report document as readable, formatted text. JSON export is also available for anyone downstream who wants to parse the findings programmatically rather than read them. (PDF and CSV export are not yet available in this release — the export panel says so plainly rather than failing silently, so nobody wastes time waiting on an export that isn't coming.)

Why this is faster than doing it by hand

The same investigation done manually would mean separately visiting CISA's KEV catalog, querying NVD, running a search for current OSINT chatter, writing a detection rule from scratch, and then remembering to document all of it somewhere durable — five or six separate tools and a self-imposed discipline to write anything down at all. Here, one search surfaced every authoritative source at once, the platform's own evidence trail let the analyst verify the severity claim instead of just trusting a score, and opening a case took the finding from "something I noticed" to "something the team can act on" in the same sitting. Search, correlate, verify, document, escalate

— collapsing those five steps into one continuous flow is what keeps a real Log4Shell-caliber finding from getting lost in a routine sweep.

System Health

A simple yes/no answer: is the platform up?

Before trusting any investigation result, it helps to know the backend itself is actually running and reachable. The platform exposes a small, unauthenticated health-check address for exactly this purpose. Opening it directly in a browser returns a short, raw JSON reply rather than a page of graphics -- it is meant to be read by a person in two seconds, or by a monitoring script in milliseconds.

Figure 52 — The backend's own health-check address, opened directly in a browser, returns a short raw JSON reply confirming the service is up and reachable.

The reply is deliberately minimal:

```
{"status":"ok","service":"HORIZON GRID"}
```

There is no login, no dashboard, and no extra detail here on purpose. This endpoint exists to answer one question only -- "is the backend up right now?" -- as unambiguously as possible. If the backend is down, still starting up, or unreachable, this address simply fails to load instead of returning the reply above; that failure to load is itself the signal that something needs attention.

Checking health without touching Docker

The platform's actual runtime is a set of Docker containers, but an administrator running the Windows-installed version never needs to know that, let alone type a Docker command, just to check on it day to day. The Windows installer adds a Start Menu group that includes a **Service Status** shortcut and a **Diagnostics** shortcut, both built specifically so a routine health check is a single click rather than a terminal session.

- **Service Status** reports on whether the platform's services are up and healthy.
- **Diagnostics** is the equivalent shortcut for digging deeper when something looks wrong, without needing to know the underlying container commands.

Between these Start Menu shortcuts and the health-check address described above, an administrator has two independent, beginner-friendly ways to confirm the platform is alive -- one click, or one URL.

Provider Health

A dedicated page for "is every provider actually working right now?"

The single-page health check above answers "is the backend up." A separate, more detailed question -- "is every individual threat-intelligence provider actually working right now, or just configured" -- gets its own dedicated page. It's reachable from the new **Dashboard** entry in the global navigation, then **"View full Provider Health status"** on the Executive Dashboard's provider-health widget (or directly via the **Provider Health** link under the Operations group of the same navigation).

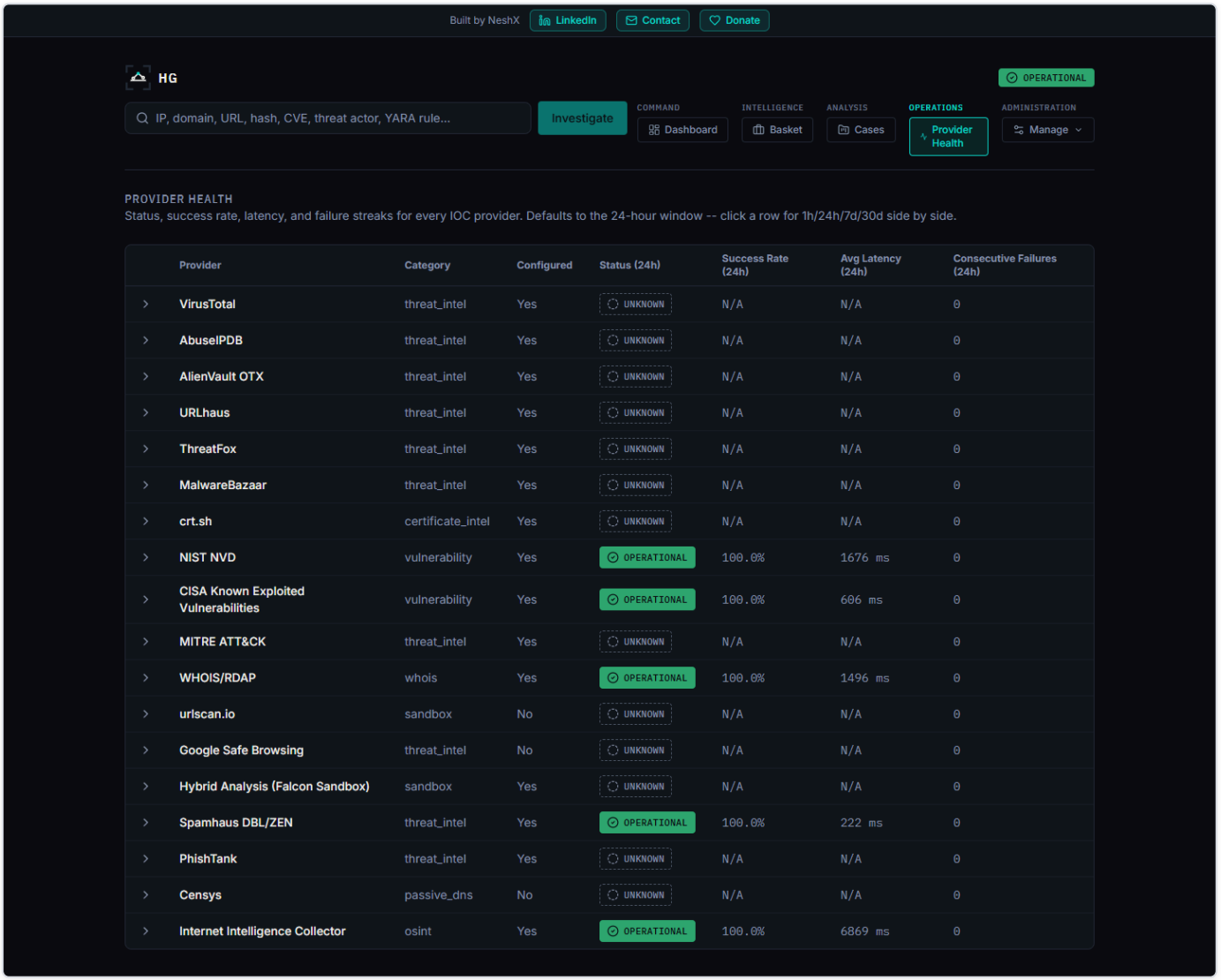

Figure 53 — The Provider Health page shows real per-provider status across four time windows, with healthy/degraded/down/unknown always paired with a distinct icon and label, not color alone.

Unlike the old provider cards shown mid-investigation (still covered below), this page is not scoped to one lookup -- it is a real, database-backed table of every registered provider's status, built from the actual outcome of every real call that provider has made

across the whole platform. Each row reports status, success rate, average latency, and consecutive-failure count, and reports all of that across four separate rolling windows -- 1 hour, 24 hours, 7 days, and 30 days -- so a brand-new problem and a month-long pattern are both visible without cross-referencing anything else; clicking a row expands all four windows side by side.

Status is always one of four values -- Healthy, Degraded, Down, or Unknown -- and each is always paired with its own distinct color, icon, and text label, never color alone, so the information doesn't depend on being able to distinguish colors.

A guarantee worth stating explicitly: a provider that was never actually exercised in a given window shows "Unknown," never "Healthy." Silence is not evidence of health -- a provider nobody has called in the last hour has no success rate to report for that hour, and reporting one anyway (even a reassuring one) would be reporting something the platform doesn't actually know.

A related guarantee, found and fixed as a real bug during this feature's own testing: a provider that correctly reports "nothing found" for a given indicator -- which is what most real-world lookups against most providers actually return, since no single provider's dataset covers every indicator -- counts as a healthy, successful outcome, not a failure. An earlier version of this page's health calculation miscounted that correct "no data" response as a failed attempt, which meant a provider working perfectly could have shown up as "Degraded" or "Down" simply for doing its job honestly. This was caught and fixed before release: a real provider (OTX) that had been reporting "Degraded" at 64% success under the flawed logic correctly reported "Healthy" at 100% once "nothing found" was counted as the successful outcome it actually is.

Troubleshooting

The list below is not a set of hypothetical problems -- every item was either deliberately induced and observed, or organically encountered, during the platform's own end-to-end quality-assurance pass, then either fixed or documented as an honest, known limitation. It is written for the first time something looks unexpected during an investigation of an IOC (Indicator of Compromise -- a piece of evidence such as an IP address, domain, URL, file hash, or CVE ID -- Common Vulnerabilities and Exposures, a public catalog number for a known software vulnerability -- that a security analyst wants to look up).

The AI summary says "Unknown" and every risk score reads 0

What you'll see: instead of a verdict like "malicious" or "benign," the final assessment reads "Unknown," the risk and confidence scores are both 0, and a note explains that AI summarization was unavailable.

What it means: the AI model the platform uses to summarize and correlate results (by default, a local model) could not be reached. Rather than guess and present a verdict it can't actually support, the platform is designed to report that failure honestly. During testing, this exact scenario was deliberately created by making the AI backend unreachable: every AI-dependent field correctly reported the failure instead of pretending to succeed, and every provider's own raw data (VirusTotal, AbuseIPDB, WHOIS, and so on) remained untouched and just as trustworthy as always -- only the AI-generated synthesis was affected. Restoring the connection to the AI backend and restarting immediately brought full AI functionality back on the next investigation.

What to do: re-run the Configuration shortcut to confirm the AI backend setting, verify the AI model/service it points to is reachable, and try the investigation again.

A related point, even when the AI is healthy: an "Unknown" verdict from an unreachable AI is a different situation from providers genuinely disagreeing with each other -- which the AI is designed to surface, not hide. Elsewhere in this guide, the IOC `8.8.8.8` is a real example where one provider (Spamhaus) flagged it malicious while two others (AbuseIPDB and VirusTotal) called it clean, and the AI's own summary spelled out that disagreement rather than silently picking a side. In every case, the AI's output is analytical assistance, not the final word -- every claim it makes can be checked against the underlying facts it was built from in the investigation's Evidence Ledger ("Show receipts"), rather than accepted on trust alone.

The page shows a real error, or an investigation won't load at all

What it means: this is consistent with a database outage. When the platform's database was deliberately stopped mid-request during testing, the result was a clear, fully logged error -- not a silent failure, a crash, or corrupted data. Restarting the database restored full functionality immediately, and previously saved cases and Basket entries were confirmed completely intact afterward.

What to do: use the Restart Platform shortcut (or Diagnostics, if you want more detail first), then try again. Your previously saved data is not at risk from this kind of outage.

A repeat lookup on the same IOC comes back noticeably faster

This is expected behavior, not a bug. Each provider's result is cached for up to an hour after it's first fetched, so looking up the same IOC again within that hour reuses the already-fetched provider data instead of re-querying every source from scratch. After that hour passes, the next lookup fetches fresh data again.

One provider card shows "Not Configured," "Error," or "Rate Limited" while others show real results

This is normal and expected, not a malfunction. Each provider's status is reported honestly and independently: a provider you haven't set up a key for shows "Not Configured," a provider that genuinely failed to respond shows "Error," and a provider that has temporarily hit its own free-tier limit shows "Rate Limited." None of these ever get mislabeled as one another, and -- just as importantly -- one provider having a problem never blocks the rest of the investigation from completing.

You closed the browser tab or lost your connection partway through an investigation

Provider results that had already come back before the disconnect are saved, not lost. Revisiting the investigation afterward shows every provider result that had completed up to that point, even though the investigation itself may show as failed if it didn't finish.

Clicking "Export PDF" or "Export CSV" doesn't produce a file

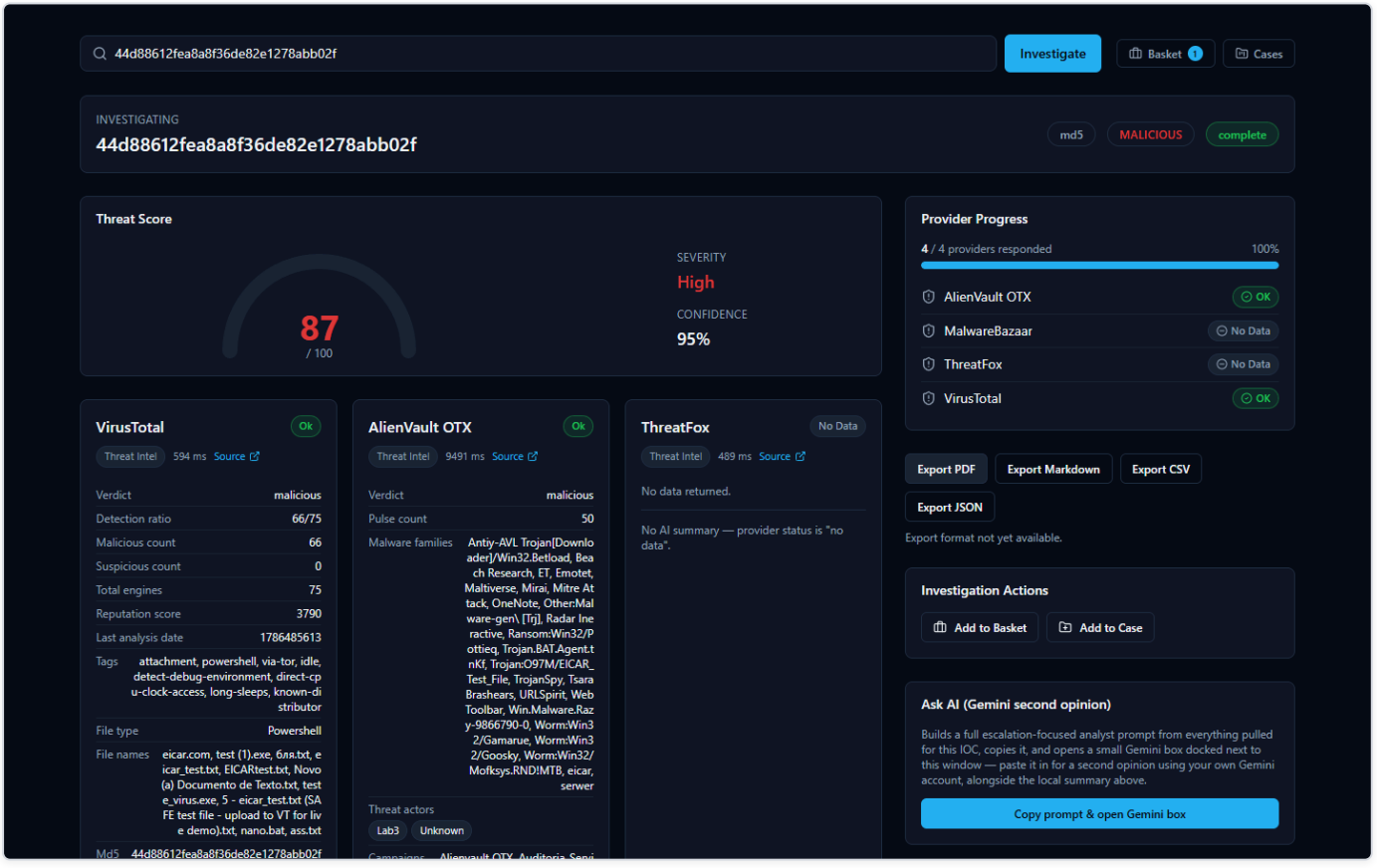

Figure 54 — The Export menu offers PDF, Markdown, CSV, and JSON, but clicking Export PDF or Export CSV shows the honest message "Export format not yet available" instead of producing a file.

This is a known, honest limitation, not a bug to work around: PDF and CSV export are not yet available in this release. Rather than fail silently or crash, the platform tells you so directly with the message shown above. **Export JSON and Export Markdown both work today** and are the way to get an investigation's results out of the platform in the meantime.

You typed something and got "Could not determine IOC type"

This means the value entered didn't match any of the IOC formats the platform recognizes (IP address, domain, URL, file hash, CVE ID, and others). This is a deliberate, clean rejection with a clear message, not a crash -- double-check the value for typos and try again.

Security and Data Handling

This section explains, in plain language, how HORIZON GRID handles your sign-in, your provider credentials, and — most importantly — the actual indicator data you investigate. It is meant to be read by anyone using the tool, not just engineers. A far more detailed, code-level Security Architecture appendix exists later in this document for readers who want the full technical picture; this section is its beginner-friendly companion.

Signing In, and Who Becomes the Administrator

The platform is a normal sign-in-protected web application: you register an account with an email and password, then sign in from the same page each time.

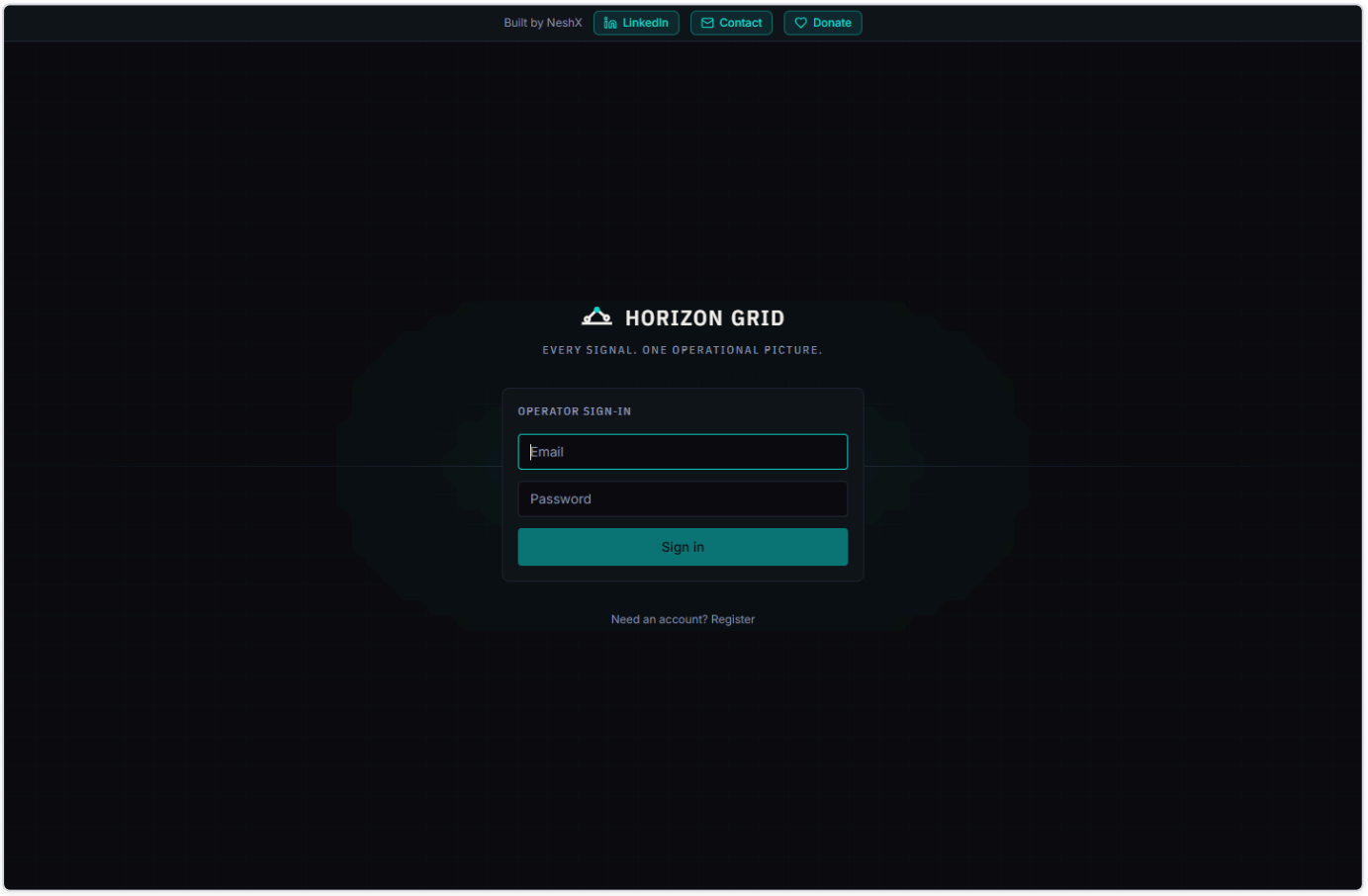

Figure 55 — The web app's Sign in page, where an existing user enters their email and password to access the platform.

A few things worth knowing about how this works:

- **Your password is never stored in plain text.** The platform keeps only a one-way scrambled (hashed) version of it, so nobody — not even someone with direct database access — can read your actual password back out.
- **Signing in issues a temporary digital pass, not a permanent one.** After you sign in, the platform gives your browser a short-lived access token that expires automatically after about 30 minutes, plus a longer-lived one (good for about 7 days) that quietly

renews your session in the background so you aren't forced to re-enter your password constantly.

- **The very first account created on a fresh install automatically becomes the administrator.** There is no separate "make someone an admin" step to remember. When the Setup Wizard runs during installation, the account you create on its Administrator Account page is that very first account — and because it's first, the platform automatically grants it administrator-level access. Once that administrator exists, the public sign-up page closes itself: anyone who tries to register through it afterward is turned away with a message pointing them at an administrator instead. Every account after the first one has to be created by an administrator from the Administration page (see below), who chooses its role — analyst or viewer, or another administrator — up front.
- **Administrators manage every other account from an Administration page**, not by editing anything by hand. Once signed in as an administrator, an "Administration" link appears in the top navigation (it's invisible to non-administrators). From there you can create a new account with a specific role from the start, promote or demote an existing analyst or viewer, disable an account instantly (their access stops working on their very next click, even if they're mid-session), re-enable one, and reset anyone's password — which also signs that person out of every device they were already signed into, so they can't keep using the old password's session. Every one of these actions is recorded in the same Audit Log a provider-configuration change would be, so there's always a record of who did what and when — never including the actual password.
- **There is always at least one administrator, and the platform won't let you accidentally remove the last one.** Trying to disable the sole remaining administrator, or demote them to analyst/viewer, is refused with a clear error rather than silently locking everyone out of account management. An administrator also can't change their own role (another administrator has to do it) — this just prevents someone from accidentally locking themselves out of the very screen they're using.

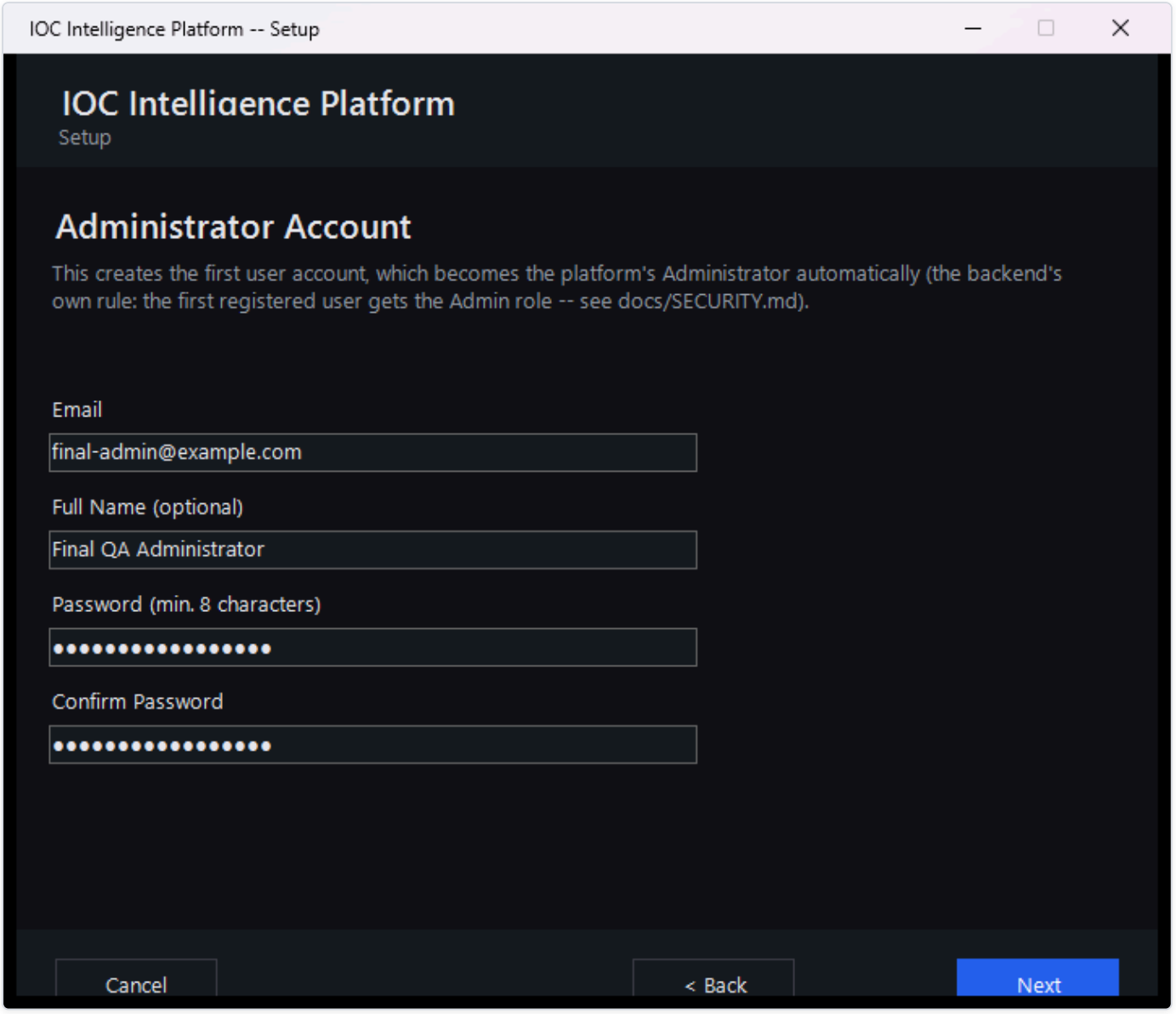

Figure 56 — The Setup Wizard's Administrator Account page during installation, where the first account on a new install is created — this account automatically becomes the administrator.

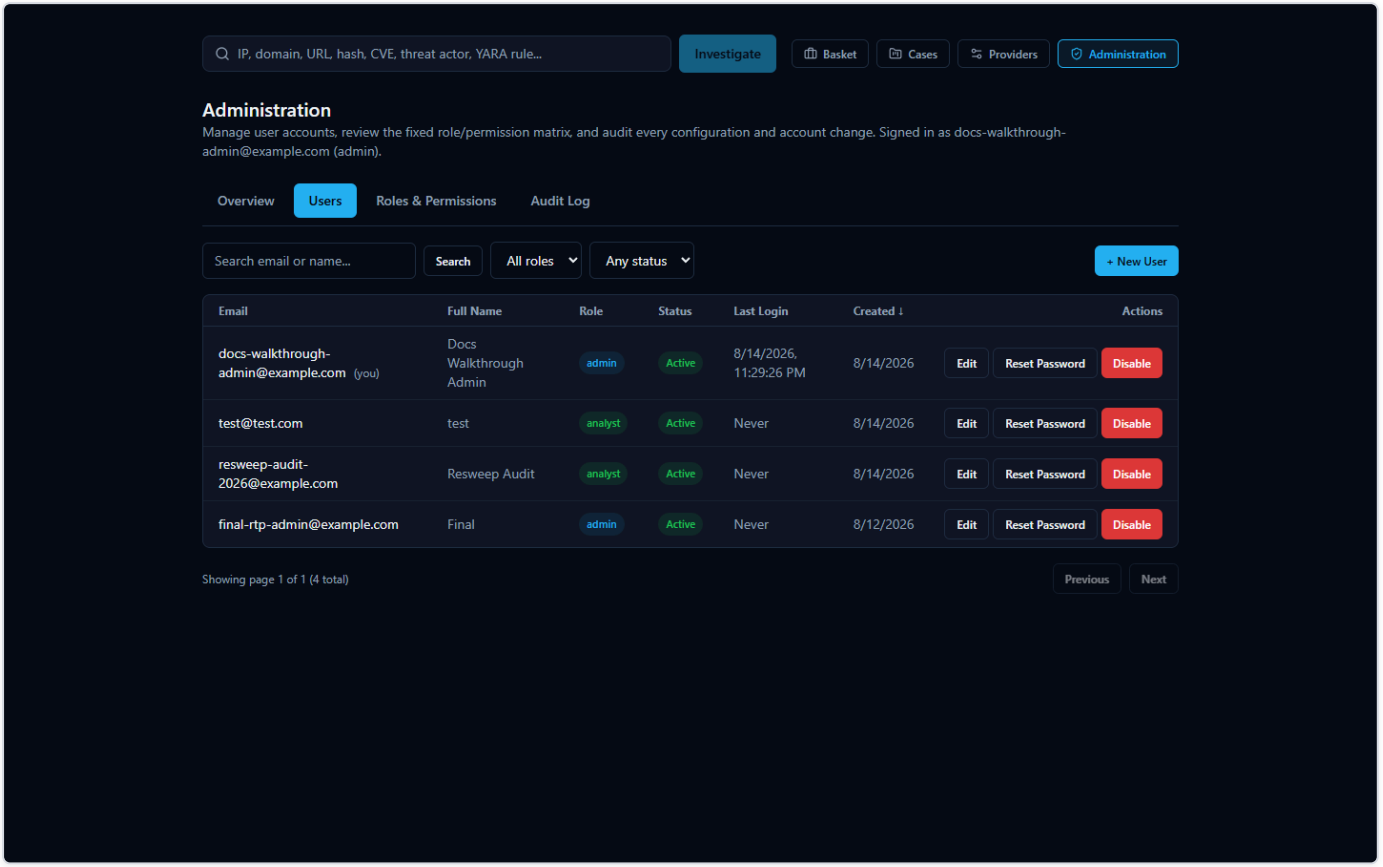

Figure 57 — The Administration page's Users tab: search, filters, and the create/edit/reset-password/enable-disable actions available to an administrator.

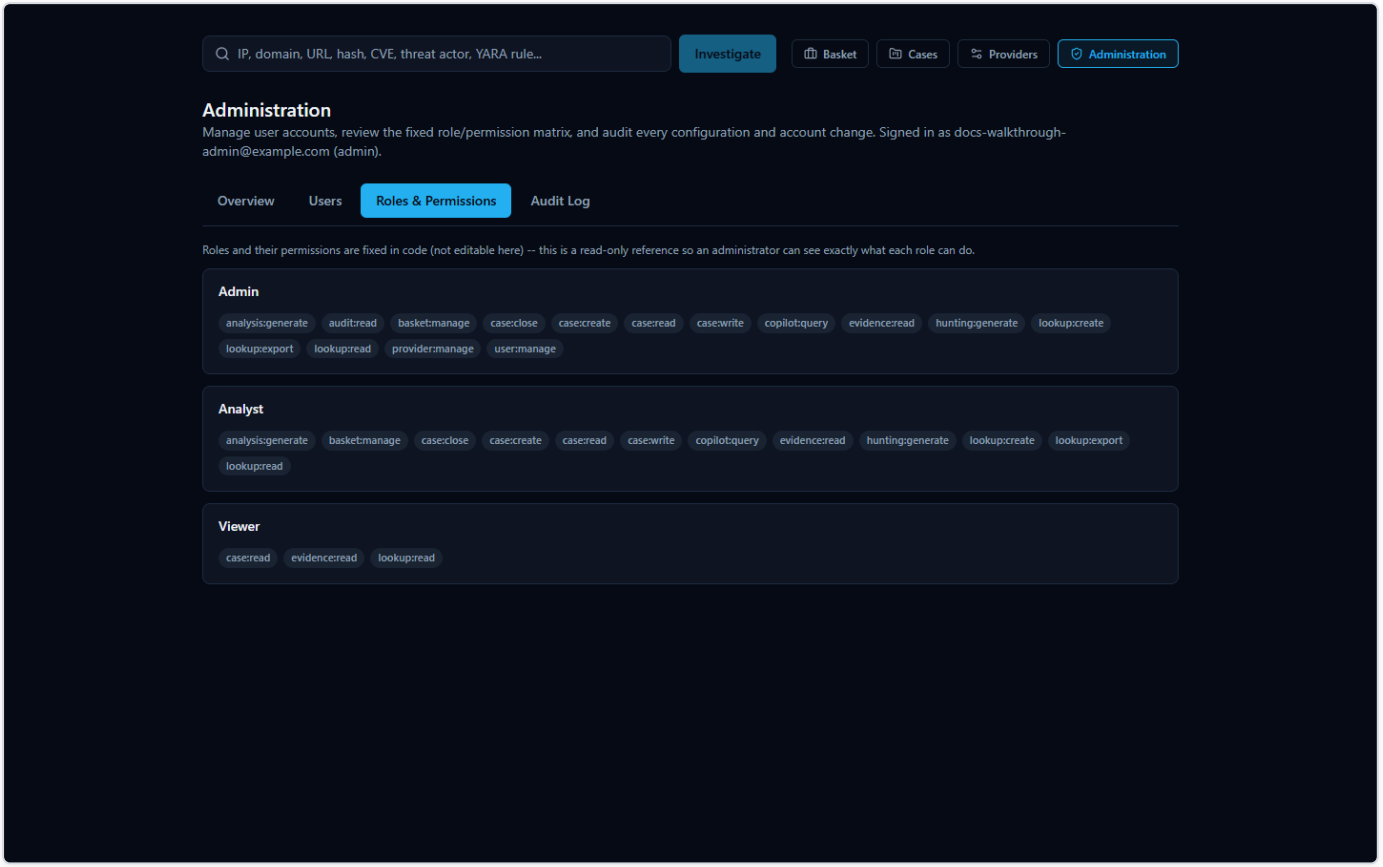

Figure 58 — The Administration page's Roles & Permissions tab: a read-only view of exactly what each role can do.

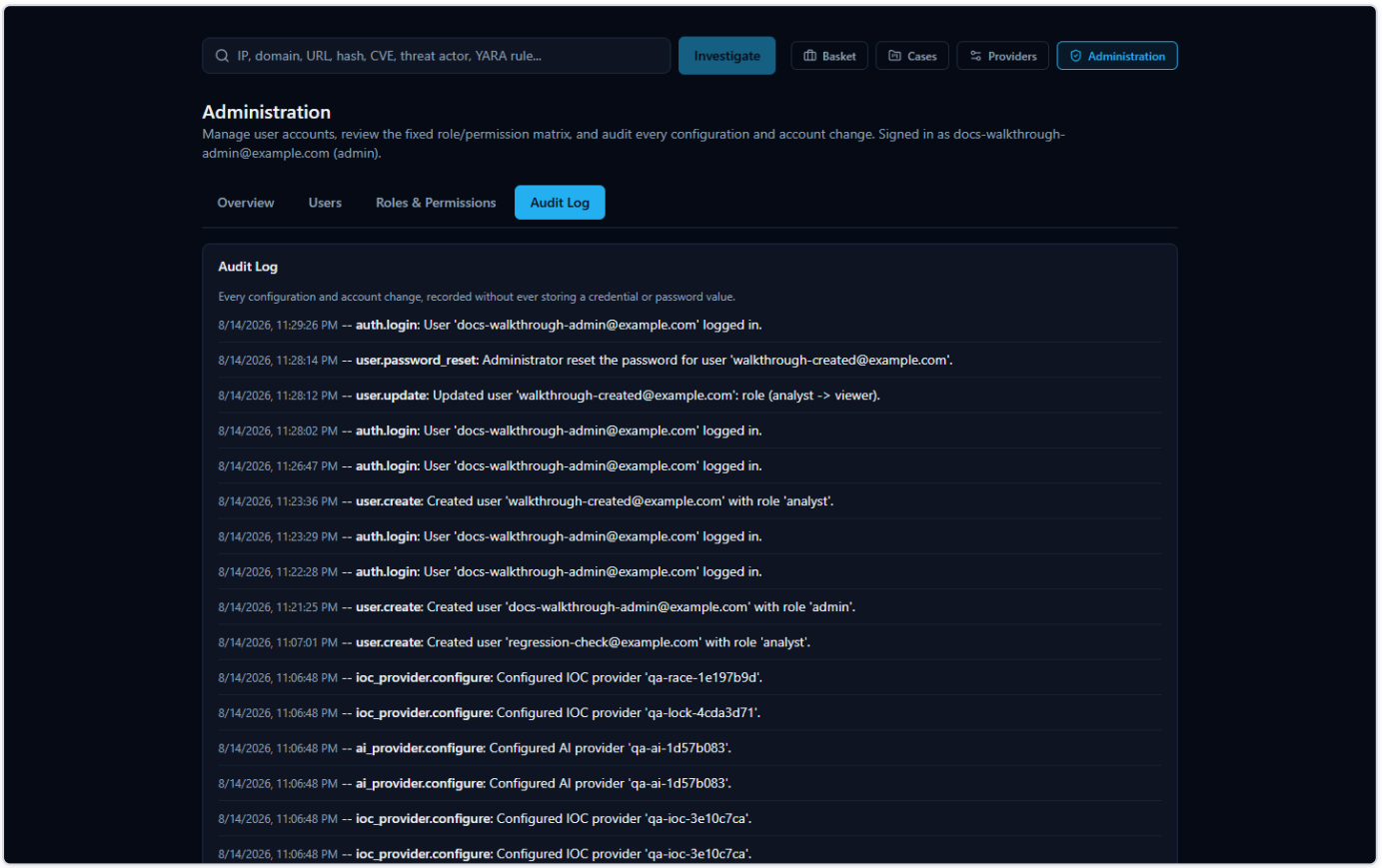

Figure 59 — The Administration page's Audit Log tab, showing account changes and provider-configuration changes together in one timeline.

Your Provider API Keys: Entered Once, Locked Away

To query the various third-party threat-intelligence services this platform aggregates (VirusTotal, AbuseIPDB, AlienVault OTX, and others), several of them require you to supply your own API key for that service. You provide these once, during installation, on the Setup Wizard's provider configuration page — not somewhere buried in a settings menu you have to hunt for.

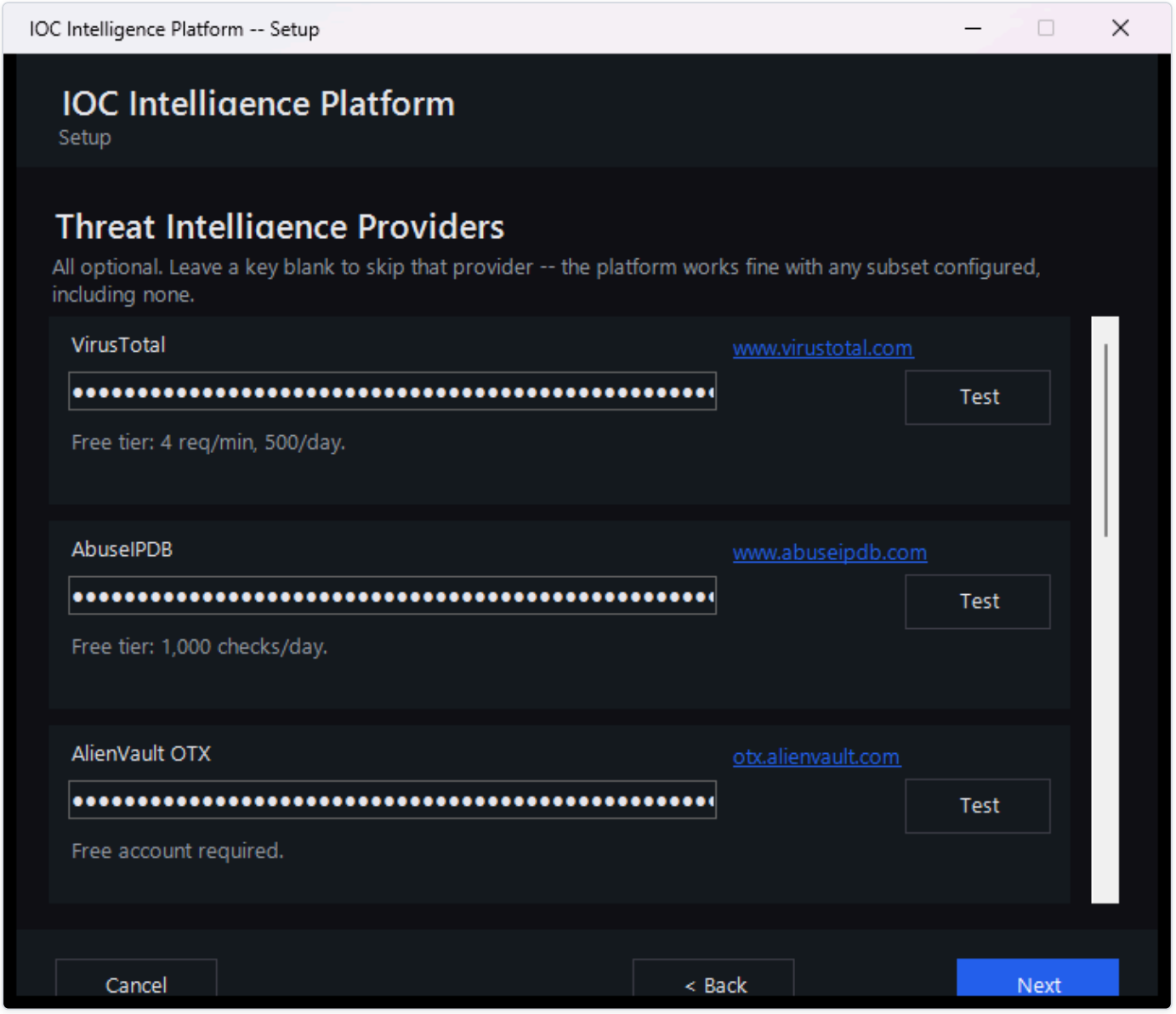

Figure 60 — The Setup Wizard's Threat Intelligence Providers page with several provider API keys entered; each key box shows masked dots on screen rather than the real characters.

A few practical points:

- **Keys are masked as you type them**, the same way a password field is — so nothing sensitive is visible on your screen by accident, including in screenshots or over someone's shoulder.
- **Keys are stored in a single configuration file, kept in a protected system folder on the machine running the platform** — not inside the web application's database, and not anywhere a regular file browse would stumble across. That folder is locked down at the operating-system level so that only Administrator-level Windows accounts (and the system itself) can open it; a standard, non-administrator user account on the same computer cannot read it.
- **You can revisit and change your keys later** using the "Configuration" shortcut the installer creates, which simply re-runs the same wizard against your existing setup.

What Happens to an IOC You Investigate

This is the part worth reading carefully, because it's central to how the platform works and it deserves an honest explanation rather than a reassuring gloss.

When you submit an IOC (Indicator of Compromise — an IP address, domain, URL, file hash, or CVE ID, i.e. a standardized identifier for a publicly known software vulnerability, that you want to investigate) for a lookup, the platform does not just search its own local database. For every configured provider that supports that type of indicator, the platform sends the literal value you entered — the actual IP address, the actual domain name, the actual file hash — out to that provider's own service, because that is the only way those services can check it against their own intelligence. This is true across essentially the whole provider lineup: threat-intelligence services like VirusTotal, AbuseIPDB, and AlienVault OTX; public lookups like WHOIS/RDAP and Certificate Transparency; and everything in between. If a provider isn't configured (no key entered, or not applicable to that indicator type), it simply isn't contacted and contributes nothing — but any provider that is configured and supports that IOC type will see the value you're investigating.

Practically, that means: **be mindful before investigating something genuinely sensitive.** If you're looking up an internal hostname, an internal IP address, or a file hash tied to an active, confidential incident, remember that value is being transmitted to whichever external providers you've enabled — the same way it would be if you pasted it into any of those providers' own websites. That's simply how multi-provider IOC lookups work, not a flaw specific to this platform, but it's a habit worth having regardless of which tool you're using.

AI Analysis: Local by Default, Cloud if You Choose

Once provider results come back, the platform uses an AI model to summarize individual provider findings and produce a consolidated assessment. You choose which AI model does that work during setup.

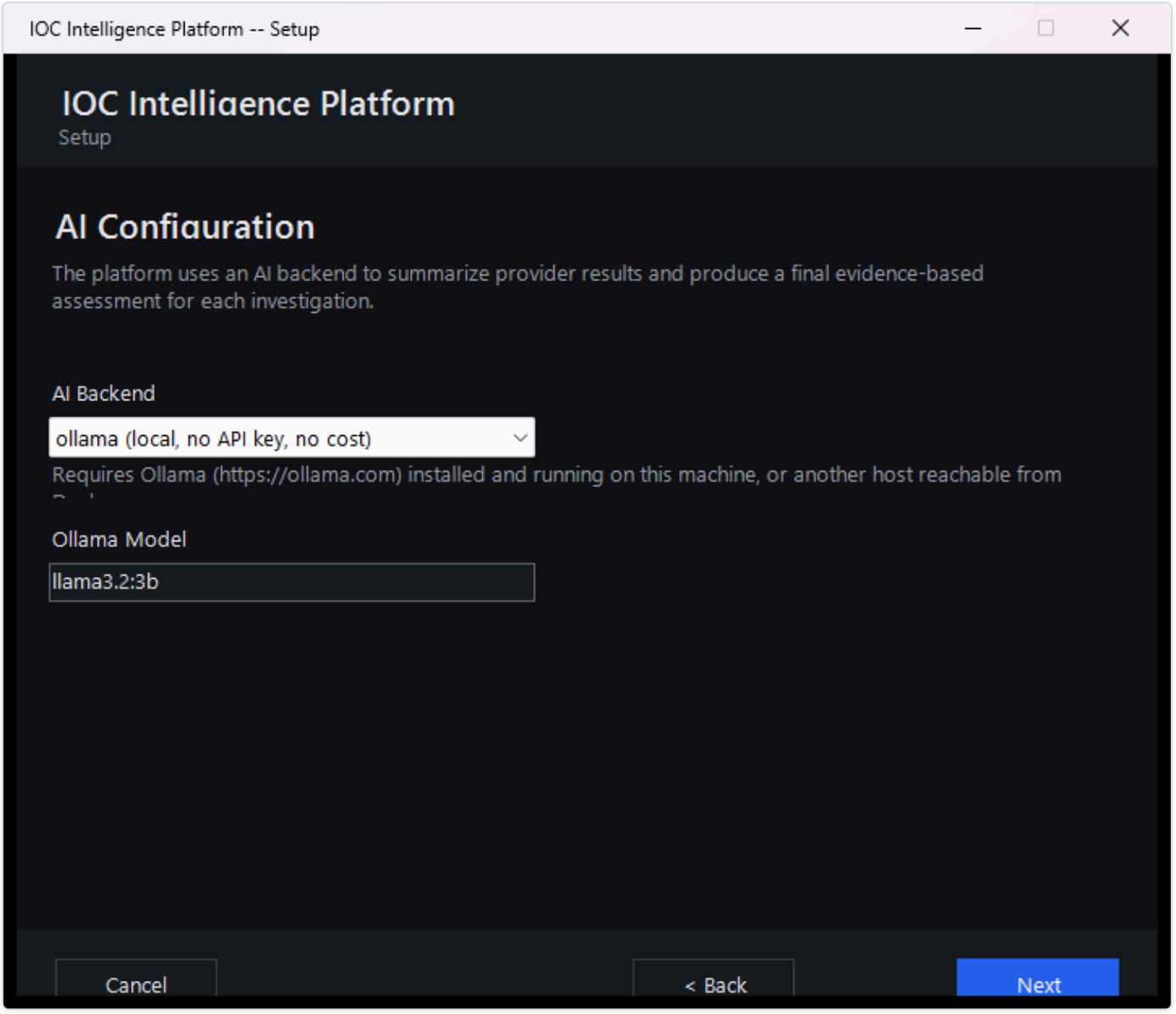

Figure 61 — The Setup Wizard's AI Configuration page, where you choose between a local AI model running on your own machine or a cloud AI provider.

- **By default, the AI runs entirely on a local model on the same machine as the rest of the platform.** In this mode, no analysis data leaves your machine at all — the model that reads the provider findings and writes the summaries and verdicts is running locally, alongside everything else.
- **Optionally, you can configure a cloud AI provider instead.** If you choose this, the provider findings gathered during your investigation are sent to that external AI service so it can generate the write-up. This is a deliberate trade-off some readers may want (for a more capable model) and others may not — it's your choice to make during setup, and it's separate from the provider-lookup data flow described above.

Either way, choosing the local model versus a cloud AI provider only affects where the *AI writing* happens — it does not change the fact, described above, that the IOC value itself is still sent to whichever threat-intelligence providers are configured, since that step happens before the AI is ever involved.

Checking the AI's Work Instead of Just Trusting It

AI-generated verdicts, summaries, and explanations in this platform are analytical assistance — a well-informed second opinion — not an unquestionable ground truth, and the platform itself is built around that idea rather than around asking you to simply trust a score.

A genuine, honest example of why this matters: a lookup on 8.8.8.8 (Google's well-known public DNS server) shows Spamhaus returning a "malicious" verdict, but for an unusual reason — its own card explains this is because open public resolvers aren't permitted to query Spamhaus directly, not because 8.8.8.8 is actually malicious. Meanwhile AbuseIPDB and VirusTotal both call it clean. The platform's own executive summary is upfront about this disagreement rather than quietly blending it into one number.

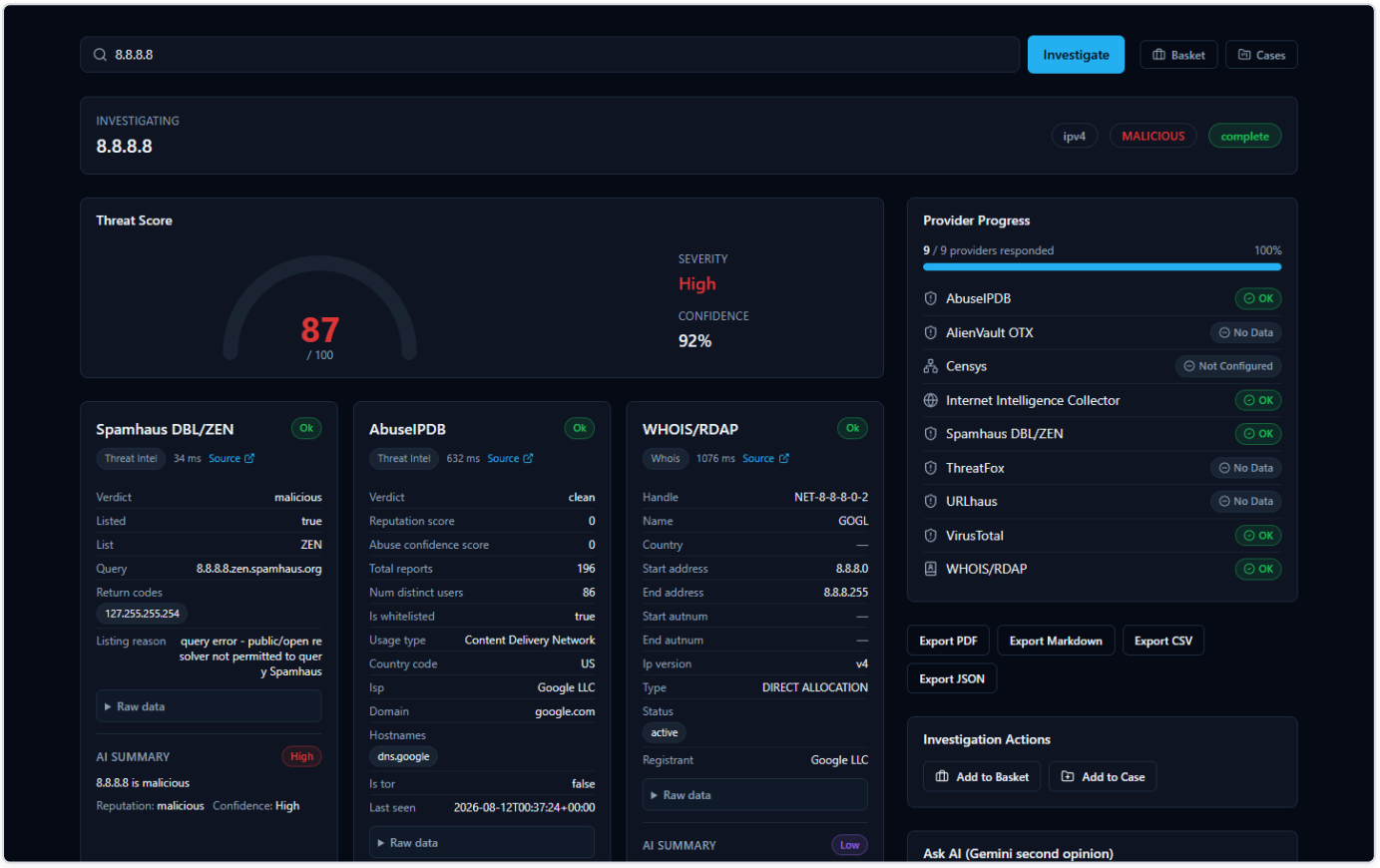

Figure 62 — An investigation of the IOC 8.8.8.8 (Google Public DNS) showing a high Threat Score alongside individual provider cards — including Spamhaus's "malicious" verdict, whose stated reason is a query error rather than an actual detection.

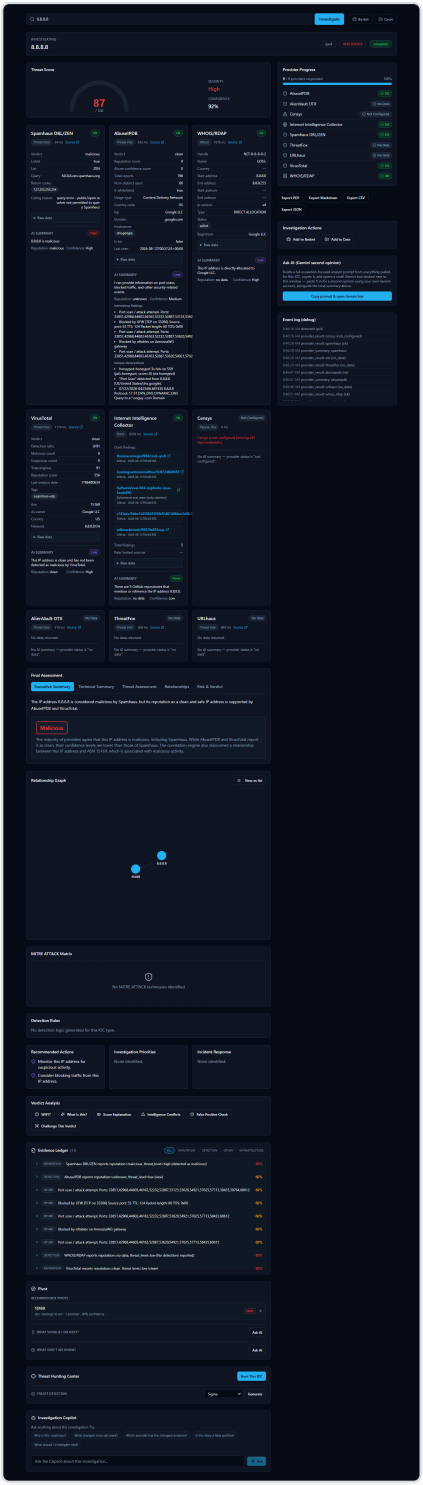

Figure 63 — The same 8.8.8.8 investigation's full results page, where the Executive Summary explicitly notes the IP is flagged malicious by Spamhaus while AbuseIPDB and VirusTotal support a clean reputation, alongside the Evidence Ledger and Verdict Analysis tabs an analyst can use to dig into why.

This is exactly the kind of AI verdict worth double-checking rather than accepting at face value — and the platform gives you the means to do that:

- The **Evidence Ledger** is a list of the underlying facts behind a verdict, each with its own confidence level. Unlike the AI's prose summaries, evidence-ledger entries are built directly and deterministically from the provider results themselves — they are not

written by the AI — so they exist specifically to give you something concrete to check the AI's claims against.

- The **Verdict Analysis tabs** (including "Why?", "What's this?", "Score Explanation", "Intelligence Conflicts", "False Positive Check", and "Challenge This Verdict") let you interrogate a verdict rather than just read it, and are designed to surface disagreements between providers — like the Spamhaus/AbuseIPDB/VirusTotal split above — instead of hiding them.
- Behind the scenes, the platform also cross-checks the AI's own output against the underlying data it was given: if the AI cites something (for example, a specific MITRE ATT&CK technique) that isn't actually backed by the real evidence collected for that IOC, the platform flags it rather than presenting it as confirmed.

The takeaway: treat AI-generated verdicts as a starting point for your own analysis, and use the Evidence Ledger and Verdict Analysis tools when a result seems surprising, high-stakes, or — as with 8.8.8.8 — genuinely contested between providers.

Frequently Asked Questions

Do I need every provider configured to use the platform? No. Every provider that needs an API key is optional -- the platform works with any subset configured, including none. Providers that need no key at all (WHOIS/RDAP, Spamhaus, crt.sh, NIST NVD, CISA KEV, MITRE ATT&CK, PhishTank, the Internet Intelligence Collector) always run regardless. An investigation with zero paid providers configured still produces a real result from whichever free sources apply to that IOC type.

Does the AI's verdict override the provider data? No, and the platform is deliberately built so you never have to take the AI's word for it. Every AI-written summary is grounded in the Evidence Ledger -- a deterministic, AI-independent record built directly from what the providers actually returned. If an AI conclusion doesn't match the underlying evidence, the Verdict Analysis tools ("Why?", "Challenge This Verdict", "False Positive Check") exist specifically so you can check.

What happens if a provider fails or isn't configured? The investigation continues with whichever providers succeeded. A failed, rate-limited, not-configured, or disabled provider is reported with a specific status (never a generic "error"), and the AI is explicitly told that provider's data is *unavailable* -- which is treated differently from "the provider ran and found nothing." A missing data point is never silently read as a clean bill of health.

Can I switch AI providers without losing my current investigation? Yes. Switching the active AI backend (via the "AI:" selector on the home page, or the AI Providers tab in Manage Providers) takes effect on the *next* investigation immediately -- no restart. For an investigation you've already run, the "AI Comparison" panel lets you re-run just the AI analysis step against a different backend, using the same already-collected evidence, without re-querying any provider.

Why does the same IOC sometimes get a slightly different AI write-up if I re-analyze it? AI-generated text is not perfectly deterministic between calls, and different backends/models can reasonably read the same evidence differently (which is exactly what the AI Comparison feature is for). The underlying facts -- provider data, correlation, Evidence Ledger -- do not change between reruns; only the AI's synthesis of them can.

Is my data sent to the third-party providers? Only the specific IOC value you investigate is sent to whichever providers are enabled and support that IOC type -- normal behavior for any threat-intelligence lookup tool. Nothing else about your environment is sent. See the Security & Data Handling chapter for the full picture.

Where do I go to change a provider's API key later? The "Providers" link in the workspace navigation opens Manage Providers, with separate tabs for AI Providers and IOC (threat-intelligence) Providers. Changes there take effect immediately -- no restart, no reinstall, no editing configuration files.

Best Practices

- **Start with the free providers, add paid ones as you need them.** Because every provider is optional, there's no reason to hold off using the platform while waiting on API key approvals -- WHOIS/RDAP, Spamhaus, and the OSINT collector already provide real signal with zero configuration.
- **Use Test Connection before relying on a newly-entered key.** It makes one real, minimal request to the provider and reports a genuine result (authenticated, invalid key, rate-limited, unavailable) -- never just "the field is non-empty."
- **Treat a disagreement between providers as a prompt to look closer, not a tie-breaker to ignore.** The correlation engine and the AI's threat assessment both explicitly track agreeing vs. disagreeing providers -- when they disagree, that is itself useful signal, not noise to average away.
- **Use the Evidence Ledger and "Why?" tools before escalating a verdict.** A high-severity verdict backed by thin evidence is exactly what these tools are designed to catch before it turns into an unnecessary incident response action.
- **Add an IOC to a Case as soon as it's part of a real investigation, not after.** Cases keep the analyst's notes, IOCs, and reasoning together as the investigation develops, rather than requiring reconstruction later.
- **If comparing AI backends, compare on the same evidence, not a fresh lookup.** The AI Comparison panel's whole point is holding the evidence constant so the only variable is which AI produced the read -- re-running a brand-new investigation instead would also change the provider data and defeat the comparison.

Quick Reference

I want to...	Where
Investigate an IOC	Home page search box, or the search bar on any page
See which AI is currently active	"AI:" selector, home page, next to the search box
Switch which AI analyzes the next investigation	Same "AI:" selector, or Manage Providers → AI Providers → Set Active
Configure or change a provider's API key	Manage Providers (workspace nav) → AI Providers or IOC Providers tab
Test a provider or AI key before trusting it	Expand its row in Manage Providers → Test Connection
Enable/disable a provider	Manage Providers → IOC Providers tab → Enable/Disable
Compare AI backends on one investigation	Completed investigation page → AI Comparison panel
See every configuration change ever made	Manage Providers → Audit Log tab
Check why a verdict was reached	Investigation page → Verdict Analysis → "Why?"
Check the platform is actually running	System Health page
Save an IOC for later without a full case	Add to Basket
Track a multi-IOC investigation with notes	Cases
Export an investigation's findings	Export menu (PDF / Markdown / CSV / JSON) on the investigation page

Technical Architecture Overview

HORIZON GRID is a self-hosted tool that looks up an **IOC** (Indicator of Compromise — e.g. an IP address, domain, URL, or file hash) against many third-party threat-intelligence sources at once, then uses an AI model to summarize and correlate the results. The backend's own self-description (`backend/app/main.py`) puts it this way:

> *"single-search IOC lookup across dozens of providers, correlated and summarized by a local Ollama model (or AWS Bedrock/Gemini/Anthropic)."*

This section describes what the platform is built from at the container/process level, not the request-level data flow (covered elsewhere in this document) or the AI/provider internals (also covered elsewhere).

Container Inventory

`docker-compose.yml` defines seven backing services — `postgres` , `redis` , `neo4j` , `opensearch` , `backend` , `celery_worker` , and `celery_beat` — plus the `frontend` container, for eight containers running the full stack.

Container	Image / stack	Role	Implementation status
`postgres`	`postgres:16-alpine`	System of record: lookups, provider results, AI summaries, correlation edges, evidence, cases, users.	Actively used — the only datastore actually written to during a lookup.
`redis`	`redis:7-alpine`	Provider-result cache (logical DB `/0`), per-user rate limiter, Celery message broker (`/1`) and Celery result backend (`/2`).	Actively used.
`neo4j`	`neo4j:5-community` + APOC plugin	Config field and container exist to "mirror" correlation edges into a graph store, per two module docstrings.	Provisioned only — no driver instantiation, Cypher query, or read/write call exists anywhere in the backend code. Not used for anything today.
`opensearch`	`opensearchproject/opensearch:2.17.0` (`DISABLE_SECURITY_PLUGIN: "true"`)	Config field and container exist for an intended full-text/search index.	Provisioned only — no indexing or search client code exists in the backend. No search functionality is implemented against it.
`backend`	FastAPI, Python 3.12	Core API server: IOC-type detection, provider orchestration, AI summarization/correlation, evidence building, auth, case management.	Actively used — the primary application process.
`celery_worker`	Celery 5.4 (same backend codebase)	Executes background jobs.	Actively used, but for exactly one job (see below).
`celery_beat`	Celery 5.4 beat scheduler	Triggers scheduled jobs on a timer.	Actively used, one scheduled job only.
`frontend`	Next.js 14.2.15	Web UI.	Actively used.

A second compose file, `docker-compose.prod.yml`, is layered on top of `docker-compose.yml` for production use (this is what the Windows installer runs): it strips the development bind-mounts and switches the `backend` / `frontend` containers to their production start commands rather than dev/hot-reload ones.

Neo4j and OpenSearch, specifically: both are real, running containers in every deployment of this stack, and both have a corresponding settings field in the backend config (`neo4j_uri` / `neo4j_user` / `neo4j_password`, `opensearch_url`). Neither is wired into any actual code path. The correlation graph that the product presents to users lives entirely in Postgres's `correlation_edges` table today. Any description of a "Neo4j-backed graph" or "OpenSearch-backed search" elsewhere should be read as an architecturally-provisioned but not-yet-integrated capability, not a working feature.

Backend

The backend is a **FastAPI** application (`backend/app/main.py`) running on **Python 3.12**. Its notable stack choices:

- **SQLAlchemy 2.0**, async, with `asyncpg` as the Postgres driver.
- **Alembic** for schema migrations (`backend/alembic/`).
- **Celery 5.4** for the one background job described below.
- **structlog** and **prometheus-fastapi-instrumentator** for logging and metrics.

The backend is also where the 16 third-party data-source connectors ("providers"), the AI summarization/correlation logic, and the authentication/RBAC (role-based access control) system live — each of those is detailed in its own section of this document.

Frontend

The frontend is a **Next.js 14.2.15** application using **React 18.3.1** and **TypeScript**, styled with **Tailwind**, with **Zustand** for client-side state, **Recharts** for charts, **react-force-graph-2d** for the relationship/correlation graph visualization, and **Radix UI** for primitive UI components (all per `frontend/package.json`).

`package.json` declares `vitest` as the test runner (`"test": "vitest run"`), but a repository-wide search found zero `*.test.*` / `*.spec.*` files under `frontend/` — the tooling is configured but no frontend automated tests currently exist. (See the Testing and Quality Assurance appendix for the full picture, including the backend's test suites.)

Executive Dashboard and Deterministic Scoring Engine

A new **Executive Dashboard** (`GET /api/v1/dashboard/kpis`, backed by `app/core/dashboard.py::get_kpis()`) gives an at-a-glance operational view of the whole platform: real-time KPI tiles for active investigations, the count of critical/high-risk IOCs, open case count and open-critical case count (reported as two separate numbers, not collapsed into one), average threat score, provider health percentage, and AI analysis success rate. Every one of these is a live SQL aggregate computed against Postgres at request time — none is hardcoded, and none is cached client-side across page loads. A companion endpoint, `GET /api/v1/dashboard/executive-summary` (`app/ai/dashboard_summary.py`), layers an AI-generated 2–4 sentence narrative on top of the exact same KPI values, grounded exclusively in those numbers as given facts. If the AI backend is unreachable, or its structured response fails validation twice in a row, the endpoint falls back to a deterministic, template-generated sentence built directly from the same real KPI numbers — never an error message, and never fabricated data. This is disclosed to the user directly on the Dashboard itself: the Executive Summary card carries an "AI-generated" badge when the narrative came from the model, or a "Template fallback" badge when it didn't, so the source of the text is never ambiguous.

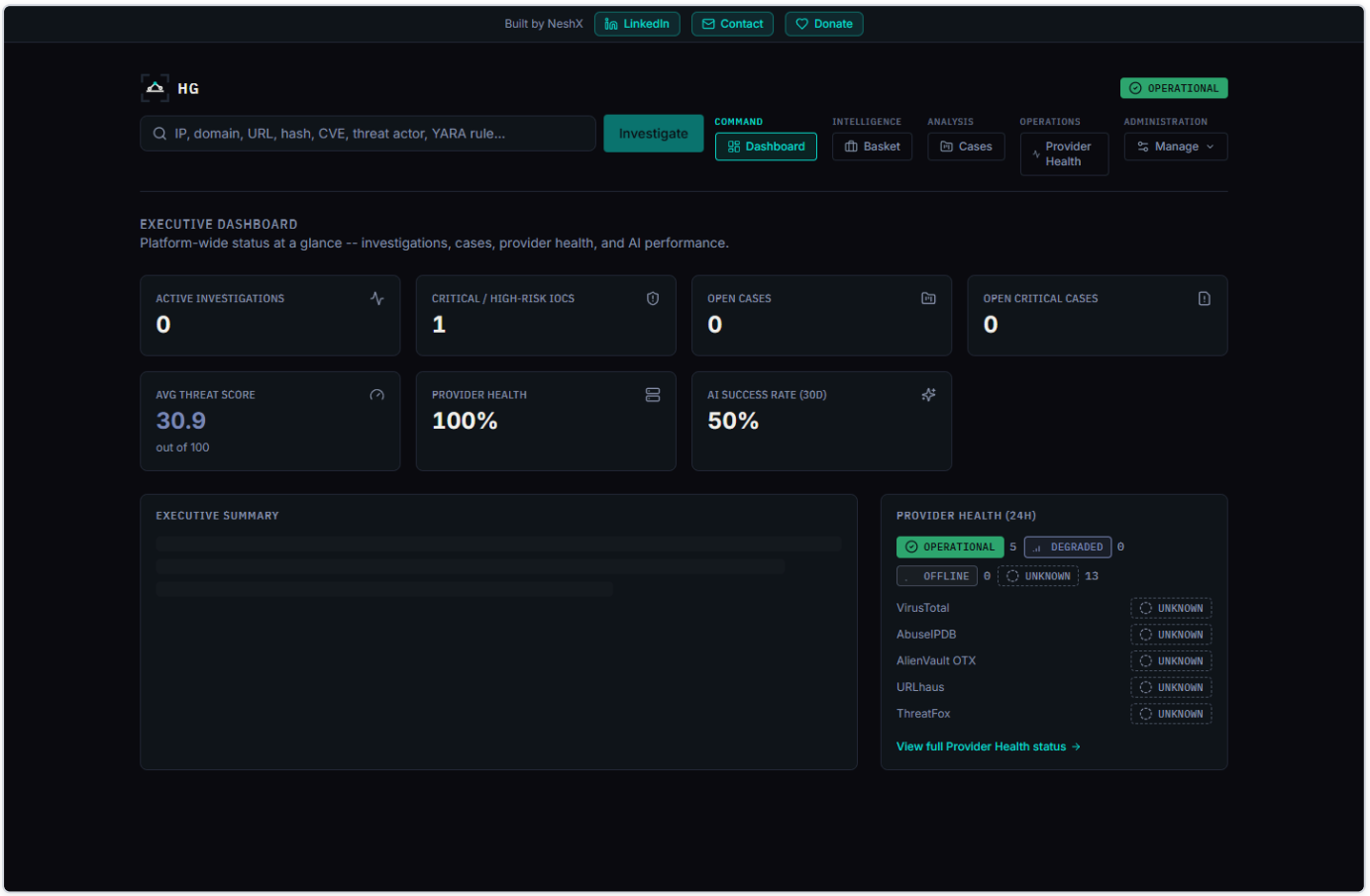

Figure 64 — The Executive Dashboard shows real KPI tiles, an AI-generated (or template-fallback) executive summary, and a provider-health summary widget -- every number sourced live, never hardcoded.

Underlying both the Dashboard's "average threat score" tile and every individual investigation's risk score is a new **deterministic (non-AI) scoring engine** (`app/scoring/engine.py`). Historically, `overall_risk_score` , `confidence_score` , `malicious_probability` , and the final verdict were 100% AI-generated in one call — prompt wording alone cannot reliably stop a model from overriding a number it was merely asked not to touch. The scoring engine instead computes all four values *before* any AI call runs, purely from already-fetched evidence: a cross-provider verdict-consensus component (weighted 65 of 100 points, with a corroboration multiplier so a single unconfirmed provider flag scores meaningfully lower than several providers agreeing) and a correlation-graph-evidence component (35 of 100 points, drawn from `app/correlation/engine.py` 's graph and given the same corroboration treatment to resist a single source flooding it with distinct, uncorroborated edges). The AI is then handed this already-decided score as a given fact and asked only to narrate a verdict/rationale consistent with it — it can never invent or override the number, and the platform's own final-assessment validation re-checks the AI's output against the deterministic result before persisting it. Every persisted assessment records `SCORING_ENGINE_VERSION` , the exact version of this weighting scheme that produced it, so any historical score can always be traced back to the logic that generated it.

Celery: Scope and a Key Boundary

`celery_worker` and `celery_beat` both run the same task application, `app.workers.celery_app` . Exactly **one** scheduled task exists in the codebase: `run_osint_crawl` , triggered hourly by `celery_beat` (`backend/app/workers/celery_app.py`), which re-crawls OSINT (open-source intelligence) sources for IOCs that were looked up in the last 24 hours (`backend/app/workers/tasks.py`).

That is the entirety of what Celery does in this platform. **The main, user-facing IOC lookup pipeline — the synchronous, streamed lookup reachable at `POST /api/v1/lookup/stream` — does not go through Celery at all.** This is stated directly in a header

comment in `celery_app.py`. A lookup request (provider fan-out, per-provider AI summarization, correlation, final AI assessment, and persistence) is handled entirely within the FastAPI backend process for the duration of that one HTTP connection, with results streamed to the browser as Server-Sent Events (SSE) as each step completes. Celery is reached only by the separate, unrelated hourly re-crawl job — never as part of answering a live lookup request.

Deployment Paths

Two deployment paths are implemented and verified in the repository:

1. **Docker Compose** — `docker-compose.yml` (development) and `docker-compose.yml` + `docker-compose.prod.yml` (production). This is the path the Windows installer automates end-to-end (covered in the Windows Deployment / Installer section of this document).
2. **Kubernetes** — the `k8s/` directory contains a parallel, **Kustomize**-based Kubernetes deployment: StatefulSets for `postgres`, `neo4j`, and `opensearch`; Deployments for `backend`, `frontend`, and `celery`; and an Ingress resource. This is an alternative to the Docker Compose / Windows-installer path. Beyond its existence and the resource types listed above, further detail on this path (replica counts, resource limits, ingress rules, or how it is intended to be operated) is **Not confirmed** — it was not otherwise inspected as part of this documentation effort.

Component Diagram

The diagram below reflects verified relationships only. Solid arrows are real, active data paths confirmed in code; dashed arrows into `neo4j` and `opensearch` represent the config fields and running containers that exist but have no consuming application code.

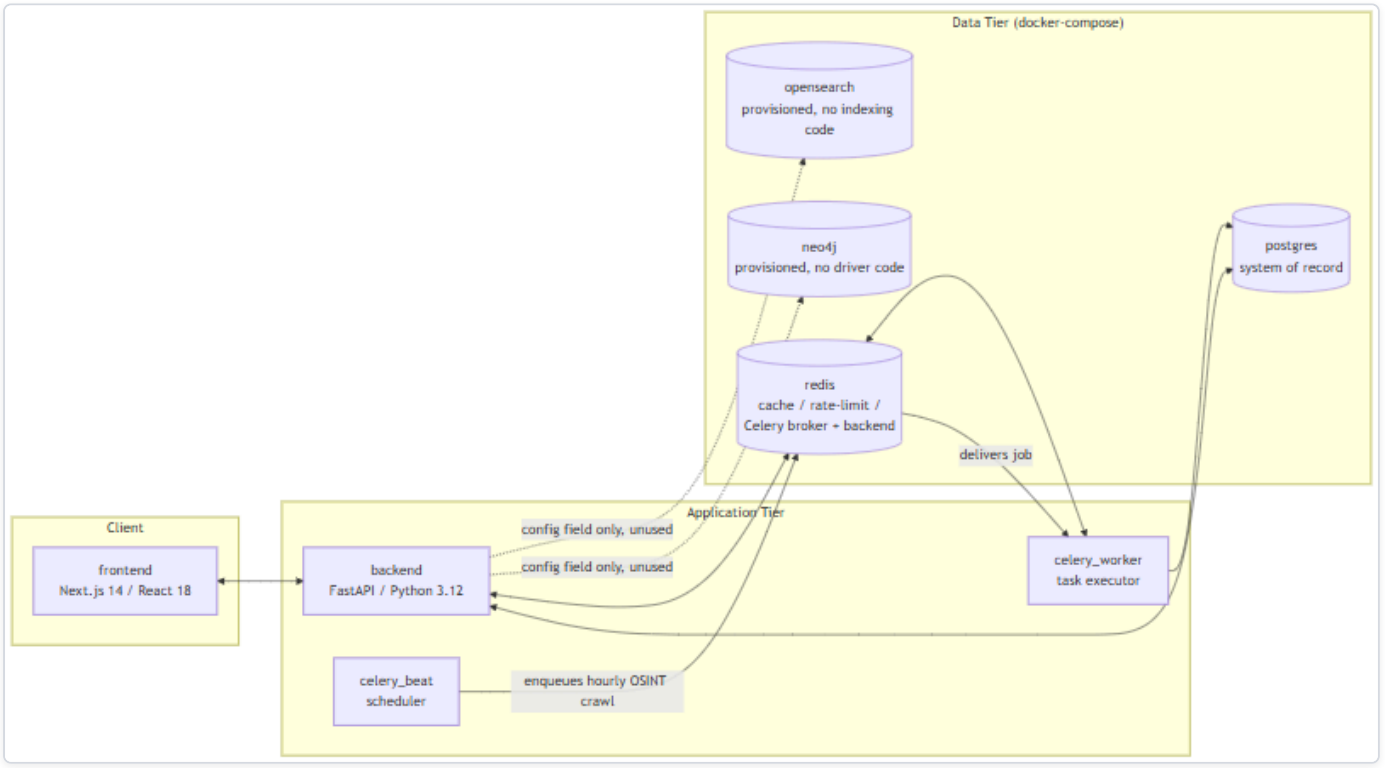

Figure 65 — Diagram: Component Diagram

Note what is *not* in this diagram: there is no edge from `backend` to `celery_worker` / `celery_beat` for the lookup pipeline, because none exists. The live IOC-lookup path is entirely `frontend <-> backend <-> postgres/redis`; the Celery containers are reached only

by the independent, hourly-scheduled OSINT re-crawl job described above.

Data Flow

This section traces exactly what happens, in code, when a user submits one IOC (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, CVE ID, or MITRE ATT&CK technique ID that a security analyst wants to investigate) to the platform, from the initial HTTP request through to the final "done" event. It then states plainly which of the platform's four datastores actually participate in that flow today, and which do not.

The entry point for a lookup is a single endpoint: `POST /api/v1/lookup/stream` (`backend/app/api/routes/lookup.py`). It is a Server-Sent Events (SSE) endpoint — the client opens one HTTP connection and receives a sequence of typed events as the lookup progresses, rather than blocking for a single response. The sequence below is verified directly against that endpoint's implementation.

The 9-Step Lookup Pipeline

1. **Rate limit check.** A Redis-backed fixed-window rate limiter (`RateLimiter` , `app/core/cache.py`) rejects the request if the calling user has exceeded the configured quota — 10 calls per 60 seconds per user by default (`app/core/config.py`), both values configurable.

2. **IOC type detection.** `detect_ioc_type()` (`app/ioc/detector.py`) classifies the raw input string using ordered regex and heuristic checks into one of 32 `IOCType` enum values (`app/ioc/types.py`) — for example, distinguishing an IPv4 address from a domain from a SHA256 hash.

3. **Lookup record created.** An `IOCLookup` row is created in Postgres immediately, with `status = RUNNING` . This is the row that will later be updated with the final verdict and risk scores.

4. **Provider fan-out.** `run_all_providers()` (`app/providers/orchestrator.py`) concurrently invokes every registered provider (of 16 total) whose `supports(ioc_type)` returns true for this IOC's type, using `asyncio.create_task` and `asyncio.as_completed` — providers run in parallel, not sequentially. A "provider" here is an external threat-intelligence, vulnerability, or OSINT source the platform queries (for example VirusTotal, AbuseIPDB, NVD). Each provider call passes through a Redis cache check first, then `BaseProvider.run()` (timeout and retry handled by `tenacity` , with configurable `provider_timeout_seconds` and `provider_max_retries`), producing a normalized `ProviderResult` .

5. **Per-provider persistence and summary.** As each provider result arrives (not waiting for all of them), it is persisted to Postgres as a `ProviderResultRecord` . If that provider's status is `ok` , `summarize_provider()` (`app/ai/service.py`) makes one AI call for that provider alone, grounded only in that provider's own JSON response, and the result is persisted as an `AISummaryRecord` tagged with that `provider_id` . Both the raw result and the summary are streamed to the client as SSE events (`provider_result` , `provider_summary`) and committed to Postgres after every single provider completes — a deliberate design so that a client disconnecting mid-lookup does not lose already-completed provider data (`lookup.py` , lines 122-136).

6. **Correlation.** Once every provider has finished, `correlate()` (`app/correlation/engine.py`) runs. This is a pure, I/O-free function: it extracts typed relationship edges from normalized provider fields (`resolved_ips` , `related_hashes` , `malware_families` , `mitre_techniques` , `cves` , and others), deduplicates and corroborates them — each additional provider that independently reports the same fact boosts its confidence score by a fixed increment (`_CORROBORATION_BONUS_PER_PROVIDER = 0.15`) — and returns a `CorrelationResult` (nodes, edges, deduplicated facts, provider agreement). The resulting edges are persisted as `CorrelationEdgeRecord` rows in Postgres and streamed as a `correlation` SSE event.

7. **Final AI assessment.** `generate_final_assessment()` (`app/ai/service.py`) makes exactly one consolidated AI call, grounded in all of the per-provider summaries plus the correlation output — never in the raw provider JSON directly. The result is validated against the `FinalAssessment` schema, cross-checked against the deterministic correlation data by `_ground_final_assessment()` (which silently strips any agreeing/disagreeing provider citation that isn't actually backed by real correlation data, and separately

flags — rather than removes — any cited MITRE technique that isn't backed by a real correlation edge, by setting that technique mapping's `grounded` field to `False`), and written onto the `IOCLookup` row as `final_verdict`, `risk_score`, `confidence_score`, and the full `final_assessment` JSON.

8. **Evidence ledger build.** `build_evidence()` (`app/evidence/builder.py`) deterministically converts the provider results, AI summaries, and correlation edges into `EvidenceItem` rows. This step is explicitly **not AI-generated** (per the module's own docstring) — it exists so that later AI-driven explanations elsewhere in the product can cite real, checkable evidence records instead of re-summarizing from memory.
9. **Final SSE events.** The stream emits a `final_assessment` event, then a `done` event, closing the connection.

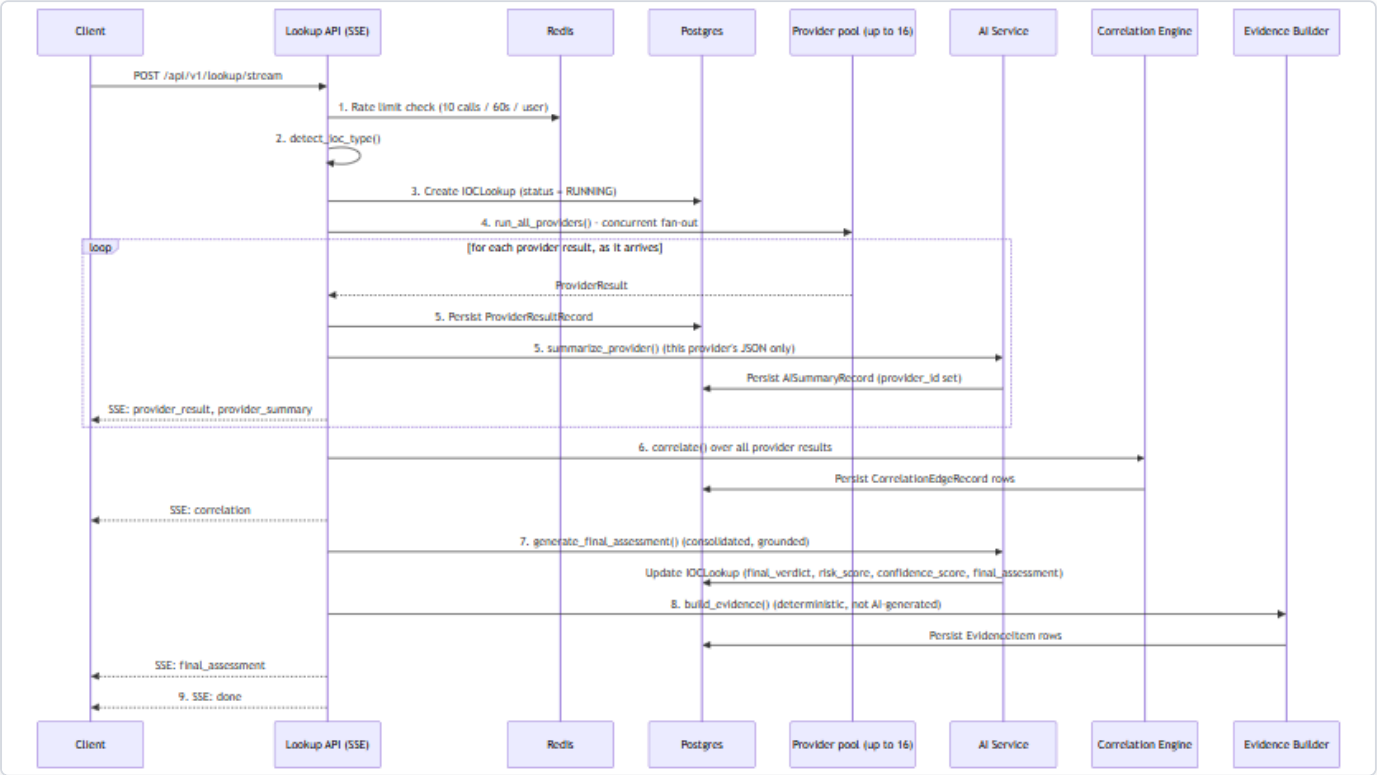

Figure 66 — Diagram: The 9-Step Lookup Pipeline

Where Data Actually Lives

The Docker Compose stack provisions four datastores — Postgres, Redis, Neo4j, and OpenSearch. Reading the pipeline above, it would be easy to assume all four are in play. They are not. This distinction matters enough that it is called out separately: **only Postgres and Redis have any application code that actually reads from or writes to them.** Neo4j and OpenSearch are running containers with nothing in the backend that talks to them.

Datastore	Provisioned in `docker-compose.yml`?	Application code that uses it?	What it actually holds today
Postgres	Yes (`postgres:16-alpine`)	Yes	Everything: `IOCLookup` rows, `ProviderResultRecord`s, `AISummaryRecord`s (per-provider and final), `CorrelationEdgeRecord`s, `EvidenceItem`s, plus users, cases, and analyst baskets
Redis	Yes (`redis:7-alpine`)	Yes	Provider-result cache (keyed `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}`, default TTL 3600s), the lookup-creation rate limiter, and the Celery broker/result-backend queues
Neo4j	Yes (`neo4j:5-community` + APOC plugin)	**No**	Nothing. Config fields only (`neo4j_uri`, `neo4j_user`, `neo4j_password`)
OpenSearch	Yes (`opensearchproject/opensearch:2.17.0`)	**No**	Nothing. A config field only (`opensearch_url`)

Postgres is the correlation graph today

Step 6 of the pipeline above talks about a "correlation graph" of nodes and edges between IOCs. It is tempting to assume that graph lives in Neo4j, since Neo4j is a graph database and is sitting right there in the stack. It does not. **The correlation graph is a set of rows in Postgres's `correlation_edges` table, queried relationally — full stop.** `CorrelationEdgeRecord`'s own docstring (`models/lookup.py`) and the correlation engine's module docstring (`correlation/engine.py`) both describe edges as being "mirrored into Neo4j," but that describes an intended future design, not current behavior. A full-repo search of `backend/app` for `neo4j`, `Neo4j`, or `GraphDatabase` (outside the Python virtual environment) turns up zero driver instantiations, zero Cypher queries, and zero read or write calls anywhere in the application. The only things Neo4j-related in the codebase are the three unused config fields and those two docstrings.

There is no search functionality

The same is true of OpenSearch, with even less ambiguity: there are no docstrings suggesting an intended integration, no indexing pipeline, and a full-repo search finds zero OpenSearch client, index, or search code anywhere in `backend/app`. The container runs; nothing in the product talks to it. Any full-text search a user might expect (e.g., searching historical lookups by free text) is not implemented against OpenSearch or anywhere else.

Practical takeaway

For a judge or engineer evaluating this build: treat Neo4j and OpenSearch as **provisioned infrastructure for a future phase**, not as working features. If asked "does this platform have a graph database backing its correlation view?" or "does it support full-text search?", the accurate answer is no — both containers exist and are healthy, but the entire correlation and evidence pipeline described in the nine steps above runs, end to end, on Postgres and Redis alone.

AI Architecture (Technical)

This section describes how HORIZON GRID's backend (`backend/app/ai/`) uses large language models to summarize and score an **IOC** (Indicator of Compromise — a hash, IP, domain, URL, CVE, etc. submitted for lookup) once raw provider data has been collected. It assumes the reader is technically literate but new to this codebase, so product-specific components — `ProviderResult` , `CorrelationResult` , `AISummaryRecord` , `IOCLookup` , `EvidenceItem` — are defined at first use.

All facts below are traceable to `backend/app/ai/service.py` , `schemas.py` , `analysis_service.py` , `hunting_service.py` , `analysis_schemas.py` , `ollama_client.py` , `anthropic_client.py` , `bedrock_client.py` , `gemini_client.py` , `groq_client.py` , `openai_client.py` , `kimi_client.py` , `deepseek_client.py` , `xai_client.py` , `mistral_client.py` , and `openrouter_client.py` .

1. Supported AI Backends and Selection

The active backend is a single configuration value, `ai_backend` (`app/core/config.py` line 69), defaulting to `"ollama"` . Eleven backends are implemented:

Backend	Config value	Mode	Auth / connection detail
Ollama	`ollama`	Local, default	No key; talks to `http://host.docker.internal:11434` , model `llama3.2:3b`
Anthropic	`anthropic`	Direct API	Direct Messages API, tool-forced JSON output
AWS Bedrock	`bedrock`	Cloud	Bedrock Converse API for Claude; supports bearer-token or IAM access-key/secret auth
Google Gemini	`gemini`	Cloud	Gemini `generateContent` REST endpoint; JSON-schema response mode
Groq	`groq`	Cloud, fast inference	OpenAI-compatible chat-completions API at `api.groq.com` ; Bearer-token auth; default model `llama-3.3-70b-versatile`
OpenAI	`openai`	Cloud	OpenAI's own chat-completions API at `api.openai.com` ; Bearer-token auth; forced tool-calling for structured output
Kimi (Moonshot AI)	`kimi`	Cloud	OpenAI-compatible chat-completions API at `api.moonshot.ai` ; Bearer-token auth
DeepSeek	`deepseek`	Cloud	OpenAI-compatible chat-completions API at `api.deepseek.com` ; Bearer-token auth
xAI	`xai`	Cloud	OpenAI-compatible chat-completions API at `api.x.ai` ; Bearer-token auth
Mistral	`mistral`	Cloud	Chat-completions API at `api.mistral.ai` ; Bearer-token auth; forced tool-calling for structured output
OpenRouter	`openrouter`	Cloud, meta-router	OpenAI-compatible chat-completions API at `openrouter.ai` , routing to many underlying model vendors; Bearer-token auth

All eleven client classes expose the **same** async method signature — `call_claude_json(system_prompt, user_prompt, json_schema, tool_name, max_tokens)` — so `app/ai/service.py` 's internal `_get_ai_client()` never has to branch on which backend is active (service.py lines 42-78). This is a straightforward adapter pattern: swapping backends is a config change, not a code change. `groq_client.py` and the five clients modeled on it (`openai_client.py` , `kimi_client.py` , `deepseek_client.py` , `xai_client.py` , `mistral_client.py` , `openrouter_client.py`) fit this pattern by targeting their vendor's own OpenAI-compatible chat-completions endpoint and using the same forced-single-tool-call technique the other backends use to coerce a structured JSON response, rather than a bespoke prompt format.

Model-list handling now splits into two groups rather than "Groq vs. everyone else": Anthropic, Gemini, and Bedrock still use a hardcoded/static model list in the codebase. Every other backend discovers its model list **live** instead — Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, and OpenRouter each query their own vendor `GET /models` endpoint (exposed to the setup wizard and API consumers via the shared `POST /api/v1/ai/{backend}/models` route), and Ollama lists whatever is actually pulled locally via `GET {base_url}/api/tags`. For the seven vendor-hosted live-discovery backends, this means the model dropdown always reflects the account's real, currently-available models rather than a list that can silently go stale as a vendor deprecates or introduces models.

Selection is **fail-fast**: if the currently selected backend's `is_configured` check returns `False` (e.g. `anthropic` selected with no API key present), the service raises a `RuntimeError` immediately rather than letting the request proceed into a confusing low-level HTTP failure.

2. Runtime AI Backend Switching (No Restart)

Until this session, `ai_backend` and every backend's credentials were read once, at process start, from `.env` via `get_settings()` (`app/core/config.py`) — a `@lru_cache`-wrapped singleton that lives for the lifetime of the container process. Switching from Ollama to Anthropic, or adding a Groq key, meant editing `.env` and restarting the Docker containers; there was no way to change the active AI backend or its credentials while the platform was running.

`_get_ai_client()` in `service.py` now resolves the active backend on **every call**, rather than once at startup, by reading a database-backed runtime-config service (`backend/app/core/runtime_config.py`) instead of the frozen Settings object. Resolution order is: (a) an explicit `backend_override`, used only by the AI-comparison feature described below; (b) the currently **active** runtime-configured backend — a fresh row read from the `provider_runtime_configs` table (kind `"ai"`, `is_active = true`) on that call; (c) the legacy Settings-based value, kept only as a fallback for installs where no runtime config has been seeded yet. Each of the eleven AI client classes (`ollama_client.py`, `anthropic_client.py`, `bedrock_client.py`, `gemini_client.py`, `groq_client.py`, `openai_client.py`, `kimi_client.py`, `deepseek_client.py`, `xai_client.py`, `mistral_client.py`, `openrouter_client.py`) gained optional constructor parameters so `_get_ai_client()` can build a fresh, correctly-credentialed instance from the decrypted runtime credentials on each call instead of reusing a stale cached singleton built from `.env` at process start.

The practical effect: `POST /api/v1/runtime/ai-active` switches the active backend, and the very next AI call — the next per-provider summary or final assessment, for any in-flight or new lookup — uses the newly active backend and its credentials. No container restart, no `.env` edit, no redeploy.

This same on-demand resolution also underlies the AI-comparison feature: `POST /api/v1/lookup/{id}/reanalyze` re-runs only the `generate_final_assessment()` step against an already-completed lookup's existing evidence, passing a `backend_override` for whichever backend the analyst wants to compare against — no provider is re-queried. Every assessment ever produced for a lookup — the original plus every later comparison run — is written to a new `final_assessment_records` table (`app/models/lookup.py`), tagged with the `ai_backend` / `ai_model` that produced it and an `is_primary` flag distinguishing the original from comparisons; a reanalysis never overwrites or deletes an earlier result. `GET /api/v1/lookup/{id}/assessments` returns the full set for side-by-side review.

One further prompt-level change accompanies this: `generate_final_assessment()` now also receives an explicit `unavailable_providers` list — providers that were applicable to the IOC but failed, timed out, were rate-limited, or were not configured/disabled for this run — rendered in the prompt as a distinct "Providers unavailable this run" section, together with a system-prompt rule never to read an unavailable provider's absence as "found nothing" or "reported clean." This closes a gap that predates runtime switching but became more visible once providers could be disabled live: without it, the AI had no way to distinguish missing coverage from a genuinely clean result.

3. Two Distinct AI Call Types

The pipeline makes two structurally different kinds of AI calls, confirmed as genuinely separate in the code (not a naming convention over one code path):

	`summarize_provider()`	`generate_final_assessment()`
Cardinality	One call **per provider** that returned OK data	**One** call per lookup, after all providers finish
Input	Only that single provider's own JSON	All per-provider `ProviderSummary` objects + the correlation engine's output — **never** raw provider JSON
Stored as	`AISummaryRecord` with `provider_id` set	`IOCLookup.final_assessment` **and** an `AISummaryRecord` with `provider_id = NULL`

The `AISummaryRecord.provider_id` column is the actual database-level marker distinguishing the two: a model-level comment (`models/lookup.py` line 93) explicitly documents that a `NULL` `provider_id` row is "the final consolidated assessment, not a per-provider one."

Why they are kept separate, per the code's own design comments (service.py lines 8-11): the final assessment is deliberately grounded in the already-summarized per-provider outputs and the deterministic correlation result rather than being handed every provider's raw JSON again. This bounds the size of the final prompt (which would otherwise grow with the number of registered providers) and forces the "master" call to reason over an already-distilled, per-provider-attributed layer instead of re-deriving everything from scratch — which is also what makes the grounding/cross-check step in §5 possible.

4. No-Evidence Short-Circuit Guard

Inside `generate_final_assessment()` (service.py lines 242-284), there is a hard guard: if `provider_summaries` is empty **and** `correlation.edges` is empty, the function returns a hardcoded, deterministic `FinalAssessment` — `final_verdict=Verdict.UNKNOWN`, all risk fields set to `0` — **without invoking the AI backend at all**.

The code comment documents the concrete, reproduced failure that motivated this guard: querying the real MD5 hash of the EICAR antivirus test file (`44d88612fea8a8f36de82e1278abb02f`) with **zero configured providers** — i.e. with no actual evidence of any kind — still caused the small local model (`llama3.2:3b`) to return `final_verdict="highly_malicious"` and `malicious_probability=92`, fabricating an "association with ransomware and trojans" purely from its pretrained knowledge. This directly violated the model's own system-prompt instruction to never fabricate findings. Rather than trying to prompt-engineer around this failure mode, the team chose to bypass the model entirely whenever there is structurally no evidence to reason over — a deterministic guard is unconditionally reliable where a prompt instruction was not.

5. Grounding / Cross-Check Mechanism

Even when the AI backend *is* called, its structured output is not trusted at face value — it is cross-checked against the deterministic correlation data. Two related mechanisms implement this:

`_ground_final_assessment()` (service.py lines 137-190), run on every `generate_final_assessment()` result:

- Filters the model's `agreeing_providers` / `disagreeing_providers` lists down to only the `provider_id`s that actually appear in the correlation edges, in `provider_agreement`, or in `known_provider_ids` (the set of providers that genuinely returned a per-provider summary, passed in by the caller). Anything else the model names is **silently stripped**.
- Sets `mapping.grounded = False` on any MITRE ATT&CK technique the model cited that is not backed by a real `uses_technique` correlation edge — so a technique mention that has no supporting evidence is flagged rather than presented as fact.

A parallel mechanism in `analysis_service.py`, applied to the analyst-facing explanation endpoints (WHY malicious, Challenge/red-team, Copilot Q&A, and others):

- `_strip_invalid_evidence_ids()` (lines 55-71) recursively removes any `evidence_ids` citation that doesn't match a real `EvidenceItem.id` belonging to that specific lookup.
- `_backfill_evidence_ids_from_prose()` (lines 74-112) does the reverse repair: it recovers citations the model correctly referenced in free-text prose but failed to place into the structured `evidence_ids` field — but it only ever adds an ID that has already been confirmed real; it never invents one.

Together, these mean every AI-authored claim that reaches storage or the UI has either been matched against a real `EvidenceItem`/correlation edge or explicitly marked/stripped as unsupported — the deterministic correlation and evidence layers act as a check on the generative layer, not the other way around.

6. Verdict and Confidence Data Model

The `Verdict` enum (`app/models/lookup.py` lines 22-34) has exactly **12 values**: `highly_malicious` , `malicious` , `suspicious` , `unknown` , `likely_benign` , `benign` , `scanner` , `tor_exit_node` , `vpn` , `cdn` , `cloud_infrastructure` , `dormant_infrastructure` . Note that several of these are not "maliciousness" gradations at all but infrastructure classifications (`tor_exit_node` , `vpn` , `cdn` , `cloud_infrastructure` , `dormant_infrastructure`) — the model is expected to distinguish "this is malicious" from "this is a category of infrastructure that merits a different interpretation."

`RiskAssessment` (`app/ai/schemas.py` lines 101-131) carries `overall_risk_score` , `confidence_score` , and `malicious_probability` , all explicitly on a **0-100 scale** (not 0-1). A `field_validator` automatically rescales any value it receives between 0 and 1 by multiplying by 100 — a defensive fix added because `llama3.2:3b` was observed defaulting to a 0-1 convention despite the 0-100 instruction.

`FinalAssessment` additionally carries a `model_validator` named `_verdict_must_agree_with_risk` (schemas.py lines 187-208) that rejects egregiously self-contradictory combinations — for example, a `final_verdict="malicious"` paired with `malicious_probability < 30` is rejected rather than silently persisted. This is a second, independent layer of internal-consistency checking, separate from the evidence-grounding checks in §5.

`FinalAssessment` also carries two traceability fields, `ai_backend` and `ai_model` (`schemas.py`), populated by `service.py` at generation time and surfaced in the UI as a badge on the final-assessment panel. With eleven interchangeable backends now selectable, and the config free to change between one lookup and the next, these fields exist so that an analyst or judge reviewing a conclusion always knows exactly which provider and model produced it — never a silent or ambiguous backend switch. When the no-evidence guard in §4 fires, or a generation error occurs (as happens on a rate-limited call), both fields are left `null` rather than populated with a misleading value, consistent with this layer's broader rule of never presenting a fabricated or guessed fact as if it were real.

7. AI Data Flow

The diagram below shows data flowing *into* the AI layer for a single lookup: per-provider JSON feeds per-provider summaries; those summaries plus the correlation engine's output feed the one consolidated final assessment; the final assessment is grounded/cross-checked before being persisted.

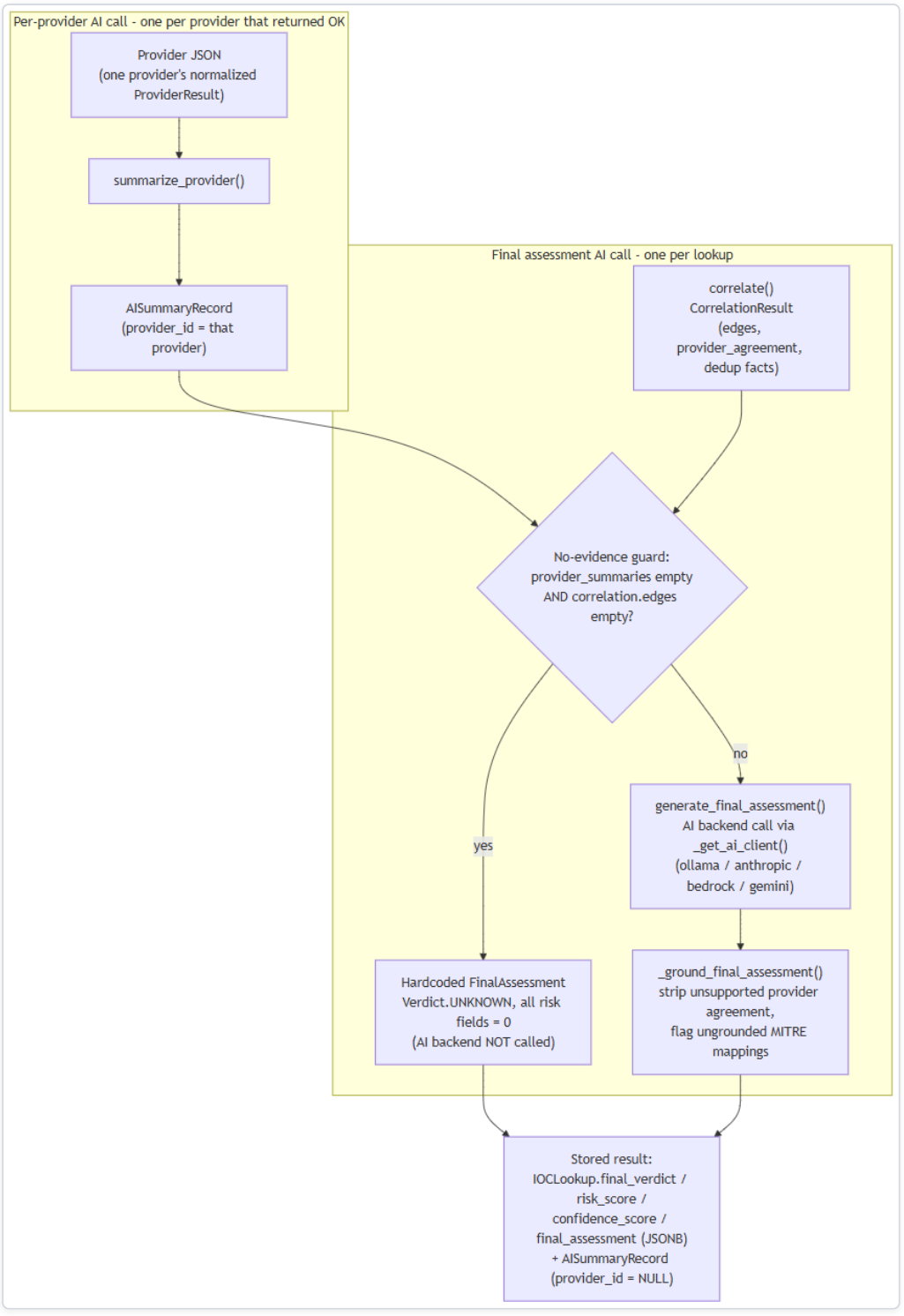

Figure 67 — Diagram: 7. AI Data Flow

8. Beyond the Core Pipeline

`analysis_service.py` implements eight further grounded, evidence-cited AI features on top of the same call/grounding pattern: WHY malicious, What Is This, Provider Disagreement, False-Positive Assessment, Challenge/red-team, Smart Next Actions, Intelligence Gaps, and Score Explanation, plus Copilot Q&A and IOC comparison. `hunting_service.py` implements hunting-query and detection-

rule generation. Both are restricted to citing only indicators/evidence that genuinely exist in the correlation graph or evidence ledger for that lookup — `hunting_service.py` (lines 74-79) explicitly strips any invented expansion target the model produces, following the same "never trust generated output at face value" principle applied throughout this layer.

Provider Architecture

A **provider** is a self-contained connector class in the platform's backend that queries one external data source — a threat-intelligence feed, a vulnerability database, a DNS blocklist, a certificate-transparency log, a sandbox scanner, or an OSINT (open-source intelligence) crawler — for information about a single IOC (indicator of compromise) submitted to a lookup. All 18 providers shipped with the platform are registered in `backend/app/providers/registry.py` (`_ALL_PROVIDERS`) and share one common interface, described below.

Common Provider Interface

Every provider implements the same three-part contract, so the orchestrator (the component that fans a lookup out to providers — see below) never needs provider-specific logic:

- **`supports(ioc_type)`** — a boolean check for whether this provider is eligible to run against the specific kind of indicator submitted. `ioc_type` is one of 31 values in the platform's `IOCType` enum (`app/ioc/types.py`), assigned to the raw input by the IOC-type detector before any provider runs. A provider is only invoked if `supports()` returns true for the lookup's detected type (e.g. `hybrid_analysis` only supports `sha256` ; `whois_rdap` supports `domain` , `ipv4` , `ipv6` , and `asn`).
- **`configured`** — a boolean property reporting whether the provider currently has what it needs to run (typically a credential). Some providers are unconditionally configured because they require no credential at all — for example `nvd` has `requires_key = False` and `configured = True` unconditionally. Others require specific fields to be present — for example `censys` is only `configured` when **both** a Personal Access Token and an Organization ID are set; either alone leaves it not configured.
- **`run()`** (via the shared `BaseProvider.run()`) — executes the actual outbound call (HTTP request, DNS query, etc.) and returns a normalized `ProviderResult` . This call is wrapped with a timeout and retry policy applied uniformly across all providers: `provider_timeout_seconds` (default 20 seconds) and `provider_max_retries` (default 2), with exponential backoff implemented via the `tenacity` library (`wait_exponential(multiplier=0.5, max=4)`).

Before `run()` is invoked, each provider call first checks the Redis cache layer (described below); a cache hit skips the outbound call entirely.

Concurrent Fan-Out Execution Model

For a given lookup, `run_all_providers()` (`backend/app/providers/orchestrator.py`) does **not** call providers one after another. It fans out **concurrently** to every registered provider whose `supports(ioc_type)` is true, using `asyncio.create_task` to launch all eligible provider calls at once and `asyncio.as_completed` to consume each result as soon as it finishes, in whatever order calls actually complete. Each provider call independently goes through: Redis cache check → `BaseProvider.run()` (timeout/retry as above) → normalized `ProviderResult` . As each result arrives, it is persisted to Postgres and streamed to the client immediately (rather than the pipeline waiting for every provider to finish before persisting or streaming anything).

The platform's integration test suite exercises this concurrency directly against fake provider stubs (never the real network-hitting connectors), verifying parallel — not sequential — fan-out, isolation of one provider's failure from the others, cache hit/miss behavior, and correct short-circuiting of unsupported or not-configured providers.

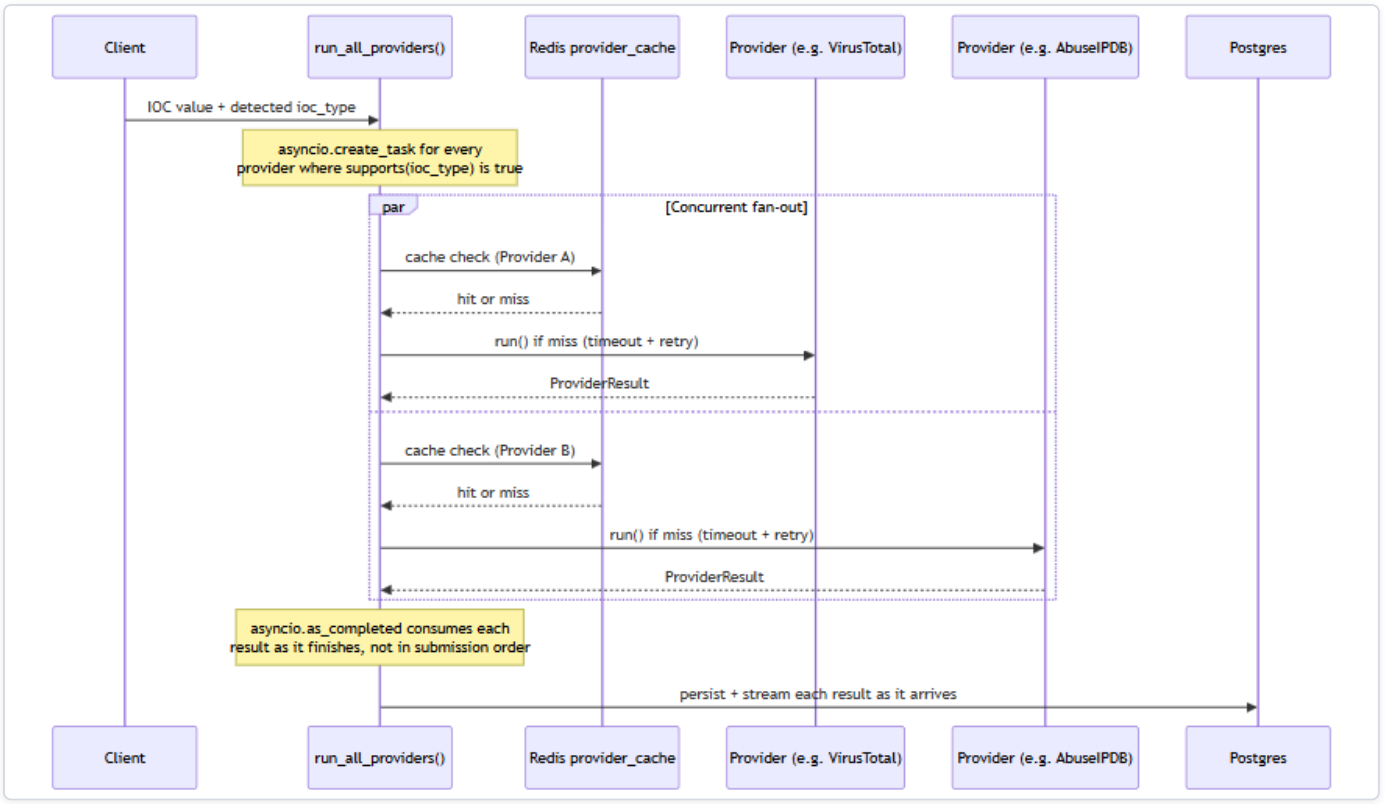

Figure 68 — Diagram: Concurrent Fan-Out Execution Model

"Provider A" and "Provider B" above are illustrative stand-ins for any two of the up to 18 registered providers eligible for a given IOC type — the fan-out is not limited to two.

Redis Caching Layer

Provider results are cached in Redis to avoid repeating identical outbound lookups. The cache key format is `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` (`app/core/cache.py`), and the time-to-live is `provider_cache_ttl_seconds`, which defaults to 3600 seconds (1 hour). This provider-result cache lives on Redis logical DB index `/0`, kept separate from the Celery broker (`/1`) and Celery result backend (`/2`) that run on the same Redis instance.

Two providers additionally maintain their own cache on top of this (visible in the Notes column of the table below): `cisa_kev` keeps an in-memory cached copy of its catalog with a 1-hour TTL, and `mitre_attack` caches its STIX bundle for 1 hour. These are provider-internal caches, distinct from the shared Redis provider-result cache described above.

The 18 Registered Providers

Provider	Category	Supported IOC types	Key requirement	Notes
`virustotal`	threat_intel	ipv4, ipv6, domain, url, hashes	API key	v3 free tier
`abuseipdb`	threat_intel	ipv4, ipv6	API key	v2
`otx`	threat_intel	ipv4, ipv6, domain, hostname, url, hashes	API key	AlienVault OTX
`urlhaus`	threat_intel	url, domain, ipv4	abuse.ch Auth-Key	shares key w/ threatfox/malwarebazaar
`threatfox`	threat_intel	ipv4, ipv6, domain, url, md5, sha256	abuse.ch Auth-Key	shares key
`malwarebazaar`	threat_intel	hash types (md5/sha1/sha256/sha512)	abuse.ch Auth-Key	shares key
`crtsh`	certificate_intel	domain, tls_certificate	none	Certificate Transparency
`nvd`	vulnerability	cve	none (optional key raises rate limit)	NIST NVD v2.0
`cisa_kev`	vulnerability	cve	none	in-memory cached catalog, 1hr TTL
`mitre_attack`	threat_intel	mitre_technique	none	STIX bundle, cached 1hr
`whois_rdap`	whois	domain, ipv4, ipv6, asn	none	python-whois (domains) + RDAP (IP/ASN)
`urlscan`	sandbox	url, domain	API key	urlscan.io; submit-then-poll scan model (`POST /scan/` , then poll `GET /result/{uuid}/` , which returns HTTP 404 while still processing); the poll loop is wall-clock timeboxed and reports a timeout rather than fabricating a result from an incomplete scan
`google_safe_browsing`	threat_intel	url, domain	API key	Google Safe Browsing Lookup API v4 (`threatMatches:find`); a confirmed HTTP 200 with an empty/absent `matches` field is the *only* input that may ever produce a "clean" verdict
`hybrid_analysis`	sandbox	sha256 only	API key	v2 overview/summary endpoint; MD5/SHA1/URL explicitly unsupported per code comment (deprecated endpoints)
`spamhaus`	threat_intel	ipv4, domain	none	DNSBL lookup, no HTTP call (raw DNS)
`phishtank`	threat_intel	url	none (optional key raises rate limit)	
`censys`	passive_dns	ipv4, ipv6	Personal Access Token **and** Organization ID both required	Platform API v3
`internet_intelligence`	osint	domain, ipv4, malware_family, threat_actor, campaign, cve,	none	OSINT crawler wrapping GitHub/Reddit/RSS/pastebin sources

Provider	Category	Supported IOC types	Key requirement	Notes
		file_name		(`app/crawler/`)

A deliberately tested safety guarantee, specific to these two newest providers: both `urlscan` and `google_safe_browsing` are built so that a genuine failure — a rejected/invalid API key, an unreachable API, or a rate limit — is always reported as an error or unknown status, never as a false "clean"/"safe" result. `google_safe_browsing.py` 's own module docstring states the invariant directly: an empty or missing `matches` field on a genuine HTTP 200 response is the *only* input that may ever produce a "clean" verdict; every other outcome — a non-200 status, a network error or timeout, or a response body that doesn't parse the way the API contract promises — routes through the same error-handling path, which always sets a `{"verdict": "unknown", ...}` payload rather than anything that could be mistaken downstream for a real clean scan. This was confirmed with a real, live chaos test that supplied deliberately invalid credentials to both providers' real external APIs: the invalid key was rejected with `status=error` , Safe Browsing's own verdict field read `"unknown"` (never `"clean"`), and the investigation's overall verdict was reported as `UNKNOWN` rather than benign or clean.

Wizard Coverage and the "6 Providers with No UI" Discrepancy

The Windows setup **wizard** (`windows/wizard/Setup-Wizard.ps1` , `$ProviderDefs`) presents a Provider Configuration screen listing exactly 8 entries: `virustotal` , `abuseipdb` , `otx` , `abusech` (a single combined entry covering the shared abuse.ch key used by `urlhaus` , `threatfox` , and `malwarebazaar`), `nvd` , `hybrid_analysis` , `censys` , and `phishtank` . Counting the three abuse.ch-backed connectors individually, those 8 wizard entries map to 10 of the platform's 16 backend providers. A sample of the wizard's entries was independently checked against the corresponding provider's code and matched exactly — for example, the wizard's Censys note ("Requires both a Personal Access Token and an Organization ID") matches `censys.py` 's configured check precisely, and the wizard's abuse.ch note ("one free Auth-Key covers all three connectors") matches all three connectors reading the same `abusech_auth_key` setting.

The remaining 6 backend providers — `crtsh` , `cisa_kev` , `mitre_attack` , `whois_rdap` , `spamhaus` , and `internet_intelligence` — have real, working backend implementations but **do not appear anywhere in the wizard's UI**. This is a documented discrepancy between the wizard and the backend provider registry, but it is flagged as **consistent behavior, not a bug**: all six of these providers have `requires_key = False` — there is no credential for a user to enter in the first place. Because the wizard's Provider Configuration screen exists specifically to collect and test credentials, a provider with nothing to configure has nothing to show there. This is corroborated on the backend side: `backend/app/providers/connection_test.py` 's handler dictionary (which powers the wizard's live "Test" button, `POST /api/v1/providers/{id}/test`) likewise defines no test handler for these six providers, falling back to a message that explicitly identifies them as requiring no credential.

In short: 8 wizard entries cover 10 providers that have a credential-related field to configure — 8 of those (`virustotal` , `abuseipdb` , `otx` , and the three abuse.ch-backed connectors, plus `hybrid_analysis` , `censys`) require a key to function at all, while 2 (`nvd` , `phishtank`) only accept an optional key to raise their rate limit rather than strictly needing one. The other 6 providers run unconditionally with no credential of any kind and correctly have no wizard entry and no connection-test handler.

Runtime IOC Provider Configuration (No Restart)

Everything described above — the `configured` property, the credential fields, and the enabled/disabled state of each of the platform's registered providers — was, prior to this session, entirely `.env` -driven and frozen for the lifetime of the running process: `app/core/config.py` 's `get_settings()` is a process-lifetime `@lru_cache` singleton, so changing a provider's key or toggling it off meant editing `.env` and restarting the Docker containers before the change had any effect.

That has been replaced with a database-backed runtime layer. A new `provider_runtime_configs` table (`backend/app/models/runtime_config.py` , kind `"ioc"`) holds one row per provider — `enabled` , `encrypted_credentials` (Fernet-

encrypted, never plaintext), `extra_config`, and the timestamp/outcome of its last connection test. Two endpoints mutate this table: `POST /api/v1/runtime/ioc-providers/{id}` configures a provider's credentials, and `POST /api/v1/runtime/ioc-providers/{id}/enabled` flips it on or off. Neither requires touching `.env` or restarting anything — the very next investigation submitted after the call picks up the change.

Because each provider connector in `registry.py` is instantiated once and reused as a long-lived singleton for the life of the process, mutating a shared instance attribute on that singleton immediately before use would be unsafe: two investigations running concurrently — one started right after an administrator changes a provider's credentials or enabled state, while another is still mid-flight — could race and see each other's configuration. Instead, `run_all_providers()` (`backend/app/providers/orchestrator.py`) takes a single, immutable snapshot of runtime configuration — `{provider_id: {enabled, configured, credentials}}` — once per investigation, at the start of the fan-out, and holds it in a Python `ContextVar` rather than on the provider object itself. Because `asyncio.create_task()` copies the current context into every task it spawns, each concurrent per-provider task launched for that investigation reads from that same frozen snapshot, regardless of what an administrator does to the underlying table while those tasks are still running. Two investigations in flight at once, each with a different configuration for the same provider, therefore never see each other's values — verified with a dedicated concurrency test simulating exactly that scenario.

The provider-availability model also gained a new state to describe this. Previously a provider was either eligible (`supports()` true) and `configured`, or not; `ProviderStatus` now also has a `DISABLED` value, distinct from `NOT_CONFIGURED` — a provider an administrator has deliberately turned off via the enabled/disabled endpoint reports `DISABLED`, while a provider with no credential entered at all still reports `NOT_CONFIGURED`. Both are, in turn, distinguished from "not applicable to this IOC type," so the final-assessment AI prompt never conflates "unavailable" with "checked and found nothing."

Finally, `POST /api/v1/lookup/stream` gained an optional `provider_ids` field: passing a list of provider IDs restricts that one investigation to exactly those providers, regardless of how many are otherwise enabled and configured platform-wide, without affecting any other investigation.

The Same Live-Test Pattern, Extended to AI Backends

The `POST /api/v1/providers/{id}/test` mechanism described above — the one behind each provider's live wizard "Test" button — has a direct counterpart for the AI layer. Until this session, none of the platform's AI backends (Ollama, Anthropic, Bedrock, Gemini, or Groq — see the *AI Architecture* chapter for the backends themselves) had any live connection test at all; a new `POST /api/v1/ai/test` endpoint (`backend/app/api/routes/ai_config.py`, `backend/app/ai/connection_test.py`) now gives every one of them a real "Test Connection" button in the wizard's AI Configuration screen, mirroring the provider-side pattern rather than introducing a new one.

The same design rule that governs the provider-side tester carries over deliberately: the endpoint always tests the *candidate* credentials currently entered in the form, never the application's stored/active configuration. Because the test path never reads from or writes to the saved config, trying out a not-yet-saved key can never race with — or accidentally disturb — whatever backend and key the running platform is actually using for real investigations.

It is also, deliberately, not a generic pass/fail. The endpoint sends a minimal real request to the backend under test and reports one of several specific, genuine outcomes: authenticated success (with real round-trip latency and the exact model name that replied), invalid/rejected key, rate-limited, model-not-found, network error, or timeout — the same level of specificity the provider-side test handlers already establish, applied to the AI layer for the first time.

Database Architecture

HORIZON GRID's `docker-compose.yml` provisions four datastore containers: `postgres` (postgres:16-alpine), `redis` (redis:7-alpine), `neo4j` (neo4j:5-community with the APOC plugin), and `opensearch` (opensearchproject/opensearch:2.17.0). This section describes what each one actually does in the running system, based on direct inspection of the backend code rather than the docker-compose comments or model docstrings that describe the *intended* design. Two of the four -- Neo4j and OpenSearch -- are running containers with no backend code that reads from or writes to them; that gap is documented plainly below rather than glossed over.

An **IOC** (Indicator of Compromise) is the value a user submits for lookup -- an IP address, domain, file hash, URL, CVE ID, etc. A **provider** is a connector to an external threat-intelligence source (VirusTotal, AbuseIPDB, and so on) that the platform queries for a given IOC.

PostgreSQL: the system of record

PostgreSQL is the only datastore the application actually writes to during a lookup. Every table below is defined under `backend/app/models/`:

Table	Holds
`users`	Account records: email, hashed password, full name, role (`admin` / `analyst` / `viewer`), active flag.
`ioc_lookups`	One row per lookup: the submitted IOC value and type, a status enum (`pending` / `running` / `completed` / `failed`), and the final AI output -- `final_verdict` , `risk_score` , `confidence_score` , and the full `final_assessment` as JSONB.
`provider_results`	One row per provider call per lookup: that provider's raw response as JSONB, plus its status and latency.
`ai_summaries`	AI-generated summaries as JSONB. A nullable `provider_id` distinguishes a per-provider summary from the single final consolidated assessment (null `provider_id` marks the latter).
`correlation_edges`	The relationship graph produced by the correlation engine: source/target type and value, relationship type, a confidence score, and provenance (which provider(s) contributed the edge).
`evidence_items`	The deterministic, non-AI-generated evidence ledger: an evidence-type enum (9 values), a source label, a claim, an interpretation, a confidence score (0-100), and the underlying raw data as JSONB.
`basket_items`	Per-analyst scratch lists of IOCs, unique per owner and IOC value.
`cases` , `case_iocs` , `case_notes` , `case_reports`	A lightweight case-management workflow: cases with status/severity enums, the IOCs attached to a case, and analyst notes. `case_reports` exists as a table, but report generation itself is not implemented -- the field is present and permanently empty rather than populated by any code path.

Two things worth calling out because they matter for how the rest of this appendix should be read:

- **The correlation graph lives in Postgres.** `correlation_edges` is a plain relational table, populated by the correlation engine after every provider has returned, and streamed to the UI as part of the lookup. There is no separate graph database behind it today -- see the Neo4j section below.
- **The evidence ledger is deliberately not AI-generated.** `evidence_items` rows are built by a deterministic conversion step so that later AI-written explanations have real, checkable records to cite against, rather than citing their own prior output.

Schema changes are managed with **Alembic** (`backend/alembic/`), SQLAlchemy's migration tool. The application itself uses SQLAlchemy 2.0 in async mode over `asyncpg` .

Neo4j: provisioned, not integrated

Neo4j (`neo4j:5-community` , with the APOC plugin) runs as a container in `docker-compose.yml` , and the backend config (`app/core/config.py`) has `neo4j_uri` , `neo4j_user` , and `neo4j_password` settings for it. Some code comments describe an intended design where correlation edges get "mirrored into Neo4j" -- this appears in the `CorrelationEdgeRecord` model docstring and in the correlation engine's module docstring.

That design was never built. A full search of the backend application code for `neo4j` , `Neo4j` , or `GraphDatabase` (excluding the Python virtual environment) turns up zero driver instantiation, zero Cypher queries, and zero read or write calls anywhere in the application. Neo4j is not used for anything today. **The correlation graph described above is, in its entirety, the `correlation_edges` table in PostgreSQL.** Anyone evaluating this platform should treat Neo4j as reserved-but-idle infrastructure, not as an active graph store.

OpenSearch: provisioned, not integrated

The same pattern holds for OpenSearch (`opensearchproject/opensearch:2.17.0` , running with `DISABLE_SECURITY_PLUGIN: "true"`). The backend config has an `opensearch_url` setting, and the container runs as part of the stack, but a full search of the backend application code found zero OpenSearch client code, zero index definitions, and zero search calls. No indexing pipeline exists, and no user-facing search feature is backed by it. OpenSearch should be described as provisioned infrastructure only -- not as an active search index.

Neo4j and OpenSearch represent the most significant gap between the design implied by the docker-compose file and docstrings, and what the running system actually does. Both are real containers consuming real resources; neither currently does any work for the platform.

Redis: three real uses

Unlike Neo4j and OpenSearch, Redis (`redis:7-alpine`) is genuinely load-bearing, in exactly three ways -- all confirmed against `app/core/cache.py` , `docker-compose.yml` , and `app/core/config.py` :

1. **Provider-result cache.** Each provider call is cached under a key of the form `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` , with a TTL controlled by `provider_cache_ttl_seconds` (default 3600 seconds / 1 hour). A repeat lookup for the same IOC within the TTL window skips the live provider call.
2. **Rate limiter.** A fixed-window rate limiter, implemented in the same `app/core/cache.py` module as the cache, guards the lookup-creation endpoint: by default, 10 calls per 60 seconds per user (both values configurable).
3. **Celery broker and result backend.** The `celery_worker` / `celery_beat` services use Redis as both the message broker and the result store for background jobs.

Redis is used across separate **logical database indexes** on the same Redis instance, rather than one shared keyspace:

Redis DB index	Purpose
`/0`	Cache (provider-result cache and rate-limiter counters, both via `app/core/cache.py`)
`/1`	Celery broker (`redis://redis:6379/1`)
`/2`	Celery result backend (`redis://redis:6379/2`)

This separation means a `FLUSHDB` or eviction event against the cache database, for example, cannot collide with in-flight Celery task state, and vice versa -- they are isolated by index even though they share one Redis process. No other use of Redis was found in the

codebase.

It's worth noting that the synchronous SSE lookup pipeline (`POST /api/v1/lookup/stream`) does **not** go through Celery at all -- it runs providers directly and streams results back over the same request. The only scheduled Celery job in the system is an hourly OSINT re-crawl (`run_osint_crawl`) for IOCs looked up in the last 24 hours; Celery's broker/result-backend role is scoped to that background job, not to the interactive lookup path.

Component overview

Figure 69 — Diagram: Component overview

Summary

Postgres is the single source of truth for every entity in the platform, including the correlation graph. Redis does three concrete jobs -- provider-result caching, rate limiting, and Celery transport -- cleanly separated by logical database index on one instance. Neo4j and OpenSearch run as containers and appear in configuration, but as of this writing carry no application code that reads or writes to them; they should be read as forward-provisioned infrastructure, not as active components of the current architecture.

Windows Deployment Architecture

This section documents the Windows installer for HORIZON GRID: an Inno Setup package (a Windows installer-authoring tool that compiles a self-extracting setup executable) plus a WinForms configuration wizard (.NET's native desktop UI framework — this is not an Electron or browser-based installer). Together they package the same Docker Compose-based runtime described elsewhere in this document for a single-machine Windows install; they do not reimplement the platform as a native Windows service. Docker Compose remains the actual runtime underneath the installer, per an explicit header comment in `windows/installer.iss`.

What the Installer Actually Does

`installer.iss` copies the following into `%ProgramFiles%\IOC Intelligence Platform\app\` (Inno Setup's `{autopf}` constant):

- `backend/`
- `frontend/`
- `docker-compose*.yml` (including the production overlay, `docker-compose.prod.yml`, which strips the development bind-mounts and switches the backend/frontend containers to production start commands)
- `docs/`
- `README.md`
- the `windows/scripts` and `windows/wizard` PowerShell used for post-install configuration and day-to-day operation

Architecture restriction. The installer sets `ArchitecturesAllowed=x64compatible`, restricting installation to 64-bit-compatible Windows systems. This is consistent with — and enforced a second time by — the prerequisite checker's own verification of a 64-bit Windows 10-or-later host (below): the same constraint is checked once at install-package level and once again at runtime. The source material does not document any further stated engineering rationale (e.g. no claim is made here about 32-bit or ARM64 support one way or the other beyond this).

Prerequisite Checks

Before any files are copied, a `[Code]`-section step in `installer.iss` runs `Check-Prerequisites.ps1`, which verifies:

- 64-bit Windows 10 or later
- Administrator privileges
- At least 8 GB RAM (soft check)
- Sufficient free disk space
- Docker Desktop installed and running
- Docker Compose v2 available
- The ports the platform needs are free

Failures surface a "Continue anyway?" prompt rather than a silent abort — the check is advisory-with-friction for at least some of these conditions, not a hard, unbypassable gate.

File Layout: Program Files vs. ProgramData

The installer follows the standard Windows split between read-mostly program binaries and writable per-machine application data (`windows/scripts/Common.ps1`, lines 28-40):

Location	Purpose	Contents
`%ProgramFiles%\IOC Intelligence Platform\app\`	Installed application code	`backend/`, `frontend/`, compose files, docs, wizard/operational scripts
`%ProgramData%\IOC Intelligence Platform\config\.env`	Real runtime secrets and configuration	The generated `.env` consumed by Docker Compose
`%ProgramData%\IOC Intelligence Platform\logs\setup.log`	Install/setup logging	Setup Wizard log output
`%ProgramData%\IOC Intelligence Platform\backups\`	Database backups	`pg_dump` snapshots; the 10 most recent are retained

ACL Protection

`Initialize-DataDirectories` locks the entire `ProgramData\IOC Intelligence Platform\` tree to the Administrators and SYSTEM accounts only, via `icacls` (Windows's command-line ACL — access control list — management tool). This is the only NTFS permission narrowing described in the source material; no per-subfolder differentiation is mentioned beyond this single restrictive ACL applied to the whole tree.

There is one additional wrinkle worth noting precisely because it affects where secrets end up on disk: `docker-compose.yml`'s `env_file: .env` directive resolves relative to the Compose *project* directory, not to whatever path a `--env-file` flag might point at. Because Compose is invoked from inside `{app}\app\`, a helper (`Sync-ComposeEnvFile`) copies the real `.env` from `ProgramData\...\config\.env` into `{app}\app\.env` on every single Compose invocation, and **re-applies the same Administrators+SYSTEM-only ACL to that copy** each time. In other words, there are effectively two on-disk `.env` files with secrets in them by design (the ProgramData original and the app-directory working copy Compose actually reads), and both are kept under the restrictive ACL rather than only the first one.

Secrets themselves are generated using a CSPRNG (cryptographically secure pseudo-random number generator) — specifically .NET's `RandomNumberGenerator` via a `New-RandomSecret` helper, not PowerShell's `Get-Random` — and written out by `Write-EnvFile.ps1`.

Setup Wizard Page Flow

`Setup-Wizard.ps1` is a WinForms application. On launch it self-elevates via UAC (User Account Control) if it isn't already running with a real, non-filtered elevated token. It then walks the operator through a fixed sequence of pages:

Welcome → Admin Account → AI Configuration → Provider Configuration → Port Review → Summary/Install → Finish

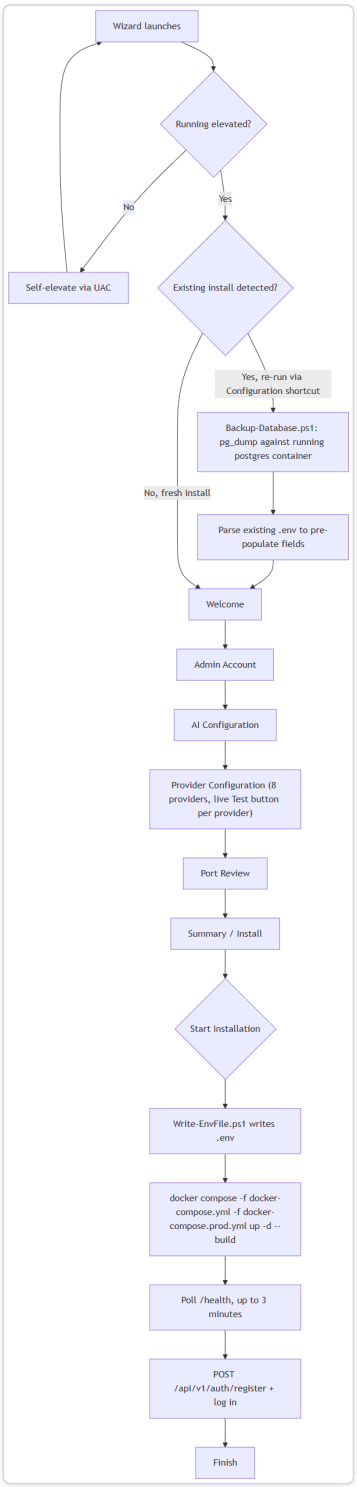

Figure 70 — Diagram: Setup Wizard Page Flow

Notes on individual pages:

- **Admin Account** — collects the credentials for the operator account the installer will register once the stack is up. Per the platform's own registration logic, the *first* account ever registered against a fresh database is automatically granted the admin role, with no manual database edit required; every registration attempt after that is rejected with `403 Forbidden` rather than silently

creating a lesser account. That bootstrap behavior is what makes it safe for this page to simply be "create the admin account" with no separate role picker.

- **Provider Configuration** — presents exactly 8 of the platform's 18 registered intelligence providers, each with a live "Test" button that calls `POST /api/v1/providers/{id}/test` against the running backend: VirusTotal, AbuseIPDB, OTX, the combined abuse.ch group (URLhaus/ThreatFox/MalwareBazaar — one free Auth-Key covers all three), NVD, Hybrid Analysis, Censys, and PhishTank. The wizard's own inline notes match the backend's behavior exactly for the cases checked: Censys requires both a Personal Access Token and an Organization ID, the abuse.ch key is shared across three connectors, and NVD works without a key at a lower rate limit. The remaining 10 backend providers are deliberately absent from this page for two different reasons: 6 (Certificate Transparency lookups, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, and the internal OSINT crawler) require no credential to collect at all, so there is nothing for this page to configure and no corresponding test handler exists for them either; the other 2 (urlscan.io, Google Safe Browsing) **do** require a credential but are still absent from the wizard on both platforms identically — both are configured after install from the app's own Providers page instead.
- **Port Review** — lets the operator confirm/adjust the host ports the stack will bind, following on from the prerequisite check's "ports free" verification.

What "Start Installation" Actually Executes

Clicking "Start Installation" on the Summary/Install page runs, in order:

1. Writes `.env` via `Write-EnvFile.ps1`, using the values collected across the previous pages plus CSPRNG-generated secrets.
2. Runs `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` — the production overlay swaps in production start commands and drops the development bind-mounts.
3. Polls the backend's `/health` endpoint for up to 3 minutes, waiting for the stack to come up.
4. Registers the admin account by calling `POST /api/v1/auth/register` with the credentials from the Admin Account page, then logs in.

Nothing here is a distinct "installer-native" install step beyond orchestrating the same Docker Compose commands and HTTP calls an operator could run by hand — the wizard's value is sequencing and validating them, not replacing them.

Start Menu Shortcuts

The installer's `[Icons]` section creates the following (plus an optional desktop icon):

Shortcut	What it does
Open Platform	Runs `Open-Platform.ps1` (also the default desktop icon action): checks stack health first, starts Docker Desktop if it isn't already running, starts the stack, then opens the browser to the platform.
Configuration	Re-runs the Setup Wizard. On an existing install this sets an upgrade flag (`$State.IsUpgrade`) that changes the wizard's behavior — see Upgrade/Reconfigure below.
Start Platform	Runs `Service-Start.ps1` : `docker compose up -d` , then polls `/health` .
Stop Platform	Runs `Service-Stop.ps1` : `docker compose stop` (no `-v` flag, so container data/volumes are preserved, not deleted).
Restart Platform	Listed in the installer's icon set; the facts available do not detail its script-level implementation beyond the name (presumably a stop-then-start sequence, not confirmed).
Service Status	Listed in the installer's icon set; specific implementation not detailed in the source material beyond the name.
Backup Database Now	Manually triggers an on-demand database backup using the same `pg_dump` -against-the-running-postgres-container mechanism (`Backup-Database.ps1`) that runs automatically before an upgrade (see below).
Diagnostics	Listed in the installer's icon set; specific implementation not detailed in the source material beyond the name.
Documentation	Opens the installed documentation (`docs/` , `README.md`) copied in at install time.
Uninstall	Invoked via `[Code]` in `installer.iss` ; offers "Remove Application" (keeps `ProgramData` and Docker volumes intact) versus "Remove Everything" (requires typing `DELETE` to confirm; runs `docker compose down -v` and deletes `ProgramData`).

Upgrade / Reconfigure Flow

Re-running the Setup Wizard from the **Configuration** shortcut on a machine that already has the platform installed sets `$State.IsUpgrade = true` internally, which changes two things before the operator sees any page:

1. **Pre-population**: the wizard parses the existing `.env` (from `ProgramData\...\config\.env`) and uses it to pre-fill the wizard's fields, so re-running Configuration is a "review and adjust" flow rather than starting from a blank slate.
2. **Pre-install backup**: before anything else changes, the wizard backs up the database by running `Backup-Database.ps1` , which executes `pg_dump` inside the already-running Postgres container. This backup lands in `ProgramData\...\backups\` , alongside the same rolling most-recent-10-retained snapshots used elsewhere.

From there the wizard proceeds through the same page flow (Welcome → Admin Account → … → Summary/Install → Finish) and "Start Installation" re-runs the same `docker compose ... up -d --build` / health-poll sequence described above. The source material does not specify whether the final admin-account registration call is skipped or altered on an upgrade path versus a fresh install — this detail is **not confirmed** and should not be assumed either way without checking the wizard script directly.

Summary

The Windows installer's job is narrowly scoped: verify the host can run the stack, lay down application files under Program Files, collect configuration through a WinForms wizard, generate and lock down secrets under a restrictive ACL in ProgramData, and drive the same `docker compose` commands and REST calls an operator could otherwise run manually. It intentionally leaves Docker Compose as the real runtime rather than replacing it with a native Windows service, and it protects the resulting secrets primarily through NTFS ACLs (Administrators + SYSTEM only) on the `.env` file(s) and CSPRNG-based secret generation, rather than through any OS-level secret store.

Security Architecture

This appendix section documents the authentication, authorization, secrets-handling, network-exposure, rate-limiting, and CORS behavior of the HORIZON GRID backend (the FastAPI service that receives IOC — Indicator of Compromise — lookup requests from the web client). All claims below are traceable to source-code inspection of the platform's backend and Windows installer; anything not directly confirmed in code is marked "Not confirmed" rather than estimated.

Authentication

The backend issues JSON Web Tokens (JWT) using `python-jose` with the HS256 signing algorithm (`backend/app/auth/security.py`). Two token types are issued on login:

Token	Lifetime
Access token	30 minutes
Refresh token	7 days

Password storage uses `passlib` with `bcrypt` hashing — plaintext passwords are never persisted; only the bcrypt hash is stored on the `users` table.

First-user-becomes-admin bootstrap

`POST /auth/register` contains a specific bootstrap rule: **the first user ever registered on a given instance is automatically granted the `admin` role; every registration attempt after that is rejected with `403 Forbidden`** ("Self-registration is closed. Ask an administrator to create your account from the Administration page.") rather than being granted a lesser role. No manual database edit is required to stand up an initial administrator account — this is also how the Windows Setup Wizard provisions the admin account at the end of installation (it calls this same registration endpoint). Every account after that first admin must be created by an existing admin, via `POST /api/v1/admin/users` or the `/admin` frontend page, which lets the admin choose the new account's role up front. `docs/SECURITY.md` describes this identical mechanism.

Role-Based Access Control (RBAC)

The platform has exactly **three roles**: `admin`, `analyst`, and `viewer`. Permissions are modeled as a permission-string matrix (`ROLE_PERMISSIONS` in `app/models/user.py`) and enforced per-route by a `require_permission()` dependency (`app/auth/rbac.py`) — a route either declares a required permission string or it doesn't; there is no implicit fallback check.

Documentation correction made during this review: `docs/SECURITY.md` states that no route enforces the `provider:manage` permission. That claim is stale. The provider-credential test endpoint, `POST /api/v1/providers/{provider_id}/test` (`backend/app/api/routes/providers.py`, line 24), does call `require_permission("provider:manage")` — confirmed directly in the route code. This endpoint is what the Setup Wizard's per-provider "Test" button invokes, so the permission gate is exercised as part of the normal install flow, not a dead code path.

Two other admin-tier permissions defined in the same matrix were checked. One remains unenforced; the other, `audit:read`, is now enforced as of the runtime provider/AI configuration feature described under Secrets Management below:

Permission	Status
`provider:manage`	Enforced — gates `POST /api/v1/providers/{provider_id}/test` and the new `/api/v1/runtime/*` provider- and AI-configuration endpoints
`user:manage`	Defined in the RBAC matrix; no route currently calls `require_permission("user:manage")`
`audit:read`	Enforced — gates `GET /api/v1/runtime/audit-log`

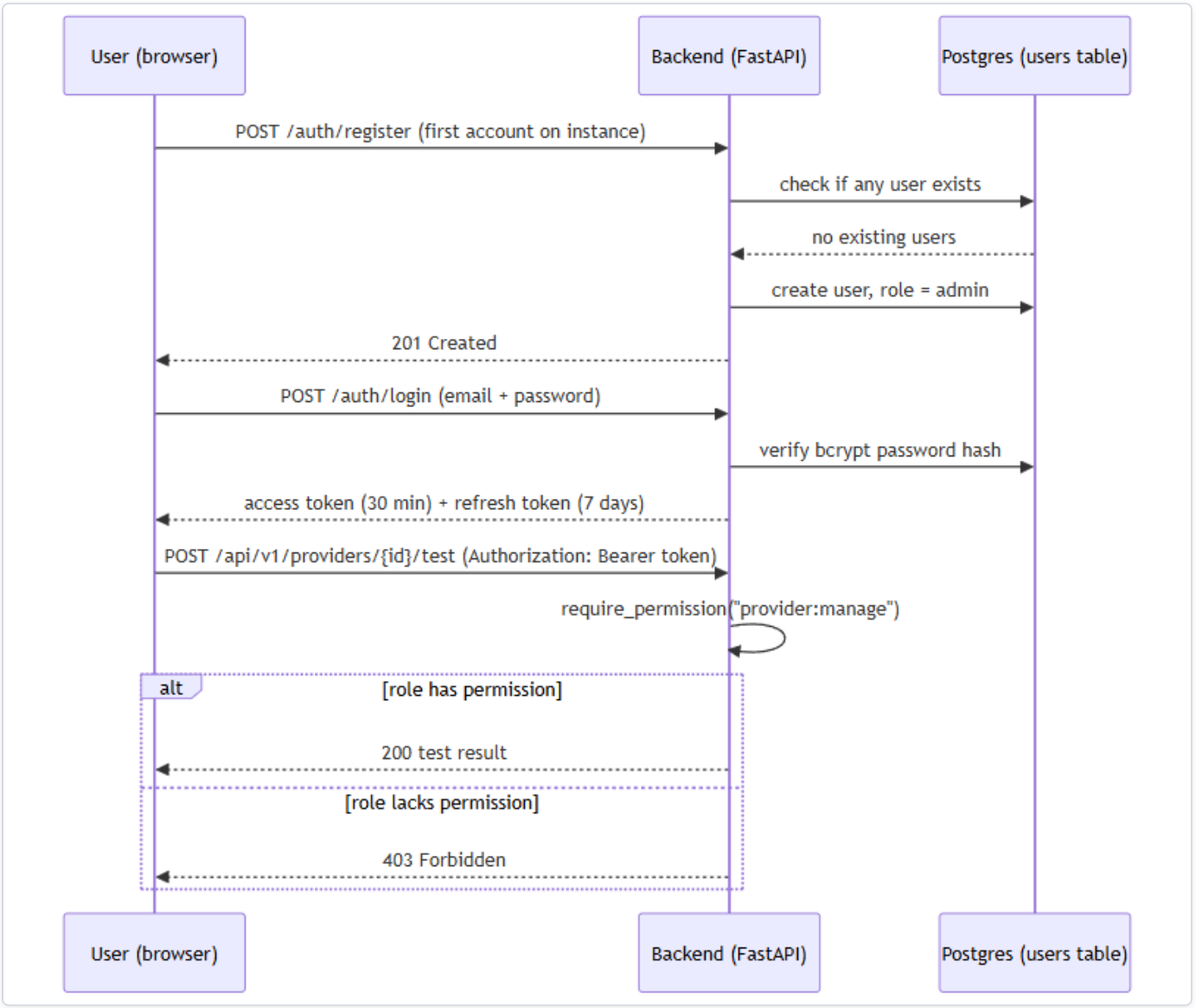

Figure 71 — Diagram: Role-Based Access Control (RBAC)

Secrets Management

Runtime configuration, including secrets, is loaded from a `.env` file via `pydantic-settings`. `.env` itself is git-ignored; only a `.env.example` template is tracked in source control.

One default value is a genuine risk if left unchanged: `jwt_secret_key` falls back to the literal string `"change-me-in-production"` when no value is set (`backend/app/core/config.py` , line 23). Any deployment that ships with this default unedited would have a publicly-known JWT signing secret.

On Windows installs, the Setup Wizard avoids this by generating secrets itself rather than relying on the code default: it uses a cryptographically secure random number generator (`New-RandomSecret` , backed by .NET's `RandomNumberGenerator` — explicitly *not* PowerShell's `Get-Random` , which is not cryptographically secure) and writes the result via `Write-EnvFile.ps1` . The resulting `.env` file's filesystem permissions are then locked down to the Administrators and SYSTEM accounts only, via `icacls` .

Setup Wizard credential-testing reliability fix

A review of the Setup Wizard's per-credential "Test Connection" buttons — used to validate both third-party provider API keys and AI-backend credentials (Ollama, Anthropic, Bedrock, Gemini, Groq) before installation is finalized — found two distinct root causes for unreliable behavior in this flow, both corrected during this review.

Session-acquisition bug. Every Test Connection button's request depends on an authenticated session token (`$State.AccessToken`). That token was previously set in exactly one place in the wizard: deep inside the "Start Installation" button's click handler on the final Summary page. As a result, no credential could be tested until installation had already been fully completed. On a reconfigure run in particular — where an administrator deliberately leaves the Admin Account page blank in order to keep an existing account unchanged — no login request was issued at all, so every Test Connection click failed with a "create the admin account first" message even though a working admin account and platform were already running. The fix is a new function, `Get-OrCreateWizardSession` (`windows/wizard/Setup-Wizard.ps1`), invoked on demand directly from each Test Connection click handler. It performs a login only, using whatever email and password are currently typed into the Admin Account page fields, and never registers or modifies an account.

`.env` **serialization hardening.** A second, independent bug existed in the writer responsible for persisting configuration values to disk, `windows/scripts/Write-EnvFile.ps1` : values were written without any escaping. A `#` character occurring anywhere inside a password or API key was interpreted by the `.env` parser as the start of a comment, silently truncating the remainder of that line, and an embedded newline in a value could similarly corrupt the file's line structure. This is fixed with a new `ConvertTo-SafeEnvValue` helper, applied to every value before it is written.

Neither issue involved the JWT signing or RBAC mechanisms described elsewhere in this document; both were specific to the Windows installer's wizard-to-backend session handling and its own configuration-file writer.

Encryption at rest for runtime-configured provider and AI credentials

A newer capability lets an administrator add, configure, enable/disable, and switch AI-backend and IOC-provider credentials at runtime through a Manage Providers UI, without editing `.env` or restarting the containers. Because this moves credential entry out of the git-ignored `.env` file and into the database, those values are encrypted before they are ever written.

Credentials submitted through this flow are encrypted with Fernet symmetric encryption (`backend/app/core/crypto.py`) and persisted as ciphertext in the `provider_runtime_configs` table's `encrypted_credentials` column — the database never holds a plaintext runtime-configured credential. This is not merely an inference from reading the code: a direct SQL query against the running database showed only Fernet-format tokens (the `gAAAAAB...` prefix), never a real key value.

The Fernet key used for this encryption is resolved one of two ways: an explicit, separately-configured `encryption_master_key` setting, if the operator has set one, or — by default — a key deterministically derived via HKDF from the platform's existing `jwt_secret_key` . The HKDF fallback means every existing installation already has a usable encryption key with no migration step required, but it is worth stating plainly what this tradeoff is: because the fallback key is derived from `jwt_secret_key` rather than generated and stored independently, it is defense-in-depth against exposure of the raw database, not HSM-grade key separation — compromise of `jwt_secret_key` would also expose the derived encryption key. An operator wanting stronger separation should set `encryption_master_key` explicitly.

Every configure, enable/disable, activate, and test-connection action taken against a provider or AI backend through this flow is separately recorded in a new `config_audit_log` table (timestamp, actor email, action, and a human-readable `detail` field). The write path for that table only ever accepts descriptive text as `detail` — it is not a place a credential value can be written. This was checked directly, not just by reading the code: after exercising the full configure/enable/disable/test/activate flow, a grep of the audit log table's contents and of the backend logs found no real secret value in either.

Network Exposure

Every datastore container's port mapping in `docker-compose.yml` — Postgres, Redis, both Neo4j ports, and OpenSearch — uses the explicit `127.0.0.1:<host_port>:<container_port>` binding form, which restricts that port to the loopback interface only (not reachable from other machines on the network as ordinarily configured).

The `backend` and `frontend` containers are published differently: their compose entries use the shorthand form (e.g. `"${HOST_PORT_BACKEND:-8000}:8000"`) with no explicit host-IP prefix. Docker's shorthand publishing binds to all host network interfaces by default — so, unlike the datastores, the backend API and frontend web app are reachable from other machines on the local network unless something else (a firewall rule, router configuration, etc.) restricts that.

Note on rigor: `docker-compose.yml` contains first-person comments asserting that live testing confirmed specific network-reachability behavior from another LAN machine. This section does not rely on that narrative comment as evidence — only the literal, machine-checkable YAML binding syntax described above was treated as verified fact.

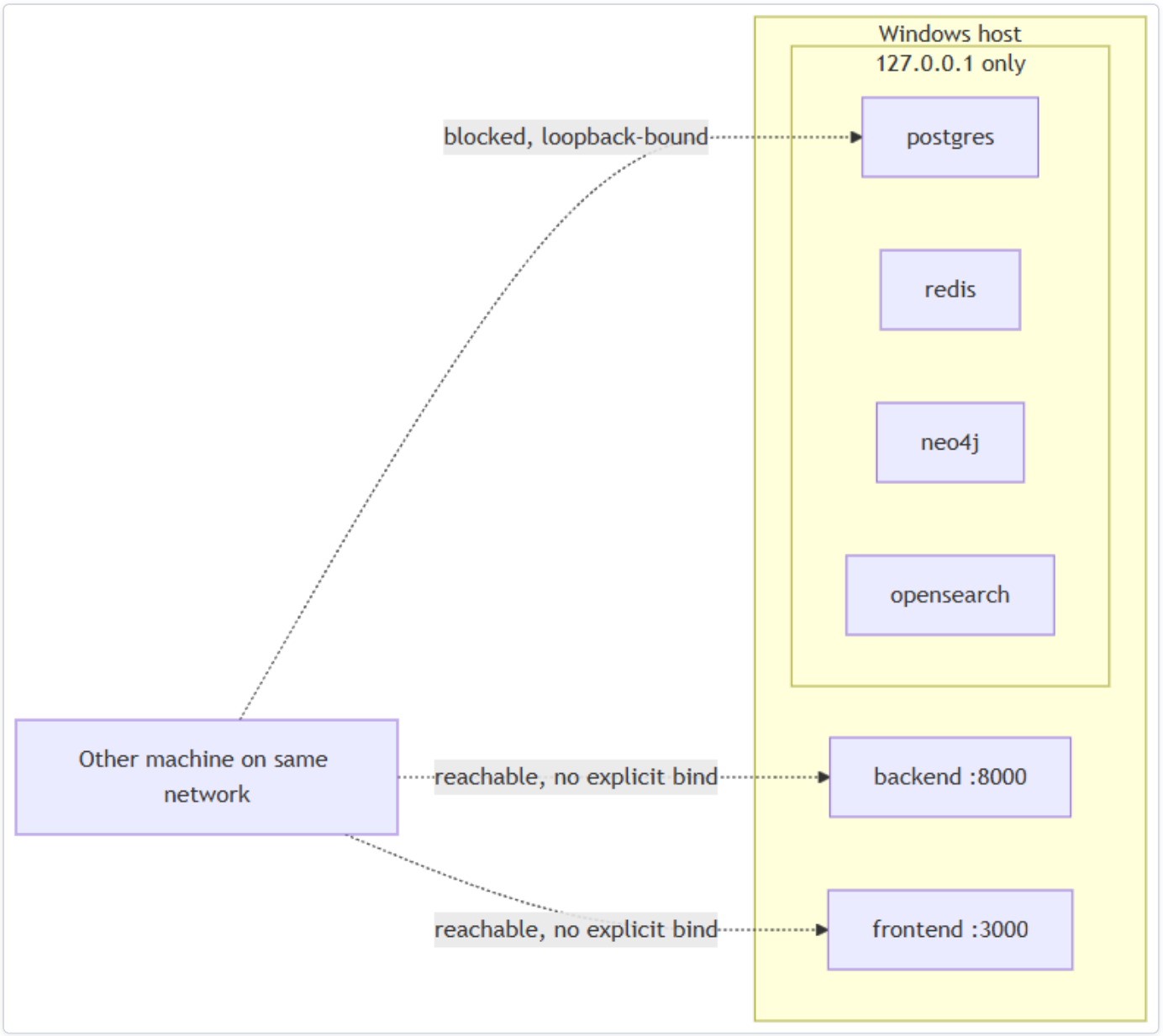

Figure 72 — Diagram: Network Exposure

Rate Limiting

The codebase implements exactly **one** rate limiter: a Redis fixed-window limiter that applies solely to `POST /lookup/stream` (the IOC lookup endpoint), defaulting to 10 calls per 60 seconds per authenticated user — both figures are configurable.

`/auth/login` has a second limiter, added in a later mission-critical-reliability review (v0.2.3): a per-account Redis fixed-window limiter keyed by email (`login:{email}` , not source IP), defaulting to 10 attempts per 60 seconds, both configurable, returning `429` once exceeded. `/auth/register` still has no rate limiting of any kind.

CORS Behavior

Cross-Origin Resource Sharing is configured in `backend/app/main.py` (lines 27-33) and its behavior depends entirely on the `debug` setting:

- When `settings.debug` is `True` (the default), `allow_origins` is set to `["http://localhost:3000"]`.
- When `debug` is `False`, `allow_origins` is set to `[]` — an empty list, meaning **zero** cross-origin browser requests are permitted.

There is no `CORS_ORIGINS` environment variable or other configuration knob to adjust this. Practically, this means a `DEBUG=false` production deployment cannot serve a browser frontend from any other origin than the backend itself without a source-code change.

Other Input-Handling Notes

Two related facts are worth surfacing here because they feed directly into the hardening list below:

- `LookupCreateRequest.value` (the raw IOC string submitted for lookup) is an unconstrained `str` on the server side. Classification into an IOC type happens via regex-based heuristics (`app/ioc/detector.py`), which is pattern classification, not input sanitization.
- `RegisterRequest.password` carries a server-side 8-character minimum (`Field(min_length=8, ...)` , `app/schemas/auth.py`), enforced by Pydantic for every account, not just the bootstrap admin — but no complexity rule (digit/symbol/case) beyond that length floor. The Setup Wizard's own UI also enforces an 8-character minimum when creating the bootstrap admin account, matching the backend constraint rather than substituting for it.

Areas for Future Hardening

Based only on the facts established above:

- **Default JWT secret is a known literal.** `jwt_secret_key` defaults to `"change-me-in-production"` if the operator never sets it explicitly — any instance that ships with this unedited has a predictable signing key.
- **No rate limiting on `/auth/register`.** `/auth/login` gained a real per-account rate limiter in v0.2.3 (see Rate Limiting above); registration has no throttling against repeated attempts.
- **No server-side password complexity rule beyond length.** The backend enforces an 8-character minimum for every account (`app/schemas/auth.py`), but no digit/symbol/case requirement on top of that.
- **One admin-tier permission still has no enforcing route.** `user:manage` is defined in the RBAC permission matrix but, unlike `provider:manage` and `audit:read` , is not currently checked by any route.
- **Backend and frontend ports are not loopback-restricted by default.** Datastores (Postgres, Redis, Neo4j, OpenSearch) are bound to `127.0.0.1` only; the backend API and frontend web app publish on all host interfaces by default, unless the operator adds their own network restriction.
- **Runtime-credential encryption key defaults to a derived, not independently-generated, secret.** Unless an operator explicitly sets `encryption_master_key` , the Fernet key protecting runtime-configured provider/AI credentials (see Secrets Management above) is deterministically derived via HKDF from `jwt_secret_key` rather than generated and stored separately. That is a reasonable defense-in-depth default — it means every existing install already has a working encryption key — but it is not equivalent to HSM-grade key separation: compromise of `jwt_secret_key` would also expose the derived encryption key.

Testing and Quality Assurance

This appendix covers two distinct, non-overlapping bodies of evidence about the quality of HORIZON GRID (a self-hosted tool that looks up an **IOC** — Indicator of Compromise, e.g. an IP address, domain, URL, or file hash — against many third-party threat-intelligence sources at once, then uses a local or cloud AI model to summarize and correlate the results):

1. **Automated test suites** that exist in the source code today (backend unit + integration tests, frontend tooling), confirmed by direct source-code inspection.
2. **Manual/QA end-to-end validation history** — a full install-to-uninstall exercise of the actual compiled Windows installer, with a bug list, fixes, and a final release verdict.

These two bodies of evidence come from different inspection passes and are reported separately below. Where a number (test count, line count, bug count) is stated, it is attributed to its source and not blended with the other source to make the two agree.

1. Automated Test Suites

1.1 Backend

The backend (FastAPI/Python) test suite lives under `backend/app/tests/` and uses **pytest**, **pytest-asyncio**, **pytest-cov**, and **respx** (an HTTP-mocking library), all declared in `requirements.txt`.

Layer	Location	File count	Approx. line count
Unit tests	`backend/app/tests/unit/`	13 files	~1,486 lines
Integration tests	`backend/app/tests/integration/`	3 files	~1,009 lines

Unit tests cover: abuse.ch status mapping, AI schemas/validators, the AI service (including its no-evidence short-circuit guard, described below), `analysis_service` grounding logic, connection-test handlers, the correlation engine, the OSINT crawler's collector/rate-limit logic, the evidence builder, the IOC-type detector, pivot logic, the provider base class, and WHOIS/RDAP handling.

Integration tests are three named files: `test_lookup_flow.py`, `test_lookup_stream_persistence.py`, and `test_api_health.py`. They exercise the real lookup **orchestrator** (the component that fans a lookup out to every applicable data-source connector, called a "provider," concurrently) end-to-end, but against:

- **Fake `BaseProvider` subclasses** standing in for real providers — the real, network-hitting connectors (VirusTotal, AbuseIPDB, etc.) are never called in these tests;
- **respx-mocked HTTP** for any HTTP calls that do occur;
- a **real Redis** instance reached via docker-compose (these tests auto-skip if Redis is not reachable, rather than failing).

With that harness, the integration tests verify: concurrent (parallel, not sequential) fan-out across providers, isolation of a single provider's failure from the rest of the investigation, cache hit/miss behavior, and correct short-circuiting for providers that are unsupported for a given IOC type or not configured (missing API key).

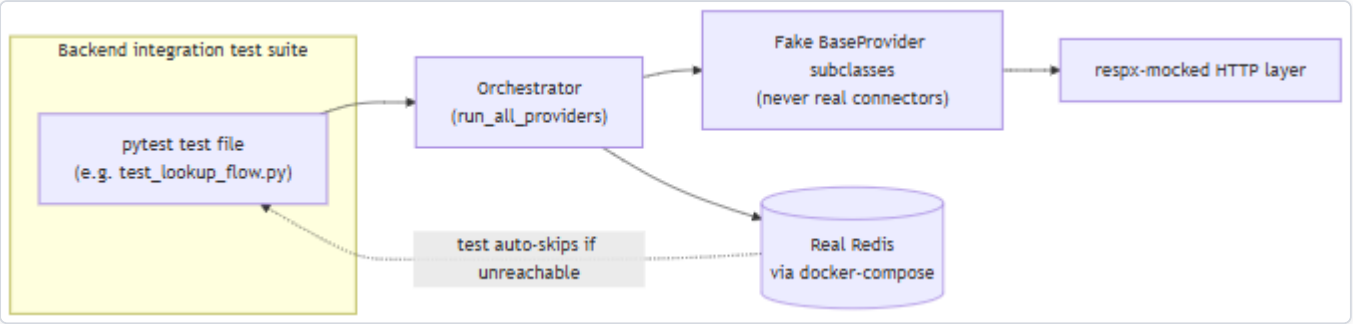

Figure 73 — Diagram: 1.1 Backend

Coverage percentage: Not measured. `pytest-cov` is present as a declared dependency, but no coverage percentage, threshold, or report is asserted anywhere in the source. Only file counts, line counts, and which behaviors the tests exercise are confirmed above.

1.2 Frontend

The frontend (Next.js/React/TypeScript) declares a `"test": "vitest run"` script in `package.json` and lists Vitest as a devDependency — the test *runner* is configured.

However, a repo-wide search for `*.test.*` / `*.spec.*` files under `frontend/` (excluding `node_modules` and `.next`) found zero test files. Stated plainly: the frontend has test tooling wired up but no automated tests actually exist. This is a real, reportable gap in the current build — running `npm test` in the frontend would execute Vitest against an empty test set.

1.3 Summary

Area	Framework(s) configured	Test files that actually exist	Coverage %
Backend unit	pytest, pytest-asyncio, pytest-cov	13	Not measured
Backend integration	pytest, respx, real Redis	3	Not measured
Frontend	Vitest (declared)	0	Not measured (no tests to measure)

1.4 Regression Tests for the API-Key / AI-Backend Configuration Fix

A follow-on engineering pass fixed a bug where entering, saving, and testing provider and AI-backend API-key credentials in the Windows setup wizard was unreliable — every "Test Connection" button required a session token that had previously only ever been set deep inside the final "Start Installation" step, so a credential could never be tested before a full install completed, and never at all on a reconfigure run where the admin-account fields were left blank to preserve an existing account. The same pass added Groq as a fifth AI backend alongside Ollama, Anthropic, Bedrock, and Gemini, and introduced a live connection-test endpoint shared by all five backends.

That fix shipped with a new regression-test file, `backend/app/tests/unit/test_ai_connection_test.py`, adding **16 automated tests** that exercise the connection-test logic for all five AI backends that existed at the time using **mock/fake credentials only** — no real API key ever appears in the test suite — with **respx**-mocked HTTP responses standing in for the real provider APIs. The 16 tests covered, across those five backends: a successful ping, an invalid-key/401 response, a rate-limit/429 response, a model-not-found/404 response, a request timeout, and a network error, each expected to be reported as its own specific, honest failure category rather than a generic error. With this file added, the full backend automated suite was reported as **144 tests total, passing with no regressions** at that point in the project's history — a count reported by this follow-on pass itself, distinct from (and not necessarily contemporaneous with) the 13-file/~1,486-line source-inspection figures in Section 1.1 above.

Current state (v0.2.3): the same file has since grown alongside the AI backend roster — six more backends (OpenAI, Kimi, DeepSeek, xAI, Mistral, OpenRouter) joined the original five, each with its own block of backend-specific test cases (including a few genuinely backend-specific scenarios: Kimi's forced-tool-call-vs-thinking-mode conflict, DeepSeek's provider-specific HTTP 402, xAI's flat non-nested error body). `test_ai_connection_test.py` now contains **53 test functions** covering all eleven backends. See `TEST_EVIDENCE_CURRENT_VERSION.md` for this release's authoritative, actually-executed full-suite numbers rather than the historical 144 figure above, which predates most of this project's test growth.

2. Manual / QA End-to-End Validation History

Separately from the automated suites above, a full manual QA pass was performed against the actual compiled Windows installer (`IOC-Intelligence-Platform-Setup-0.1.0.exe`, version 0.1.0, ~62.6 MB final build, SHA256 `b32b1cc02205a0a53dda200d6bc336943a71df6b930cecd2dff45b28d84e81e8`), dated 2026-08-11, and documented in `FINAL_END_TO_END_TEST_REPORT.md`. This pass treated the build as never-before-tested and covered install, configuration, normal use, deliberate failure injection, a real Windows reboot, uninstall, reinstall, and a final independent clean install — exercised against the real installer and running containers, not source/dev-server shortcuts.

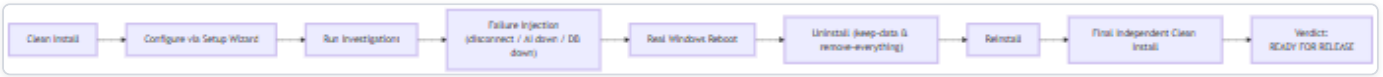

Figure 74 — Diagram: 2. Manual / QA End-to-End Validation History

2.1 Final Verdict

READY FOR RELEASE. The report's stated justification: every defect found — including the single most severe one (AI evidence-free fabrication, below) — was root-caused, fixed, covered by a regression test, and re-verified against the real running product. The three highest-severity findings (AI fabrication, LAN-exposed unauthenticated datastores, and silent data loss on client disconnect) were all closed with direct, live re-confirmation. Remaining open items are explicitly scoped-out unbuilt features (Report generation, PDF/CSV export, Timeline) and small-model AI-quality limitations, not incorrect or unsafe behavior.

2.2 Specific Bugs Found and Fixed

#	Severity	Area	Description
1	High	AI safety	Fabricated `final_verdict: "highly_malicious"`, `malicious_probability: 92`, and invented "association with ransomware and trojans" for the EICAR test hash (`44d88612fea8a8f36de82e1278abb02f`) queried against zero real provider evidence — the small local model answered from its own pretrained knowledge instead of refusing.
2	High	Security	Postgres, Redis, Neo4j, and OpenSearch were all published on `0.0.0.0` (every network interface), reachable unauthenticated from another device on the same LAN — confirmed live via the machine's real LAN IP. OpenSearch had no authentication at all.
3	High	Data integrity	A client disconnect mid-investigation (simulated via `curl --max-time 3`) silently discarded all already-fetched provider results from the database, even though six providers (including a live Spamhaus hit) had already completed and been streamed to the client.
4	High	Recovery UX	The "Open Platform" shortcut was a bare `.url` file with no check for whether Docker/containers were running, no auto-start, no health wait — directly failing the requirement that the shortcut verify services, start if needed, and open the browser automatically. Caught by the real post-reboot test.
4b	Medium	Regression in fix #4	The launcher's own fix checked whether the platform was configured (via `Test-Path` on an ACL-locked `.env` file) *before* elevating privileges, which threw an access-denied error that read as "not configured" for an already-fully-configured install.
5	Medium	AI correctness	`evidence_ids`, `agreeing_providers`, and `disagreeing_providers` structured fields came back empty even when the AI's own prose cited a specific evidence ID or named a specific provider — because some schema fields lacked field descriptions, and the grounding filter only recognized providers that produced a correlation-graph edge (excluding the OSINT crawler provider, `internet_intelligence`, which does neither).
6	Medium	AI correctness	A provider's raw `"listed": false` boolean field was misread by the AI summarization step as "the IOC **is** listed," inverting a clean result into a false "has been blocked" claim.
7	Medium	Configuration	`docker-compose.yml` hardcoded `OLLAMA_BASE_URL` in the `environment:` block, which always overrides `env_file:`, so the setup wizard's AI Configuration page could write the correct value to `.env` but the platform would never actually use it.
8	Medium	Data/feature gap	`GET /api/v1/lookup/{id}` never returned the persisted relationship-graph data, so revisiting a completed investigation always showed an empty graph even though `CorrelationEdgeRecord` rows existed in Postgres.
9	Low	Installer UX	Five WinForms label-overlap/text-clipping bugs across the Administrator Account, AI Configuration, Threat Intelligence Providers, and Network Ports wizard pages.
10	Low	Installer UX	The account-creation failure path dumped a raw FastAPI/Pydantic validation-error JSON blob into the user-facing log and unconditionally printed "Setup complete." even when the admin account was never created.
11	Low	Cosmetic	Routine `docker compose` stderr progress output was rendered as a fake red "NativeCommandError" on every Start/Stop/Restart, even on full success.
12	Low	Branding accuracy	The landing page hardcoded "Claude" as the AI backend name regardless of which backend was actually configured.

Bugs #1–12 (including 4b) were all root-caused, fixed, covered by a regression test where the fix was in backend Python code, rebuilt into a real running instance, and re-verified live against the actual product. Per the QA report, the regression tests added for these fixes brought the backend unit-test suite to **128/128 passing** at the time of that pass, plus the targeted stream-persistence integration tests — a count reported by the QA pass itself, distinct from (and not necessarily contemporaneous with) the 13-file/3-file source-inspection counts in Section 1 above. One named regression test is called out specifically: `test_completed_lookup_returns_rebuilt_correlation_graph`, added for bug #8.

2.3 Findings Documented but Not Fixed (Not Blocking Release)

#	Severity	Area	Description
13	Info	AI quality	A detection-rule-generation call mapped MITRE ATT&CK technique `T1053` to "Create a Backdoor"; the real T1053 is "Scheduled Task/Job." Documented as a limitation of the bundled small local model (`llama3.2:3b`), not fixed.
14	Info	AI quality	The OSINT crawler provider occasionally surfaces keyword-matched but semantically unrelated content, and the AI summarization step sometimes repeated it as if it were a specific finding. Confidence/verdict fields stayed appropriately cautious in every case observed; judged a data-quality/polish issue rather than a safety-relevant fabrication.
15	Info	Feature scope	Report generation does not exist (backend field is permanently empty, no frontend UI). PDF and CSV export are wired to a backend export endpoint that returns a genuine 404; the frontend shows a graceful "not yet available" message rather than crashing (JSON and Markdown export do work end-to-end). Timeline does not exist anywhere in the codebase. All three are unbuilt features, not defects.

2.4 Failure-Recovery Testing

Beyond the bug list, the QA pass deliberately injected failures and confirmed recovery:

- **AI backend down**: Ollama made genuinely unreachable (bad `OLLAMA_BASE_URL`, confirmed via `docker exec ... printenv`). Provider data stayed intact; AI-dependent fields honestly reported the failure (`final_verdict: "unknown"` , risk scores zeroed) rather than pretending to succeed. Reverting restored full functionality with no other intervention.
- **Database down**: stopping the Postgres container mid-request produced a real, fully logged HTTP 500 (not a silent failure or corruption); restarting Postgres restored functionality with prior data intact.
- **Bulk workflow**: eight IOCs processed sequentially (mixed types, one duplicate, one malformed value). The malformed entry was cleanly rejected with HTTP 422; the duplicate correctly created an independent investigation record (investigations are not deduplicated by design) while still benefiting from provider-level caching.
- **Real Windows reboot**: a genuine `Restart-Computer` was executed (with user approval) on the tester's own workstation; post-reboot, login, prior case/watchlist data, and a brand-new investigation all worked without any manual component restarts — this is the scenario that surfaced bugs #4 and #4b above.
- **Uninstall/reinstall**: both the keep-data and "Remove Everything" (`docker compose down -v`) uninstall paths, plus a reinstall and a fully independent final clean install, were each verified against real data (existing cases, watchlist entries, admin login all confirmed intact where expected).

2.5 Performance Observations (as measured in that pass)

Idle-baseline container memory was recorded as: backend ~96 MB, frontend ~39 MB, celery_worker ~695 MB, postgres ~38 MB, redis ~5 MB, neo4j ~553 MB, opensearch ~1.07 GB, all containers under 1% CPU at rest. After an eight-IOC bulk run, backend memory grew to ~104 MB (attributed to normal request handling, not a leak) and settled; no other container changed. A typical single-IOC investigation completed in 5–15 seconds end-to-end, with the OSINT crawler's multi-source fan-out consistently the slowest single component (~6 seconds). These are the specific figures reported by that one QA pass on that one machine, not a benchmark suite, and are not restated as general performance guarantees.

2.6 Live End-to-End Verification of the API-Key / AI-Backend Fix

Beyond the automated regression tests in Section 1.4, the same fix was also verified live against the real running product, start to finish: a full uninstall, a wipe of the database and Docker volumes, a fresh install, a complete run through the setup wizard, a fresh administrator-account bootstrap, and a live IOC investigation all passed. A genuine container-restart persistence test was performed separately: after a real container restart, admin login and all prior investigation records were confirmed to have survived intact. Finally, a live side-by-side comparison ran one identical IOC through both the Ollama and Groq AI backends to compare the quality of each backend's AI-generated output against identical underlying provider evidence. As with the rest of Section 2, these are the results observed in that specific verification pass, not a repeatable benchmark.

3. What This Means Together

The two bodies of evidence answer different questions and should not be conflated: Section 1 shows that the codebase has a real, if partial, automated regression safety net on the backend (13 unit-test files, 3 integration-test files exercising the orchestrator against fake providers plus real Redis) and none yet on the frontend. Section 2 shows that, independent of those automated tests, a full manual lifecycle pass against the actual shipped installer found and fixed 12 real defects — including one severe AI-safety issue (evidence-free fabrication of a malicious verdict) and one severe security issue (unauthenticated datastores reachable from the LAN) — and reached a "ready for release" verdict with two remaining categories of known, documented, non-blocking gaps: small-model AI-quality quirks, and three genuinely unbuilt features (Report generation, PDF/CSV export, Timeline).

Performance

An **IOC** (Indicator of Compromise) is the value a user submits for lookup — an IP address, domain, file hash, URL, CVE ID, etc. A **provider** is a connector to an external threat-intelligence source (VirusTotal, AbuseIPDB, and so on) queried for a given IOC. A lookup runs over **SSE** (Server-Sent Events) — a single streamed HTTP response that pushes result events to the client as they become available, rather than one request/response per step.

This section reports two different kinds of number, and is explicit about which is which: (1) values actually observed during the one documented end-to-end QA pass (`FINAL_END_TO_END_TEST_REPORT.md`, run 2026-08-11 on a single Windows 11 workstation), and (2) hard-configured limits read directly from the backend code, which bound worst-case behavior even though no live measurement of hitting those limits exists. With one exception — the Executive Dashboard / Provider Health endpoint load test described later in this section — no dedicated load-testing, benchmarking, or repeated-trial statistical harness exists for this platform as of this writing; the rest of the QA pass was a single functional/regression walkthrough (clean install through uninstall/reinstall), not a performance study.

Scope and Methodology

- The only performance data available comes from **one** test pass on **one** machine, observed manually, not from an automated benchmark suite or multiple repeated trials.
- The bulk-load exercise referenced below was **eight** sequential IOC lookups, not a stress test — it was designed to check for resource leaks during a routine batch, not to find a breaking point or measure throughput.
- Where the two source documents do not contain a number for something a reader would reasonably want to know (e.g., percentile latencies, throughput, total install duration), the table below says "Not measured" rather than supplying an estimate.

Measured and Configured Values

Every row below is either a number actually observed during the single documented test pass or a hard-configured limit found in the code; rows for commonly-expected performance metrics that neither source actually measured (percentile latency, throughput, install duration) are marked "Not measured" rather than estimated.

Metric	Value	Basis	Source
Single-IOC investigation duration, end-to-end (SSE stream open to `done` event)	5–15 seconds, varying by which providers make real network calls	Measured (one test pass)	`FINAL_END_TO_END_TEST_REPORT.md` §12
OSINT crawler (`internet_intelligence` provider) fan-out time	~6 seconds — consistently the slowest single component observed	Measured (one test pass)	`FINAL_END_TO_END_TEST_REPORT.md` §12
p50 / p95 / p99 lookup latency	Not measured	—	—
Throughput (concurrent lookups per second/minute)	Not measured	—	—
`backend` container idle memory (RSS)	~96 MB	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`frontend` container idle memory	~39 MB	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`celery_worker` container idle memory	~695 MB	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`postgres` container idle memory	~38 MB	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`redis` container idle memory	~5 MB	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`neo4j` container idle memory	~553 MB (provisioned container with no consuming backend code — see architecture-facts.md §2/§5/§9)	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`opensearch` container idle memory	~1.07 GB (provisioned container with no consuming backend code — see architecture-facts.md §2/§5/§9)	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
CPU utilization across all eight containers at idle	<1%	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
`backend` memory after an 8-IOC sequential bulk run	~104 MB (~9% increase over idle baseline; the tester attributed this to normal request handling, not a leak)	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12
Leak/stability check across the bulk run	No memory-leak pattern, no CPU pegged at 100%, no zombie processes observed (qualitative, one run)	Measured	`FINAL_END_TO_END_TEST_REPORT.md` §12/§11
Installer/wizard total wall-clock duration	Not measured	—	—
Setup-wizard health-check poll ceiling after `docker compose up -d --build` (before admin registration)	Up to 3 minutes (a ceiling the wizard will wait, not an observed typical duration)	Configured limit	architecture-facts.md §6
"Open Platform" launcher: Docker Desktop cold-start wait ceiling	Up to 3 minutes (a ceiling the launcher will wait, not an observed typical duration)	Configured limit	`FINAL_END_TO_END_TEST_REPORT.md` §5
Post-reboot interactive Windows logon delay observed during the one real-reboot test	~18 minutes after boot completed (an OS-level startup-throttling effect on the test workstation, not a product behavior —	Measured (single	`FINAL_END_TO_END_TEST_REPORT.md` §10

Metric	Value	Basis	Source
	noted because the launcher has to tolerate it)	observation, environmental)	
Per-provider call timeout (`provider_timeout_seconds`)	20 seconds	Configured limit	architecture-facts.md §9
Per-provider retry policy (`provider_max_retries`)	2 retries, exponential backoff (`wait_exponential(multiplier=0.5, max=4)`, via tenacity)	Configured limit	architecture-facts.md §9
Provider-result cache TTL (`provider_cache_ttl_seconds`), Redis-backed	3600 seconds (1 hour)	Configured limit	architecture-facts.md §2/§9
AI-backend client timeout, Ollama (`_TIMEOUT_SECONDS`) — applies only when `ai_backend=ollama` (the default); no equivalent hardcoded value is documented for the Anthropic/Bedrock/Gemini clients	120 seconds, hardcoded	Configured limit	architecture-facts.md §9
Lookup-creation rate limit, Redis fixed-window, per user	10 calls / 60 seconds	Configured limit	architecture-facts.md §2/§9

Where These Limits Apply in the Lookup Pipeline

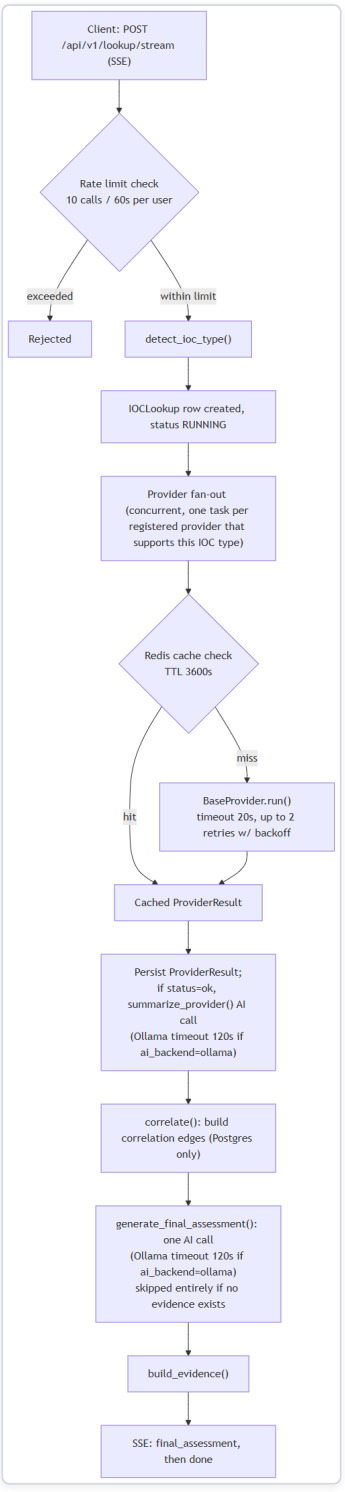

Figure 75 — Diagram: Where These Limits Apply in the Lookup Pipeline

Dashboard and Provider Health Endpoint Load Testing

Unlike the rest of this section, the Executive Dashboard (`GET /api/v1/dashboard/kpis` , `GET /api/v1/dashboard/executive-summary`) and Provider Health (`GET /api/v1/providers/health`) endpoints received a dedicated, real concurrent-load test — a repeated-trial exercise, not a single-pass manual walkthrough.

The first such test found a genuine complete-failure mode: 25 concurrent requests to the Provider Health endpoint timed out 100% of the time. Two real bugs were found and fixed as a direct result:

1. **Excessive sequential database round-trips.** The endpoint's aggregation code was issuing on the order of 300 sequential, awaited database round-trips in a single request (multiple metrics across multiple time windows, for every registered provider). This was consolidated into a small number of aggregate queries per request, using SQL conditional aggregation grouped by provider so the database computes every window's numbers for every provider in one round-trip instead of one round-trip per provider per window per metric.
2. **Database connection-pool sizing that didn't account for real per-request connection usage.** The pool had been sized around the ORM's un-tuned defaults, which didn't account for a single authenticated request actually holding two simultaneous connections at once (one for the authentication dependency, one opened separately by the endpoint's own service logic). The pool was resized to account for that.

After both fixes, the same 25-concurrent-request test against the Provider Health endpoint was re-run live: it went from 100% timeout to 100% success, completing in under 2 seconds — a complete reversal of the earlier failure, not just an incremental improvement.

A separate, honestly disclosed characteristic, not a bug: the AI-generated executive summary can become significantly slower under heavy *concurrent* load specifically when the platform is configured to use a local AI backend (e.g. Ollama) — a single local model processes generation requests one at a time, so concurrent requests for the executive summary queue up behind one another. This does not affect data correctness: the underlying KPI numbers behind the summary are always the current, real, deterministic values, regardless of how long the AI narrative itself takes to generate. It also does not affect the Dashboard's core KPI tiles or the Provider Health page, both of which are pure database reads, unaffected by AI backend choice or load. An administrator expecting many concurrent users should consider a cloud AI backend (Claude/Bedrock/Gemini/Groq) rather than a local model, for a consistently fast executive summary under concurrent load.

Caveats

- **No formal performance-testing infrastructure exists, with one exception.** Every measured value above comes from a single manual QA pass on a single workstation, not from a repeatable benchmark harness, and the "bulk" exercise it drew from was eight sequential lookups — enough to sanity-check for leaks, not to establish throughput or latency distributions. Treat the measured rows as one anecdotal data point each, not as guaranteed or typical performance. The one exception is the dedicated concurrent-load test against the Dashboard/Provider Health endpoints described above, which was a genuine repeated-trial (25 concurrent requests) exercise rather than a single manual pass.
- **A meaningful share of idle memory is currently unused infrastructure, not active workload.** Neo4j (~553 MB idle) and OpenSearch (~1.07 GB idle) are provisioned containers with no consuming backend code (see architecture-facts.md §2/§5/§9) — together they account for roughly 1.6 GB of the platform's idle memory footprint while doing no work for the running system today.
- **The two "3-minute" figures are ceilings, not typical waits.** Neither source document reports how long the setup wizard's post-install health poll or the "Open Platform" launcher's Docker-Desktop-startup wait actually took in the tested runs — only the configured maximum each will tolerate before giving up or erroring.
- **The Ollama 120-second timeout is backend-specific.** It applies when the platform is configured to use the default local `ollama` AI backend (model `llama3.2:3b`); the facts reviewed do not document an equivalent hardcoded timeout for the alternative `anthropic`, `bedrock`, or `gemini` backends.

Limitations

This appendix is a code-verified list of HORIZON GRID's known limitations, compiled by direct inspection of the backend source rather than by restating aspirational design documents. An **IOC** (Indicator of Compromise) is the value a user submits for lookup — an IP address, domain, URL, file hash, CVE identifier, and so on. A **provider** is the platform's term for one integrated external intelligence source (e.g., VirusTotal, AbuseIPDB) that the lookup **orchestrator** queries in parallel for a given IOC. Where the platform's own documentation (`docs/SECURITY.md`) or design docstrings describe something that the code does not actually do, that gap is stated plainly below rather than smoothed over.

Provider Limitations

The platform registers 18 providers (`backend/app/providers/registry.py`), including urlscan.io and Google Safe Browsing (both configured after install, not via the setup wizard — see the Provider Guide). Several have quirks, verified directly in their connector code, that affect what results a user can expect from them.

Provider	Quirk	Verified detail
Censys	Dual-credential requirement	`configured` requires **both** a Personal Access Token **and** an Organization ID; either alone leaves the provider `NOT_CONFIGURED` (`app/providers/stubs/censys.py` lines 30-35).
NVD (NIST)	Key is optional, not required	`requires_key = False` , `configured = True` unconditionally. An API key does not unlock the provider — it only raises the unauthenticated rate limit (~5 requests/30s) to ~50 requests/30s (`app/providers/nvd.py` lines 24-27).
URLhaus / ThreatFox / MalwareBazaar (abuse.ch)	Single shared key across three providers	All three connectors read the same `abusech_auth_key` setting. A missing or invalid key is deliberately mapped to `ProviderStatus.ERROR` , not `NO_DATA` — so a misconfigured key cannot be mistaken for "this IOC is clean" (`abusech.py` lines 6-8).
Hybrid Analysis	SHA256-only sandbox lookups	The connector's own code comment states the VirusTotal-style hash-search endpoint (`/api/v2/search/hash`) is deprecated (HTTP 410), and that URL-search support was removed after every plausible request-body field name failed against the live API. MD5, SHA1, and URL lookups are therefore unsupported (`stubs/hybrid_analysis.py` lines 6-17).
PhishTank	Intermittent 403s are treated as inconclusive	PhishTank is documented to sometimes return HTTP 403 to any generic client regardless of key validity. The platform's connection test treats a 403 as inconclusive/"ok" rather than a hard authentication failure, since PhishTank never requires a key in the first place (`connection_test.py` lines 158-183).

Beyond individual quirks, every provider call is bounded — but not eliminated — by a uniform timeout and retry policy: `provider_timeout_seconds` (20s) and `provider_max_retries` (2), with exponential backoff (`wait_exponential(multiplier=0.5, max=4)`) (`app/core/config.py` lines 89-92). A provider result is also cached in Redis for `provider_cache_ttl_seconds` (3,600s / 1 hour) to reduce repeat outbound calls (`app/core/cache.py`).

Internet / provider-availability dependency. Most of the 18 providers are external third-party services reached over the internet, so a lookup's completeness depends on those services (and the host machine's network path to them) being reachable at query time. This extends to the two providers backed by periodically-refreshed cached catalogs — `cisa_kev` and `mitre_attack` , both cached in-memory for a 1-hour TTL — and to the `internet_intelligence` provider, which wraps an OSINT crawler over GitHub, Reddit, RSS, and pastebin sources (`app/crawler/`). Because providers run concurrently rather than sequentially (`app/providers/orchestrator.py`), one unreachable or rate-limited provider does not block the others, but its own result for that lookup will be missing or marked as an error rather than substituted with any cached or inferred data.

AI Limitations

The platform's AI layer defaults to a small local model (`llama3.2:3b` via Ollama, no API key, no internet dependency for inference itself) rather than a large hosted model; `anthropic` , `bedrock` (AWS), `gemini` , and `groq` are also supported as alternative backends, all of which do require internet connectivity and a configured credential (`app/core/config.py` line 69; client files in `app/ai/`).

Known characteristic: a small local model can fabricate findings if not constrained. The code comments document a specific, reproduced failure: looking up the real MD5 hash of the EICAR antivirus test file (`44d88612fea8a8f36de82e1278abb02f`) against **zero configured providers** — i.e., with no actual evidence — still caused `llama3.2:3b` to return `final_verdict="highly_malicious"` and `malicious_probability=92` , fabricating an "association with ransomware and trojans" purely from its own pretrained knowledge, in direct violation of its own system-prompt instruction never to fabricate (`service.py` lines 242-284). This is stated here as a documented, actively-mitigated characteristic of the current small-model default, not a hidden defect. The engineering response was not to trust prompt wording alone but to add deterministic, code-level guards around the model's output:

- **No-evidence short-circuit guard** — if both `provider_summaries` and `correlation.edges` are empty, `generate_final_assessment()` returns a hardcoded, deterministic result (`final_verdict=Verdict.UNKNOWN` , all risk fields `0`) **without calling the AI backend at all** (`service.py` lines 242-284).
- **Grounding pass (`_ground_final_assessment()` , `service.py` lines 137-190)** — cross-checks the AI's own structured output against the deterministic correlation data: it silently strips any `agreeing_providers` / `disagreeing_providers` entry that isn't backed by a real correlation edge, provider-agreement record, or per-provider summary, and marks `mapping.grounded = False` on any cited MITRE ATT&CK technique not backed by a real `uses_technique` correlation edge.
- **Evidence-citation stripping/backfill (`analysis_service.py` , lines 55-112)** — `_strip_invalid_evidence_ids()` removes any `evidence_ids` citation on the analyst-facing explanation endpoints (WHY malicious, Challenge/red-team, Copilot Q&A, and others) that doesn't match a real `EvidenceItem.id` for that lookup; `_backfill_evidence_ids_from_prose()` only ever recovers citations already confirmed real, never invents new ones.
- **Hunting/detection-rule generation (`hunting_service.py` lines 74-79)** — explicitly strips any invented expansion target the model produces before it reaches the user.
- **Score-scale and consistency validators** — `RiskAssessment` fields are defined on a 0-100 scale, with a `field_validator` that auto-rescales any 0-1-range value by x100, added because `llama3.2:3b` was observed defaulting to a 0-1 convention (`app/ai/schemas.py` lines 101-131); a separate `model_validator` rejects self-contradictory combinations such as `final_verdict="malicious"` with `malicious_probability < 30` (schemas.py lines 187-208).

A related operational limitation: the Ollama client has a hardcoded 120-second timeout (`ollama_client.py` line 35), with a code comment noting that a model which has spilled from VRAM to CPU/mmap has been observed taking several minutes per response — i.e., local-model latency is not bounded tightly and can occasionally approach the timeout. (The full AI grounding pipeline, including both call types and every safeguard above, is described in the "AI Architecture (Technical)" appendix section; this section states it specifically as a limitation-and-mitigation pair.)

Groq backend: a real free-tier rate limit. Unlike Ollama, the hosted `groq` backend (`app/ai/groq_client.py`) depends on Groq's own API quota. At the time of testing, Groq's free tier enforced a per-minute token limit of 12,000 tokens/minute, and a real investigation was observed to hit it under only moderate use — this is a genuine constraint an analyst can run into, not a theoretical one. The platform's handling of it was verified directly rather than assumed: the resulting HTTP 429 did not crash the request, `executive_summary` was correctly set to a plain "AI-generated assessment unavailable (generation error)" message, and the `ai_backend` / `ai_model` traceability fields (`app/ai/schemas.py` , populated in `service.py`) were correctly left `null` rather than fabricated with a fake success. A judge or analyst running a burst of investigations against Groq's free tier should expect occasional 429s and brief retries as normal behavior, not a platform defect.

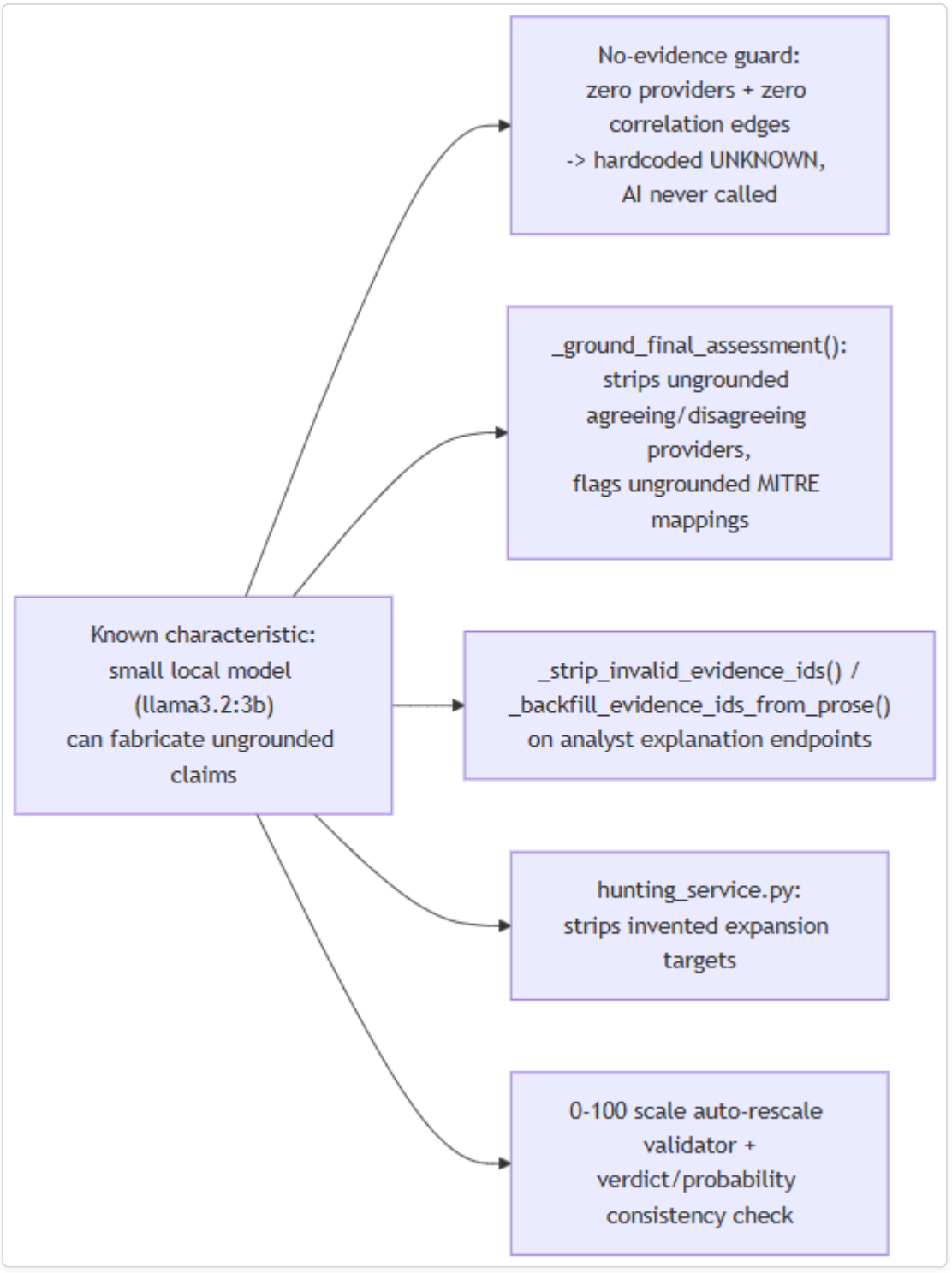

Figure 76 — Diagram: AI Limitations

Data Layer Limitations

Postgres is the only datastore actually written to during a lookup today. Lookups, provider results, AI summaries, correlation edges, evidence items, cases, and users all live there (`backend/app/models/`).

Neo4j and OpenSearch are provisioned infrastructure with no consuming application code. Both run as containers in `docker-compose.yml` (`neo4j:5-community` with the APOC plugin; `opensearchproject/opensearch:2.17.0`) and both have configuration fields already defined (`neo4j_uri` / `neo4j_user` / `neo4j_password` , `opensearch_url` in `app/core/config.py`). `CorrelationEdgeRecord` 's model docstring and the correlation engine's own module docstring describe correlation edges as being "mirrored into Neo4j" — but a full-repo grep of `backend/app/**/*.py` for `neo4j` / `Neo4j` / `GraphDatabase` found **zero** driver instantiation, zero Cypher queries, and zero read/write calls anywhere in the application code. The same is true for OpenSearch: the container runs, but zero indexing or search client code exists in `backend/app` . Concretely, this means: the relationship graph a user sees is rebuilt from Postgres's `correlation_edges` table on every read, not served from a graph database; and there is no full-text search functionality anywhere in the product today. Anything describing a Neo4j-backed graph store or OpenSearch-backed search as a working feature would misstate the current build — both are architecturally provisioned, not yet integrated.

Redis has exactly three confirmed real uses, and no others were found: the provider-result cache (keyed `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` , TTL 3,600s), the fixed-window rate limiter (both the per-user limiter on lookup creation and the per-account `login:{email}` limiter added in v0.2.3 share the same `RateLimiter` mechanism, just different key prefixes), and the Celery broker (logical DB `/1`) and result backend (`/2`), with the cache using a separate logical DB (`/0`).

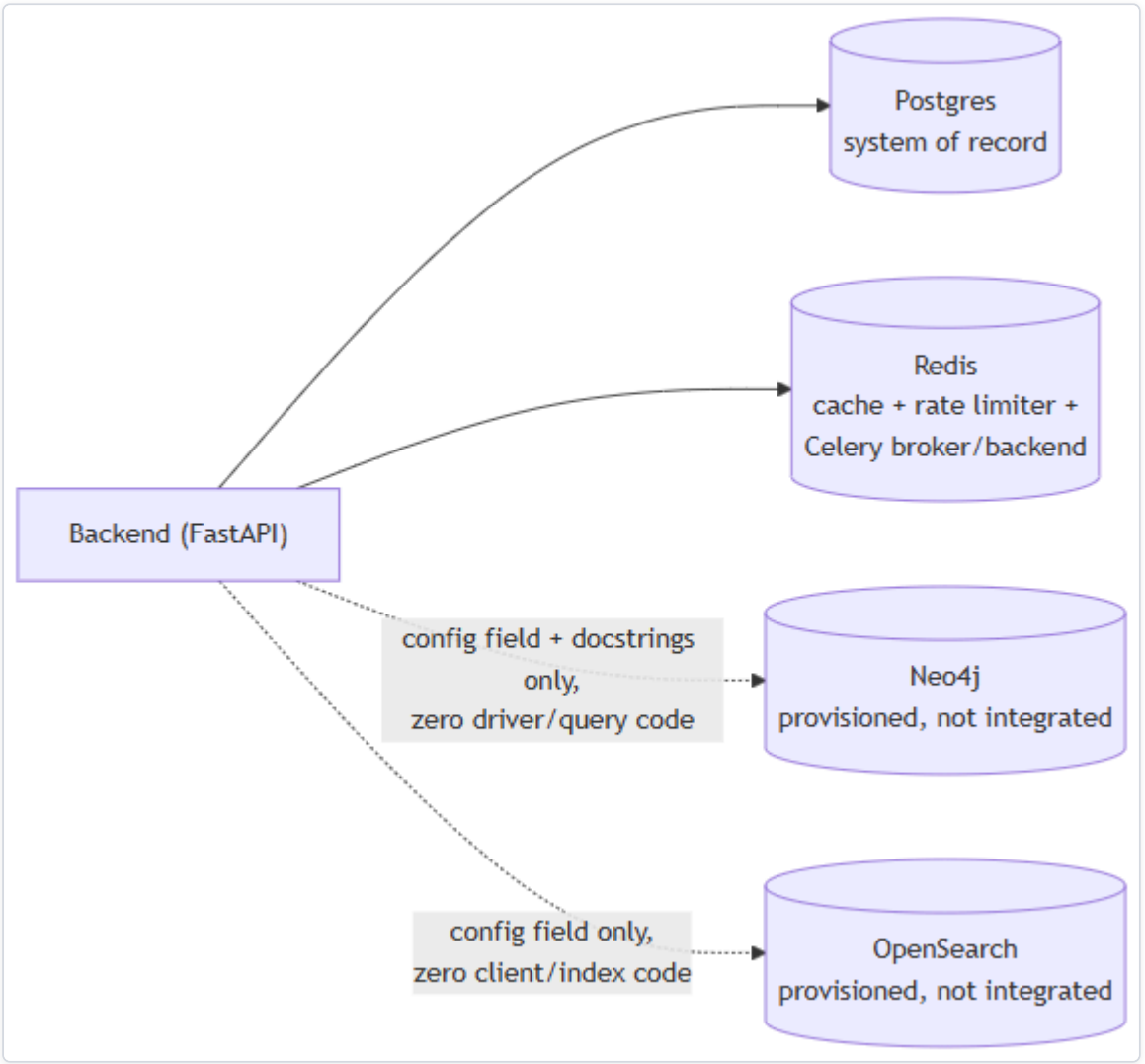

Figure 77 — Diagram: Data Layer Limitations

Security Considerations

- **Default JWT secret.** `jwt_secret_key` defaults to the literal string `"change-me-in-production"` if left unset (`app/core/config.py` line 23) — a real risk if that default ships unchanged into a running deployment. The Windows installer path mitigates this by generating a secret via a CSPRNG (`New-RandomSecret`, .NET `RandomNumberGenerator`) and writing it into an ACL-locked `.env`, but the underlying application-level default itself remains unsafe if that generation step is ever bypassed.
- **No rate limiting on `/auth/register`.** Two Redis fixed-window rate limiters exist in the codebase: one on `POST /lookup/stream` (10 calls/60s per user) and one added in a later mission-critical-reliability review (v0.2.3) on `POST /auth/login` (10 attempts/60s per account, keyed by email), both configurable and both returning an error once exceeded. `POST /auth/register` still has no rate limiting at all — no built-in throttling against repeated registration attempts.
- **CORS lockdown outside debug mode.** `app/main.py` (lines 27-33) sets `allow_origins=["http://localhost:3000"]` only when `settings.debug` is `True` (the default); otherwise `allow_origins=[]`. A `DEBUG=false` deployment therefore permits **zero** cross-origin browser requests until the source code itself is edited — there is no `CORS_ORIGINS` environment variable to configure this without a code change.
- **Network exposure nuance.** Every datastore port binding in `docker-compose.yml` (`postgres`, `redis`, `neo4j` x2, `opensearch`) uses the `127.0.0.1:<port>:<container_port>` form, which is syntactically bound to loopback only. The `backend` and `frontend` ports, by contrast, are published without an explicit host binding (`"${HOST_PORT_BACKEND:-8000}:8000"`), which is Docker's shorthand for publishing on all host interfaces.
- **Minimal input validation on the lookup value.** `LookupCreateRequest.value` is an unconstrained string server-side (`app/schemas/lookup.py`); IOC-type detection (`app/ioc/detector.py`) is regex/heuristic classification, not sanitization. `RegisterRequest.password` has a server-side 8-character minimum (`Field(min_length=8, ...)`, `app/schemas/auth.py`) applied to every registration, though no complexity constraint beyond that length floor — the Setup Wizard's own 8-character minimum (`Setup-Wizard.ps1` line 350) matches this backend constraint rather than substituting for it.

Testing Coverage

Automated test coverage exists on the backend but not on the frontend:

- **Backend**: `backend/app/tests/unit/` (13 files, ~1,486 lines) and `backend/app/tests/integration/` (3 files, ~1,009 lines), using pytest, pytest-asyncio, pytest-cov, and respx (HTTP mocking). Integration tests exercise the real orchestrator end-to-end against **fake** `BaseProvider` subclasses (never the real network-hitting connectors) with respx-mocked HTTP and a real Redis dependency reached via docker-compose (auto-skipped if Redis is unreachable) — covering parallel (not sequential) fan-out, partial-provider-failure isolation, cache hit/miss behavior, and unsupported/not-configured short-circuits.
- **Frontend**: `package.json` declares `"test": "vitest run"` and lists Vitest as a devDependency, but a repo-wide search for `*.test.*` / `*.spec.*` files under `frontend/` (excluding `node_modules` / `.next`) found **zero test files**. The test runner is configured; no actual frontend tests exist today.
- **No coverage percentages are asserted anywhere in the source.** Consistent with this document's no-fabrication policy, this appendix states only which test files and frameworks exist, not an estimated or inferred coverage percentage — anything not explicitly measured in the source is "not measured," not approximated.
- **A scripted-UI-testing caveat, not a product defect.** While automating the Setup Wizard's AI-backend dropdown for this session's testing, driving it via external Win32 message injection (synthetic `SendMessage` / `PostMessage`-style input, not a real mouse and keyboard) did not reliably trigger the panel-switch visual update for the newly-selected backend. This was root-caused to a cross-process WinForms event-notification quirk specific to synthetic input delivery, not to the underlying panel-switch code — which is ordinary WinForms event-handler logic, identical in pattern to the four backends that shipped before Groq, and was not observed to misbehave under real, mouse-driven use. It is recorded here in the same spirit as the rest of this appendix: this document does not claim exhaustive UI verification beyond what was actually exercised, and a testing-tool artifact should not be mistaken for an application bug.

Future Roadmap

This section closes the technical appendix by separating what HORIZON GRID actually does today from what its existing architecture is positioned to grow into. An **IOC (Indicator of Compromise)** is the artifact a user submits for lookup — an IP address, domain, URL, file hash, CVE ID, etc. A **provider** is a connector to one external threat-intelligence, vulnerability, or OSINT data source that the platform queries for a given IOC. Everything in "Current Capabilities" below is a recap of functionality documented elsewhere in this appendix and verified directly against source code; it introduces no new claims. Everything in "Future Possibilities" is explicitly speculative and is scoped strictly to work that the existing architecture already provisions for.

Current Capabilities

(Updated for v0.2.4 — the counts below were re-verified directly against `backend/app/ioc/types.py`, `backend/app/providers/registry.py`, and `backend/app/core/runtime_config.py` at documentation time; the AI-backend and provider counts in earlier revisions of this page were stale and have been corrected.)

- **Streaming lookup pipeline**: `POST /api/v1/lookup/stream` is a Server-Sent Events (SSE) endpoint, rate-limited to 10 calls/60s per user by default (Redis fixed-window limiter). It classifies the submitted value into one of 32 `IOCType` values, then fans out concurrently to every registered provider that supports that type.
- **18 working providers** — `virustotal`, `abuseipdb`, `otx`, `urlhaus`, `threatfox`, `malwarebazaar`, `crtsh`, `nvd`, `cisa_kev`, `mitre_attack`, `whois_rdap`, `urlscan`, `google_safe_browsing`, `hybrid_analysis`, `spamhaus`, `phishtank`, `censys`, and the Internet Intelligence Collector — each normalized behind a common `BaseProvider` interface with per-provider timeout/retry and Redis-backed result caching. See the Provider Guide for the full per-provider breakdown.
- **Eleven interchangeable AI backends**: Ollama (local, no key), Anthropic, AWS Bedrock, Google Gemini, Groq, OpenAI, Kimi (Moonshot AI), DeepSeek, xAI (Grok), Mistral AI, and OpenRouter — all exposed through one identical `call_claude_json()` method, so the rest of the application never branches on which backend is active. Backend selection fails fast with a clear error if the chosen backend isn't configured, and switching is a runtime action with no restart.
- **A deterministic, non-AI threat-scoring engine** (versioned, currently `1.0`) computes `overall_risk_score`, `confidence_score`, `malicious_probability`, and a severity band from provider-verdict consensus and correlation-graph evidence *before* the AI's final assessment runs — the AI receives that score as a fixed input to narrate and is validated against it, rather than being the source of it. See the Threat Scoring guide for the full formula.
- **Two distinct, separately-persisted AI call types**: a per-provider summary (grounded only in that provider's own returned data) for every provider that returned data, and one consolidated final assessment grounded only in the per-provider summaries, the correlation engine's output, and the already-computed threat score — never in raw provider JSON directly.
- **Anti-fabrication safeguards**: a no-evidence short-circuit that returns a hardcoded "unknown" verdict without calling the AI at all when there is nothing to summarize; a grounding pass that strips any AI-cited provider agreement/disagreement that isn't backed by real correlation data, and flags any AI-cited MITRE ATT&CK technique that isn't backed by a real correlation edge as ungrounded; an equivalent evidence-citation stripping/backfill pass on the analyst-facing explanation features; a persisted, honest AI-outcome signal (real success / correct no-evidence skip / disclosed failure) that feeds the Executive Dashboard's AI success-rate KPI instead of a fabricated number.
- **Correlation engine**: a pure, I/O-free function that extracts typed relationship edges (resolved IPs, related hashes, malware families, MITRE techniques, CVEs, and more) from normalized provider fields, boosts confidence per corroborating provider, and returns deduplicated facts and provider-agreement data. Edges are persisted to Postgres and streamed live.
- **Deterministic evidence ledger**: evidence items are built directly from provider results, AI summaries, and correlation edges — explicitly not AI-generated — so downstream AI explanations have real, checkable records to cite.

- **Grounded AI analyst features** (WHY malicious, What Is This, Provider Disagreement, False-Positive Assessment, Challenge/red-team, Smart Next Actions, Intelligence Gaps, Score Explanation) plus Copilot Q&A, IOC comparison, and hunting-query/detection-rule generation — all restricted to citing indicators and evidence that genuinely exist in the correlation graph and evidence ledger.
- **Executive Dashboard and Provider Health**: live KPI tiles (active investigations, critical/high-risk IOC count, open case counts, average threat score, provider health percentage, AI success rate) and a dedicated Provider Health page tracking real per-provider status across 1h/24h/7d/30d rolling windows, backed by real query aggregates — never a hardcoded display value.
- **Security Assessment Toolkit**: an active (not passive) Nmap-based port/service scan, DNS record lookup, TLS certificate inspection, and HTTP security-header check against a target the analyst explicitly confirms authorization for, with real cancellation support. Findings feed the investigation's risk/confidence floor without inflating `malicious_probability` on their own.
- **Postgres as the sole system of record**: lookups, provider results, AI summaries, correlation edges, evidence items, cases, users, and security-assessment runs/findings are all written there today; it is the only datastore actually populated during a lookup.
- **Redis in three confirmed real roles**: provider-result cache, per-user rate limiter on lookup creation, and Celery broker/result backend.
- **Scheduled OSINT re-crawl**: a single Celery Beat job (`run_osint_crawl`, hourly) re-crawls OSINT sources for IOCs looked up in the last 24 hours; the synchronous lookup pipeline itself does not go through Celery.
- **Role-based access control and auth**: JWT-based authentication (first registered user automatically becomes admin; self-registration is then rejected for everyone else, who must be created by an admin from the Administration console with their role chosen up front — genuinely supporting multiple administrator accounts, not one hardcoded superuser), a 3-role permission matrix enforced via a `require_permission()` dependency on every route, not just hidden in the UI.
- **Windows and Linux deployment paths**: an Inno Setup installer and WinForms Setup Wizard (Windows) and a `.deb` package (Linux) that configure providers (each with a live "Test" button; some require an API key/token, several work without one), write secrets via a CSPRNG, and run Docker Compose as the actual runtime — plus a parallel Kubernetes/Kustomize deployment alternative.
- **Automated backend test coverage**: an actively maintained pytest unit + integration suite (383 passed, 39 skipped as of the v0.2.4 regression run) covering IOC detection, provider connectors, AI grounding logic, the correlation engine, the deterministic scoring engine, and the real orchestrator end-to-end.

Future Possibilities

None of the items below exist today. They are described here only as architecturally plausible next steps.

The architecture already provisions infrastructure and patterns that go further than what is currently wired up. The items below are scoped strictly to that existing provisioning — nothing here is a new subsystem.

Provisioned infrastructure with no consuming code yet

Capability	Already provisioned today	What "completing" it would require
Neo4j graph integration	The `neo4j:5-community` container (with the APOC plugin) already runs in `docker-compose.yml`; `neo4j_uri` / `neo4j_user` / `neo4j_password` config fields already exist; the `CorrelationEdgeRecord` model and the correlation engine's own module docstring already describe edges as intended to be "mirrored into Neo4j."	Actual driver instantiation and Cypher read/write code — none exists anywhere in the backend today — so the graph database consumes the correlation edges it was already provisioned to store, rather than those edges living only in Postgres as they do now.
OpenSearch indexing	The `opensearchproject/opensearch:2.17.0` container already runs in `docker-compose.yml`; an `opensearch_url` config field already exists in `app/core/config.py`.	An indexing pipeline and search/query code against the already-running container — zero such code exists today, so no search functionality is implemented against it.

The diagram below reflects today's actual wiring (solid lines) versus the provisioned-but-unconsumed datastores (dashed lines):

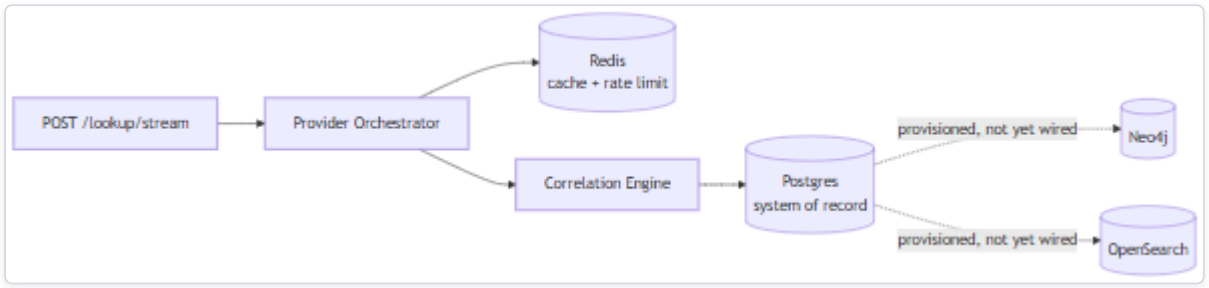

Figure 78 — Diagram: Provisioned infrastructure with no consuming code yet

Extending existing patterns

Two items that appeared in earlier revisions of this roadmap as *future* work — expanding past 4 AI backends and past 16 providers — have since actually shipped (eleven backends, eighteen providers as of this release) and have been moved into "Current Capabilities" above rather than left here as stale forward-looking claims.

- **Provider "Test" UI coverage for the remaining credential-free providers** (`crtsh`, `cisa_kev`, `mitre_attack`, `whois_rdap`, and the stub-implemented `spamhaus` / `phishtank`) — these already have real, working backend implementations and are already `requires_key = False`, so extending the Setup Wizard's existing "Test" button pattern to cover them (even as a simple "no configuration needed" confirmation) would be a bounded extension of a UI pattern that already exists for the keyed providers, not a new mechanism.
- **Further provider and AI-backend growth** — the registry already normalizes 18 heterogeneous sources and the AI layer already normalizes 11 backends behind one shared interface each (`BaseProvider` / `supports(ioc_type)` and `call_claude_json()` respectively), so onboarding further sources or backends remains a natural extension of a pattern already proven at this scale, rather than a redesign.
- **Promoting the two remaining `stubs/`-namespaced providers** (`censys`, `hybrid_analysis`) to the same top-level treatment as the other sixteen, once their real API integration depth is verified — they are registered and functional today but live in `app/providers/stubs/` alongside `spamhaus` and `phishtank`, a namespace this project uses for lighter-weight or credential-optional connectors.